

Step 1 Summary — Reinforcement Learning Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Contents

1 Step 1 — Reinforcement Learning Basics	1
1.1 How Reinforcement Learning Became a Discipline	1
1.2 Core Principles — Agent, Action, Environment	2
1.3 The MDP Formulation	3
1.4 Information Sets — When You Cannot See Everything	4
1.5 Dynamic Programming — When You Have the Model	4
1.6 Temporal-Difference Learning — Learning Without a Model	5
1.7 DQN — Deep Q-Networks	6
1.8 PPO — Proximal Policy Optimization	7
1.9 Practical Validation — Custom Implementations vs Stable-Baselines3	8

1 Step 1 — Reinforcement Learning Basics

This is a condensed summary of the foundational reinforcement learning material covered in Step 1. It is written to serve two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis.

1.1 How Reinforcement Learning Became a Discipline

Reinforcement learning did not emerge fully formed. Its roots stretch across multiple fields that were largely unaware of each other for decades.

The earliest thread comes from **animal psychology** in the early 1900s — Thorndike’s “Law of Effect” (1911) proposed that actions followed by satisfying outcomes become more likely. This is, in essence, the reward signal idea. A parallel thread ran through **optimal control theory** in the 1950s, where Bellman developed dynamic programming to solve sequential decision problems. He introduced the value function and the principle of optimality, both of which sit at the core of modern RL. A third thread came from **trial-and-error learning** in early AI research — Samuel’s checkers program (1959) learned by playing against itself, adjusting its evaluation function based on wins and losses.

These threads stayed separate until the 1980s and 1990s. Sutton’s PhD work on temporal-difference learning (1988) bridged the gap between animal learning theories and Bellman’s dynamic programming. Then Watkins formalized Q-learning (1989), giving the field its first clean, model-free control algorithm with convergence guarantees. By the time Sutton and Barto published their textbook in 1998 (updated 2018), reinforcement learning had consolidated into a recognizable discipline with its own notation, problem taxonomy, and algorithmic toolbox.

The deep learning revolution hit RL in 2013–2015 when DeepMind’s DQN learned to play Atari games from raw pixels. That demonstration made it clear that RL algorithms, combined with neural function approximators, could handle high-dimensional real-world problems. PPO followed in 2017 as a practical policy gradient method, and since then RL has expanded into robotics, game AI, recommendation systems, and language model alignment (RLHF).

The important takeaway is that RL is not just “machine learning with rewards.” It is a distinct framework for sequential decision-making under uncertainty, with its own theoretical foundations drawn from control theory, psychology, and statistics.

Read more: Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition — Chapter 1.

Free: <http://incompleteideas.net/book/the-book-2nd.html>

1.2 Core Principles — Agent, Action, Environment

The entire RL framework rests on a simple loop. An **agent** observes the current state of an **environment**, selects an **action**, and receives a **reward** signal plus a new state. Then it repeats. The agent’s goal is to learn a **policy** — a mapping from states to actions — that maximizes the cumulative reward over time.

A few terms to keep straight:

- **State** (s): a description of the environment at a particular moment. In CartPole, this is the cart position, cart velocity, pole angle, and pole angular velocity. In poker, it would include the cards dealt and the betting history.
- **Action** (a): what the agent can do. Discrete (push left or right) or continuous (apply a specific torque).
- **Reward** (r): a scalar feedback signal. The agent gets it after every action. Rewards can be sparse (only at the end of a game) or dense (every timestep).
- **Policy** (π): the agent’s strategy. Can be deterministic ($\pi(s) = a$) or stochastic ($\pi(a|s)$ gives a probability distribution over actions).
- **Value function** ($V^\pi(s)$): the expected cumulative reward from state s onward, if the agent follows policy π . This tells the agent how good a state is in the long run, not just immediately.
- **Discount factor** (γ): a number between 0 and 1 that controls how much the agent cares about future rewards versus immediate ones. $\gamma = 0.99$ means the agent is patient; $\gamma = 0.5$ means it is

myopic.

The critical design challenge in RL is the **exploration-exploitation tradeoff**. The agent must explore unfamiliar actions to discover potentially better strategies, but it also must exploit its current knowledge to collect reward. Too much exploration wastes time; too much exploitation gets stuck at local optima.

Two broad families of algorithms exist. **Value-based** methods (like DQN) learn the value of states or state-action pairs and derive a policy from those values. **Policy-based** methods (like PPO) learn the policy directly. Both have strengths; much of the RL landscape is about combining them effectively.

Read more: OpenAI Spinning Up — “Key Concepts in RL”

https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

1.3 The MDP Formulation

A **Markov Decision Process** (MDP) is the formal mathematical framework that most RL algorithms are built on. It consists of five components:

- S : a set of states
- A : a set of actions
- $P(s'|s, a)$: the transition function — probability of moving to state s' given state s and action a
- $R(s, a, s')$: the reward function
- $\gamma \in [0, 1)$: the discount factor

The “Markov” part means that the future depends only on the current state, not on the history of how you got there. This is a strong assumption — and one that does not hold in many interesting settings (like poker, where the history of bets matters). We deal with that in Section 4.

The objective is to find a policy π^* that maximizes the expected discounted return:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

The key recursive relationship is the **Bellman equation**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

This says: the value of a state equals the expected immediate reward plus the discounted value of wherever you end up. All of dynamic programming, TD learning, and deep RL build on variations of this equation.

Read more: Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 3 — Finite Markov Decision Processes.

Free: <http://incompleteideas.net/book/the-book-2nd.html>

1.4 Information Sets — When You Cannot See Everything

Standard MDPs assume the agent can fully observe the state. Real problems often violate this. In poker, you do not see the opponent’s cards. In real-time strategy games, you have fog of war. These are **partially observable** settings.

The formal extension is the **Partially Observable MDP** (POMDP), which adds an observation function — the agent sees observations that are noisy or incomplete views of the true state. But POMDPs are notoriously hard to solve optimally.

In game theory, the same idea is captured through **information sets**. An information set groups together all game states that a player cannot distinguish between, given what they have observed. In Kuhn Poker (3 cards, 2 players), when you hold a Jack and no betting has occurred, you are in an information set — you know your card but not your opponent’s, so you cannot tell if the true state is “I have Jack, opponent has Queen” or “I have Jack, opponent has King.”

Algorithms that work in these settings — like CFR (Counterfactual Regret Minimization, covered in Step 2) — operate over information sets rather than individual states. This is a fundamentally different computational problem than standard RL, and it is why game-playing AI requires its own set of tools beyond what DQN or PPO provide out of the box.

Understanding information sets now matters because the later steps (5–8) deal entirely with imperfect-information games. The algorithms change, but the core concept stays: you are making decisions based on what you can observe, reasoning about what you cannot.

Read more: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, Chapter 5.

Free: <http://www.masfoundations.org/download.html>

1.5 Dynamic Programming — When You Have the Model

Dynamic programming (DP) is the oldest approach to solving MDPs. It works when you have complete knowledge of the environment: the transition probabilities $P(s'|s, a)$ and the reward function $R(s, a, s')$. In practice, this means you know the rules of the game perfectly.

The two main DP algorithms are:

Policy Iteration alternates between two steps. First, *policy evaluation*: given a fixed policy, compute the value of every state by repeatedly applying the Bellman equation until convergence. Second, *policy improvement*: update the policy to be greedy with respect to the newly computed values. Repeat until the policy stops changing. It is guaranteed to converge to the optimal policy.

Value Iteration is a shortcut. Instead of fully evaluating a policy before improving it, it combines evaluation and improvement into a single update. At each step, for every state, pick the action that maximizes the expected one-step return plus the discounted value of the next state. It converges to the optimal value function, from which the optimal policy is extracted.

DP is beneficial because it gives exact solutions and has clean convergence guarantees. The downside is obvious: you need the full model, and the computation scales with the number of states. For Kuhn Poker with 12 states, DP is trivial. For Go with 10^{170} states, it is impossible.

The reason DP matters even though we rarely use it directly is that every subsequent method — Monte Carlo, TD learning, DQN, PPO — can be understood as an approximation to the DP solution when the model is unknown or the state space is too large. DP is the theoretical ceiling that practical algorithms try to approach.

Read more: Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 4 — Dynamic Programming.
Free: <http://incompleteideas.net/book/the-book-2nd.html>

1.6 Temporal-Difference Learning — Learning Without a Model

Temporal-difference (TD) learning is the breakthrough that made RL practical. Unlike DP, it does not require a model of the environment. Unlike Monte Carlo methods, it does not have to wait until the end of an episode to learn. It updates value estimates after every single step, using a bootstrap: the current estimate of the next state’s value stands in for the true future return.

The basic TD update (TD(0)) for the value function is:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

The term in brackets is the **TD error** — the difference between what happened (reward + estimated future value) and what was predicted. If the TD error is positive, the state was better than expected; if negative, worse.

The control version of this idea is **Q-learning** (Watkins, 1989):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Q-learning learns the value of state-action pairs and uses the max operator to be **off-policy** — it learns about the optimal policy even while following an exploratory one. This is what makes it so practical.

TD learning is beneficial compared to DP because it needs no model. It is beneficial compared to Monte Carlo because it learns online (every step, not every episode) and has lower variance. The bootstrapping

introduces some bias, but in practice this tradeoff is overwhelmingly favorable. TD methods are the foundation that DQN and most modern value-based algorithms are built on.

SARSA is the on-policy cousin of Q-learning — it updates using the action the agent actually took next, rather than the best possible action. This makes it more conservative and sometimes safer in practice, but it cannot learn an optimal policy while following a suboptimal exploratory policy.

Read more: Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 6 — Temporal-Difference Learning.
Free: <http://incompleteideas.net/book/the-book-2nd.html>

1.7 DQN — Deep Q-Networks

DQN (Mnih et al., 2015) is the algorithm that proved deep neural networks could work as function approximators in RL at scale. Before DQN, Q-learning was limited to tabular settings or hand-crafted features. DQN showed it could learn to play 49 Atari games from raw pixel input, surpassing human performance on many of them.

How it works. DQN replaces the Q-table with a neural network that takes a state as input and outputs Q-values for all actions. The network is trained by minimizing the squared TD error: the difference between the predicted Q-value and the target (reward + discounted max Q-value of the next state). At each step, the agent picks the action with the highest Q-value (exploitation) or a random action with probability ϵ (exploration).

Two tricks make this stable enough to work:

1. **Experience replay.** Instead of training on sequential transitions (which are highly correlated), transitions are stored in a large buffer and sampled randomly in mini-batches. This breaks the correlation, reduces variance, and lets each experience be reused multiple times. It is the same principle as shuffling your training data in supervised learning.
2. **Target network.** A separate copy of the Q-network is maintained and updated only periodically (every few thousand steps or episodes). The TD target is computed using this frozen copy. Without it, the target shifts with every training step — you are chasing a moving target, which causes oscillation and divergence. The target network decouples the update and stabilizes learning.

Why DQN matters compared to what came before: tabular Q-learning cannot handle high-dimensional states (images, continuous observations); function approximation with Q-learning alone was known to be unstable (the “deadly triad” of off-policy + function approximation + bootstrapping). DQN’s two tricks — replay and target networks — addressed the instability, making deep value-based RL practical for the first time.

Limitations. DQN handles only discrete action spaces. It can overestimate Q-values (addressed by Double DQN). It is off-policy, which is sample-efficient but can lead to stale data in the replay buffer.

And fundamentally, it learns a greedy deterministic policy — there is no natural way to express stochastic policies, which matters in games where mixing strategies is optimal.

In our Step 1 implementation, DQN solved CartPole-v1 (achieving a 100-episode average of 477.5 against a 475 target) after ~1000 episodes of training. The most impactful hyperparameters were network size, epsilon minimum, and target network sync frequency.

A note on neural networks in this step. The Q-network inside DQN is a standard multi-layer perceptron (MLP) used here as a practical tool — a function that maps states to Q-values. For Step 1, the neural network internals (weight initialization, gradient flow, activation functions) were treated as a black box: we used PyTorch’s defaults and focused on the RL algorithm itself. A more thorough treatment of neural network architectures and how they interact with equilibrium-finding methods will come in Step 5 (Neural Equilibrium Approximation).

Read more: Mnih, V. et al. (2015). “Human-level control through deep reinforcement learning.” *Nature*, 518(7540), 529–533.
arXiv: <https://arxiv.org/abs/1509.06461>

1.8 PPO — Proximal Policy Optimization

PPO (Schulman et al., 2017) takes a fundamentally different approach to RL than DQN. Instead of learning the value of actions and deriving a policy from those values, PPO **learns the policy directly** — a neural network outputs a probability distribution over actions, and the network is trained to increase the probability of actions that lead to high returns.

The problem PPO solves. Policy gradient methods are powerful in theory but notoriously unstable in practice. The basic REINFORCE algorithm suffers from high variance and can take destructively large steps that ruin a good policy. Trust Region Policy Optimization (TRPO, Schulman et al., 2015) addressed this by constraining how much the policy can change per update, but TRPO requires expensive second-order optimization (computing the Hessian).

How PPO works. PPO keeps the stability idea from TRPO but replaces the constraint with a much simpler mechanism: **clipping**. The key quantity is the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — how much more (or less) likely the new policy is to take the same action the old policy took. PPO clips this ratio to the range $[1 - \epsilon, 1 + \epsilon]$ (typically $\epsilon = 0.2$), which means:

- If an action turns out to be good (positive advantage) and the ratio grows beyond $1 + \epsilon$, the gradient is zeroed — the policy is not allowed to become too eager for that action.
- If an action turns out to be bad (negative advantage) and the ratio drops below $1 - \epsilon$, the gradient is also zeroed — preventing the policy from overcorrecting.

This is the **clipped surrogate objective**, and it is what makes PPO work. It gets TRPO-like stability using only first-order gradients (no Hessian), making it substantially cheaper to compute and easier to

implement.

Architecture. PPO is actor-critic: a **policy network** (actor) outputs action probabilities, and a **value network** (critic) estimates state values. The value network enables Generalized Advantage Estimation (GAE), which provides low-variance advantage estimates for the policy gradient. Both networks are trained simultaneously.

Why PPO matters compared to DQN: PPO naturally handles both discrete and continuous action spaces. It can learn stochastic policies, which is critical in game settings where mixing strategies is necessary. It is on-policy (uses fresh data from the current policy), which avoids stale data issues. And the clipped objective makes it remarkably robust — PPO is the default choice for most RL applications today, including RLHF for language models.

Limitations. Being on-policy, PPO is less sample-efficient than off-policy methods like DQN (each experience is used for only a few update epochs before being discarded). It requires careful tuning of the advantage estimation (GAE lambda), entropy bonus, and network architecture. And it can still struggle with sparse rewards — if the agent rarely encounters success, there is little signal to learn from.

In our Step 1 implementation, PPO solved LunarLander-v3 (100-episode average of 202.2 against a 200 target) at ~264K environment steps. The decisive factors were network capacity (doubling from [64,64] to [128,128]) and advantage normalization.

A note on neural networks in this step. Same as with DQN — the policy and value networks here are standard MLPs whose internals were not the focus of this step. We adopted a practical “it works” stance toward the neural components and concentrated on understanding the PPO algorithm logic (clipping, GAE, rollout structure). The deeper interplay between neural network design and RL convergence — particularly in the context of game-playing agents — is deferred to Step 5.

Read more: Schulman, J. et al. (2017). “Proximal Policy Optimization Algorithms.”
arXiv: <https://arxiv.org/abs/1707.06347>

1.9 Practical Validation — Custom Implementations vs Stable-Baselines3

To verify that our from-scratch implementations actually work, we compared them against Stable-Baselines3 (SB3) — a widely used, well-tested RL library. Both our implementations and SB3 were given matched hyperparameters and equivalent training budgets.

1.9.1 DQN on CartPole-v1

Our custom DQN solves CartPole by episode ~1011, reaching a 100-episode rolling average of 477.5 (above the 475 target). SB3’s DQN, given 750K environment steps (17,176 episodes), never converges — its rolling average stays around 30 and never approaches the target.

The gap comes from three implementation-level differences:

DQN on CartPole-v1 — Custom vs SB3

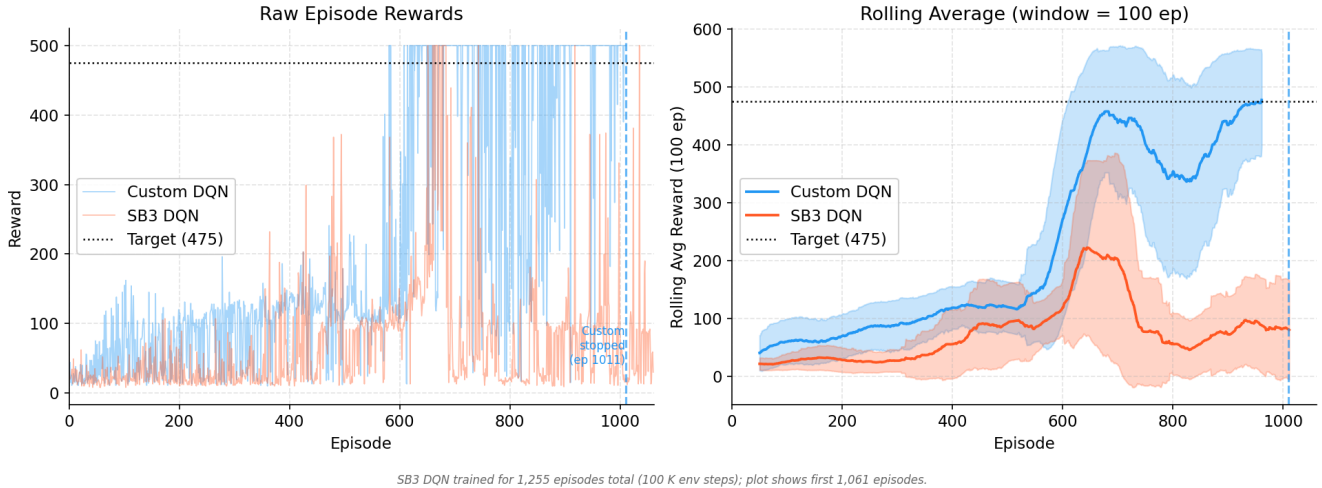


Figure 1: DQN comparison — Custom vs SB3

- **Exploration schedule.** Our epsilon decays per episode ($\times 0.995$ each), concentrating exploration where it matters. SB3 decays epsilon linearly over steps, which in short-episode environments like CartPole wastes many episodes in near-random mode.
- **Target network sync.** Our code syncs every 5 episodes — an adaptive schedule that gives faster feedback during early learning. SB3 uses a fixed 1000-step interval.
- **Early stopping.** Our agent saves the best model and stops at peak performance, avoiding catastrophic forgetting. SB3 runs the full budget, and the policy oscillates.

These are genuine algorithmic choices, not hyperparameter unfairness. SB3’s defaults are designed for robust performance across diverse, long-training domains like Atari games — not brief episode classic control environments like CartPole; our task-specific choices simply work better on CartPole.

1.9.2 PPO on LunarLander-v3

Our custom PPO crosses the 200 target around episode 543 (264K steps). SB3’s PPO with the same 500K-step budget peaks at a best rolling-100 average of 131.2 — still climbing but not converged.

The gap is narrower here than with DQN. Both use identical PPO hyperparameters. The remaining difference likely comes from SB3’s orthogonal weight initialization and built-in observation normalization, which help long runs but may slow early convergence. Given more training budget (1–2M steps), SB3 PPO would likely reach the target.

1.9.3 Takeaway

A clean from-scratch implementation is 300–500 lines versus SB3’s ~10K. The tradeoff: fewer features but complete transparency and debuggability. For research purposes — particularly the opponent exploitation work planned for Steps 7–8 — custom implementations allow surgical modifications (e.g., injecting belief-

PPO on LunarLander-v3 — Custom vs SB3

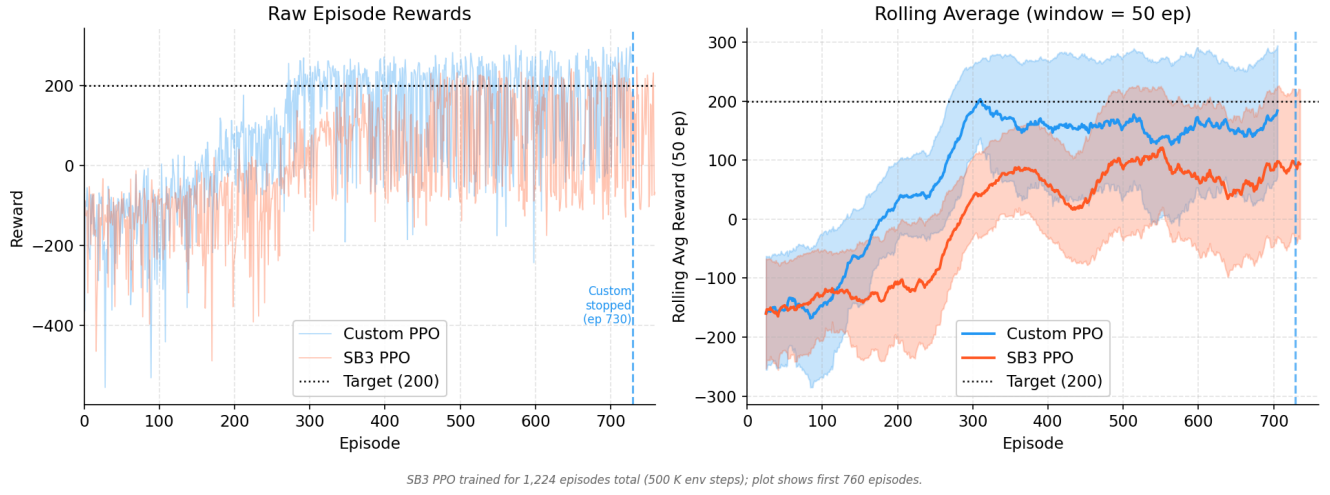


Figure 2: PPO comparison — Custom vs SB3

state observations) that would require deep SB3 subclassing. That practical flexibility is why we built from scratch.

Step 2 Summary — Game Theory & CFR Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Contents

2 Step 2 — Game Theory & CFR Basics	1
2.1 From Single-Agent RL to Multi-Agent Strategic Interaction	1
2.2 Extensive-Form Games and Information Sets	2
2.3 Minimax Theorem	2
2.4 Nash Equilibrium	3
2.5 Regret Matching — The Building Block	4
2.6 Counterfactual Regret Minimization (CFR)	4
2.7 Kuhn Poker — The Simplest Testbed	5
2.8 Exploitability — Measuring Distance from Nash	6
2.9 Connections to Step 1 and Forward Pointers	7
2.10 Empirical Visualizations	7

2 Step 2 — Game Theory & CFR Basics

This is a condensed summary of the game theory and counterfactual regret minimization material covered in Step 2. It serves two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis.

2.1 From Single-Agent RL to Multi-Agent Strategic Interaction

Step 1 treated environments as passive — the cart-pole doesn’t fight back, the lander doesn’t try to crash you. The agent optimized against fixed dynamics. Step 2 introduces a fundamentally different problem: the environment includes another *strategic* decision-maker whose actions depend on yours.

This shift breaks the MDP framework. In a single-agent setting, the optimal policy is fixed — there exists a single best way to act regardless of what strategy you considered before. In a multi-agent setting, the “best” strategy depends on what the opponent does, and the opponent’s best strategy depends on what you do. This circularity is the core challenge of game theory.

The formalization begins with **normal-form games** (simultaneous moves, like Rock-Paper-Scissors) and extends to **extensive-form games** (sequential moves with a tree structure, like poker). In both cases,

the solution concept shifts from “optimal policy” to **Nash equilibrium** — a strategy profile where no player can improve by unilaterally changing their own strategy.

Read more: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems*, Chapters 3–4.
Free: <http://www.masfoundations.org/download.html>

2.2 Extensive-Form Games and Information Sets

An **extensive-form game** represents strategic interaction as a tree. Each node is a decision point for one player (or “chance” for random events like card deals). Edges represent actions, and leaves carry payoffs for each player.

The critical distinction is between **perfect** and **imperfect** information:

- **Perfect information** (chess, Go): every player sees the full game state. The game tree has no grouped nodes — each node is its own information set. Minimax and alpha-beta pruning solve these games.
- **Imperfect information** (poker, many real-world problems): players cannot observe some aspects of the state. In poker, you see your own cards but not your opponent’s. This means multiple game states are *indistinguishable* to the acting player.

An **information set** groups all game states that a player cannot tell apart. At an information set, the player must choose the same strategy for all states in the group (since they cannot distinguish them). This constraint is what makes imperfect-information games fundamentally harder than perfect-information ones — you cannot simply pick the best action for each state; you must pick one action that works well *on average* across all states in the information set.

In Kuhn Poker, the information set “2pb” contains two game states: “I hold Queen, opponent holds Jack, history is pass-bet” and “I hold Queen, opponent holds King, history is pass-bet.” The player holding Queen cannot distinguish these and must use the same strategy (call probability) for both.

Read more: Shoham & Leyton-Brown (2008). *Multiagent Systems*, Chapter 5 — Extensive-form games.

2.3 Minimax Theorem

The Minimax Theorem, established by John von Neumann in 1928, provides a foundational solution concept for two-player zero-sum games. It states that for every finite, two-player, zero-sum game, there exists a strategy for both players where the maximum expected loss is minimized. In other words, each player can guarantee a specific expected payoff, known as the “value of the game,” regardless of the opponent’s strategy. This creates a scenario where one player’s optimal strategy is to maximize their minimum reward (maximin), while the opponent seeks to minimize the first player’s maximum reward

(minimax). When these two values equal, the game is in equilibrium. This algorithm directly relates to Nash Equilibrium: in the specific case of two-player zero-sum games, a Minimax strategy profile is equivalent to a Nash equilibrium. Discovering this Minimax solution guarantees safety against any exploitative strategy the opponent might employ.

2.4 Nash Equilibrium

A **Nash equilibrium** is a strategy profile where each player’s strategy is a best response to every other player’s strategy. Formally, for each player i :

$$u_i(\sigma_i^*, \sigma_{-i}^*) \geq u_i(\sigma_i, \sigma_{-i}^*) \quad \forall \sigma_i$$

No player can improve their expected payoff by unilaterally deviating. This does not mean everyone is happy with the outcome (Prisoner’s Dilemma), just that no one can do better by changing only their own strategy.

In games extended to more than two players (N-player games) or general-sum games, the properties of Nash equilibrium become significantly more complex. In these settings, finding an exact Nash equilibrium is known to be computationally intractable, often falling into the complexity class PPAD-complete. Furthermore, the equilibria lose their safety guarantee: playing a Nash strategy in a 3-player game does not protect against two opponents who might deviate from equilibrium, potentially forming temporary coalitions that exploit the third player. This dynamic necessitates entirely different evaluation and training frameworks (such as those discussed in Steps 9 and 11) since the direct equivalence between Nash and minimax optimality no longer holds.

Key properties:

- **Existence:** Nash (1950) proved that every finite game has at least one Nash equilibrium (possibly in mixed strategies). This is a foundational theorem but does not help with computation.
- **Uniqueness:** Games may have multiple equilibria. Kuhn Poker has a one-parameter *family* of Nash equilibria indexed by $\alpha \in [0, 1/3]$.
- **Mixed strategies:** In many games, the equilibrium requires randomization. In Kuhn Poker, the Nash equilibrium has Player 2 bluffing with Jack exactly 1/3 of the time — any other frequency is exploitable.

For 2-player zero-sum games (where one player’s gain is the other’s loss), Nash equilibria have a special property: they are **minimax optimal**. Playing your Nash strategy guarantees you at least the game value, regardless of what the opponent does. This is the safety guarantee that makes Nash equilibrium the baseline for poker AI.

Read more: Nash, J.F. (1950). “Equilibrium points in n-person games.” *PNAS*, 36(1), 48–49.

2.5 Regret Matching — The Building Block

Before CFR can minimize regret across a game tree, we need a mechanism to minimize regret at a single decision point. **Regret matching** is that mechanism.

The idea: after many rounds of play, for each action a , compute the **cumulative regret** — how much better you would have done if you had always played a instead of your actual mixed strategy. Then set your next strategy proportional to the *positive* regrets:

$$\sigma^{T+1}(a) = \begin{cases} \frac{R^{T,+}(a)}{\sum_{a'} R^{T,+}(a')} & \text{if } \sum > 0 \\ \frac{1}{|A|} & \text{otherwise} \end{cases}$$

Convergence guarantee: Blackwell (1956) and Hart & Mas-Colell (2000) proved that regret matching ensures average regret converges to zero at rate $O(\sqrt{T})$. In a two-player zero-sum game, if both players use regret matching at their respective decision points, the average strategy profile converges to a Nash equilibrium.

A simple example is Rock-Paper-Scissors. If you play regret matching against a fixed opponent who always plays Rock, your cumulative regret for Paper will grow (Paper beats Rock), while regret for Scissors will stay at zero. Your strategy will gradually shift toward playing Paper with increasing probability, which is the correct best response.

Read more: Neller, T.W. & Lanctot, M. (2013). “An Introduction to Counterfactual Regret Minimization,” Section 1–2.

2.6 Counterfactual Regret Minimization (CFR)

CFR (Zinkevich et al., 2007) extends regret matching to extensive-form games. The key insight: decompose the total regret of a game strategy into **local regrets** at each information set, then minimize each local regret independently using regret matching. If each information set’s regret converges to zero, the overall strategy converges to Nash equilibrium.

The “counterfactual” part is crucial. At each information set I , the regret for action a is not simply “how much better a would have been.” It is the *counterfactual* regret — the improvement assuming the player had intentionally played to reach I (reach probability = 1 for the player) but everything else (opponent strategy, chance events) stayed the same. Formally:

$$R^T(I, a) = \sum_{t=1}^T \pi_{-i}^t \cdot (v^t(I, a) - v^t(I))$$

where π_{-i}^t is the opponent’s reach probability and $v^t(I, a)$ is the counterfactual value of action a at I on iteration t .

Why opponent reach probability? The regret at an information set matters more when the opponent is likely to have played such that we reach that information set. Weighting by π_{-i} ensures that regret is proportional to the frequency with which the information set is “relevant” to the game outcome.

Algorithm (vanilla CFR with chance sampling):

1. Initialize all cumulative regrets and strategy sums to zero
2. For T iterations:
 - a. Sample a random card deal (chance sampling)
 - b. Recursively traverse the game tree from root
 - c. At each info set, compute strategy via regret matching
 - d. For each action, recurse into the subtree (negating utility for zero-sum)
 - e. Update cumulative regrets weighted by opponent reach probability
 - f. Accumulate strategy weighted by player’s own reach probability
3. Output the **average** strategy (not the final iteration’s strategy)

Convergence: The average strategy profile converges to Nash equilibrium at rate $O(1/\sqrt{T})$ for exploitability, where T is the number of iterations (Theorem 4, Zinkevich et al. 2007). The convergence bound is:

$$\text{exploit}(\bar{\sigma}^T) \leq O\left(\Delta\sqrt{|I|/T}\right)$$

where Δ is the maximum payoff range and $|I|$ is the number of information sets.

Read more: Zinkevich, M. et al. (2007). “Regret Minimization in Games with Incomplete Information.” *NeurIPS*. Neller & Lanctot (2013). Sections 3–5 (full Kuhn Poker walkthrough).

2.7 Kuhn Poker — The Simplest Testbed

Kuhn Poker (Kuhn, 1950) is the standard minimal example for imperfect-information game algorithms. With only 3 cards, 2 players, and 12 information sets, it is small enough to solve analytically yet rich enough to exhibit the key phenomena: bluffing, information asymmetry, mixed equilibria.

Rules: 3 cards {J, Q, K}, each player antes 1 chip, each receives 1 private card. Player 0 acts first: pass or bet. Player 1 responds: pass or bet. If Player 0 passed and Player 1 bet, Player 0 gets to respond again. Showdown with higher card winning.

Nash equilibrium (parameterized by $\alpha \in [0, 1/3]$):

- **Player 0 with J:** Bet (bluff) with probability α . If facing bet after passing, always fold.
- **Player 0 with Q:** Always pass. If facing bet after passing, call with probability $1/3 + \alpha$.
- **Player 0 with K:** Bet with probability 3α . If facing bet after passing, always call.
- **Player 1 with J:** After pass, bet (bluff) with probability $1/3$. After bet, always fold.

- **Player 1 with Q:** After pass, always pass. After bet, call with probability $1/3 + \alpha$.
- **Player 1 with K:** Always bet, always call.

Game value: Player 0’s expected payoff at Nash is $-1/18 \approx -0.0556$. Player 0 has a structural disadvantage from acting first.

Why Kuhn matters: Every key concept in imperfect-information game solving appears here in miniature — bluffing (J bets despite being worst card), information asymmetry (Player 2 sees Player 1’s action before deciding), mixed strategies (exact $1/3$ bluff frequency), indifference (Q is indifferent between call and fold at certain info sets). If your algorithm cannot solve Kuhn correctly, it cannot solve anything.

Read more: Kuhn, H.W. (1950). “Simplified Two-Person Poker.” *Contributions to the Theory of Games*, Vol. 1.

2.8 Exploitability — Measuring Distance from Nash

Exploitability is the primary evaluation metric for strategies in imperfect-information games. It measures how much a strategy can be exploited by a worst-case opponent:

$$\text{exploit}(\sigma) = BR_0(\sigma_1) + BR_1(\sigma_0)$$

where $BR_i(\sigma_{-i})$ is the expected payoff of player i ’s best response to the opponent’s strategy.

At Nash equilibrium, exploitability is exactly zero — neither player can improve by deviating. For approximate Nash equilibria, exploitability quantifies the approximation quality.

Best response computation in imperfect-information games is subtle. The best-responding player must choose the same action at all states within an information set (they cannot distinguish them). A naive approach that picks the best action per game state (using knowledge of the opponent’s private card) computes an “oracle” value, not a true best response. For Kuhn Poker, the correct approach enumerates all $2^6 = 64$ pure strategies and evaluates each.

Convergence rate: For vanilla CFR, exploitability decreases as $O(1/\sqrt{T})$. On a log-log plot, this appears as a line with slope ≈ -0.5 . Our measured slope was -0.489 , confirming theoretical predictions.

Why not just game value? A strategy can achieve the correct game value while still being exploitable. Consider a Kuhn strategy where Player 1 always bluffs with J (instead of $1/3$ of the time). The average game value might be close to $-1/18$, but the strategy is highly exploitable — Player 0 can always call with Q and profit. Exploitability catches this; game value alone does not.

This metric recurs throughout the thesis. Steps 7–8 (opponent modeling, safe exploitation) use exploitability as the primary measure. Step 14 builds a general evaluation framework around it.

2.9 Connections to Step 1 and Forward Pointers

Local optimization → global convergence. In Step 1, DQN minimizes TD error at each state independently, yet the overall Q-function converges to optimal. In Step 2, CFR minimizes regret at each information set independently, yet the overall strategy converges to Nash. Both demonstrate the same principle: local updates, when properly structured, achieve global objectives.

Strategy accumulation experience replay. CFR outputs the *average* strategy, not the last iteration’s strategy. This averaging stabilizes convergence, just as experience replay in DQN stabilizes learning by preventing the agent from over-fitting to recent transitions. Both are memory mechanisms that smooth out noise.

Forward: Step 2 covers *full tree traversal* CFR — every information set is visited every iteration. For larger games, this is intractable. Step 3 introduces Monte Carlo CFR (MCCFR), which samples parts of the tree, trading exactness for scalability. Step 4 adds game abstraction, reducing the game size. Step 5 replaces tabular strategies with neural networks.

Open questions: - Why does the *average* strategy converge but not the current strategy? (Intuition: the current strategy overshoots corrections, while averaging dampens oscillations — same reason as Polyak averaging in optimization.) - CFR is proven for 2-player zero-sum games. What about N -player or general-sum? (Steps 9 and 11 explore this frontier.) - How do we handle games too large for full tree traversal? (Step 3 — Monte Carlo CFR.)

Read more: Bowling, M. et al. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. This paper used CFR+ (a variant) to solve heads-up limit Texas Hold’em — the first non-trivial imperfect-information game to be essentially solved.

2.10 Empirical Visualizations

To validate the theoretical guarantees of CFR and its application to Kuhn Poker, several metrics were tracked during the training process:

Game Value Convergence

This graph illustrates how the empirical average game value approaches the theoretical expectation of $-1/18$ as CFR iterations increase, confirming the strategy’s stabilization.

Exploitability Convergence

Tracking exploitability over time shows the $O(1/\sqrt{T})$ convergence rate. The log-log plot demonstrates a linear decay, validating that the average strategy forms an approximate Nash equilibrium.

Strategy Analysis

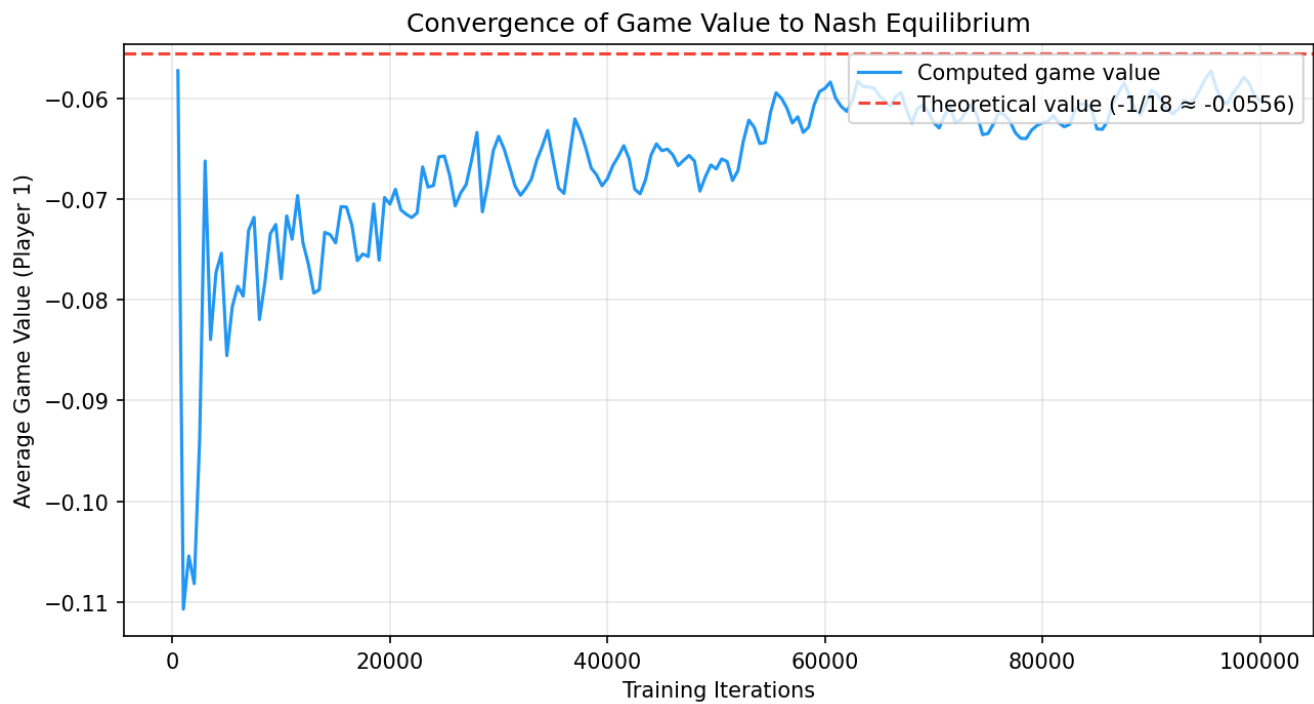


Figure 1: Game Value Convergence

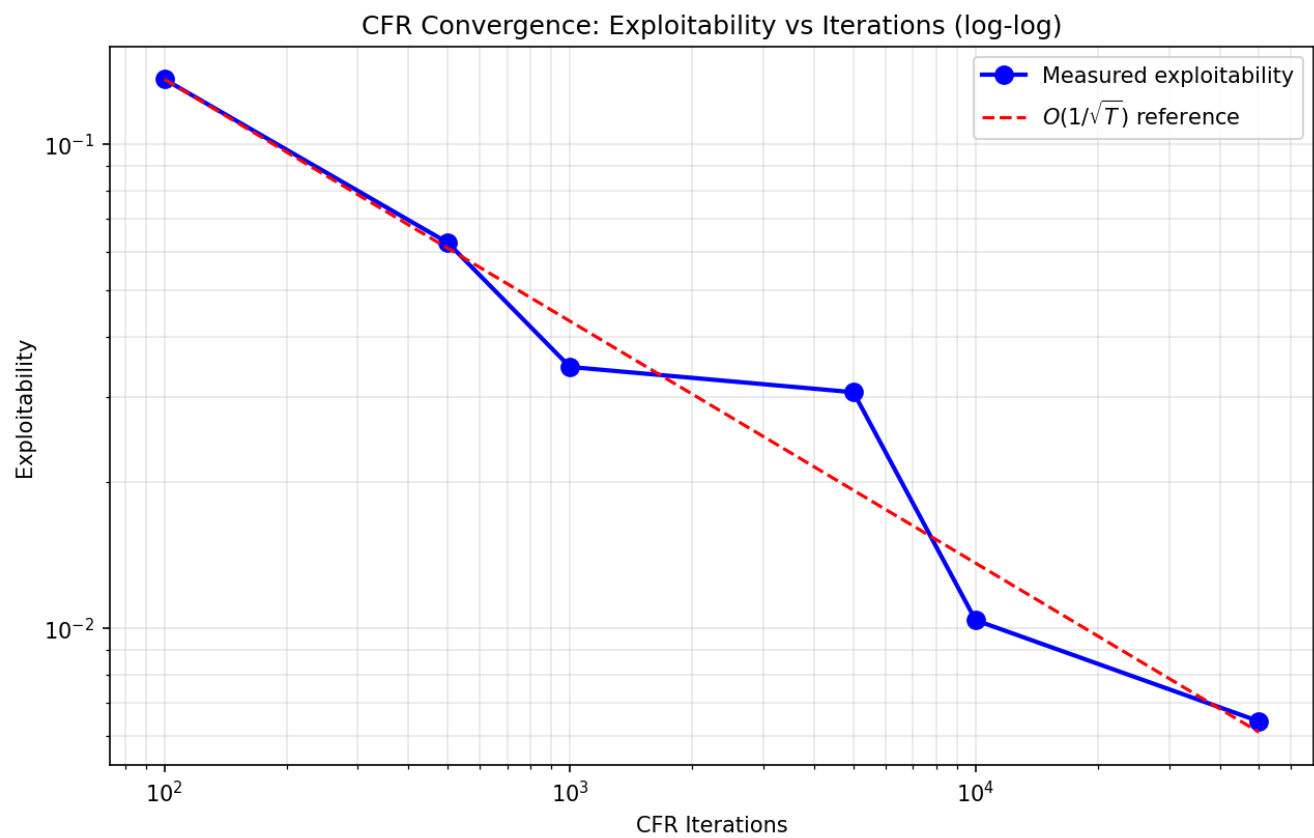


Figure 2: Exploitability Convergence

Kuhn Poker — CFR Strategy Analysis

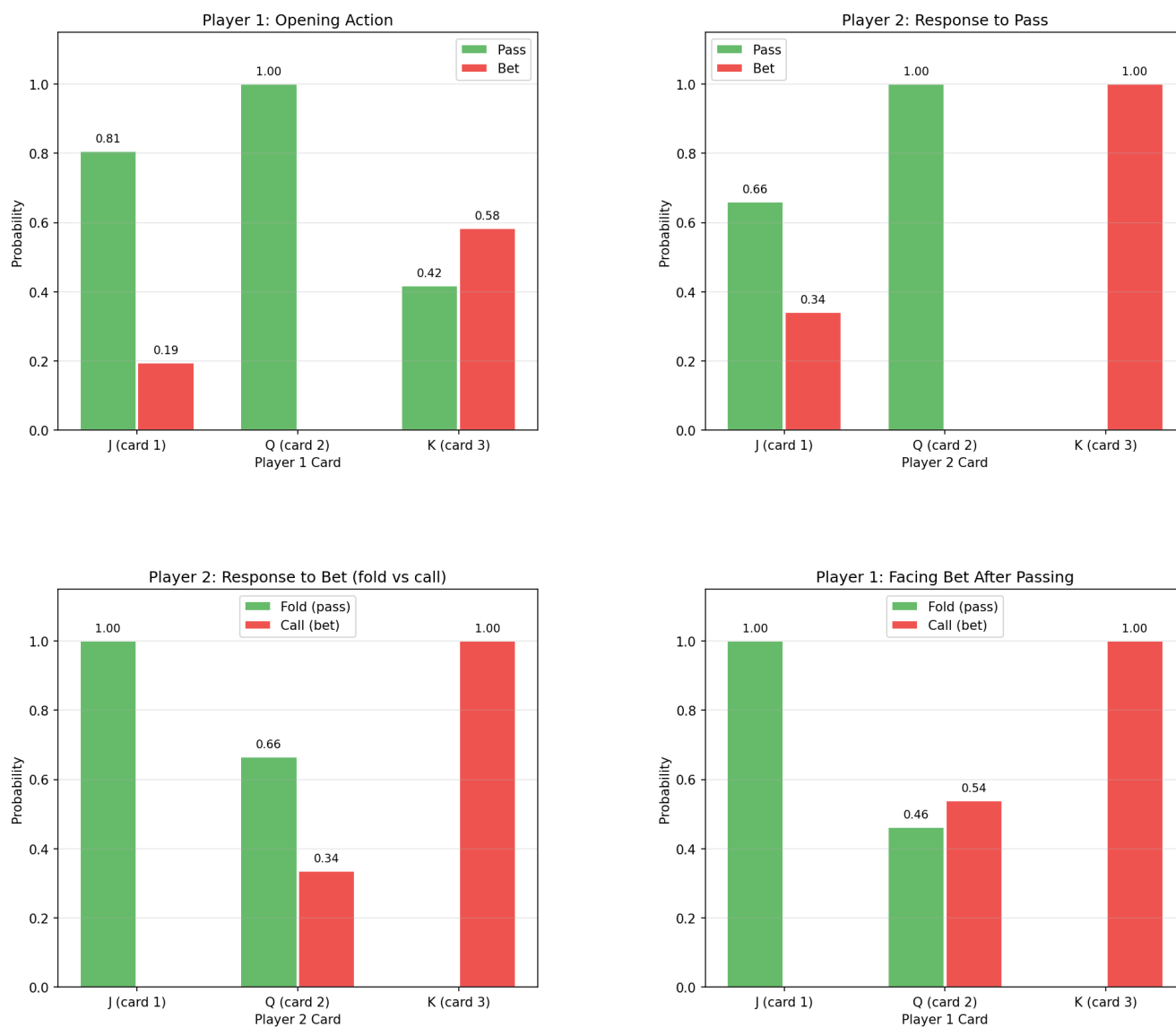


Figure 3: Strategy Analysis

This visualization breaks down the specific action probabilities at various information sets, demonstrating the emergence of the optimal bluffing and calling frequencies dictated by the Kuhn Poker Nash equilibrium family.

Step 3 Summary — CFR Variants & Monte Carlo Methods

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Contents

3 Step 3 — CFR Variants & Monte Carlo Methods	1
3.1 Why Vanilla CFR Breaks at Scale	1
3.2 Monte Carlo Tree Search as Inspiration	2
3.3 Markov Chains and the Law of Large Numbers	3
3.4 The Mathematics of Poker	4
3.5 Leduc Poker — Scaling Up the Benchmark	4
3.6 CFR+ — The Regret Flooring Trick	5
3.7 MCCFR — Sampling the Game Tree	7
3.8 The Variance-Speed Tradeoff	8
3.9 When MCCFR Wins — The Crossover Point	10
3.10 Connections to Step 2 and Forward Pointers	12

3 Step 3 — CFR Variants & Monte Carlo Methods

This is a condensed summary of the scalability material covered in Step 3. It serves two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis.

3.1 Why Vanilla CFR Breaks at Scale

Step 2 established a working Vanilla CFR solver on Kuhn Poker — 12 information sets, 58 terminal nodes, solvable in milliseconds. That experience is deceptive. Vanilla CFR requires a **full traversal of the game tree on every iteration**, and real games are not Kuhn. Leduc Poker has 936 information sets ($\sim 78\times$ larger); Limit Texas Hold'em has over 10^{14} . No computer can enumerate that tree even once.

Step 3 closes the scale gap through two complementary mechanisms, developed independently in the game-theory literature and later combined in practice:

1. **CFR+ (Tammelin, 2014)** — a modification of vanilla CFR that achieves dramatically faster convergence through regret flooring, linear strategy averaging, and alternating updates. CFR+ was

the algorithm that “solved” Heads-Up Limit Texas Hold’em in 2015 — the first non-trivial poker variant to be essentially solved.

2. **Monte Carlo CFR (Lanctot et al., 2009)** — instead of traversing the entire game tree each iteration, MCCFR samples a small portion. Each iteration is orders of magnitude cheaper, at the cost of noisier updates. This is the only tractable approach for games where full traversal is physically impossible.

For the thesis, MCCFR is the algorithm that computes the baseline Nash strategy — the “play it safe” anchor from which an adaptive agent deviates based on opponent observations (Contribution #1). CFR+ makes that computation tractable on the intermediate benchmark games used in Steps 5–8.

Read more: Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149.

3.2 Monte Carlo Tree Search as Inspiration

Monte Carlo Tree Search (MCTS) is the canonical family of algorithms that use **random sampling** to evaluate positions in a game tree without exhaustive enumeration. Its design philosophy — “evaluate some branches to full depth instead of all branches to fixed depth” — directly motivates MCCFR.

MCTS builds an asymmetric search tree through four repeated phases:

1. **Selection** — descend from the root using a policy (typically UCB1, Upper Confidence Bound) that balances exploitation of known-good moves with exploration of under-visited ones.
2. **Expansion** — at a leaf, add one or more child nodes to the tree.
3. **Simulation (rollout)** — from the new node, play a random game to completion.
4. **Backpropagation** — propagate the terminal result back up the visited path, updating visit counts and cumulative rewards.

After many iterations, the child of the root with the highest visit count is selected as the best move. Random rollouts, aggregated over thousands of iterations, converge to accurate value estimates — even without any domain knowledge beyond the game rules.

Classical alternatives like **Minimax with alpha-beta pruning** evaluate every relevant node to a fixed depth, then apply a heuristic at the leaves. Both approaches have their regimes:

Property	Minimax + −	MCTS
Tree coverage	All branches to depth d	Selected branches to terminal
Evaluation	Heuristic at depth limit	Exact (terminal payoff) or neural net

Property	Minimax + −	MCTS
Branching factor sensitivity	Exponential: $O(b^d)$	Graceful: focuses on promising branches
Domain knowledge needed	Strong eval function	None (random rollout) or learned
Best suited for	Moderate branching, good heuristics	High branching, weak/no heuristics

For Go (branching factor ~250, no good evaluation function before neural networks), Minimax is impractical. MCTS was the breakthrough that made Go AI competitive; the same “sample instead of enumerate” principle carries over to imperfect-information game trees via MCCFR.

Read more: Browne, C. et al. (2012). “A Survey of Monte Carlo Tree Search Methods.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

3.3 Markov Chains and the Law of Large Numbers

The “Monte Carlo” in MCCFR refers to a broad family of computational methods that use **repeated random sampling** to obtain numerical results. The name originates from the Manhattan Project, where Stanislaw Ulam and John von Neumann used random sampling to simulate neutron diffusion — a problem too complex for analytical solution.

The mathematical foundation rests on **Markov chains** — stochastic processes where the next state depends only on the current state, not on the history of how it was reached (the Markov property). A game tree traversal is a Markov chain: from any game state, the transition depends only on the current state and the action chosen.

The key theoretical guarantee is the **Law of Large Numbers**: if we sample enough trajectories through the Markov chain, the average of the sampled values converges to the true expected value. For MCCFR:

$$\frac{1}{T} \sum_{t=1}^T \hat{v}_I^{(t)} \xrightarrow{T \rightarrow \infty} v_I$$

where $\hat{v}_I^{(t)}$ is the sampled counterfactual value at information set I on iteration t , and v_I is the true value that vanilla CFR computes exactly. The sampled values are **unbiased estimators** — their expected value equals the true value — which guarantees convergence to the same Nash equilibrium as full-traversal CFR.

The cost is **variance**. Individual samples can deviate substantially from the true value, requiring more iterations to reach the same precision. That variance-for-speed tradeoff is the central theme of this step.

Read more: Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, Chapter 5 — Monte Carlo Methods.

3.4 The Mathematics of Poker

Poker is the canonical testbed for imperfect-information game theory because it combines three sources of complexity that rarely co-occur:

1. **Hidden information** — each player sees only their own cards. The same observable game state (information set) can correspond to many different underlying game states.
2. **Stochastic elements** — card deals introduce chance nodes into the game tree. An optimal strategy must account for all possible deals, not just the observed one.
3. **Strategic deception** — unlike perfect-information games, poker rewards mixed strategies. A player who always bets with strong hands and checks with weak ones is trivially exploitable. Nash equilibrium requires **randomized bluffing at mathematically precise frequencies**.

The extensive-form framework from Step 2 carries over: game tree, information sets, strategies (mappings from information sets to action distributions), Nash equilibrium. The key quantities for evaluating strategies are:

- **Expected value (EV)** — average payoff under a given strategy pair.
- **Best response** — the optimal counter-strategy against a fixed opponent; the strongest possible exploitation.
- **Exploitability** — the average gain an opponent could obtain by playing a perfect best response. Nash equilibrium has exploitability zero.

Chen & Ankenman formalise these concepts through toy-game solutions where Nash bluffing frequencies can be derived analytically. Their half-street and full-street models build intuition for why CFR's output strategies contain the precise bluff/call ratios they do — a balanced player must bluff with a frequency proportional to the pot odds they offer.

Read more: Chen, B. & Ankenman, J. (2006). *The Mathematics of Poker*. ConJelCo — analytical Nash derivations for toy poker games.

3.5 Leduc Poker — Scaling Up the Benchmark

Kuhn Poker captures the essence of imperfect information in the smallest possible game tree. Leduc Poker scales that up by roughly two orders of magnitude — still solvable exactly, but large enough that algorithm differences become meaningful:

Property	Kuhn Poker	Leduc Poker
Cards	3 (J, Q, K)	6 ($\{J, Q, K\} \times \{ , \}$)
Rounds	1	2 (pre-flop + community card)
Community card	None	1 revealed between rounds
Hand ranking	High card only	Pair (private = community) beats high card
Bet sizes	Fixed (1 chip)	Variable (2 in round 1, 4 in round 2)
Max raises/round	1	2
Information sets	12	936
Game tree nodes	58	10,200
Chance outcomes	6	120

Three qualitatively new features appear:

- **Multi-round structure** — information changes between rounds (community card reveal), requiring strategies that adapt to new information.
- **Pair hands** — the hand ranking now depends on the community card; a Jack can become the best hand if a Jack is dealt on the board.
- **Larger bet sizes in later rounds** — the 4-chip raise in round 2 means decisions in later rounds carry more weight.

Despite being $\sim 78\times$ larger than Kuhn, Leduc remains small enough for exact computation (a full tree traversal takes ~ 50 ms). That makes it the ideal benchmark: large enough for performance differences to be meaningful, small enough that all algorithms can be run to near-convergence within minutes.

Read more: Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*.

3.6 CFR+ — The Regret Flooring Trick

CFR+ makes three small changes to vanilla CFR, each trivial to implement but dramatic in effect.

Regret flooring. In vanilla CFR, cumulative regrets can become arbitrarily negative:

$$R^{T+1}(I, a) = R^T(I, a) + r^T(I, a)$$

CFR+ floors regrets at zero after every update:

$$R^{T+1}(I, a) = \max(R^T(I, a) + r^T(I, a), 0)$$

This prevents actions from accumulating large negative regret that takes many iterations to “pay off”

before the action is reconsidered. The analogy to ReLU activations in neural networks is direct: both clip negative values to zero, preventing dead units (or dead actions) from requiring a long recovery period.

Linear strategy averaging. Vanilla CFR weights every iteration’s strategy equally in the running average. CFR+ weights iteration t by t itself:

$$\bar{\sigma}^T(I, a) = \frac{\sum_{t=1}^T t \cdot \sigma^t(I, a)}{\sum_{t=1}^T t}$$

Later iterations — which have lower regret and better strategies — contribute more to the average. Early, noisy iterations are down-weighted.

Alternating updates. Instead of updating both players’ regrets every iteration, CFR+ updates player 0 on odd iterations and player 1 on even iterations. This halves per-iteration work and provides a small convergence benefit by avoiding simultaneous strategy shifts.

Combined effect: empirical convergence improves from $O(1/\sqrt{T})$ to approximately $O(1/T)$. CFR+ with 1,000 iterations achieves what vanilla CFR needs $\sim 1,000,000$ iterations for. This is what made it feasible to solve Heads-Up Limit Texas Hold’em in 2015. Note: while the $O(1/T)$ rate is consistently observed, it lacks a formal proof — Tammelin et al. give the algorithm but not a convergence theorem matching the observed rate.

Empirically, on Leduc at 5,000 iterations CFR+ reaches exploitability 5.4×10^{-4} while vanilla CFR is still at $\sim 7.6 \times 10^{-3}$ — a $\sim 140\times$ improvement from three small modifications.

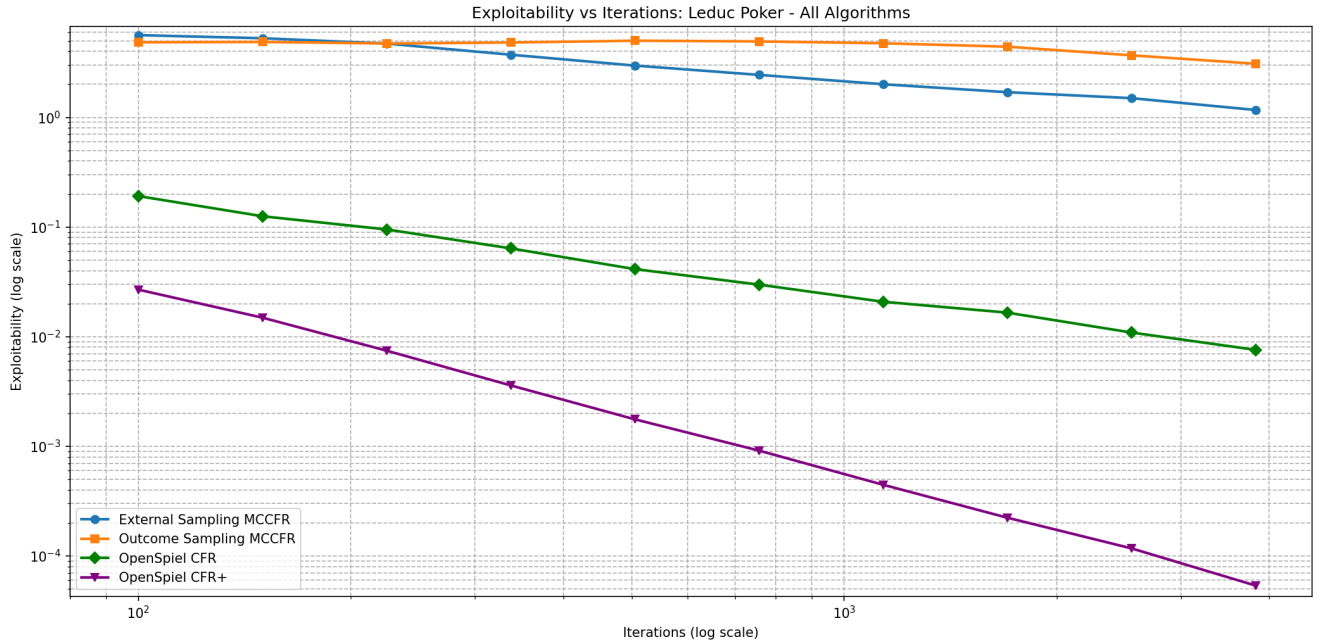


Figure 1: Leduc Poker — Exploitability vs Iterations

Read more: Tammelin, O. (2014). “Solving Large Imperfect Information Games Using

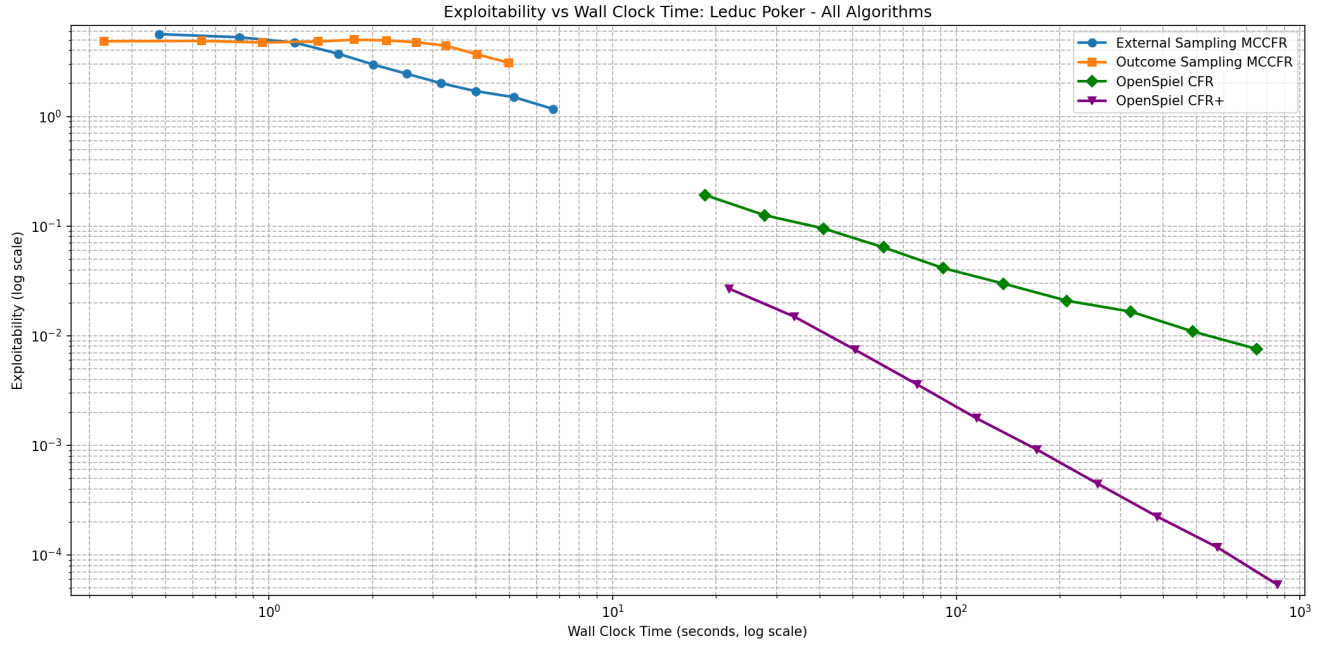


Figure 2: Leduc Poker — Exploitability vs Wall-Clock Time

CFR+.” AAAI.

3.7 MCCFR — Sampling the Game Tree

MCCFR replaces full tree traversal with partial sampling. Two variants sit on the same variance-speed curve at different points:

Variant	Chance nodes	Traverser’s nodes	Opponent’s nodes	Cost per iteration
External Sampling	Sample one deal	Explore all actions	Sample one action	~42 nodes (~945× faster than full)
Outcome Sampling	Sample one deal	Sample one action (-on-policy)	Sample one action	~5.5 nodes (~2,228× faster)

External Sampling explores every action at the updating player’s nodes but samples at chance and opponent nodes. Regrets for the traversing player are updated based on the sampled subtree. Since only one deal and one opponent path are visited, each iteration touches a tiny slice of the tree.

Outcome Sampling pushes sampling further: a single root-to-terminal trajectory is drawn. Because actions at the updating player’s nodes are also sampled, the regret update must be corrected by **importance sampling** — the ratio of the true reach probability to the sampling probability. An -on-policy mixture (with probability , choose uniformly; otherwise follow the current strategy) ensures all actions are explored even when the current strategy assigns zero probability.

The critical theoretical property for both: sampled counterfactual values are **unbiased estimators** of the true values (Lanctot et al., Theorem 1). This guarantees convergence to the same Nash equilibrium as vanilla CFR, despite the sampling noise. The cost, as always with Monte Carlo, is variance. Each sampled update deviates from the true counterfactual value because it reflects only one possible deal, not the expectation over all deals.

On Kuhn Poker’s 12 information sets, all four algorithms (vanilla CFR, CFR+, both MCCFR variants) reach near-Nash within seconds — the variance-speed tradeoff is invisible at this scale, and the choice of algorithm is academic:

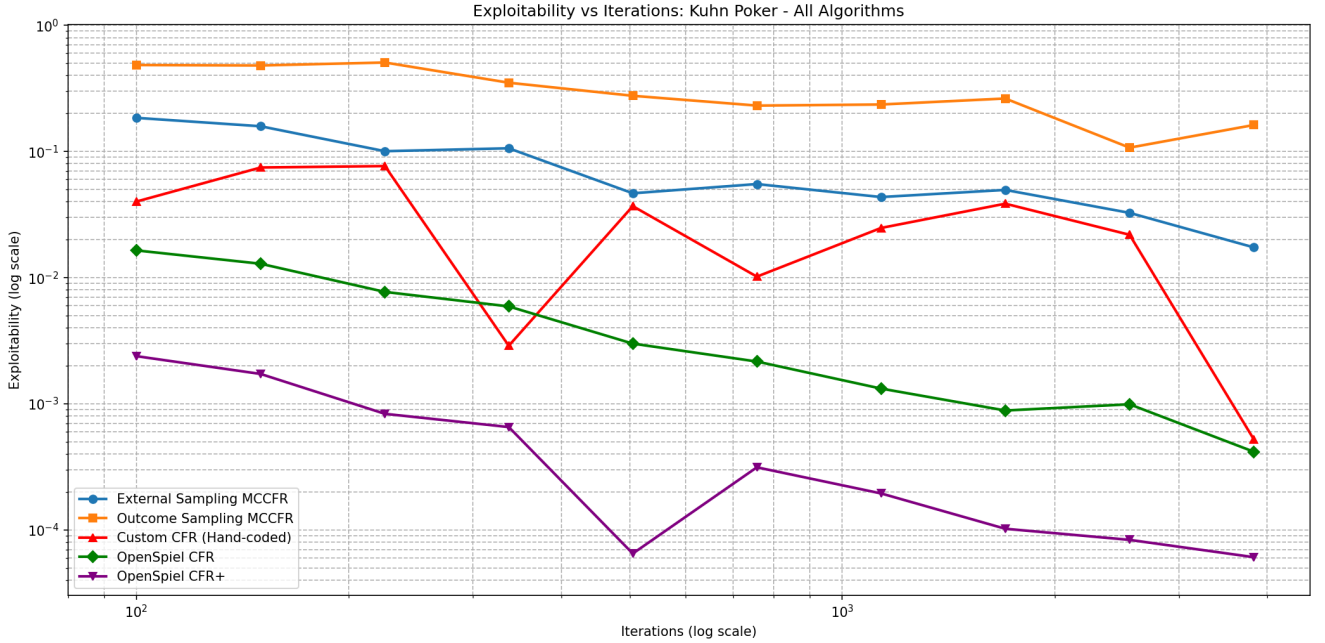


Figure 3: Kuhn Poker — Exploitability vs Iterations

To see the tradeoff emerge, the game tree has to be large enough that per-iteration cost matters. That is Leduc’s role.

Read more: Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Monte Carlo Sampling for Regret Minimization in Extensive Games.” *NeurIPS*.

3.8 The Variance-Speed Tradeoff

All four algorithms follow $O(1/\sqrt{T})$ convergence, but the constants differ enormously. Writing exploitability as $\epsilon(T) = C/\sqrt{T}$, and converting to wall-clock via $C_w = C/\sqrt{\text{speed}}$:

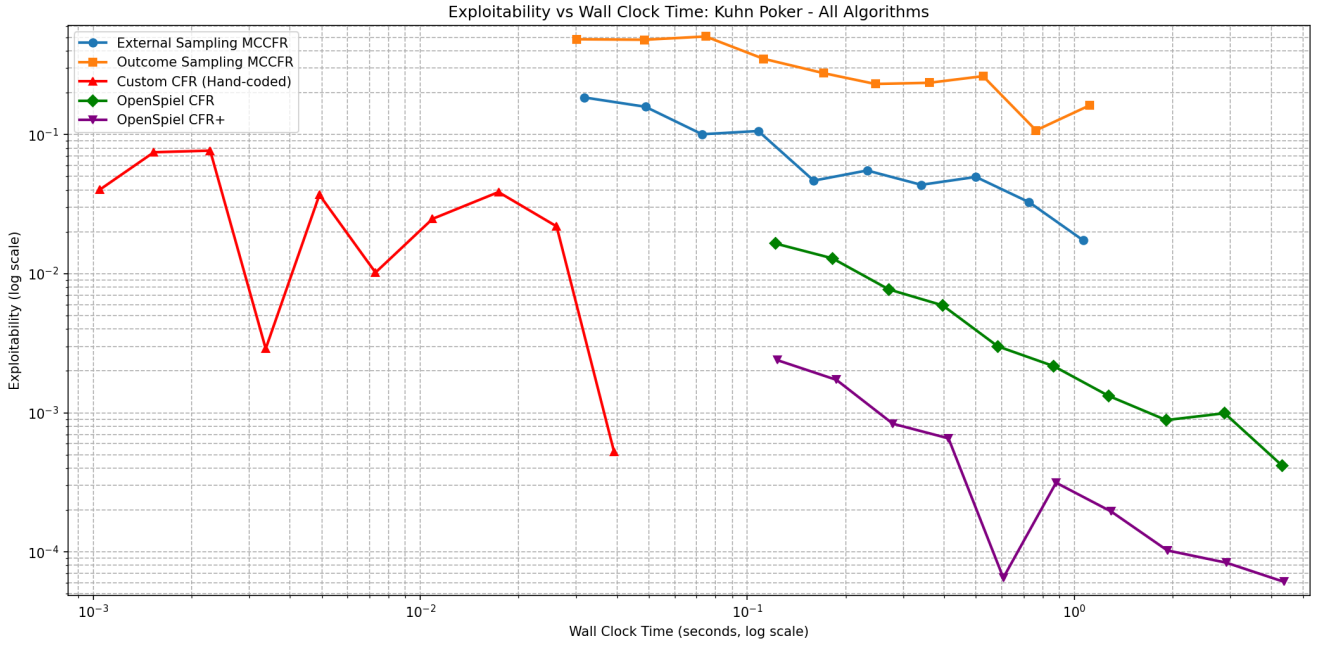


Figure 4: Kuhn Poker — Exploitability vs Wall-Clock Time

Algorithm	C_{iter}	Speed (iter/s)	C_w (wall-clock)	Relative to Vanilla
Vanilla CFR	0.34	20.6	0.075	1.0×
CFR+	0.34	20.6	~0.002	~38× faster
MCCFR	107	19,470	0.767	10.2× slower
External MCCFR	245	45,901	1.143	15.3× slower
Outcome				

The wall-clock constant is what determines real-world performance. Despite being 945× faster per iteration, External Sampling’s variance constant (107 vs 0.34) is 315× larger. The speed advantage **does not overcome the variance penalty**:

$$\frac{99,000 \times \text{more iterations needed}}{945 \times \text{faster per iteration}} \approx 105 \times \text{more wall-clock time}$$

Why does the variance exist? Vanilla CFR computes **exact** counterfactual values by enumerating all 120 deals. Its regret update has zero variance. MCCFR samples one deal per iteration. Even if it explored the full subtree for that deal, it would still have variance across deals — a consequence of the hidden information itself, not a deficiency of the sampling scheme:

$$\text{Var}[\hat{v}_I] = \mathbb{E}_{\text{deal}} [(\hat{v}_I - v_I)^2] \propto \frac{|N|}{|I|}$$

A formal 180-second timed benchmark of all four variants on Leduc makes the wall-clock comparison concrete. Per-iteration view (sampling variants complete millions of updates, full-traversal variants a few thousand):

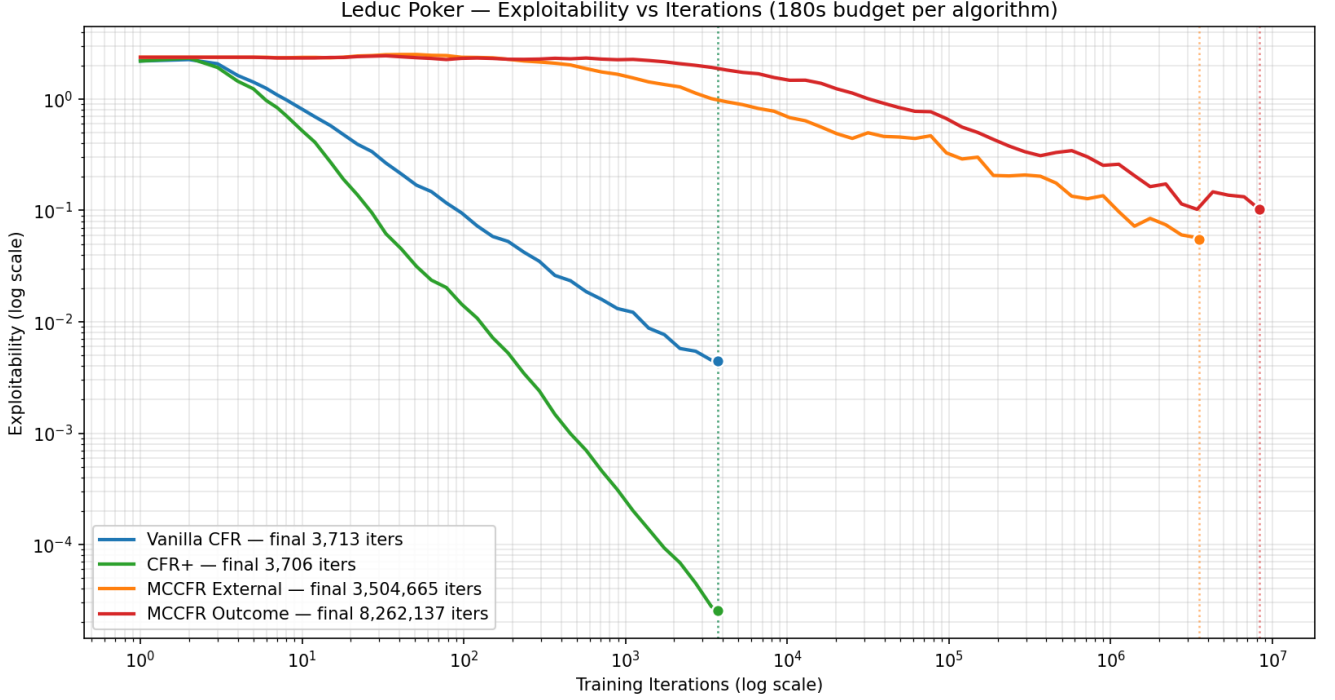


Figure 5: Exploitability vs Iterations

Wall-clock view — the fair comparison (given equal compute, which algorithm produces the best strategy?):

CFR+ reaches exploitability 2.6×10^{-4} — nearly exact Nash — in 3 minutes. Vanilla CFR follows at 4.4×10^{-3} . Both MCCFR variants lag by multiple orders of magnitude on this game size.

3.9 When MCCFR Wins — The Crossover Point

The pattern above raises an obvious question: if MCCFR is always worse on Leduc, why was it invented? The answer is that vanilla CFR’s per-iteration cost grows linearly in the tree size $|N|$, while MCCFR’s is roughly constant. At sufficient scale the linear cost wins. The critical tree size is:

$$|N|_{\text{crossover}} = |N|_{\text{Leduc}} \cdot \left(\frac{C_{mc}}{C_v} \right)^2 \cdot \frac{\text{speed}_v}{\text{speed}_{mc}}$$

Plugging in measured values:

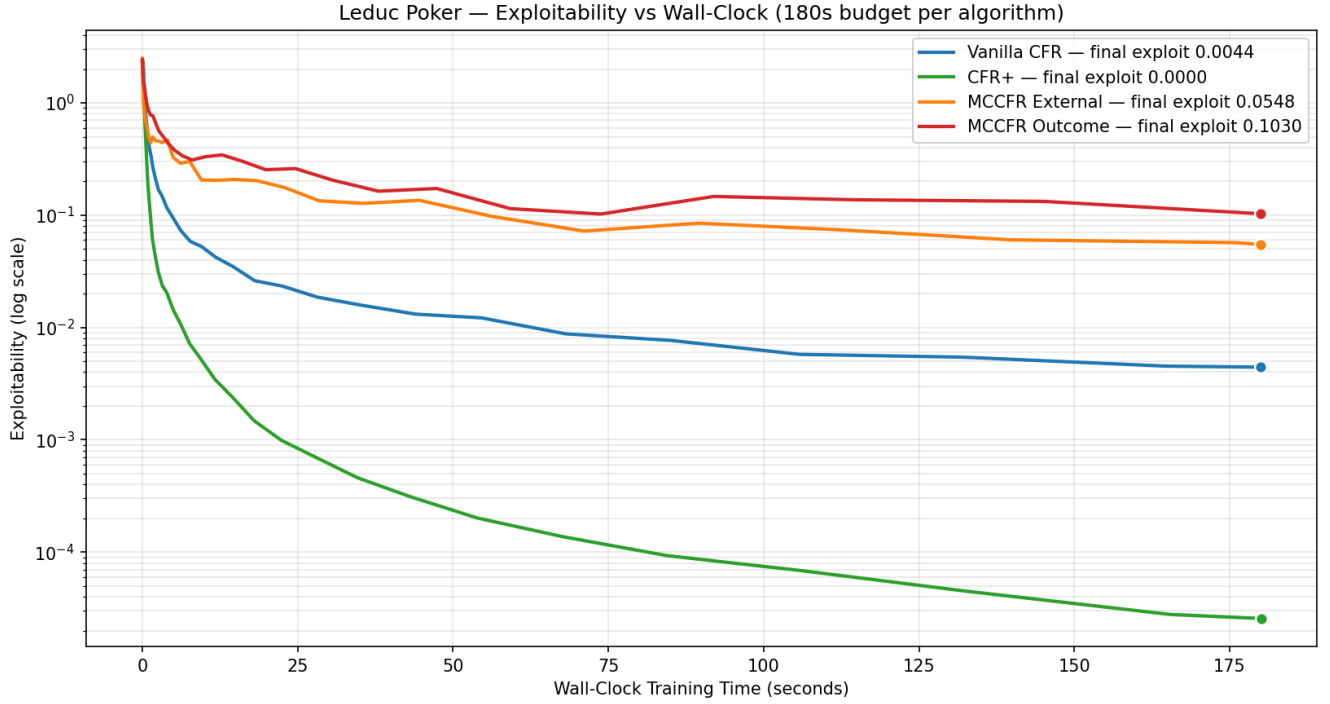


Figure 6: Exploitability vs Wall-Clock Time

Variant	$ N _{\text{crossover}}$
External Sampling	~2.1 million nodes
Outcome Sampling	~4.8 million nodes

Compared against actual games:

Game	$ N $	External wins?	Outcome wins?
Leduc Poker	10,200	No ($210\times$ too small)	No ($466\times$ too small)
Limit Texas Hold'em	$\sim 10^{14}$	Yes ($10^7\times$ above threshold)	Yes
No-Limit Texas Hold'em	$\sim 10^{17}$	Yes ($10^{10}\times$ above threshold)	Yes

Leduc sits $210\times$ below the External Sampling crossover and $466\times$ below the Outcome Sampling crossover. Full traversal dominates because the entire game tree fits in memory and can be enumerated in milliseconds. At real poker scales the conclusion reverses completely — even one full-traversal iteration of Hold'em would take longer than the age of the universe. At that scale, MCCFR (with variance-reduction extensions) becomes the **only** tractable approach.

3.10 Connections to Step 2 and Forward Pointers

Same principle, different regime. Step 2 established full-traversal CFR on Kuhn and proved that minimizing regret locally at each information set drives the global strategy to Nash equilibrium. Step 3 preserves that principle but introduces two independent refinements: CFR+ modifies how regrets accumulate (flooring + linear weighting), and MCCFR modifies how regrets are measured (sampled estimator instead of exact expectation). Both still output the **average** strategy, not the final iteration — the same averaging mechanism that stabilizes vanilla CFR stabilizes every variant.

Convergence rates. Step 2 measured a log-log slope near -0.5 for Kuhn’s exploitability, confirming $O(1/\sqrt{T})$. Step 3 shows that this rate is a property of the algorithm family, not of any one implementation — only CFR+’s empirical $O(1/T)$ is faster, and it lacks a formal proof. MCCFR stays at $O(1/\sqrt{T})$ with a dramatically larger constant.

Forward:

- **Step 4 (Game Abstraction & Scaling)** — attacks scalability from the opposite direction: instead of sampling the tree, reduce the tree itself through state and action abstraction. Combines with MCCFR to make real poker tractable.
- **Step 5 (Neural Equilibrium / Deep CFR)** — replaces tabular strategy storage with neural function approximators, using MCCFR’s sampling framework as the data-generation engine. The variance properties established here explain why Deep CFR requires specific variance-reduction techniques (advantage networks, importance-weighted targets).
- **Step 6 (End-to-End Game AI)** — builds on both CFR+ and MCCFR for complete agents. The Leduc engine built for this step becomes the benchmark environment through Step 8.

Open questions:

- Why does CFR+ empirically achieve $O(1/T)$ when no proof matches that rate? The best current hypothesis is that regret flooring reshapes the trajectory of the regret-minimization dynamics in a way that classical convergence bounds do not capture.
- At what tree size does variance reduction (weighted average sampling, variance-reduced MCCFR) become essential rather than optional? The crossover formula tells us where plain MCCFR wins — but not where it stops winning without additional tricks.

Read more: Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS* — bridges tabular CFR with subgame decomposition for real poker.

Step 4 Summary — Game Abstraction & Scaling Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

May 2026

Contents

4 Step 4 — Game Abstraction & Scaling Imperfect-Information Games	1
4.1 From Enumerated Games to Abstraction	1
4.2 Why Abstraction Is Needed	2
4.3 Two Axes of Abstraction	4
4.4 The Exploitability Gap	5
4.5 Earth Mover’s Distance: A Primer	6
4.6 The Merging Criterion	7
4.7 The Error Budget	8
4.8 The Build-Time Pipeline	9
4.9 Runtime Patching	11
4.10 Architecture: Blueprint + Live Patches	12
4.11 Implicit Route: Deep RL Counterparts	13
4.12 Practical Validation	13
4.13 Connections and Forward Pointers	14

4 Step 4 — Game Abstraction & Scaling Imperfect-Information Games

This is a condensed summary of the game-abstraction material covered in Step 4. It serves two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis.

4.1 From Enumerated Games to Abstraction

Steps 1–3 built the algorithms (DQN/PPO; Vanilla CFR; CFR+ and MCCFR external/outcome). Each was demonstrated on a game small enough to enumerate exactly: Kuhn (12 information sets), Leduc (936). That toolkit is complete — but it only works when the entire game tree fits in memory.

Step 4 is the bridge from “toy games we can enumerate” to “games we cannot.” The mechanism is

abstraction: deliberately collapse parts of the game so the same algorithms can run on a smaller, structurally simpler proxy, and measure what that costs in strategy quality. By the end of this step the deliverable contains a quantitative answer to the central question of every practical poker AI since 2007:

How much can the game be shrunk before the abstract Nash strategy stops being a good strategy in the real game?

That answer takes the form of a Pareto curve: on one axis, the size of the abstracted game; on the other, the **exploitability gap** — how much worse the abstract strategy is than the real game’s exact Nash. Both metrics are introduced formally in the section below.

4.2 Why Abstraction Is Needed

A game tree’s size is the product of three factors: (a) the number of distinct hidden states the chance node can produce, (b) the branching factor at each decision point, and (c) the depth (decisions until terminal). Each of the three blows up independently:

- **Hidden states** — Texas Hold’em deals 2 hole cards from 52, then up to 5 board cards. The number of hand-vs-board distinguishable situations is on the order of 10^{17} before counting betting history.
- **Branching factor** — No-limit poker allows any bet size from “min raise” to “all-in.” Even discretising to a handful of sizes pushes per-node branching from 3 (fold/call/raise in fixed-limit) to 5–10.
- **Depth** — Multiple betting rounds, each potentially with several raises, multiply.

The combined number of information sets in heads-up no-limit Hold’em is $\sim 10^{161}$ — more than there are atoms in the observable universe by a factor of $\sim 10^{80}$. None of Step 3’s algorithms can run on that tree.

The recipe is the same across every approach: *build a smaller game whose strategies translate back into playable strategies for the real game, run the Step 3 algorithms on that smaller game, then bound the damage.* That recipe has two routes.

4.2.1 Two Routes to Abstraction

The word “abstraction” in this literature actually covers two operationally different things. Both are used in modern game AI; both will appear in this thesis; conflating them is a category error. This section pins the boundary, and the abstraction pipeline follows it strictly: every phase has an *explicit* subsection (the tabular / algorithmic route) and an *implicit* subsection (the Deep-RL counterpart in the same conceptual slot).

Aspect	Explicit / tabular	Implicit / IB-style (Deep RL)
Where compression lives	A partition over info sets / a finite chosen set of actions	A continuous latent vector $z = f_\theta(s)$
Who does it	A human-designed rule or a clustering pass on hand features	The optimiser, via gradient descent on a loss
The “knob”	k in k-means buckets, the bet-set, the suit-isomorphism rule	β multiplying $I(S; Z)$ in the loss
Guarantee	Bounded exploitability gap in the same game (formal exploitability bound)	Information-theoretic bound on $I(S; Z)$; no Nash-preservation theorem
When it is computed	Before solving — fixed input to CFR/MCCFR	During solving — the network <i>is</i> the strategy
Output type	A discrete bucket id per info set	A real vector
Where it appears in this thesis	This step (4)	Step 5 (Deep CFR), step 6 (end-to-end), steps 11–12 (sequence models)

Both routes share the same **intuition** — compress while preserving value — and that intuition is best captured by the Information Bottleneck Lagrangian, with β as the exchange rate between memory and value:

$$\mathcal{L} = \underbrace{\text{Complexity}(Z)}_{\text{memory cost}} - \beta \cdot \underbrace{\text{Value}(\pi_Z)}_{\text{strategic worth}}$$

When $\beta = 0$, the algorithm compresses the entire game into one state and plays terribly. When $\beta \rightarrow \infty$, it refuses to merge anything and uses the full game. Every algorithm in the abstraction pipeline is one specific instantiation of that knob:

- **Lossless merging** — corner case $\beta \rightarrow \infty$: only merges when $\text{Value}(\pi_Z) = \text{Value}(\pi)$ exactly.
- **Bounded lossy** — middle of the curve with a guarantee: pick β such that $|\text{Value}(\pi_Z) - \text{Value}(\pi)| \leq \varepsilon_{\text{abs}}$.
- **Empirical similarity (HSD + EMD)** — measures the curve itself: EMD between abstract and full distributions is a proxy for the value-loss term.
- **Runtime patching** — orthogonal: accept the value loss from any chosen β , then patch it at runtime via subgame solving.

The implementations in this step are all explicit; the implicit subsections are forward pointers, not work performed here.

Throughout the rest of this summary, the strategy produced by solving the abstract game (and used as the starting point at runtime) is referred to as the **blueprint**.

4.3 Two Axes of Abstraction

Every concrete abstraction technique falls onto one of two orthogonal axes — *what states the agent treats as the same* (information abstraction) and *what moves the agent considers* (action abstraction). A third axis, *runtime refinement*, is orthogonal to both and lives in its own section (Runtime Patching, below).

4.3.1 Information Abstraction

Group together info sets that the agent will treat as the same state. This is the central operation of the pipeline — every phase from the merging criterion through the build-time pipeline is about *how* and *when* to perform information merges.

Note on regimes. Under explicit information abstraction in vanilla CFR there is a subtle distinction between abstracting (a) the **node map** (memory only) and (b) the **traversal** (wall-clock per iteration). Only the second produces a wall-clock speedup.

Remember: information abstraction decides which hidden situations the agent treats as the same.

4.3.2 Action Abstraction and the Translation Problem

Restrict the set of actions the solver considers, run CFR on the restricted game, then handle whatever the real opponent does that lies outside that restriction. Two regimes:

- **Discrete action spaces.** The base set is already finite. Abstraction means *pruning* (drop dominated or uninteresting actions before solving) or *macro-actions* (group sequences of primitives into one abstract action — a forward pointer to temporal abstraction in steps 1 and 11). Translation is usually trivial: if the abstract action set is a subset of the legal one, the agent’s own moves are always in-abstraction.
- **Continuous action spaces.** The base set is uncountable (bet sizes in no-limit poker, joint torques in continuous control). For tabular methods, action abstraction is *mandatory* — CFR cannot run on a continuous tree without first collapsing it to a finite proxy (typical poker grid: {**fold**, **call**, **0.5×pot**, **1×pot**, **2×pot**, **all-in**}). The Deep RL alternative is a parameterised actor that outputs a continuous distribution over actions directly (PPO/SAC), eliminating the translation problem at the cost of formal guarantees.

The translation problem is **acute** in the explicit-discretisation route: when the opponent plays a $0.7\times\text{pot}$ bet but the abstraction only contains $0.5\times\text{pot}$ and $1\times\text{pot}$, the agent must convert that bet to a node it has trained on. Three translators in common use:

1. **Nearest-action** — round to the closest abstract bet on the linear scale. Worst on equity loss when bets cluster between abstract sizes.

2. **Probability-split (linear)** — assign mass to the two nearest abstract bets in proportion to their distance from the actual bet.
3. **Pseudo-harmonic mapping** — interpolate in *pot-fraction-odds* space rather than linear bet-amount space, which corresponds to how strategically equivalent two bets actually are. State-of-the-art for poker bet translation.

Translation errors compound across betting rounds — an opponent who detects rounding can systematically bet just below or above grid points to coerce wrong-sized responses. This is empirically where practical poker AIs lose the most equity, and is the operational reason runtime patching (below) exists: re-solve the actual subgame with the actual bet rather than relying on the translator.

Remember: action abstraction decides which moves the agent can reason about before translation or resolving.

4.4 The Exploitability Gap

For a strategy σ played in game G , exploitability is the standard step-3 metric:

$$\text{exploit}_G(\sigma) = \frac{1}{2}[v_G(\text{BR}(\sigma_1), \sigma_1) + v_G(\sigma_0, \text{BR}(\sigma_0))]$$

where σ_0 and σ_1 are the two players’ strategies, and $\text{BR}(\sigma)$ (Best Response) is the strategy that optimally maximizes profit specifically against the strategy σ .

Step 4 introduces a derived metric, the **exploitability gap**: how much worse an abstract strategy is than the real game’s exact Nash, *measured in the real game*.

Let G be the real game and $\hat{G}$ its abstraction. Let $\hat{\sigma}^*$ be the Nash of $\hat{G}$, and let $T(\hat{\sigma}^*)$ be its translation back into a playable strategy for G (identity if $\hat{G}$ is purely an information abstraction; non-trivial if it is also an action abstraction — see action abstraction). Then

$$\Delta_{\text{abs}}(\hat{G}) = \text{exploit}_G(T(\hat{\sigma}^*)) - \text{exploit}_G(\sigma_G^*)$$

with σ_G^* the exact Nash of G . By definition $\Delta_{\text{abs}} \geq 0$, with equality iff the abstraction is lossless (covered in the merging criterion below).

This is the central quantity of Step 4. Every Pareto plot in the Pareto frontier has Δ_{abs} on one axis. Every “abstraction quality” claim is checked by computing it.

Two complementary tools quantify Δ_{abs} before solving the abstract game outright:

- **Reach-weighted error bound** (covered in the error budget below) — sums per-merge utility/probability errors weighted by reach probability to give an *upper bound* on Δ_{abs} from the abstraction’s structural properties alone.

- **Earth Mover’s Distance between abstract and real distributions** (covered in the error budget below) — a *measured proxy* for Δ_{abs} that does not depend on solving anything: just compute EMD between the hand-strength distributions of merged info sets and read off how much information the merge is throwing away. Empirically the strongest predictor of post-solve exploitability.

The two are complementary: the bound is rigorous but loose; the EMD proxy is empirical but tight. The Pareto view reports both alongside the directly-measured Δ_{abs} for every abstraction configuration.

Remember: exploitability gap is the price paid for solving the smaller game instead of the real one.

4.5 Earth Mover’s Distance: A Primer

EMD shows up across nearly every section that follows, so it earns a short standalone introduction.

What it measures. Given two probability distributions over the same support, EMD is the minimum amount of “work” needed to transform one into the other, where *work* = *mass moved* $\times$ *distance moved*. Imagine each distribution as piles of sand on a number line — EMD is the smallest total effort to reshape pile A into pile B.

Origins and other names. The idea traces to Monge’s 1781 *optimal transport* problem (moving piles of earth to fill holes with minimum effort), formalised by Kantorovich in 1942 as a linear program. The same quantity appears in machine learning under several names — **Wasserstein-1 distance**, **Kantorovich-Rubinstein distance**, or simply **Wasserstein distance**. Rubner, Tomasi & Guibas (2000) popularised the “Earth Mover’s Distance” name in computer vision for image retrieval. More recently it underpins Wasserstein GANs and is a workhorse in document similarity, domain adaptation, and any task where comparing distribution *shape* matters.

Why it beats mean-based comparisons. EMD respects the *geometry* of where mass lives, not just average values. Two distributions concentrated near each other are close; two distributions concentrated at opposite ends are far apart, regardless of where their means happen to coincide. Expected-value comparisons throw all this structure away.

In this step’s context. Each information set has a *Hand Strength Distribution (HSD)* — the histogram of end-of-game win probabilities computed by rolling out unseen cards. Two HSDs with the same mean can have completely different shapes: a peaked “stable medium hand” vs a bimodal “boom-or-bust” hand. Expected-hand-strength clustering merges these together; EMD does not. That is why every quality criterion, every error proxy, and every clustering pipeline below uses EMD between HSDs as the core similarity signal.

Cheap on 1-D histograms. When the support is one-dimensional and ordered (binned win probability $[0, 1]$), EMD reduces to the L1 distance between cumulative distributions — a single linear sweep over the bins. This is why HSD + EMD is fast enough to cluster millions of info sets.

Remember: EMD compares distribution *shape*, not just the mean — that’s why two hands with identical win rates can still be strategically different.

4.6 The Merging Criterion

Further reading: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

When can two information sets be collapsed into one? Three nested levels of strictness, weakest at the top.

4.6.1 Level 1 — Lossless

Merge two info sets only when they are *strategically identical*: same probability of being reached, same recursive structure, and same utility consequences against every possible opponent continuation. The last condition is the load-bearing one — if any opponent reaction can distinguish them, they cannot be losslessly merged.

When this holds, the merge is free: any optimal strategy in the abstract game lifts to an optimal strategy in the original game with **zero** exploitability cost.

Intuition: A red Jack and a black Jack in a game where suits don’t matter (no flushes possible) are losslessly mergeable. A red Jack in a game *with* flushes is not — the suit silently affects the opponent’s flush odds through card removal.

Remember: lossless abstraction is free compression: same strategic meaning, fewer stored information sets.

4.6.2 Level 2 — Bounded lossy

Relax “identical utilities” to “utilities differ by at most ε ”; same idea for chance probabilities. The price is a quantifiable error — each merge contributes a slack budget that propagates into a bound on overall exploitability (used by the next section). Action sequences along merged paths still need to agree, so this is *controlled* forgetting, not arbitrary.

Intuition: Putting J and Q into a single “low card” bucket forces the agent to play them identically, even though optimally they differ slightly. The system absorbs that mistake in exchange for a much smaller game.

Remember: bounded lossy abstraction is controlled forgetting: the game shrinks, but each merge receives an error price.

4.6.3 Level 3 — Empirical similarity

When the analytical bound is too pessimistic or the inputs too high-dimensional to enumerate (Texas hold'em has $\sim 10^9$ canonical river boards), drop the per-merge bound and replace it with a *learned distance function*:

- Compute a feature vector per info set — typically the **Hand Strength Distribution (HSD)**: the histogram of end-of-game win probabilities after rolling out unseen cards.
- Use **Earth Mover's Distance (EMD)** between feature vectors as the similarity metric.
- Cluster info sets whose feature distributions are close; merge within clusters.

There is no formal exploitability guarantee here — quality is checked *after* solving by measuring exploitability on the resulting strategy.

Remember: empirical abstraction is a budgeted guess that must be measured after solving.

4.6.4 Picking among the three

The decision order is: first take every lossless merge available; then accept bounded lossy merges only when the error budget is tolerable; when exact checks are too pessimistic or too expensive, cluster with HSD + EMD and verify the resulting abstraction empirically.

4.7 The Error Budget

Further reading: https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf>

Three quantification tools, increasing in tightness and decreasing in formal rigour.

4.7.1 Tool 1 — Analytical bound

Given a bounded-lossy abstraction with per-merge slack constants, sum them — weighted by how often each info set is actually reached during play — to get an upper bound on the exploitability gap.

The reach-weighting is the key idea. Rare info sets can be merged aggressively without hurting overall exploitability; sloppy merges in dense, frequently-visited regions are catastrophic. This is what every practical poker abstraction since 2010 exploits.

Intuition: The error of a bucket merge is weighted by how often players actually reach that situation. A sloppy merge in a rare endgame scenario costs almost nothing overall, allowing aggressive abstraction without losing money in expectation.

Remember: the analytical bound is rigorous but usually loose; its most important idea is reach-weighting.

4.7.2 Tool 2 — EMD proxy

A *measured* upper bound that does not require enumerating leaves: just compute EMD between the hand-strength histograms of two info sets. EMD is a *proxy*, not a bound — it correlates well with post-solve exploitability but carries no formal guarantee. (See the EMD primer above for the underlying mechanics.)

Remember: EMD remembers distribution shape; expected hand strength remembers only the mean.

4.7.3 Tool 3 — CFR-BR direct evaluator

The strongest measurement: it returns the closest representable Nash approximation the abstraction can express, *isolating* abstraction error from solving error.

In experiments, CFR-BR strategies show exploitability as low as 1/3 of the corresponding plain-CFR strategies on the *same* abstraction. The takeaway: a large fraction of measured “abstraction error” in the literature is actually *solving error in disguise* — the abstraction itself was capable of better, but the solver hadn’t converged.

Remember: CFR-BR asks what the abstraction can express, not how well one solver happened to train.

4.7.4 Empirical findings worth memorising

Two robust comparisons on equal info-set budgets:

- **Distribution-aware vs expectation-based.** HSD + EMD strictly dominates expected-win-rate clustering on both exploitability and head-to-head win rate against fixed opponents.
- **Imperfect vs perfect recall.** Imperfect-recall abstractions outperform perfect-recall ones at matched info-set budget — the bucket-reallocation freedom is empirically worth more than the lost continuity.

Both are reasons the build-time pipeline defaults to imperfect recall + HSD + EMD.

4.8 The Build-Time Pipeline

Further reading: <https://www.cs.cmu.edu/~gilpin/papers/extensive.JACM.pdf> · <https://poker.cs.ualberta.ca/publications/AAMAS13-abstraction.pdf> · https://www.cs.cmu.edu/~sandholm/imperfect_recall_abstraction.arxiv14.pdf

Two complementary pipelines, one per criterion-strictness level.

4.8.1 Pipeline 1 — GameShrink (lossless)

Exhaustive merger that walks the *signal tree* — a structure smaller than the game tree, enumerating only sequences of public + private signals — bottom-up. At each level it checks pairs of sibling subtrees for

strategic equivalence and merges every passing pair.

Why the signal tree is smaller: the game tree multiplies every distinct hidden-state signal by every distinct betting sequence that could lead there, while the signal tree collapses all those betting paths into a single signal node. On Rhode Island Hold'em the signal tree is $\sim 6.6\text{M}$ nodes vs $\sim 3.1\text{B}$ game-tree nodes — roughly $500\times$ compression before any lossy step is applied. GameShrink is *complete* over its merging criterion: every lossless merge expressible by the criterion is found.

This pipeline + linear programming was the engine that solved Rhode Island Hold'em in 2007, four orders of magnitude beyond any poker game previously solved.

Remember: GameShrink searches the signal tree for every free merge before any lossy compression is considered.

4.8.2 Pipeline 2 — HSD + EMD + k-means + imperfect recall (lossy)

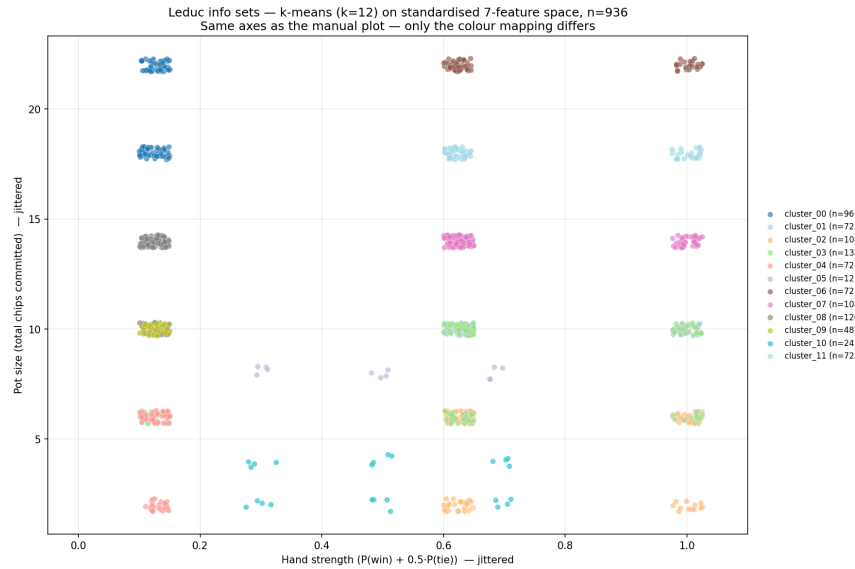


Figure 1: InfoSet grouping via K-means

When the lossless merger has run to convergence and the game is still too large, switch to clustering:

The pipeline computes an HSD for each information set, clusters those distributions with EMD as the distance metric, and maps each information set to a bucket id. Two recall regimes:

- **Perfect recall** — the bucket identity includes the past bucket trail.
- **Imperfect recall** — the later-round bucket can forget the earlier trail, freeing more buckets for the round where new information matters most.

Imperfect recall consistently wins at a fixed bucket budget — capacity is spent on rounds that matter rather than on remembering history.

Remember: HSD + EMD decides *what looks strategically similar*; imperfect recall decides *where to spend the bucket budget*.

4.8.3 A note on hardness

Picking the partition that minimises the analytical bound is **NP-complete**, even for a tiny single-player game two levels deep. So nobody minimises the bound exactly — practical pipelines approximate level-by-level (one round at a time) rather than globally. Under reasonable conditions, a single level reduces to k-centre clustering in a metric space, which has polynomial-time approximation algorithms with constant-factor guarantees.

This is *why* every practical poker abstraction since 2010 is a level-by-level clustering pipeline rather than a global optimiser.

4.9 Runtime Patching

Further reading: <https://arxiv.org/pdf/1705.02955v3>

When play descends into a subgame and the abstract blueprint is too coarse, re-solve the subgame at higher fidelity *in real time*. Two patches matter — together they were the load-bearing components of Libratus, the first AI to defeat top humans in heads-up no-limit Texas hold'em.

4.9.1 Why subgame solving cannot be done in isolation (Coin Toss)

A simple counterexample called *Coin Toss*: a coin lands Heads or Tails with equal probability, only P_1 sees the outcome. P_1 chooses *Sell* (with payoff that depends on the coin) or *Play* (where P_2 guesses the side). The optimal P_2 strategy in the *Play* subgame is **not** a function of the *Play* subgame alone — it depends on the value P_1 would have gotten by choosing *Sell* instead. Change *Sell*'s payoff and the optimal *Play* strategy flips, even though the *Play* subgame itself is unchanged.

This is the central pathology that all naive imperfect-information subgame solving walks into. The fix: solve an *augmented subgame* that includes the original subgame plus extra “alternative-payoff” nodes encoding what each player could have achieved by *not entering* this subgame.

4.9.2 Patch 1 — Safe subgame solving

The augmented subgame is anchored to blueprint values: each top-of-subgame information set gets an alternative payoff equal to what the blueprint promised that player at this point in the game. Solving the augmented game yields a refined strategy with a safety guarantee — exploitability is provably no higher than the blueprint, and strictly lower whenever local conditions allow.

A practical refinement (*Reach*) carries forward “gifts” — value differences from earlier points along the path where the player could have done strictly better — for further improvement.

In practice, the conservative blueprint payoffs can be replaced by *estimates* of equilibrium value. This drops the strict guarantee but typically yields lower real-play exploitability, since the blueprint's conservative payoffs are themselves loose.

Remember: safe subgame solving works because the subgame is not solved alone; it is anchored by blueprint values.

4.9.3 Patch 2 — Nested subgame solving (the action-translation killer)

When the *opponent* plays an action a outside the abstraction (a $0.7 \times \text{pot}$ bet when the abstraction has only $0.5 \times \text{pot}$ and $1 \times \text{pot}$), instead of rounding a to a known action via a translator, *re-solve a fresh subgame that contains a .*

The inexpensive version builds a subgame just after the off-tree action, re-solves it with the safe-subgame scaffold, and appends the new sub-strategy to the blueprint. If another off-tree action appears later, the process repeats — the blueprint grows only where play actually goes.

Empirical impact. On heads-up no-limit Texas hold'em, nested subgame solving's exploitability against off-tree opponent bets is **10–100× lower** than every prior action-translation method, depending on abstraction size.

The recursion is shallow in practice. Most real off-tree actions do not chain — the opponent plays one weird bet, the agent re-solves, and the new abstract tree absorbs it. Static action translators remain the right choice only when latency cannot afford a live CFR solve (online play, embedded apps).

Remember: action translation rounds the opponent's move; nested solving keeps the move and solves around it.

4.10 Architecture: Blueprint + Live Patches

The full pipeline now adds up to a single architectural pattern that every competitive heads-up no-limit poker AI since 2017 (Libratus, Modicum, Pluribus) has used:

1. **Build-time** — apply lossless and lossy abstraction to shrink the game; solve the resulting abstract game with CFR / CFR+ / MCCFR; freeze the resulting strategy as the **blueprint**.
2. **Runtime** — when play descends into a subgame the blueprint covers coarsely, re-solve it with safe subgame solving (Patch 1). When the opponent plays an action outside the abstraction, re-solve a fresh subgame containing that action (Patch 2).

The two halves complement each other: abstraction makes the build-time problem tractable; live patches recover the precision lost to abstraction at the points of play where it actually matters. Step 6 covers the full integrated systems; this step provides the abstraction-and-patch primitives they are built from.

Remember: the production architecture is one part precomputed blueprint, one part live re-solving — neither half stands alone.

4.11 Implicit Route: Deep RL Counterparts

Each phase of the explicit pipeline has a Deep RL counterpart that occupies the same conceptual slot but trades formal guarantees for end-to-end learning. These are forward pointers — actual implementations land in steps 5–6.

- **Merging criterion** — the three strictness levels map to network architectures: lossless permutation-equivariant networks (Deep Sets), bounded lossy information bottlenecks in recurrent/transformer policies, empirical similarity end-to-end learned latent embeddings (as in Deep CFR).
 - **Error budget** — no explicit per-merge bounds. Quality is monitored via rate-distortion curves (Information Bottleneck) and test-loss tracking, accepting asymptotic convergence in place of algorithmic guarantees.
 - **Build-time pipeline** — there is no separate build phase. The network’s latent space learns its own abstract representation, shaped directly by gradient descent on the regret-minimization loss.
 - **Runtime patching** — exact equilibrium re-solvers are replaced by depth-limited heuristic search backed by neural-network value functions (DeepStack, ReBeL).
-

4.12 Practical Validation

The implementation phase converted the summary’s abstractions into a small but complete Leduc-family pipeline:

- **Lossless suit isomorphism** on fixed-limit Leduc and Extended Leduc.
- **Lossy card bucketing** using hand-strength features, EMD-style distances, and configurable bucket counts.
- **Action abstraction** on Mini-NL Leduc, with nearest, probability-split, and pseudo-harmonic translation code.
- **Extended Leduc** with four ranks and two suits, giving a larger imperfect-information testbed.
- **Combined abstraction** composing suit reduction, action reduction, and card buckets.
- **Quality evaluation** through exploitability gaps, EMD proxies, OpenSpiel cross-validation, and a Pareto frontier.

The main empirical lesson matches the theory: lossless abstraction is almost free strategically and very useful computationally, while lossy information and action abstraction introduce persistent exploitability floors. Under 180-second CFR+ budgets, suit isomorphism reduced fixed-limit Leduc from 936 to 288 information sets and reached lower exploitability than the full game because it completed many more iterations. On Extended Leduc, the same idea reduced the game from 10,304 to 2,968 information sets and improved final exploitability from about 2.7×10^{-2} to about 1.3×10^{-3} .

The lossy bucket runs show the other side of the tradeoff. Smaller bucketed games train faster, but the error does not disappear with more CFR+ iterations because the strategy is solving the wrong game.

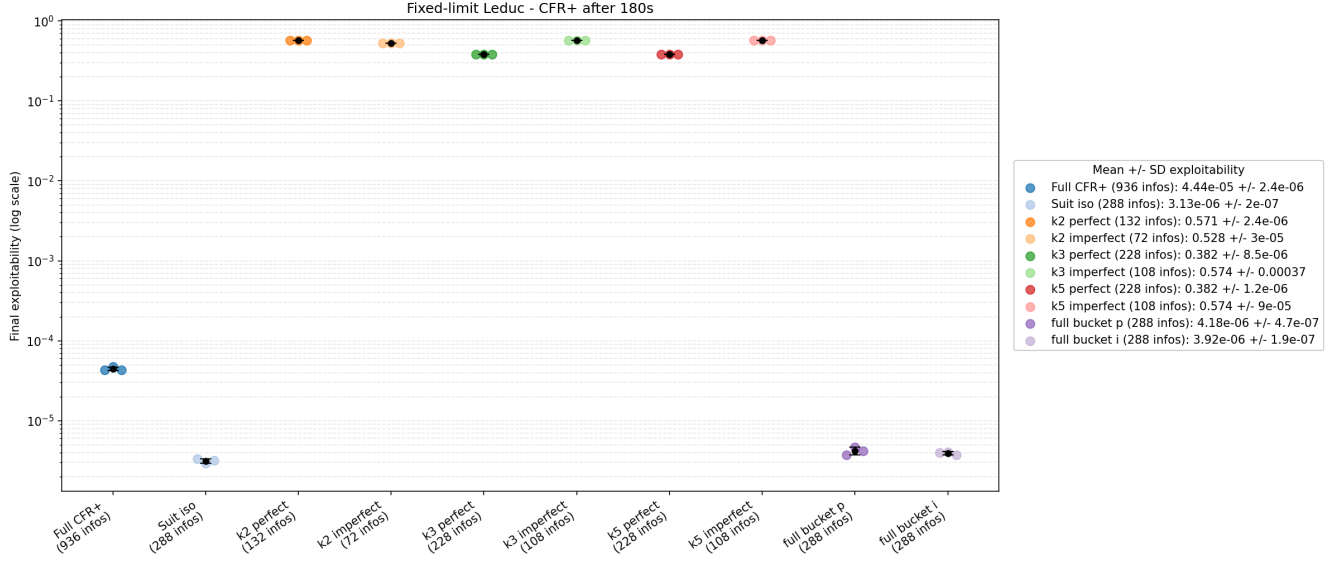


Figure 2: Fixed-limit Leduc CFR+ abstraction results

In fixed-limit Leduc, coarse bucket abstractions remained around 0.38-0.57 exploitability, even though they completed more iterations than full CFR+. This is the practical meaning of the exploitability gap: abstraction error is not optimizer error.

Action abstraction was the riskiest part of the step. In Mini-NL Leduc, restricting the action set reduced the information-set count from 4,704 to 936 and produced many more CFR+ iterations under the same time budget, but exploitability stayed high. In Extended Leduc, adding action abstraction on top of suit isomorphism produced a compact tree, but the translated strategy was highly exploitable. This is why the literature moves from static action translation toward nested subgame solving: the full action actually played by the opponent often matters too much to round away.

The Pareto view is the right final diagnostic. Each point asks: how much smaller did the game become, and how much exploitability did that compression buy or cost? Lossless suit isomorphism lands on the attractive part of the frontier. Coarse buckets and action abstraction can reduce the game further, but they move onto a different regime where smaller size is paid for with strategy quality.

4.13 Connections and Forward Pointers

Step 4 continues the progression started in Steps 2 and 3. Step 2 introduced CFR as local regret minimization over information sets. Step 3 made that solver faster and cheaper through CFR+ and MCCFR. Step 4 changes the input game itself: before solving, it asks which states and actions can be merged without destroying the strategic signal.

The bridge to Step 5 is the distinction between explicit and implicit abstraction. Step 4's abstractions are hand-built or clustering-built partitions over information sets and action sets. Deep CFR and neural equilibrium approximation replace those tables with learned function approximators: the representation is still compressed, but the compression lives inside a network rather than in a fixed bucket map.

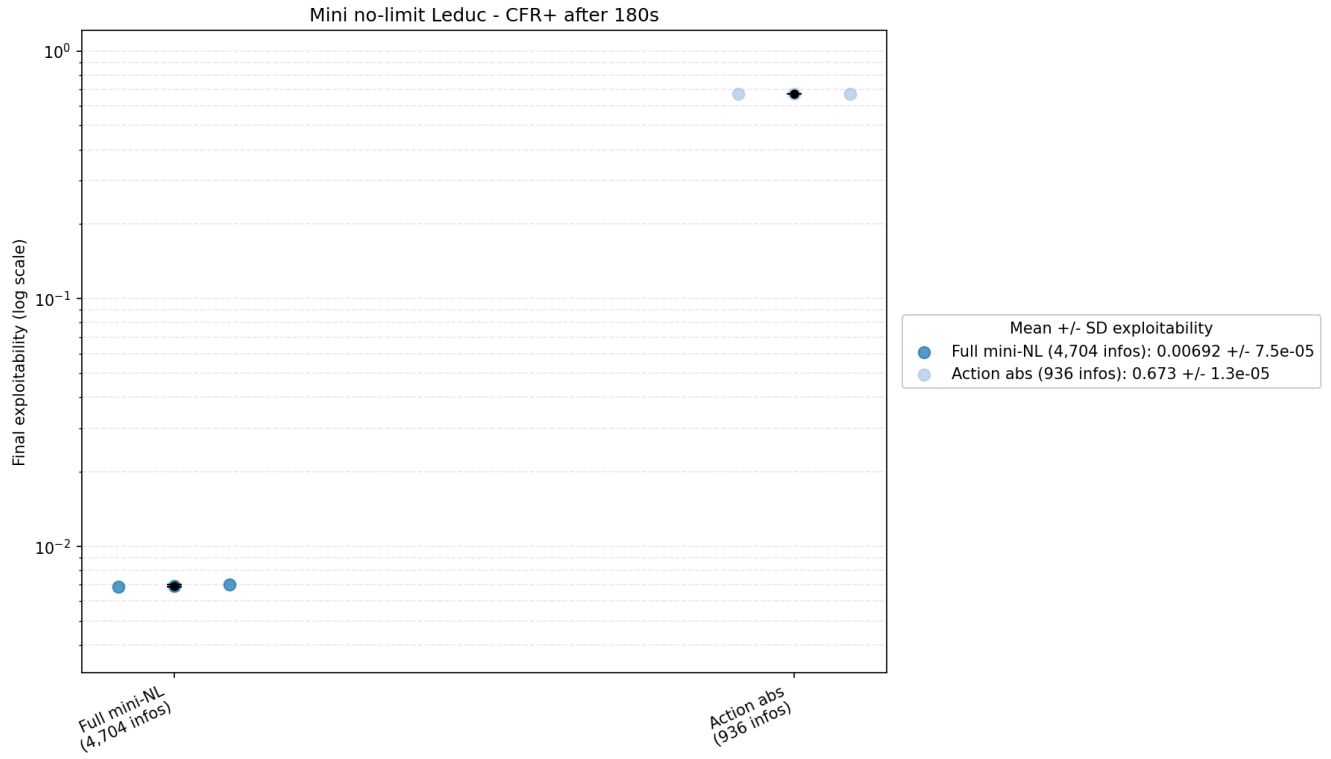


Figure 3: Mini-NL Leduc CFR+ abstraction results

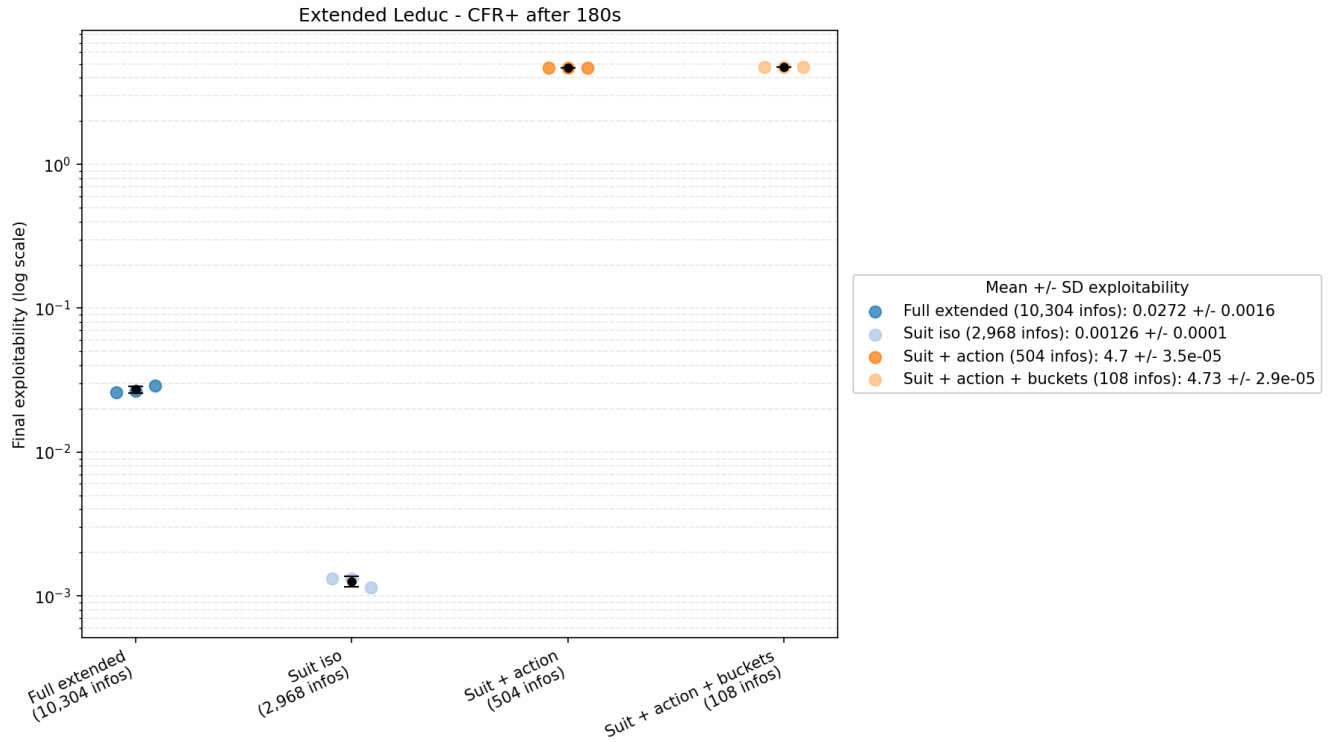


Figure 4: Extended Leduc CFR+ abstraction results

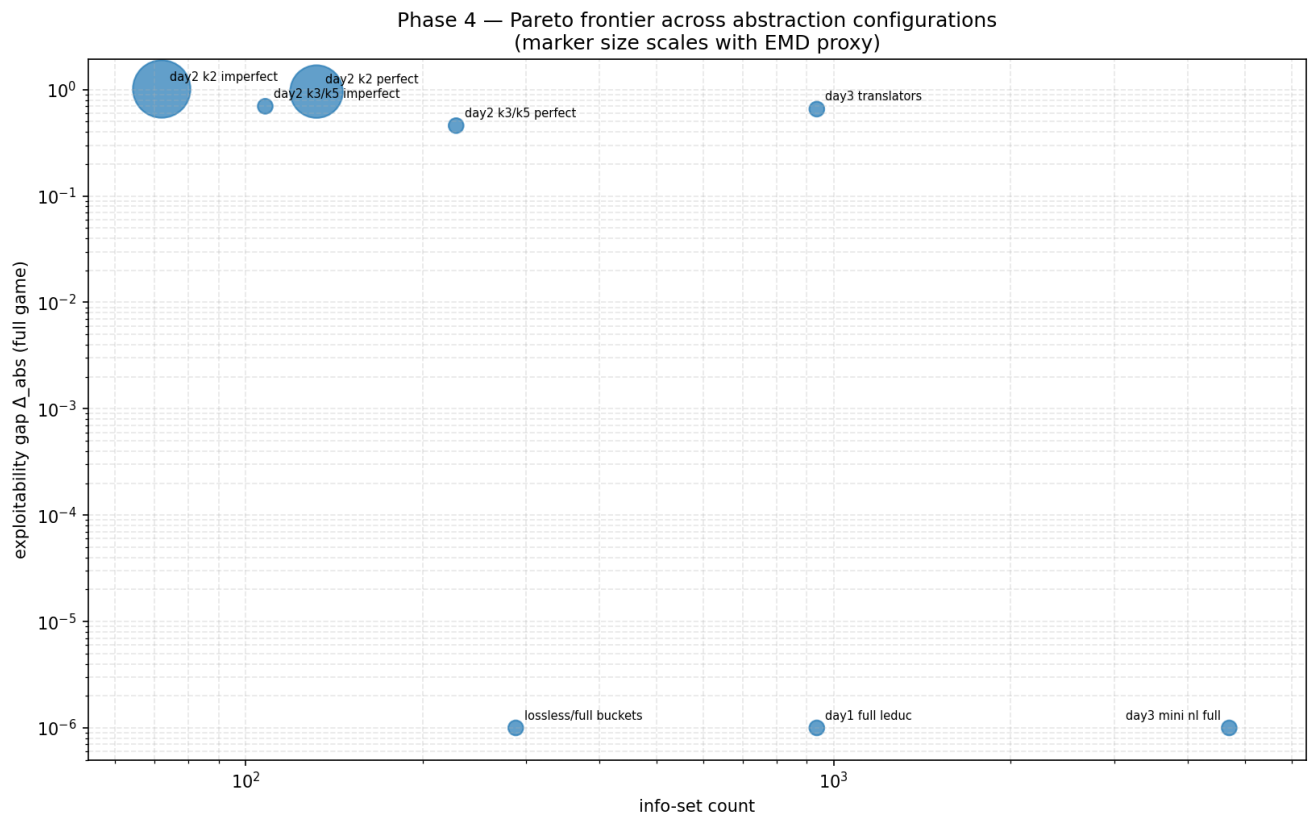


Figure 5: Abstraction Pareto frontier

The bridge to Step 6 is the blueprint architecture. Modern poker agents solve a coarse abstract game first, then patch weaknesses online with subgame solving. Step 4 supplies the vocabulary: blueprint, action translation, exploitability gap, Pareto frontier, and safe/nested refinement. Step 6 turns those pieces into complete game-playing systems.

For the thesis, abstraction matters because opponent adaptation only works at the resolution the representation preserves. If the abstraction merges two strategically distinct opponent-facing states, no downstream opponent model can recover that distinction. Conversely, a representation that is too fine may be too expensive to solve or evaluate. The Step 4 Pareto frontier therefore becomes part of the evaluation methodology: strategy quality must be reported together with the size and granularity of the game representation that produced it.

Step 5 Summary — Neural Networks for Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

May 2026

Contents

5 Step 5 — Neural Networks for Imperfect-Information Games	1
Why Neural Networks	1
5.1 Part 1 — Fundamentals	2
5.2 Part 2 — Architecture & Composition	4
5.3 Part 3 — Neural Networks in CFR	10
5.4 Part 4 — Other Neural Applications	13
5.5 Part 5 — Experimental / Frontier	15
5.6 Synthesis	15

5 Step 5 — Neural Networks for Imperfect-Information Games

This is a condensed survey of neural-network methods for imperfect-information games — the material that Step 5 widened from its original Deep CFR focus into a broader treatment of how neural networks are built, trained, and deployed in games where players cannot see the full state. It serves two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis. Step 5 is a deliberate one-off deviation from the usual learning cycle — the from-scratch implementation phase is deferred and the theory is broadened in its place — so this document is a breadth-first map of the field rather than a record of code. Mathematics is kept light: key equations are stated where they sharpen intuition, but derivations and proofs are left to the cited sources.

Why Neural Networks

Tabular CFR and its Monte Carlo variants (Steps 2–4) store a regret value and a strategy value at each information set, so their memory cost is proportional to the number of information sets, which grows with the size of the game. The Step 5 exploration quantified this limit on Leduc poker, a game small enough to hold in a table: tabular MCCFR reached an exploitability of 0.097, whereas Deep CFR under a comparable wall-clock budget reached only 1.70. At this scale the tabular method is preferable; the neural method offers no advantage, because the table already fits.

A neural network replaces the table with a function f_θ that maps a state’s features to a value or an action distribution and generalizes to states never visited during training. This inverts the cost structure: memory becomes independent of the number of information sets, at the price of more expensive iterations and weaker convergence guarantees. The two regimes meet at a crossover in game size — below it, enumeration is cheaper and exact; above it, the table cannot be constructed and function approximation is the only option. The games this research targets lie above the crossover, which is why the remainder of the chapter concerns neural methods.

5.1 Part 1 — Fundamentals

5.1.1 Neural Network Fundamentals

A feed-forward network is a composition of layers, each applying an affine map followed by an elementwise nonlinearity:

$$h = \sigma(Wx + b),$$

where x is the layer input, W and b are the trainable weight matrix and bias vector, and σ is a fixed nonlinearity such as the rectifier $\text{ReLU}(z) = \max(0, z)$. Stacking L such layers gives the network function f_θ , with θ collecting every W and b . The nonlinearities are what allow the composition to represent functions that no single affine map can.

ReLU (rectified linear unit): the standard activation — it passes positive values through unchanged and clamps negatives to zero, which is cheap to compute and avoids the vanishing gradients of older saturating activations.

Training requires a scalar measure of error. For a regression target — for example, a counterfactual value — the mean squared error is standard:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N (f_\theta(x_i) - y_i)^2,$$

where N is the number of examples in a batch, x_i an input, and y_i its target. When the target is an action distribution, the cross-entropy loss $-\sum_a y_a \log f_\theta(x)_a$ is used instead, with y_a the target probability of action a .

Learning is gradient descent on this loss. Each parameter moves a short distance in the direction that most reduces the loss:

$$\theta \leftarrow \theta - \eta \nabla_\theta L,$$

where η is the learning rate and $\nabla_\theta L$ the gradient of the loss with respect to the parameters. *Backpropagation* computes that gradient: it applies the chain rule layer by layer from the output back to the input,

reusing each layer’s stored intermediate values, so one gradient costs about as much as one forward pass. Automatic differentiation performs this procedure mechanically.

In practice the gradient is estimated from a small random batch rather than the full dataset (*stochastic gradient descent*) — noisier, but far cheaper per step. Momentum reduces that noise by stepping with a running average of gradients, $v \leftarrow \beta v + \nabla_{\theta} L$ and $\theta \leftarrow \theta - \eta v$, where $\beta \in [0, 1)$ sets how strongly past gradients persist. Adam, the default optimizer in the methods of this chapter, additionally divides each parameter’s step by a running estimate of its gradient magnitude, which yields stable progress with little manual tuning of η .

Adam (adaptive moment estimation): combines momentum with per-parameter step sizes derived from running gradient statistics, which is why it trains reliably with little tuning.

A network with sufficient capacity can fit a training batch exactly, including its noise, and then perform poorly on unseen states — *overfitting*. Regularization counters this. Weight decay adds a penalty $\lambda \|\theta\|^2$ to the loss, with λ controlling how strongly large weights are discouraged; dropout randomly zeroes a fraction p of activations during training so the network cannot depend on any single unit; and early stopping halts training once error on held-out data begins to rise.

One reinforcement-learning idea recurs throughout the chapter and is worth stating explicitly: *bootstrapping*. A temporal-difference update estimates a state’s value from the network’s own estimate at the next state rather than from a final outcome:

$$V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)],$$

where s and s' are consecutive states, r the reward, $\gamma \in [0, 1]$ the discount factor, and α the step size; the bracketed quantity is the temporal-difference error, the gap between the current estimate and the bootstrapped target. Bootstrapping makes learning sample-efficient and, as the next section shows, also less stable. The underlying material is in Sutton & Barto (2nd ed.) chapters 9 and 11; any standard deep-learning reference covers backpropagation and optimization in full.

5.1.2 Putting the Fundamentals Together on Leduc Poker

These pieces combine into the simplest neural solver for a game. Take Leduc poker, the testbed from Steps 3–4: its information state — the private card, the betting history, and the pot — encodes as a vector of about thirty numbers. A small network maps that vector to a value for each action; backpropagation with Adam fits it to target values generated during play, and light regularization prevents it from memorizing the noise in those targets. Used well, the network provides two capabilities the table lacks: it generalizes, so a value learned at one information set informs similar unseen ones, and its memory cost is fixed by the number of parameters rather than by the number of information sets.

The same machinery degrades when overdriven, and the exploration made the failure modes concrete. Capacity is the clearest. On Leduc the smallest network tested, two hidden layers of thirty-two units,

reached a lower exploitability than networks several times larger, because at a limited sample budget the larger networks held too many parameters for too little data and fit noise rather than structure. A learning rate set too high destabilizes the descent; excessive regularization flattens the sharp, near-deterministic strategies CFR converges to, since heavy weight decay or dropout pulls the output toward the uniform distribution. The general principle is that capacity and regularization must be matched to the amount and quality of the data rather than maximized by default — a point Part 2 develops into explicit sizing guidance.

5.1.3 Non-Stationarity and the Deadly Triad

Combining the components above introduces instability. Sutton & Barto identify a *deadly triad*: when function approximation, bootstrapping, and off-policy learning are used together, the parameters can diverge rather than converge, because each update’s bootstrapped target depends on the same parameters being modified. Any one component is safe in isolation; the combination is not. The remedies are practical rather than theoretical. A *target network* — a delayed copy of the parameters used only to compute bootstrap targets — and a *replay buffer* that decorrelates consecutive samples are together what make Deep Q-Networks trainable, and both reappear in the neural game solvers of Parts 3 and 4.

Games introduce a second source of instability absent from single-agent learning. In supervised learning the data distribution is fixed, and in single-agent reinforcement learning it stabilizes once the policy converges. In a game the opponent is also adapting, so the distribution of situations a network encounters changes as training proceeds; the function being approximated is itself moving. This non-stationarity is not incidental — it is the central difficulty of the setting this research addresses. Several methods in later parts are best understood as responses to it: self-play against frozen or time-averaged opponents restores a stationary target, population-based methods replace the moving target with an explicit set of fixed opponents, and the opponent-modeling and fast-adaptation techniques of Parts 4 and 5 attempt to track it directly.

5.2 Part 2 — Architecture & Composition

5.2.1 Inductive Bias: Matching Architecture to Game Structure

Every architecture encodes assumptions about the structure of its input before it sees any data; these built-in assumptions are its *inductive bias*. A fully connected network assumes nothing — every input feature may interact with every other — which is general but data-hungry, since the network must learn any structure from scratch. Choosing an architecture whose bias matches the game’s structure reduces the number of parameters and the amount of data needed to reach a given accuracy, and it improves generalization to states absent from training. The architecture decisions in the rest of Part 2 are, at bottom, choices of inductive bias.

Three correspondences cover most cases in imperfect-information games. When the state is spatial — a board or a grid — a convolutional network assumes that the same local pattern is meaningful wherever it appears, so a feature learned in one region transfers to others; this *translation equivariance* is why

convolutions dominate grid domains. When part of the state is an unordered collection — a hand of cards, or the set of other players — the network should be *permutation-invariant*, producing the same output regardless of the order in which those elements are presented, so that it does not expend capacity learning that the order is irrelevant. When the state is a history revealed over time, as under partial observability, a recurrent network or an attention mechanism lets the network summarize the sequence and carry forward what earlier observations implied. Matching these biases to the game is the difference between a network that learns efficiently and one that merely could in principle.

5.2.2 Layer Types in Practice: MLP, CNN, RNN, Attention/Transformer

The families differ in their *wiring* — which units connect to which — and in whether the connecting weights are *shared* across positions or time. Everything else (the neuron, the nonlinearity, backpropagation) is identical. In the figure below, edge colour denotes a distinct weight, so a repeated colour marks a shared weight; the paragraphs that follow read off each panel.

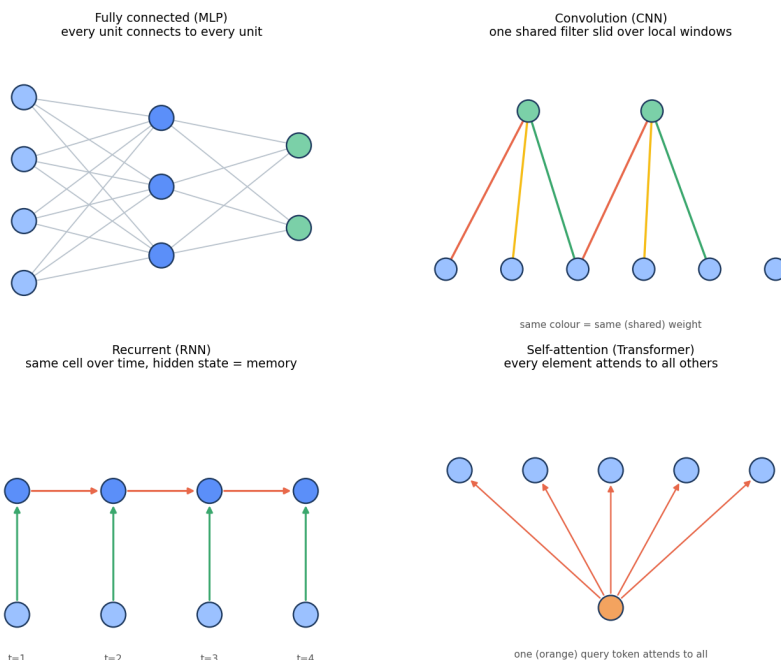


Figure 1: Connectivity of the four layer families: a fully connected MLP, a convolution that reuses one filter across local windows, a recurrent cell reused across time, and self-attention.

A **fully connected layer** — the multilayer perceptron, or MLP — connects every unit to every unit in the next layer, each connection carrying its own weight. It assumes no structure and can represent any relationship in principle, but the number of weights is the product of the two layer sizes, and all structure must be learned from data. It is the right choice for a flat feature vector with no spatial or sequential meaning, such as a poker information state.

A **convolutional layer** (CNN) connects each output unit to only a small local window of the input — its *receptive field* — and applies the same small set of weights, the *filter*, at every position. Two consequences follow from this one change in wiring: the number of weights depends on the filter size rather than the

input size, and a pattern learned at one location is recognized at every location (translation equivariance). Stacking convolutions enlarges the effective receptive field, so later layers combine local patterns into larger ones. This is the standard choice for boards and grids.

Weight sharing: reusing the same weights at many positions (CNN) or time steps (RNN).

It is what makes these layouts parameter-efficient and is the mechanism behind their equivariance.

A **recurrent layer** (RNN) applies the same cell repeatedly along a sequence: at each step it takes the current input together with the previous step’s *hidden state* and produces a new hidden state, which is passed forward. Because the cell is shared across steps, the number of weights is independent of sequence length, and the hidden state acts as a memory of everything seen so far — the property needed when the state is a history revealed over time, as under partial observability. Gated variants (LSTM, GRU) add learned gates that control what to keep and what to discard, which lets them retain information across longer spans.

A **self-attention layer** (the Transformer) lets every element interact with every other directly. Each element emits a *query*, a *key*, and a *value*; the output for an element is a weighted average of all elements’ values, with the weights set by the similarity between its query and each key. Because the operation ignores element order, it is permutation-equivariant, and order must be supplied explicitly through positional encodings when it matters. Attention captures long-range dependencies that a recurrent layer reaches only indirectly, at the cost of computation that grows with the square of the number of elements. It suits variable-length histories and set-valued inputs.

A genuinely unordered input — a hand of cards, or the set of other players — calls for **permutation invariance**, which comes not from a new layer type but from a layout: apply the same small network to each element, then combine the results with an order-independent operation such as a sum, mean, or maximum. This is the recipe behind Deep Sets, and it is the natural encoder for a collection whose order carries no meaning.

In practice a network rarely uses one family alone. The usual design is a pipeline: a separate *encoder* matched to each part of the input, the encoder outputs *fused* by concatenation into a shared *trunk*, and one or more *heads* that read predictions off the trunk — commonly a policy head and a value head. A Deep Recurrent Q-Network (DRQN) composes a convolutional encoder, an LSTM, and an output MLP, the recurrence supplying the memory a single feed-forward pass lacks under partial observability — the natural template for a fog-of-war grid agent. AlphaStar goes further, encoding a set of units with a Transformer, the spatial minimap with a CNN, and scalar statistics with an MLP, then fusing all three and passing them through an LSTM core before its action heads. The practical lesson for the rest of Part 2 is that “which architecture” is usually “which encoder for each input, and how to combine them,” rather than a single choice.

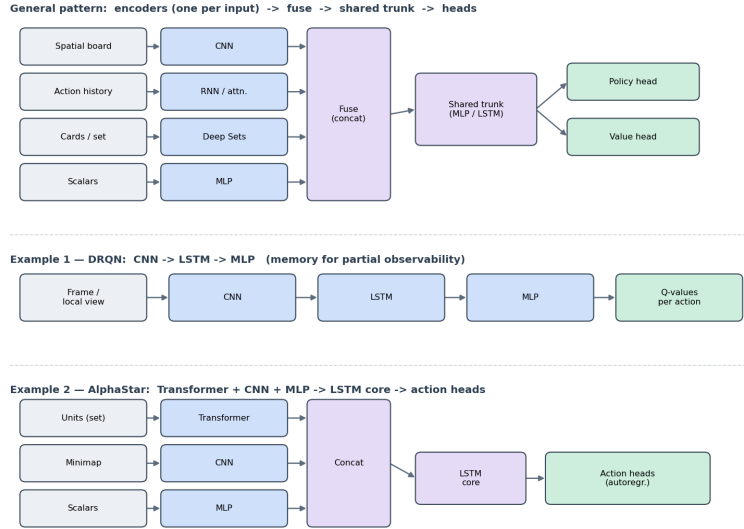


Figure 2: The general encoder / trunk / heads pattern (top); two instances below — DRQN and AlphaStar.

5.2.3 Encoding Game State & History

Before any of these architectures can run, the game state must be turned into numbers. The information state — everything the acting player may legally observe — has to become a fixed-shape tensor, and the way it is laid out determines which architecture can exploit it. Categorical facts are encoded either as *one-hot* vectors — a length- k vector with a single 1 marking which of k categories holds — or, when k is large, as *embeddings*: short learned vectors looked up per category and trained alongside the network. Leduc poker is small enough for one-hot throughout: the private card, the public card, and the betting history each become one-hot fields and the pot a single scalar, producing the roughly thirty-dimensional vector used in Steps 3–4.

One-hot vs. embedding: one-hot is exact but its length grows with the number of categories; an embedding compresses a large vocabulary — every card, or every board tile — into a few learned dimensions and lets similar categories share structure.

Richer inputs call for richer layouts. A spatial state is encoded as a stack of grids, one *channel* per entity type, so that a convolution can read it directly. Pommerman — a grid-based, Bomberman-style game in which agents move around a board placing bombs to destroy walls and trap opponents — is a clean example: its 11x11 board is split into a separate binary plane for each entity type (passages, wooden walls, rigid walls, bombs, flames, power-ups, the acting agent, and each opponent), stacked into an (11, 11, channels) tensor; scalar quantities such as remaining ammunition and blast range are then appended as a short vector or broadcast as constant planes. A history whose order matters is encoded as a *sequence* of per-step vectors for a recurrent layer or a Transformer, rather than flattened into one vector that discards the order. An unordered collection, such as the current set of opponents, is encoded element by element and pooled. In every case the encoding is where the inductive bias of the previous sections is

actually applied: a grid is presented as a grid precisely so that a convolution’s assumptions hold.

One discipline is specific to imperfect-information games and easy to violate: the encoding must contain only what the player can observe, and nothing hidden. If any privileged information — the opponent’s private card, or the true contents of a fogged cell — leaks into the input tensor, the network will learn to use it and effectively see through the fog, yielding a policy that cannot be deployed against a real opponent and an exploitability figure that is meaningless. This is why frameworks such as OpenSpiel define an explicit per-player information-state representation; getting that representation right is a correctness requirement, not a tuning choice.

Two smaller points complete the craft. Continuous features such as the pot or a stack size are normalized to a comparable scale before entering the network, so that no single feature dominates the gradient. And the encoder need not be designed by hand end to end: a network can learn to compress a long history into a compact summary vector — an idea Part 4 develops into learned abstraction and Part 5 into learned belief states.

5.2.4 Sizing & Capacity

A network’s *capacity* — set by its width, the number of units per layer, and its depth, the number of layers — is the size of the family of functions it can represent. Too little capacity and the network *underfits*, unable to express the target strategy; too much and it *overfits*, fitting the noise in a limited sample rather than the underlying structure. The right capacity is the one matched to how much informative data the training procedure actually supplies — which, in a game solver, is often far less than the raw number of states suggests, because the values being fit are themselves noisy estimates.

The Step 5 exploration showed this directly. Running Deep CFR on Leduc under a fixed budget but with different network sizes, the smallest network — two hidden layers of thirty-two units — reached a lower exploitability than networks two and four times wider; the larger networks held more parameters than the few thousand traversals could constrain and spent the surplus fitting sampling noise. The lesson is not that smaller is always better, but that capacity must track the data budget: on a larger game, or with far more traversals, the ranking would reverse.

This argues for sizing a network deliberately rather than defaulting to the largest that fits in memory. A workable recipe is to start small and increase width and depth only while a held-out metric — validation loss, or exploitability for a solver — keeps improving, stopping when it plateaus. Width lets a network represent more distinct patterns at a given level of abstraction; depth lets it compose simple patterns into more abstract ones, which is why spatial and sequential problems reward several layers while a flat feature vector often does not. Capacity should scale with the size and richness of the game, not be maximized by default.

Width need not stay constant across a network. Layers can *shrink* toward the output, compressing the input into a small summary (an encoder, or *funnel*), or *expand* from a small input to reconstruct a structured output (a decoder). Combining the two gives the *hourglass*, whose narrow middle — the *bottleneck* — forces all information through a low-dimensional code; because the network must act or

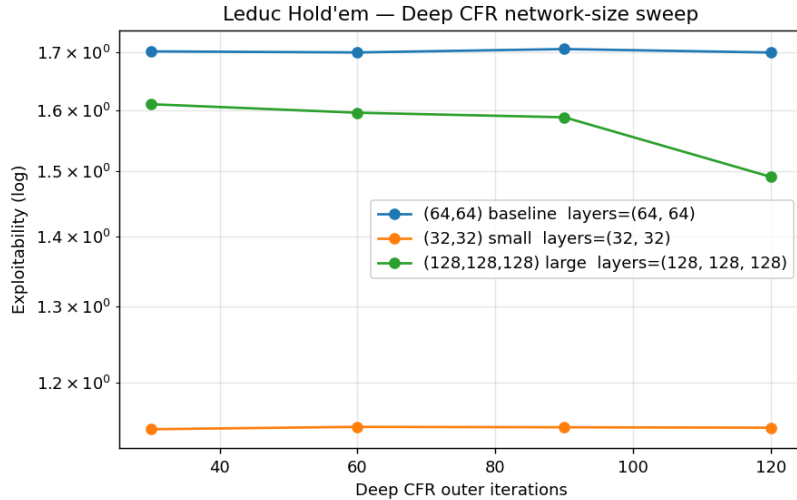


Figure 3: Deep CFR exploitability on Leduc across network sizes (32x32, 64x64, 128x128x128).

reconstruct from that code, it keeps only what matters, which regularizes the model and yields a compact, reusable representation — the seed of a learned abstraction or belief state (Parts 4 and 5). The same pinch dropped into an otherwise uniform stack is a *collar*. The figure below sketches these profiles.

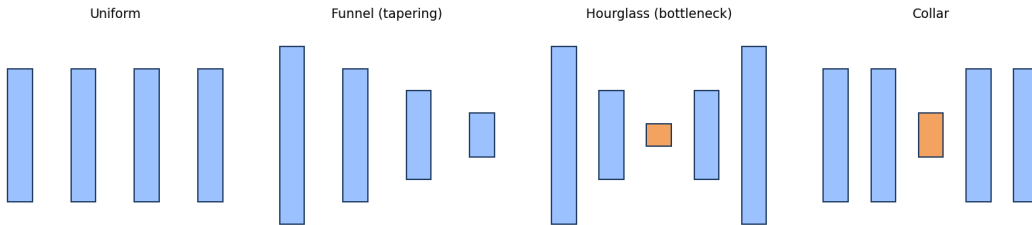


Figure 4: Common network shapes: a uniform stack, a tapering funnel, an hourglass with a central bottleneck (orange), and a uniform stack with a single narrow collar (orange). Bar height represents layer width.

The bottleneck constrains *width*; *skip (residual) connections* address *depth*, adding a layer's input to its output so gradients reach deep layers and very deep networks become trainable — the idea behind ResNets and the deep value networks of Part 4. Both complement the dropout, weight decay, and normalization mentioned earlier: each shapes how capacity is used, not just how much there is.

Skip (residual) connection: a shortcut that adds a layer's input to its output, so the layer only has to learn a correction; it keeps gradients strong through deep networks and is what makes very deep models trainable.

5.2.5 Training Stability & Diagnostics

Two stabilizers from the deadly-triad discussion — the target network and the replay buffer — do most of the work of keeping value-based training from diverging. Two more recur specifically in the game solvers ahead. *Reservoir sampling* maintains a fixed-size, uniformly random sample of all data seen so far, which

matters when a quantity must be averaged over the entire history of training rather than over recent data alone; Deep CFR uses it so that its strategy network trains on an unbiased sample across all iterations (Part 3). *Variance reduction* — subtracting a baseline from sampled returns, or using control variates — lowers the noise that sampling injects into the gradient, so the network converges from fewer samples. Input normalization and sensible weight initialization, mentioned earlier, complete the list: both keep activations and gradients in a usable range from the first step.

Reservoir sampling: a method that keeps a fixed-size sample in which every item seen so far is equally likely to be retained, so a running average stays unbiased without storing all the data.

Even with these in place, a neural solver fails quietly more often than loudly, so reading the diagnostics matters. A loss that falls steadily while the evaluation metric — exploitability, for a solver — stalls is the signature of a network that is *alive but not converged*: training is working, but the budget is too small for the average strategy to settle, which is what the Step 5 networks showed on Leduc at a few thousand traversals. The opposite and more dangerous case is training that is not running at all while appearing to: in the exploration, a one-character bug in an OpenSpiel routine left the advantage networks untrained, yet the program ran to completion and produced plausible output, with exploitability simply frozen at the random-strategy level. The lesson is to confirm that the loss actually moves and that the evaluation metric responds before trusting any result — a silent no-op is indistinguishable from a hard problem unless the curves are checked.

5.3 Part 3 — Neural Networks in CFR

5.3.1 From Tabular CFR to Function Approximation

Tabular CFR keeps two numbers at every information set: a cumulative counterfactual regret for each action, from which the current strategy is derived by regret matching, and a cumulative strategy, whose running average is the Nash approximation the algorithm ultimately returns (Steps 2–4). Both tables grow with the number of information sets — the wall described at the start of this chapter.

Replacing them with neural networks is conceptually simple: a function approximator takes an information-state tensor as input and predicts what the table would have stored. One network can learn the regrets — equivalently, the counterfactual advantages — that determine the current strategy; another can learn the average strategy directly. Because a network generalizes across similar information states, it never has to enumerate them, and its size is fixed by its parameters rather than by the game. The methods below differ mainly in *which* of the two tables they approximate and *how* they generate the data to train it: Deep CFR learns advantages from sampled tree traversals, while NFSP learns the average strategy from self-play without traversing the tree at all.

5.3.2 Deep CFR and Its Single-Network Variants

Deep CFR (Brown et al., 2019) is the direct neural translation of MCCFR. On each iteration it traverses the game tree by external sampling, and at every information set it visits it computes the sampled

counterfactual advantage of each action — how much better that action did than the current strategy on average. These (information-state, advantage) pairs go into a reservoir buffer, and an *advantage network*, one per player, is trained to predict them; the current strategy at any information set is then read off the network’s predicted advantages by regret matching, exactly as the table would have been used. A separate *strategy network*, trained from its own reservoir buffer, learns the average strategy across all iterations — the Nash approximation returned at the end.

The one equation that distinguishes Deep CFR from tabular MCCFR is the advantage network’s objective: a mean squared error between its prediction and the sampled advantages,

$$L(\theta) = \frac{1}{|B|} \sum_{(I, \tilde{a}) \in B} \|f_{\theta}(I) - \tilde{a}\|^2,$$

where B is a batch drawn from the reservoir, I an information-state tensor, $\tilde{a}$ the vector of sampled advantages for its actions, and f_{θ} the advantage network (in practice the samples are weighted by iteration, mirroring linear CFR). Everything else — the traversal, the regret-matching step, the averaging — carries over unchanged from the tabular algorithm.

Two refinements cut the cost. *Single Deep CFR* drops the separate strategy network and recovers the average policy directly from the stored advantage models, halving the networks to train. *DREAM* replaces external sampling with outcome sampling — one trajectory per iteration — and adds a learned baseline as a control variate to absorb the extra variance that outcome sampling brings, the same variance-reduction idea from Part 2. On large poker games Single Deep CFR matches or slightly outperforms Deep CFR while training a single network instead of two, as the original paper’s comparison shows.

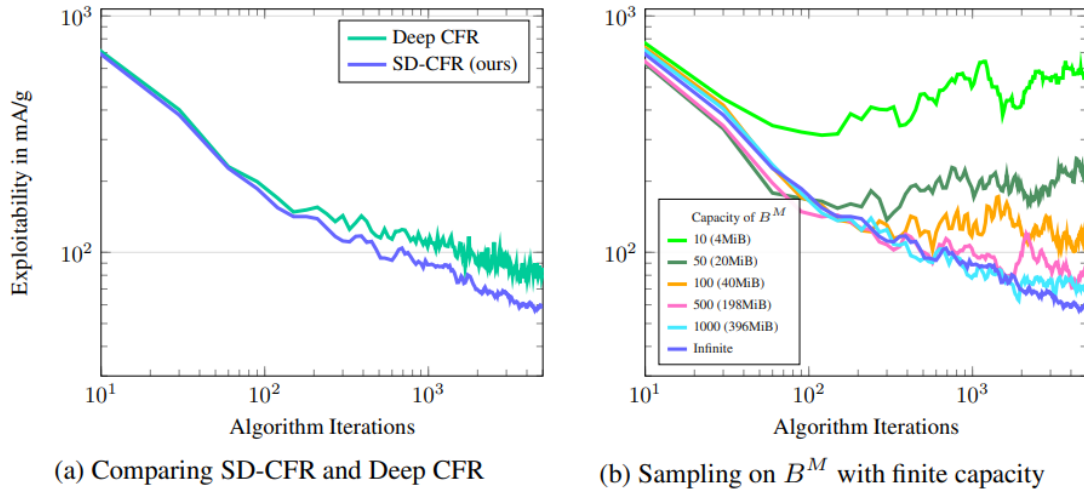


Figure 5: Single Deep CFR (SD-CFR) matches or slightly beats Deep CFR while training only one network (left); the right panel shows how reservoir-buffer capacity affects convergence. From the Single Deep CFR paper — not an experiment from this work.

On Leduc the exploration confirmed both the promise and the catch. Deep CFR trained correctly — its

advantage losses fell as expected — but at 120 iterations it reached an exploitability near 1.70, against 0.097 for tabular MCCFR at comparable wall-clock time. This is not a defect: Leduc is small enough that the table wins outright, and Deep CFR needs several times more iterations to approach Nash. Its advantage appears only when the game is too large for the table to exist — the regime the closing section of this part makes precise.

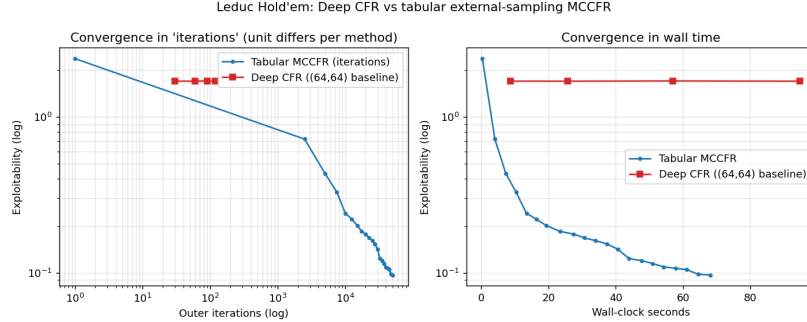


Figure 6: Deep CFR vs. tabular MCCFR exploitability on Leduc.

5.3.3 NFSP — Neural Fictitious Self-Play

NFSP — Neural Fictitious Self-Play (Heinrich & Silver, 2016) — reaches an approximate Nash equilibrium from the reinforcement-learning side, without ever traversing the game tree. Each player keeps two networks. A *best-response network*, a DQN, learns the greedy best response to the opponent’s current behaviour; an *average-policy network*, trained by supervised learning on the player’s own past actions, learns the time-average of those best responses. The average policy is what converges toward Nash — the same averaging that CFR performs, reached through play rather than regret.

The two networks are tied together by the *anticipatory parameter* η : at each step the agent acts from its best-response network with probability η and from its average-policy network otherwise. The fraction of best-response play keeps generating fresh, on-distribution data for the average policy to absorb, while the average policy supplies the stable opponent against which the best response is computed. No reservoir of tree traversals is required — the data comes entirely from episodes of self-play.

The price is sample efficiency. On Leduc the exploration ran NFSP for tens of thousands of episodes and saw exploitability drift around 2.5 rather than fall, because reaching a usable equilibrium even on this small game needs on the order of a million episodes or more — OpenSpiel’s own example uses far more still. NFSP scales to large games, where its model-free simplicity is an asset, but on a teaching benchmark it converges far more slowly than either tabular CFR or Deep CFR.

5.3.4 Trade-offs: When Neural CFR Pays Off

Stepping back: what do neural networks actually buy in CFR, and at what cost? The benefit is generalization and bounded memory — a network shares structure across similar information states and stores a strategy in a fixed number of parameters, so equilibrium computation no longer requires enumerating, or even visiting, every information set. The cost is steep per-iteration overhead and weaker guarantees:

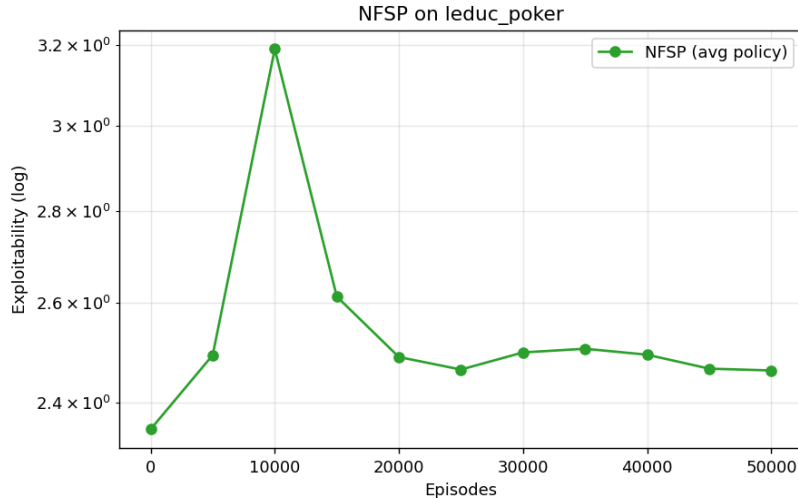


Figure 7: NFSP exploitability on Leduc over training episodes.

each iteration trains networks rather than incrementing counters, convergence is noisier, and the clean monotonic behaviour of tabular CFR becomes approximate. The exploration’s Leduc numbers are the small-game face of this trade-off: where the table fits, it wins.

The trade-off reverses with scale. As a game grows, the tabular tables eventually do not fit in memory at all, while a network’s footprint is unchanged; beyond that crossover the neural method is not merely competitive but the only option. This is why Deep CFR and its relatives matter for full-scale poker even though they lose on Leduc.

One structural caveat bounds where regret-based neural methods apply. CFR computes counterfactual values by reasoning about the whole subtree below an information set, which is cheap when the tree is *wide but shallow* — poker has an enormous number of hands but ends after a few betting rounds. In games that are *deep* — hundreds of sequential moves, as in a gridworld — that reasoning becomes prohibitively expensive, and the model-free self-play methods of Part 4 become the more natural fit. The regret-based family keeps growing within its niche — NeuRD, for instance, recasts the regret update as a policy-gradient step to combine CFR’s stability with neural policy optimization — but its sweet spot remains wide, shallow, imperfect-information games.

5.4 Part 4 — Other Neural Applications

5.4.1 The Four SOTA Algorithm Families

Part 3 covered one way to put neural networks to work in imperfect-information games — approximating CFR. It is one of four broad families that define the current state of the art, and choosing among them is largely a matter of matching the family to the game’s size, tree shape, and number of players. The four are *regret-based* methods (Deep CFR, DREAM, NeuRD) from Part 3; *population-based* methods (PSRO and its relatives, including NFSP); *search-plus-RL* methods (ReBeL, Player of Games); and *model-free self-play* (PPO, DQN, MAPPO, QMIX). The table places each; the sections that follow take the three

not yet covered in turn.

Family	Core idea	Best for	Cost
Regret-based (Deep CFR, DREAM, NeuRD)	Approximate CFR with networks	Wide, shallow games; unexploitability	High
Population-based (PSRO, NFSP, XFP)	Best-respond to a growing opponent pool	n-player; opponent modeling	Medium
Search + RL (ReBeL, Player of Games)	Learned value plus online search at play	High-stakes; inference-time compute	High
Model-free self-play (PPO, DQN, MAPPO, QMIX)	Optimize reward via self-play	Spatial, long-horizon (grids)	Low–medium

5.4.2 Population-Based: PSRO, NFSP, XFP

Population-based methods reframe equilibrium-finding as an iterative tournament. Starting from a small pool of policies, a *meta-game* is formed by playing the pool members against one another; a new policy is then trained as an approximate best response to the current population, added back to the pool, and the process repeats. This is the idea behind PSRO (Policy-Space Response Oracles) and its scalable variants such as Pipeline PSRO; fictitious self-play, including the NFSP of the previous part, is the special case where the best response targets the time-average of the pool. Because the population is an explicit set of opponents rather than a single moving target, these methods extend naturally beyond two-player zero-sum into the n -player, opponent-modeling setting this thesis is concerned with — and they are how AlphaStar reached grandmaster level at StarCraft II. The price is running and storing many agents, and the equilibrium guarantees are weaker than CFR’s.

5.4.3 Search + RL: ReBeL & Player of Games

Search-plus-RL methods are the AlphaZero idea adapted to hidden information. In a perfect-information game one can search forward from the current position because the state is known; under imperfect information the searching player does not know the true state, so the search must range over a distribution of possible states. ReBeL (Brown et al., 2020) makes this precise with the *public belief state* — a probability distribution over the players’ private information given everything public — and runs CFR over a small, depth-limited subgame at play time, with a learned value network supplying the leaf values. Player of Games (Schmid et al., 2023) generalizes the recipe into one algorithm that handles both perfect and imperfect information. These systems reach the strongest play but are the most complex to build and depend on having compute available *during* play, not only during training; they are the subject of Step 6, so this section only places them in the landscape.

5.4.4 Model-Free Self-Play: PPO, DQN, MAPPO, QMIX

Model-free self-play methods ignore the game tree entirely and optimize the reward signal directly, improving a policy from episodes of play against copies of itself. They are the workhorses of multi-agent

reinforcement learning: PPO and DQN for the single-agent core, and the multi-agent extensions MAPPO and QMIX for cooperative and mixed settings. Because they never reason about the subtree below a state, they are unbothered by the depth that defeats regret methods, which makes them the natural choice for spatial, long-horizon games — gridworlds, Pommerman, and the like. To cope with partial observability they rely on the recurrent and attention architectures of Part 2, giving the agent a memory of what the fog has hidden. Their weakness mirrors their strength: with no notion of counterfactual regret they carry no built-in pressure toward unexploitability, so a model-free agent can be strong on average yet remain exploitable by a worst-case opponent — which is precisely why exploitability has to be measured separately, the subject of a later section.

5.4.5 NN for Abstraction, Opponent Modeling & Belief Representation

5.4.6 Evaluating Neural Policies: Exploitability at Scale

5.4.7 Generalizing Beyond Poker: Environments & Tooling

5.5 Part 5 — Experimental / Frontier

5.5.1 Memory & Time

5.5.2 Latent Belief Models

5.5.3 Fast Adaptation

5.5.4 Modeling Other Minds & Relations

5.5.5 Uncertainty & Symmetry

5.5.6 Deliberately Out of Scope

5.6 Synthesis

5.6.1 A Decision Map: Which NN Method for Which Game

5.6.2 Connections & Forward Pointers

Step 6 Summary — End-to-End Game AI Architectures

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

June 2026

Contents

6 Step 6 — End-to-End Game AI Architectures	1
6.1 Introduction	1
6.2 DeepStack (2017)	3
6.3 Libratus (2017/2018)	8
6.4 Pluribus (2019)	15
6.5 ReBeL (2020)	22
6.6 Student of Games / SoG (2023)	30
6.7 Synthesis	39

6 Step 6 — End-to-End Game AI Architectures

6.1 Introduction

Chapter 6 is the keystone of this study plan. Steps 1–5 assembled the parts in isolation — the game-theoretic vocabulary of extensive-form games and Nash equilibria, counterfactual regret minimization (CFR) and its Monte-Carlo variants, game abstraction, and the neural function approximators that replace tabular storage once a game outgrows it — and this chapter is where those parts converge into complete, competition-grade systems. The treatment is deliberately **architectural**: rather than re-deriving the mathematics, which lives in the technical chapters and the cited papers, each system is studied at the level of how it is *put together* — what it computes offline, what it computes in real time, where learning sits, and what it can actually guarantee. A short per-system scorecard opens each section so the same nine dimensions line up at a glance, and the prose then traces the design decisions, the engineering compromises, and the abandoned approaches behind them.

The five systems span seven years and, read in order, trace the evolution of superhuman game AI: **DeepStack** (2017), **Libratus** (2017/2018), **Pluribus** (2019), **ReBeL** (2020), and **Student of Games** (2023). It is tempting to read such a list as a leaderboard — each entry strictly stronger than the last — but that is neither what happened nor how this chapter is organized. The progression is a sequence of *deliberate trades*: each system bought a new capability by giving something else up, and the most instructive moments are the ones where the field stepped sideways or even backward. DeepStack and Libratus are near-contemporaries that proposed *opposite cures* for the same disease — DeepStack discarded the

abstraction-and-blueprint paradigm that had dominated computer poker for nearly two decades, while Libratus kept it and cured its one fatal flaw with real-time, provably safe subgame solving. Pluribus then broke the *player-count* barrier, reaching superhuman six-player play, but only by abandoning safety guarantees altogether and remaining entirely tabular. ReBeL stepped back to two players yet leapt forward on generalization — learning belief-state values with AlphaZero-style self-play, dissolving both the abstraction and the blueprint, and *recovering* the safety Pluribus had surrendered. Student of Games, the capstone, unified perfect- and imperfect-information play in a single algorithm — and paid for that breadth with peak strength, losing decisively to a specialist AlphaZero at Go. Read this way, the chapter is a study of what each advance cost, not a ranking of winners.

Beneath the individual trades, three axes of motion run through all five systems and give the chapter its spine. The first is **representational**: the move from hand-built *abstraction* — bucketing similar hands and allowing only a few bet sizes — toward *learned neural approximation*, in which a network generalizes across situations a table could never enumerate. The second is **temporal**: the move from *solving the entire game offline* into a stored strategy toward *learning a compact model and searching with it in real time*, so that computation is spent at the moment of decision rather than baked in advance. The third axis appears only at the very end, with Student of Games: the unification of **perfect- and imperfect-information** play in one sound algorithm, joining the two great traditions of game AI — the minimax / Monte-Carlo-tree-search / AlphaZero line and the CFR / poker line — that had run on separate tracks for seventy years. The first two axes describe the four poker systems; the third is what makes the fifth the chapter’s capstone.

One idea ties these threads together and deserves to be named before the systems themselves, precisely because it is *not* one of them: **depth-limited solving**. Formalized by Brown & Sandholm (2018), it is the principle that one may search only a little way ahead in an imperfect-information game and substitute a *learned or precomputed value* for the remainder — provided the substitution is done so that hidden information does not render it unsound. It is the theory that retroactively unifies DeepStack’s continual re-solving with Libratus’s nested subgame solving, that underwrites Pluribus’s continuation strategies, and that, in belief-state form, becomes the inner loop of ReBeL and Student of Games. We treat it as connective tissue — referenced wherever a system instantiates it — rather than as a sixth entry, because depth-limited solving is the mechanism *through which* the offline-to-search axis actually operates.

Finally, because this chapter is the hinge between the fundamentals of Steps 1–5 and the opponent-modelling and exploitation work of Steps 7–15, it is worth flagging at the outset the single thread the synthesis returns to. Every system here is, by deliberate design, **opponent-blind**: each computes a strategy that is hard to beat *in the worst case* and then plays it without regard to who is actually across the table — Pluribus does not even know its opponents’ identities, and both it and Libratus refuse on principle to model or adapt to them, so as never to be counter-exploited in return. This robustness-first stance is the field’s great strength and, for a dissertation about *adaptive* play, its defining limitation: it is exactly the opponent-awareness these systems omit that Steps 7–15 set out to add. The chapter therefore closes not with a winner but with a synthesis — a map of what the five systems share, what each gave up, and where the open problems that motivate the rest of this work actually lie.

6.2 DeepStack (2017)

DeepStack (Moravčík et al., 2017), from the University of Alberta computer-poker group with collaborators in Prague, was the first program to defeat professional poker players at heads-up no-limit Texas hold'em (HUNL) with statistical significance, and the first to put heuristic search — the engine behind chess and Go — on a theoretically sound footing in a game of imperfect information. Over a four-week study it beat a pool of 33 professionals by 492 milli-big-blinds per game (mbb/g, the standard poker win-rate unit; 50 mbb/g is a sizable professional edge) across 44,852 hands, and no known technique could find a flaw in its play. Its guiding *what-if* is the one that frames this whole chapter: **could a program play poker the way AlphaGo plays Go — searching locally from the current situation and trusting a learned value function for everything beyond the horizon — even though, in poker, the situation is itself partly hidden?**

At a glance	DeepStack (2017)
Players	2 (heads-up)
Game type	HUNL — heads-up no-limit Texas hold'em (2-player zero-sum)
Blueprint (offline)?	No — offline work trains value nets, not a stored strategy
Neural component	Deep counterfactual value networks (flop, turn, aux); value-only
Search mechanism	Continual re-solving (depth-limited CFR look-ahead, every decision)
Abstraction?	None constrains play; 1,000-bucket clustering only at the net input, plus sparse betting + river action bucketing in look-ahead
Perfect-info too?	No (imperfect-information only)
Compute	Offline-heavy (~175 CPU-core-years to label the turn network); play-time runs on one GPU, < 5 s/decision
Key innovation	Continual re-solving + learned counterfactual values: the first <i>sound</i> heuristic search for imperfect-information games

6.2.1 The gap it closed

Every prior game-AI milestone — backgammon, chess, Go — rested on *local search*: from the current position, look a few moves ahead and substitute a heuristic value for the rest. That recipe assumes the position is known, and poker breaks the assumption — the right action depends on the distribution over the opponent's hidden cards, revealed only through their betting, which in turn depends recursively on what they believe about your cards. The open question DeepStack set out to answer is whether heuristic

search can be made *sound* under hidden information at the scale of HUNL, a game with roughly 10^{160} decision points — comparable to Go, but imperfect-information.

For nearly two decades the dominant answer had been something else entirely: **abstraction plus offline equilibrium plus translation**. Shrink HUNL’s 10^{160} situations into roughly 10^{14} abstract ones by bucketing similar hands (*card abstraction*) and allowing only a few bet sizes (*action abstraction*), solve that smaller game offline with counterfactual regret minimization (CFR) to produce a complete strategy — a *blueprint* — store it, and at play time *translate* each real situation and opponent bet into the nearest abstract one. The compression is lossy, and the loss shows up as exploitability — how much a worst-case opponent can win, the field’s quality metric, zero at a Nash equilibrium. In 2015 the abstraction-based program Claudico lost to professionals by 91 mbb/g, and a local-best-response probe (LBR, a tractable lower bound on exploitability) later showed top competition bots exploitable by more than 3,000 mbb/g — four times worse than folding every hand. Blueprints were also enormous (a single no-card-abstraction strategy took about 2 TB and 14 CPU-years to compute and was *still* exploitable through off-tree bets), and the translation step was itself a source of weakness. DeepStack closes this gap by discarding the whole edifice: it never builds a full-game abstraction and never stores a blueprint, reasoning about each situation *as it actually arises* and replacing only the distant remainder of the game with a learned estimate.

6.2.2 Architecture

DeepStack splits cleanly into an **offline** phase that learns intuition and an **online** phase that searches with it (Figure 6.1).

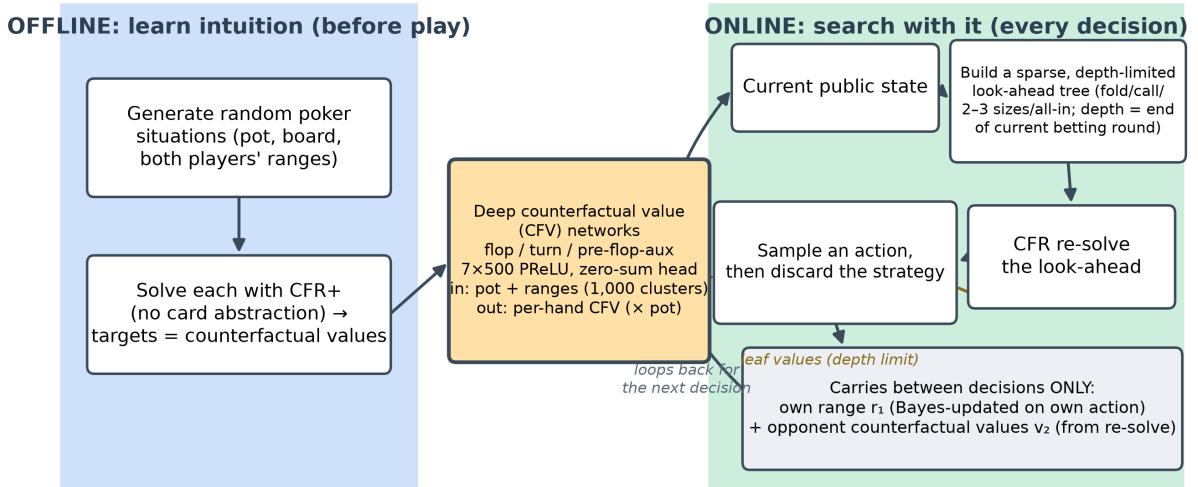


Figure 1: DeepStack architecture: offline intuition-building (left) feeding a shared deep counterfactual-value network, reused as the leaf evaluator inside the online continual-re-solving loop (right).

Offline, the system generates millions of random poker situations and solves them with a CFR solver to obtain target *counterfactual values* — conditional “what-if” payoffs for holding each possible hand — and these (situation → value) pairs train the **deep counterfactual value (CFV) networks**. Online, DeepStack carries no blueprint: between decisions it remembers only two vectors — its own *range* (the

distribution over the hands it could be holding) and the opponent’s *counterfactual values* — and at each turn it runs CFR over a small look-ahead tree rooted at the true current state, using a CFV network to supply leaf values at the depth limit. CFR already computes both ranges and counterfactual values, which is why it slots so naturally into this loop.

This is heuristic search with three ingredients: a sound local strategy computation (continual re-solving), depth-limited look-ahead with a learned value function in place of the rest of the game, and a restricted set of look-ahead actions to keep each solve fast. The neural network sits *only* at the depth limit as a leaf evaluator; CFR does the searching; and — crucially — there is no full-game abstraction in the loop.

6.2.3 Key innovation: continual re-solving with learned counterfactual values

DeepStack’s contribution is the marriage of two ideas, each of which makes the other practical.

The first is **continual re-solving**: reconstructing a fresh local strategy at every decision from the maintained range and opponent counterfactual-value vector, then discarding it, so the agent never stores or commits to a global strategy. Classical re-solving (Burch et al., 2014) shows that to reconstruct a strategy for a subgame you do not need the whole strategy — you need only your own range entering the subgame and a vector of the opponent’s counterfactual values. DeepStack pushes this to its limit: it *never* holds a strategy for the whole game. Every time it must act, it re-solves the current public state from those two vectors, plays one action, and throws the strategy away.

What makes this work is the bookkeeping. After each event DeepStack updates its two vectors by simple rules: on its **own action** it swaps in the re-solved counterfactual values for the chosen action and Bayes-updates its range; on a **chance event** it swaps in that card’s values and zeroes the now-impossible hands; and on the **opponent’s action** it does *nothing at all*. That last point is the quiet masterstroke: because it tracks the opponent’s *values* rather than their *range*, and never needs the opponent’s specific action to maintain those values, it sidesteps the action-translation step that crippled abstraction-based bots. The opponent can make any bet of any size; DeepStack simply re-solves from the state that bet produced.

The second idea is the **learned counterfactual value network** — DeepStack’s “intuition.” In a perfect-information game a leaf evaluator maps one state to one number; under imperfect information it must map a *whole public state plus both players’ ranges* to a *vector* of counterfactual values, one per hand, because the values at a node shift with the ranges that reach it. DeepStack learns this map with a feed-forward network of seven hidden layers of 500 units, taking the pot size and the two ranges (compressed into 1,000 hand clusters) as input and emitting per-hand counterfactual values as fractions of the pot. A bespoke outer layer enforces the zero-sum constraint: it forms the two implied game values from the ranges and raw outputs and subtracts half their sum, so the estimates are mutually consistent and the whole thing stays differentiable. With this network supplying values at the end of the current betting round, the depth-limited re-solve shrinks the game from 10^{160} decision points to about 10^7 — small enough to solve in under five seconds on a single GPU.

The pairing is provably sound. If the value network’s error is at most ϵ and the re-solve runs T CFR iterations, the resulting strategy’s exploitability is bounded by

$$\text{exploitability} < k_1 \epsilon + k_2 / \sqrt{T},$$

with game-specific constants k_1, k_2 . The first term is the price of imperfect intuition; the second is ordinary CFR convergence. This bound is the theoretical heart of the paper — the guarantee that heuristic search can be carried into imperfect information without the strategy quietly becoming exploitable — and the same depth-limit-plus-learned-value template, formalized further by Brown & Sandholm’s depth-limited solving the following year, underlies every system in this chapter.

6.2.4 Caveats, dead-ends, and what the paper under-describes

The clean story above is the main text; the engineering reality lives in the supplement, and it is where the spine’s “caveats and evolution” emphasis earns its keep.

The most important asterisk is that **the deployed system is not the one the theorem covers**. To play at human speed DeepStack restricts its look-ahead to a sparse betting set (fold, call, two or three bet sizes, all-in), and the paper states plainly that this “voids the soundness property of Theorem 1.” So the shipped guarantee is empirical — supported by the LBR results below — not proven. A second deviation compounds this: the soundness proof assumes *best-response* constraint values, but DeepStack actually uses *self-play* values, which “lack a theoretical justification” yet were less exploitable in early tests. The proven algorithm and the winning algorithm are, strictly, different algorithms.

Abstraction also creeps back at the margins. DeepStack advertises that it uses no card abstraction to constrain play — but it *does* cluster hands into 1,000 buckets at the value network’s input, and on the river it abandons the network entirely, solving to the end of the game while using a **bucketed action abstraction** for tractability. To speed up re-solving it also **warm-starts** the opponent’s range, in a conservative blend most of the time but in an aggressive variant (when acting first) that “sacrifices the re-solving guarantees when the opponent’s range estimate is wrong.” And the pre-flop, far from cheap, requires enumerating all 22,100 possible flops through the flop network, mitigated only by caching repeated betting sequences. Finally, the choice of re-solving gadget (the CFR-D gadget over the max-margin alternative) was made because it “performed better in early testing,” not derived — and, echoing Step 5, the network’s useful depth was capped by data rather than architecture, with validation error flattening past five layers at the ten-million-sample budget. None of these undermine the result, but together they show that the headline “sound, abstraction-free search” is an aspiration the implementation approximates rather than attains.

6.2.5 Compute & accessibility

DeepStack’s cost is almost entirely **offline, and almost entirely in the CFR solving used to label the value networks** — not in the neural training, and not in play. Generating the targets meant approximately solving millions of random subgames: the turn network alone consumed about **175 CPU-core-years** on a 6,144-core cluster, with the flop network adding roughly half a GPU-year on 20 GPUs; by contrast, the networks themselves trained in about two days each on a single GPU, and at play time

DeepStack runs on **one commodity GPU at under five seconds per decision**. The shape of that bill is the real lesson: the “intuition” is bought once, up front, by brute-force equilibrium solving, after which deployment is cheap — the mirror image of a system whose cost is dominated by inference. In accessibility terms this put a *from-scratch* build within reach only of a well-resourced lab at the time (the offline solve is cluster-scale), even though the trained agent then ran on hardware any enthusiast owned; a decade on, that offline cost is far more tractable on commodity cloud compute and with faster modern solvers, and a public reference implementation on Leduc lowers the entry barrier further.

6.2.6 Strengths and limitations

DeepStack’s central strength is **soundness with low exploitability**: it is the first imperfect-information search method with a real guarantee, and empirically LBR — which exposes competition bots as losing thousands of mbb/g — cannot find any way to beat it, itself losing by over 350 mbb/g. It needs **no full-game abstraction and no action translation**, so off-tree opponent bets are handled exactly rather than rounded; its **play-time footprint is modest**; and it learns from **no human data and little domain knowledge**. A neat bonus is evaluation synergy — DeepStack’s own value function is exactly what the AIVAT variance-reduction estimator needs, cutting the human-study standard deviation by 85% and making significance achievable in only a few thousand hands.

The limitations set the agenda for the rest of the chapter. DeepStack is **two-player zero-sum only**; its soundness leans on that structure. It retains a **sparse betting abstraction** in the look-ahead and an action abstraction on the river, and it inherits **value-network approximation error** (the $k_1\epsilon$ term), worst on the flop where it leans hardest on the network. It pays the **heavy offline cost** noted above and **re-solves from scratch** at every decision, and it was validated only against humans and LBR — never head-to-head against the strongest abstraction bots.

6.2.7 Legacy and modern relevance

Strip away the poker specifics and DeepStack’s core idea is **depth-limited search with a learned value function at the leaves, adapted to hidden information** — the imperfect-information counterpart of the value-guided search behind AlphaGo and AlphaZero. That idea did not age into obsolescence; it became the template. Brown & Sandholm formalized the depth-limited-solving theory the next year, and the paradigm was then generalized into full reinforcement-learning-plus-search frameworks: ReBeL (2020) recast the leaf-value learning around public belief states with AlphaZero-style self-play, and Student of Games (2023) — which shares several DeepStack authors — unified perfect- and imperfect-information play in a single algorithm. The same “planning in the loop with a learned value model” recipe later reached beyond two-player zero-sum poker, for instance in CICERO’s human-level Diplomacy play. More broadly still, DeepStack is an early, clean instance of the principle now central to frontier AI: **spend compute at decision time via search guided by learned intuition, rather than baking everything into one giant precomputed policy** — the same “test-time compute” thesis behind today’s reasoning models, with the field’s own caveat (voiced by Noam Brown) that imperfect information needed *belief-aware* search precisely because the plain Monte-Carlo tree search that works for Go does not work for poker.

Several concrete subsystems remain reusable today: the depth-limit-plus-learned-leaf-value pattern itself; the differentiable **zero-sum-consistency output head** for two-player value networks; **AIVAT-style variance reduction**, which turns a learned value model into a control variate for low-variance evaluation of any stochastic agent; and the small but transferable trick of expressing values as **fractions of the pot** for scale-invariant generalization. What is genuinely superseded is the hand-engineered machinery *around* the idea — the 1,000-bucket k-means clustering, the separate per-round networks, the hand-tuned re-solving gadget — which the more general learned representations of ReBeL and Student of Games replace. The honest verdict: **as a deployed poker system DeepStack is superseded, but it is far from a mere stepping stone** — its central paradigm won and is now mainstream, and for this thesis specifically its (range, opponent counterfactual values) state and its error-propagation bound are direct seeds for belief-based opponent modelling (Contribution 1) and safe-exploitation analysis (Contribution 2).

6.3 Libratus (2017/2018)

DeepStack answered its *what-if* by throwing the abstraction-and-blueprint edifice away — yet it still kept a sparse betting abstraction inside its own look-ahead, and it was never tested head-to-head against the strongest prior bots or against HUNL specialists in a long, rigorous match. Libratus (Brown & Sandholm, 2017; Carnegie Mellon) — Latin for *balanced*, as in approximating a Nash equilibrium, and *forceful*, for its play — was built independently and announced the same year, and it made the opposite bet: keep the abstraction-and-blueprint paradigm and cure its one fatal disease. Its guiding question is the mirror image of DeepStack’s: **what if we solve a coarse blueprint of the whole game offline, then *repair it in real time* wherever the abstraction is too crude — with a provable guarantee that the repair never leaves us more exploitable?** In January 2017 Libratus became the first program to beat top human HUNL specialists in a long match, defeating four professionals by 147 mbb/g (milli-big-blinds per game, the win-rate unit from the previous section; ~50 mbb/g is a sizable professional edge) over 120,000 hands at 99.98% significance — and, unlike DeepStack, it first dismantled the prior best poker AI head-to-head.

At a glance	Libratus (2017/2018)
Players	2 (heads-up)
Game type	HUNL — heads-up no-limit Texas hold’em (2-player zero-sum)
Blueprint (offline)?	Yes — an abstracted full-game strategy solved offline with MCCFR (detailed early, coarse late)
Neural component	None — purely tabular CFR / abstraction (no neural networks anywhere)
Search mechanism	Nested safe subgame solving (real-time CFR+ re-solve of the late game, re-run for every off-tree opponent bet)

At a glance	Libratus (2017/2018)
Abstraction?	Yes — card abstraction (turn/river, blueprint only) + asymmetric action abstraction; dissolved to <i>no card abstraction</i> inside the real-time subgames
Perfect-info too?	No (imperfect-information only)
Compute	Offline- <i>and</i> online-heavy: ~25M CPU core-hours on the Bridges supercomputer; ~50 nodes and tens of seconds per late decision; no GPUs
Key innovation	Blueprint + real-time nested <i>safe</i> subgame solving + self-improvement: exact, provably-safe responses to off-tree bets in place of action translation

6.3.1 The gap it closed

The previous section laid out the paradigm DeepStack discarded — abstraction plus offline equilibrium plus translation — and named its fatal flaw. Libratus targets exactly that flaw without discarding the paradigm. The blueprint is solved in advance over a *fixed* menu of bet sizes; at play time any opponent bet that is not on that menu is *translated* — rounded to the nearest size the bot knows — after which the bot replies as though the opponent had made the rounded bet. That rounding is the single largest exploitable seam in abstraction-based poker: a local-best-response probe had shown the leading competition bots losing thousands of mbb/g to a worst-case adversary, and in 2015 Libratus’s own predecessor Claudico lost the first *Brains vs. AI* match to professionals by 91 mbb/g, in good part because opponents could feel out and punish its translation boundaries.

The conceptual question Libratus answers is therefore narrower and more surgical than DeepStack’s: *can the decades-old abstraction paradigm be made superhuman by repairing only its real-time behaviour — responding to off-menu bets exactly rather than by rounding — and can that repair be done with a guarantee that local fixes never increase global exploitability?* The answer is the three-module architecture below. Where DeepStack removed the blueprint entirely and leaned on a learned value function, Libratus keeps the blueprint as a cheap scaffold and adds a real-time solver that, whenever the opponent steps off the tree, builds and solves a finer subgame *containing the actual bet* and stitches the result back into the blueprint. The same year, then, produced two opposite cures for the same disease — one neural and blueprint-free, one tabular and blueprint-based — and Libratus is the proof that the older paradigm, properly repaired, was still enough to reach superhuman play.

6.3.2 Architecture

Libratus is a pipeline of three modules that operate on three different timescales — **offline** (before the match), **online** (during each decision), and **overnight** (between days of play) — and, unlike every other system in this chapter, it contains **no neural network at all** (Figure 6.2).

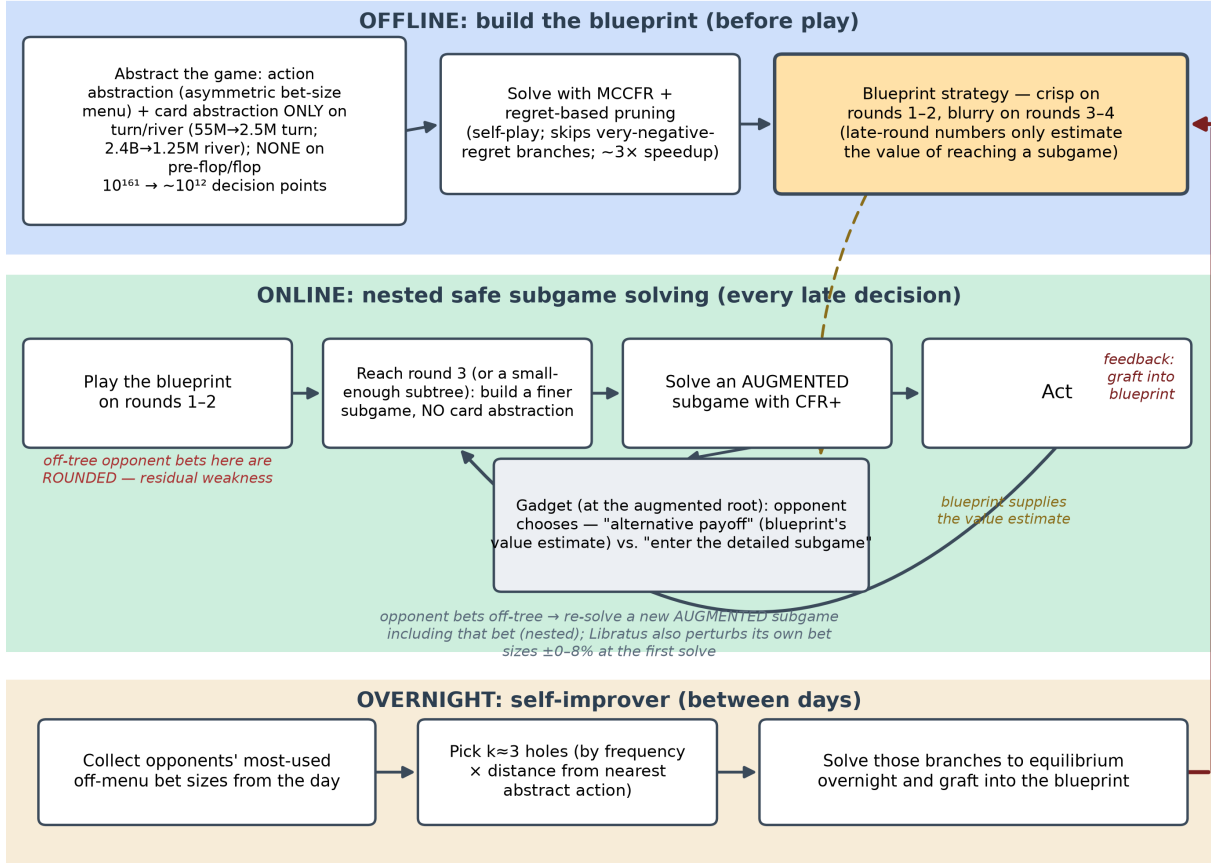


Figure 2: Libratus's three-module pipeline: an offline blueprint (top), the online nested safe-subgame solver that plays it (middle), and the overnight self-improver that grafts solved holes back into the blueprint (bottom, feedback arrow).

Module 1 — the blueprint (offline). Libratus first compresses HUNL’s roughly 10^{161} decision points to about 10^{12} with two kinds of abstraction: an *action abstraction* that keeps only a discrete menu of bet sizes (mostly round fractions and multiples of the pot, drawn from the sizes top competition bots favour, with a few early sizes tuned by a parameter-optimization algorithm), and a *card abstraction* that groups strategically similar hands. Crucially it uses **no card abstraction on the first two betting rounds** — small enough to afford full resolution — and buckets only the turn and river, and even there only in the blueprint. It then solves this abstract game by self-play with an improved **Monte Carlo counterfactual regret minimization (MCCFR)** that probabilistically prunes very-negative-regret branches, a roughly three-fold speedup that also quietly heals a pathology of imperfect-recall abstractions, in which several distinct situations share one bucket’s strategy and “fight” over it. The result is the **blueprint**: a complete but uneven strategy — detailed early, coarse late — whose late-round numbers are used not to play but only to *estimate the value of reaching a subgame*.

Module 2 — nested safe subgame solving (online). Libratus plays the blueprint only in the early rounds. On reaching the third betting round — or any earlier point where the rest of the hand is small enough — it discards the coarse late-game blueprint and instead **builds a fresh, finer-grained subgame with no card abstraction and solves it in real time** with a heavily optimized **CFR+** (a fast, deterministic CFR variant). Each hand is thus played individually in the late game. The defining feature is what happens when the opponent makes a bet that is not on the blueprint’s menu: rather than round it, Libratus solves a new subgame that *contains that exact bet*, and repeats this for every subsequent off-menu action — *nested* subgame solving. This is the module that the next subsection dissects.

Module 3 — the self-improver (overnight). Because real-time solving is skipped on the first two rounds (solving so early would itself demand heavy abstraction), off-menu opponent bets *there* are still rounded. The self-improver narrows this residual seam between days: it tallies the opponents’ most-used off-menu bet sizes, chooses a few of the most damaging holes, computes proper game-theoretic responses to them overnight, and grafts those branches into the blueprint. Pointedly, this is **not opponent exploitation** — it never tries to model and punish the humans’ mistakes (which would expose Libratus to counter-exploitation). Instead it uses the opponents’ bets only as a hint about *which of Libratus’s own holes to patch*, and the patches are universal, improving play against any future opponent.

The classic building blocks of earlier steps sit in plain view: game abstraction and MCCFR build the blueprint offline, CFR+ does the online subgame solving, and “search” is the per-decision real-time resolve. The conspicuous absence — no value network, no policy network, nothing learned by gradient descent — is the architectural signature that separates Libratus from every other system in this chapter.

6.3.3 Key innovation: nested safe subgame solving

Libratus’s central contribution is making **real-time subgame solving both *safe* and *nested*** — strong enough to beat humans and provably unable to make the strategy much more exploitable than the blueprint it refines. Both adjectives matter, and each fixes a specific prior failure.

The difficulty is that an imperfect-information subgame *cannot be solved in isolation*. In chess, once you

reach an endgame you can solve it on its own, because the board is fully known. In poker the right strategy for a subgame depends on the strategies in *other, unreached* subgames, because those determine the probability distribution over the hidden hands the opponent could hold here. Earlier real-time solvers sidestepped this by assuming the opponent had played the blueprint up to this point — **unsafe** subgame solving — which the opponent can punish simply by deviating, sometimes producing a refined strategy far *worse* than the blueprint.

Libratus’s **safe** subgame solving removes that assumption with a small gadget, the *augmented subgame*. At the root of the subgame the opponent is handed a choice for every hand they might hold: either take a fixed **alternative payoff** — the blueprint’s estimate of what that hand is worth here — or *enter* the detailed subgame and play it out. Solving this augmented game forces Libratus’s refined strategy to make the opponent **no better off than that estimate for every possible hand**, which is exactly the meaning of “safe”: the local repair cannot be exploited relative to the blueprint, whatever the opponent does. Libratus sharpens this with a variant the authors call **Estimated-Maxmargin** — it maximizes the *smallest* safety margin across all opponent hands — and with two refinements that make it strong rather than merely safe: it uses blueprint *estimates* of opponent values instead of conservative *upper bounds* (less timid play), and it *de-emphasizes hands the opponent could only hold by having made an earlier mistake* (it is pointless to defend against a hand a rational opponent would never have brought to this spot), freeing capacity to defend against the hands they realistically have.

The pairing comes with a guarantee that parallels DeepStack’s. If σ^* is the least-exploitable strategy that differs from the blueprint only inside the solved subgames, and the blueprint’s estimate of the opponent’s subgame values is off by at most Δ , then the refined strategy’s exploitability obeys

$$\text{exploitability}(\sigma_{\text{refined}}) \leq \text{exploitability}(\sigma^*) + 2\Delta.$$

The 2Δ term is the price of imperfect value *estimates*, and it is the structural twin of DeepStack’s $k_1\epsilon$ term — both bound how much an error in the values handed to the solver can cost in worst-case exploitability. The two systems thus arrive at the same kind of soundness statement from opposite directions: DeepStack’s ϵ is a *neural network’s* approximation error at the depth limit; Libratus’s Δ is a *tabular blueprint’s* estimation error at the subgame boundary.

“Nested” is the second half. Rather than expand the blueprint’s bet menu (which would balloon the offline solve), Libratus crafts a **distinct response to each off-menu bet in real time**: when the opponent bets a size it has never enumerated, it builds an augmented subgame whose alternative payoff is the best *in-menu* action the opponent could have taken instead, solves it, and does so afresh for every later off-menu bet down the hand. In the controlled experiments this beat the previous standard — rounding the bet to the nearest menu size — by more than an order of magnitude in worst-case exploitability (119 versus 1465 mbb/g in the reported small-game test). And because each subgame is solved live, Libratus can also *change its own bet sizes* between hands — it perturbs them by a random 0–8% at the first solve — so the humans never face a fixed target to dissect. This is the precise mechanism by which the abstraction

paradigm’s worst weakness, action translation, is excised from the late game.

6.3.4 Caveats, dead-ends, and what the paper under-describes

As with DeepStack, the clean three-module story hides where the engineering — and the honesty about limitations — actually lives, except that here much of it is not even in the *Science* paper but in the companion IJCAI paper and the authors’ course notes.

The most revealing caveat is that **the blueprint alone is not superhuman — it does not even beat the prior bot**. Against Baby Tartanian8, the 2016 competition winner, Libratus’s raw blueprint *lost* by 8 mbb/g; only when nested subgame solving was switched on did the same system win by 63 mbb/g. The offline strategy, in other words, is a scaffold, and essentially all of Libratus’s edge comes from real-time search — a result that quietly reframes the whole system and foreshadows the field’s pivot toward search-at-inference.

Several other asterisks matter. **Action translation is not eliminated, only contained**: on the first two betting rounds Libratus still rounds off-menu opponent bets, and the entire self-improver module exists to chip away at that residual seam, three holes per night, never closing it. **Safe subgame solving had to be re-engineered to be usable at all** — for three years it was considered impractical because, in head-to-head play, textbook-safe solving lost to the theoretically unjustified *unsafe* variant; Libratus’s use of value *estimates* rather than conservative *bounds*, together with its de-emphasis of hands an opponent would only hold by mistake, is what made safety competitive, and even so Libratus deliberately uses *unsafe* solving once, at its first entry into the third round, because it is cheaper and empirically fine there. The **Estimated-Maxmargin** choice itself trades a little theoretical purity for strength: using estimates rather than upper bounds can in principle push exploitability *above* the blueprint’s, bounded only by the 2Δ of the theorem. And the imperfect-recall abstraction underneath it all means several distinct situations share a strategy and “fight” over it — a known distortion the regret-based pruning only partly mitigates. Finally, the **compute is almost absent from the main paper**: the headline article gives essentially no figures, and the true cost (next) surfaces only in the secondary sources, as does the fact that the **code was never released**, leaving independent verification to rest on the published pseudocode.

6.3.5 Compute & accessibility

Where DeepStack’s bill is paid almost entirely offline and its play is cheap, Libratus is **expensive at both ends**. The project consumed roughly **25 million CPU core-hours** on the Bridges supercomputer at the Pittsburgh Supercomputing Center over a year — of which about 6 million went to building and solving the blueprint, about 3 million to real-time subgame solving during the match, about 3 million to the self-improver, and the remaining ~13 million to exploratory experiments and evaluation. Operationally the blueprint runs occupied roughly 195 nodes for one to eight weeks at a time; each **real-time subgame solve used about 50 nodes and took on the order of tens of seconds**; the overnight self-improver ran on up to several hundred nodes for hours; and the strategies and snapshots consumed about 2.6 petabytes of disk. There were **no GPUs and no neural training anywhere** — every core-hour is CFR or abstraction.

The shape of that bill is the lesson. Libratus does not buy cheap deployment with an expensive one-time training run, the way DeepStack does; its *play-time* cost is itself supercomputer-scale, because the strength comes from solving a fresh subgame at almost every late decision. In accessibility terms this made Libratus, as deployed, essentially impossible to reproduce outside a major supercomputing centre at *both* training and play time — a far higher bar than DeepStack’s single-GPU agent — and the absence of any code release reinforced that. The underlying techniques are application-independent and run far more cheaply on modern hardware and solvers today, but the 2017 artefact was a demonstration of what was possible with abundant compute, not of what was broadly attainable.

6.3.6 Strengths and limitations

Libratus’s signal strength is **decisive, rigorously demonstrated superhuman play**. It is the first AI to beat top HUNL *specialists* in a long match — 147 mbb/g over 120,000 hands at 99.98% significance, beating each of the four professionals individually — and, unlike DeepStack, it also beat the **prior best poker AI head-to-head** by 63 mbb/g, disentangling its strength from the question of human skill. Its core technique, nested safe subgame solving, is both **provably safe** (the 2Δ bound) and **empirically an order of magnitude less exploitable** than the action translation it replaces, and it uses **no human data and no domain-specific knowledge**. The modular design is itself a contribution: a cheap blueprint, a real-time solver that repairs it where it matters, and an overnight loop that patches its own holes — each module covering another’s weakness — and the whole system is **robustness-first**, refusing to exploit opponents so as never to be counter-exploited.

The limitations are equally clear and set up the rest of the chapter. Libratus is **two-player zero-sum only**; its safety guarantees lean on that structure. It remains **abstraction-based**, and still **translates off-menu bets on the early rounds**, an exploitable seam the self-improver only narrows. It uses **no neural generalization** whatsoever: nothing transfers across situations, the blueprint is enormous (petabytes of stored strategy), and play time is **supercomputer-scale** rather than the single GPU DeepStack needed. The Estimated-Maxmargin trade-off accepts a small, bounded rise in exploitability for strength, one unsafe solve slips into the pipeline, and — because both humans and AI adapted over the match — even the headline significance is, strictly, an “as-if-independent” figure, though a 147 mbb/g margin over 120,000 hands leaves no real doubt.

6.3.7 Legacy and modern relevance

Strip away the abstraction machinery and Libratus’s enduring idea is **real-time search layered on a coarse precomputed strategy, made safe under hidden information**: solve the local situation exactly at decision time, consistently with a cheap global plan, and respond to whatever the opponent actually does rather than to a rounded approximation of it. That idea did not just survive — it became, with DeepStack’s continual re-solving, half of the foundation the rest of the chapter is built on. Libratus’s nested safe subgame solving is the direct ancestor of the real-time search in **Pluribus** (2019), it is unified with DeepStack’s re-solving by the **depth-limited solving** theory the authors published the following year, and the same “refine a global value estimate with a local solve” pattern reappears, now with *learned*

values replacing the tabular blueprint, in **ReBeL** (2020).

Libratus is also, in a precise sense, the **high-water mark of a paradigm that the field then left behind**. It reached superhuman play with **no deep learning at all** — pure abstraction and CFR — and yet the subsequent systems are all neural, because hand-crafted abstraction and petabyte blueprints neither generalize across situations nor scale beyond two players. The honest verdict is that Libratus is *superseded as an architecture but vindicated as a thesis*: its bet that **real-time search matters more than a bigger precomputed strategy** was exactly right, even as its bet on tabular abstraction was overtaken. That first thesis has since become a central theme of frontier AI — the observation that adding search at decision time was worth far more than scaling the offline computation is an early, concrete instance of the “test-time compute” argument now made for reasoning models, and Libratus’s blueprint-then-search split prefigures the modern pretrain-then-search recipe.

Several subsystems remain directly reusable: **safe subgame / endgame solving** as a way to locally refine a global policy *without breaking its guarantees*; the **augmented-subgame gadget** — the clean “take a known alternative value or enter the subgame” construction for pinning a local solution to a global value estimate; **regret-based pruning** for scaling CFR; and the **self-improver pattern** of using observed play to find and *fix one’s own holes* rather than to exploit the opponent. What is genuinely obsolete is the surrounding edifice — the hand-tuned card and action abstraction, the multi-petabyte tabular blueprint, and the cluster-at-play-time deployment — all of which the learned representations of later systems dissolve. For this thesis specifically, Libratus contributes two seeds: its safe-subgame-solving exploitability bound is a template for **safe exploitation under value error (Contribution 2)**, and its deliberate refusal to exploit — “fix your own weaknesses, do not model the opponent” — is the precise foil against which a *bounded, deliberate* opponent adaptation (Contribution 1) can be defined.

6.4 Pluribus (2019)

Libratus settled two-player no-limit hold’em, but its safety guarantees — and indeed the very meaning of “solving” the game — rested on two-player zero-sum structure: there a Nash equilibrium is unbeatable, and a real-time solve could be made provably *safe* against it. Poker as humans actually play it, though, seats six. Pluribus (Brown & Sandholm, 2019; Carnegie Mellon and Facebook AI) confronted the multiplayer question directly: **what becomes of the blueprint-plus-real-time-search recipe when you remove the two-player crutch and sit at a six-handed table — a setting where a Nash equilibrium is neither unique, nor efficiently computable, nor even a guarantee that you will not lose?** Its answer was empirical and emphatic. Across two formats — five professionals seated with one copy of Pluribus, and one professional against five copies — it beat a rotating cast of thirteen elite pros, several of them World Series or World Poker Tour champions, winning by about **48 milli-big-blinds per game** against five humans at once (mbb/g, the win-rate unit from the previous sections — roughly five big blinds per hundred hands, a decisive six-handed margin) at 95% statistical significance. And it did so after training for **eight days on a single 64-core server for about \$150 of cloud compute** — on the order of a thousandth of what the supercomputer behind Libratus consumed.

At a glance	Pluribus (2019)
Players	6 (six-max) — the first superhuman AI in any benchmark game with more than two players/teams
Game type	6-max NLHE — six-player no-limit Texas hold'em (imperfect-information; multiplayer, <i>not</i> 2-player zero-sum)
Blueprint (offline)?	Yes — a full-game blueprint solved offline by Linear MCCFR; played <i>directly</i> only on the first betting round, a scaffold thereafter
Neural component	None — purely tabular CFR / abstraction (no neural networks anywhere, as in Libratus)
Search mechanism	Real-time <i>depth-limited</i> search with $k=4$ continuation strategies at the leaves (nested <i>unsafe</i> solving, re-solved from the start of each betting round); Linear CFR inside subgames
Abstraction?	Yes — action (1–14 blueprint bet sizes; 1–6 in search) + information abstraction (lossless first round; lossy buckets later, finer in search than in the blueprint)
Perfect-info too?	No (imperfect-information only)
Compute	Famously cheap: blueprint ~12,400 core-hours / 8 days / < 512 GB on one 64-core server (~\$144); play on 2 CPUs (28 cores), < 128 GB, no GPUs, 1–33 s/decision
Key innovation	Superhuman six-player play via blueprint + depth-limited search with continuation strategies — won <i>empirically</i> , on a tiny budget, with no N-player safety guarantee

6.4.1 The gap it closed

Every superhuman game AI before Pluribus — checkers, chess, Go, and both prior poker programs — shared a hidden assumption: two players, zero sum. In that setting a Nash equilibrium carries a property that makes it the obvious target: any player who adopts one is *guaranteed not to lose* in expectation, whatever the opponent does, and if two players independently compute equilibria, their strategies still combine into an equilibrium. “Solving” such a game therefore *means* approximating a Nash equilibrium, and DeepStack and Libratus were, at bottom, two ways to do exactly that. Six-handed poker dissolves all of it. With more than two players, finding — even approximating — a Nash equilibrium is computationally intractable in general; there are typically *many* equilibria; and, fatally, if each player independently selects

one, the resulting joint strategy need not be an equilibrium at all, so a player can *lose* while playing an impeccable equilibrium strategy. Brown and Sandholm illustrate this with the Lemonade Stand Game, in which players space themselves around a ring: there are infinitely many equilibria, and independently chosen ones rarely mesh. The conclusion is stark — in multiplayer poker a Nash equilibrium is *neither unique nor a safety guarantee*, and the field’s two-decade definition of success simply evaporates.

Pluribus closes this gap by changing the goal and then meeting it. Rather than chase a solution concept it cannot trust, it aims only to *empirically and consistently defeat elite humans*, and it accepts up front that its algorithms carry no guarantee of converging to an equilibrium outside two-player zero-sum play. That concession is the entire point — and it is the precise gap this thesis’s Contribution 2 sets out to fill. The other half of the gap is mechanical. Libratus reached superhuman play by solving every late-game subgame *to the end* in real time; with six players the subgames explode exponentially and solving to the end becomes infeasible, while DeepStack’s alternative — leaf values conditioned on belief distributions — was already costly in two-player poker and worsens as players multiply. Pluribus therefore needed a real-time search that looks only a little way ahead and stops — in a game where, as the next sections show, stopping early is exactly what naïve search cannot safely do.

6.4.2 Architecture

Like Libratus, Pluribus splits into an offline phase that builds a blueprint and an online phase that searches — but the balance of power between them is inverted, and, again like Libratus, there is **no neural network anywhere** (Figure 6.3).

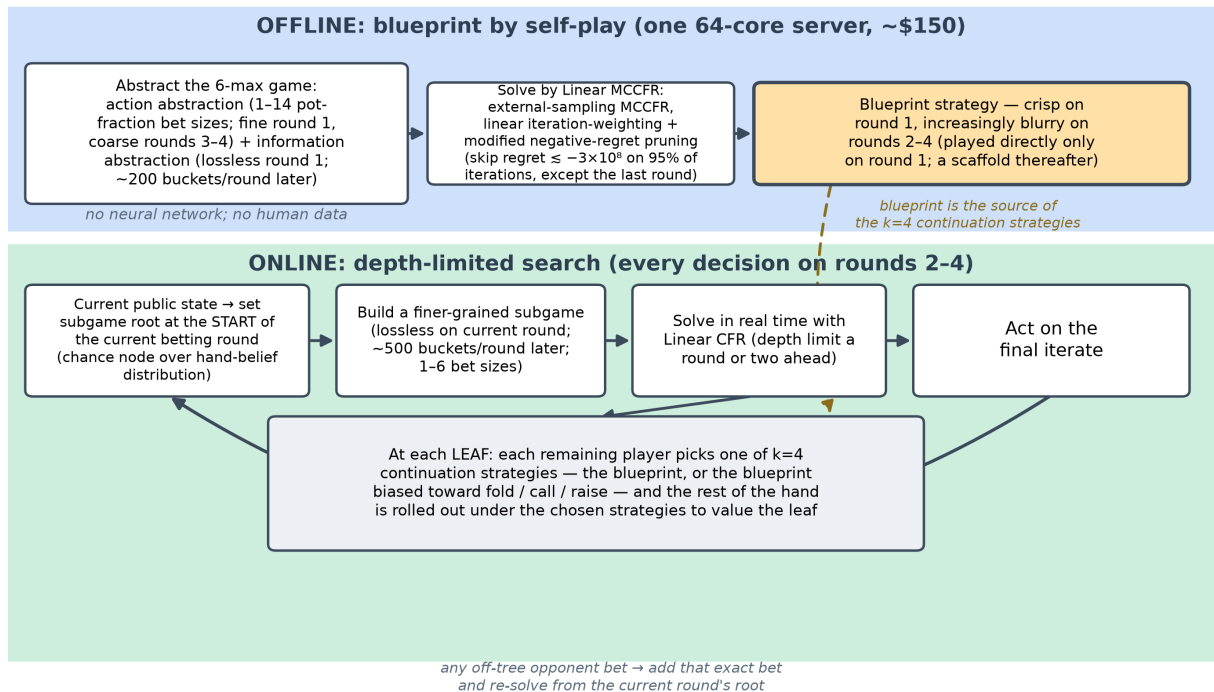


Figure 3: Pluribus’s two-phase architecture: an offline Linear-MCCFR blueprint (top) reused as the source of $k=4$ continuation strategies inside the online depth-limited search (bottom).

Offline, Pluribus computes a blueprint for the whole game by self-play, starting from random play and improving through an **external-sampling Monte Carlo CFR (MCCFR)** that, each iteration, picks one player as the “traverser,” simulates a hand, and nudges that player toward the actions that did better. It abstracts the game two ways — a handful of bet sizes (*action abstraction*) and buckets of strategically similar card situations (*information abstraction*) — but only coarsely, and it plays this blueprint *directly* in just the first of the four betting rounds, where the action abstraction is kept fine and no card bucketing is used. Everywhere else — flop, turn, river — Pluribus discards the coarse blueprint locally and **searches in real time** for a finer strategy, exactly as Libratus did, with one decisive difference: it does not solve to the end of the game. It looks only a round or two ahead to a depth limit and stops, which is what makes six players affordable.

The familiar building blocks are all in view — abstraction and MCCFR build the blueprint, Linear CFR does the online solving, and “search” is the per-decision re-solve — and the two architectural signatures are both *absences*. There is **no neural network** (the system is purely tabular, like Libratus), and, unlike Libratus, there is **no self-improver**: Pluribus never patches its blueprint between sessions, because depth-limited search on three of the four rounds already does the repairing. Two ingredients make stopping early sound, and they are the subject of the deep dive below: a careful answer to what a “leaf value” can even mean under hidden information, and a search that begins not at the current decision but at the *start* of the current betting round.

6.4.3 Key innovation: depth-limited search with continuation strategies

Heuristic search in chess or Go works by looking a fixed distance ahead and reading a value off the horizon, on the assumption that both sides play well from there. That assumption fails completely under hidden information, and Brown and Sandholm make the failure vivid with a one-shot sequential Rock–Paper–Scissors: if the searcher assumes the opponent will play the equilibrium (each throw one-third of the time) beyond the horizon, then every action looks equally valued at zero, so the searcher might settle on “always play Rock” — whereupon the opponent switches to always Paper and the true value plummets. A leaf in an imperfect-information game therefore has *no single value*: its worth depends on the strategy the searcher will adopt, which is precisely what the search is trying to determine.

Pluribus’s central contribution is the way it gives leaves a value anyway. When search reaches the depth limit, instead of freezing one continuation, **each player still in the hand chooses among four different continuation strategies** for the remainder of the game, and may mix over them. The four are deliberately simple: the blueprint itself, and three biased copies that multiply the probability of *folding*, of *calling*, and of *raising* (each then renormalized). Because an opponent can always switch to whichever continuation punishes the searcher most, an unbalanced strategy — the poker equivalent of always playing Rock — is no longer rewarded, and the searcher is driven toward balance. This idea was first proven in a two-player precursor, Modicum (Brown, Sandholm & Amos, 2018), which beat two former champion bots while running on a 4-core laptop with 16 GB of memory — a striking sign that depth-limited search with continuation strategies could stand in for a supercomputer.

Pluribus generalizes the idea from two players to six and adds a subtle but important twist. In Modicum only the *opponent* chose among continuation strategies while the searcher always played the blueprint — sound in two-player zero-sum, but it hands the opponent the initiative and leaves the searcher timid and low-value. Pluribus lets the **searcher choose among the continuation strategies too**, which balances the players and, in the authors’ words, is “more effective, easier, and more elegant.” It also re-solves not from the current decision point but from the *start of the current betting round*, holding fixed only the actions it has already taken. This “unsafe” search — so named because, unlike Libratus’s, it carries no exploitability guarantee — is cheaper (most six-handed hands are folded immediately, so few need a strategy at all) and, because it begins just after a high-branching chance event, turns out to be hard to exploit in practice.

What is conspicuously missing from all of this is a *guarantee*. DeepStack bounded its exploitability by $k_1\epsilon + k_2/\sqrt{T}$ and Libratus by 2Δ ; Pluribus offers no such bound, and the omission is principled rather than careless. CFR’s engine still does, in any finite game, drive each player’s *average regret* to zero,

$$\frac{R_i^T}{T} \rightarrow 0 \quad (\text{no-regret}),$$

but the inference that carried the two predecessors from no-regret to safety — *no-regret play converges to a Nash equilibrium, and a Nash equilibrium cannot be beaten* — holds only when there are two players and the game is zero-sum. With six players the first implication fails (self-play need not approach an equilibrium) and the second is meaningless (an equilibrium is not unbeatable). The very iterations that *proved* DeepStack and Libratus safe therefore buy Pluribus only empirical strength: superhuman in practice, with nothing certified. To make those iterations cheap enough to run six-handed, Pluribus leans on **Linear CFR** — weighting iteration t ’s contribution by t so the bad early iterations wash out about three times faster — and a **modified negative-regret pruning** that skips hopeless actions on most iterations (everywhere but the final round); together with lazy memory allocation, these are what shrink the bill to a single server.

6.4.4 Caveats, dead-ends, and what the paper under-describes

As with both predecessors, the published article is the clean story and the engineering honesty lives in the supplement — only more so here, because Pluribus is a six-page *Science* paper whose architecture is almost entirely relegated to its supplementary materials and to two companion papers. The most consequential admission is that Pluribus uses **unsafe** subgame solving — it assumes opponents have played the strategy it computes *for* them — which, the authors state plainly, “lacks theoretical guarantees on performance even in two-player zero-sum games and there are cases where it leads to highly exploitable strategies.” Safe alternatives exist, but in head-to-head play they did worse, so Pluribus takes the empirical win and mitigates the risk only by always re-solving from the start of the betting round. A second seam is inherited and never fully closed: on the *first* betting round, opponent bets too far off the blueprint’s menu are still **rounded** by action translation — the very weakness Libratus’s self-improver existed to chip at — and Pluribus, having no self-improver, simply lives with it.

Several other asterisks matter. Pluribus’s headline innovations are **never individually ablated**: the authors concede that the variance of no-limit poker and the cost of human trials make it “too expensive” to measure each one’s contribution, so the reported component speedups (roughly $3\times$ from Linear CFR, $2\times$ from the modified pruning, more than $2\times$ in memory from lazy allocation) are estimates, and the overall winning margin is not decomposed. The supplement does, however, dispatch one tempting misconception: assuming a *single* blueprint continuation at the leaves — the obvious thing to try, and the way the underlying study notes sometimes gloss the method — was shown in the two-player precursor to *lose* to both champion bots (by 10 and 1 mbb/g), whereas the four-continuation version *won* (by 6 and 11); the continuation-strategy set is doing real work, not decoration. Smaller curiosities round out the picture: Pluribus plays its **final** search iterate rather than the usual time-average, to avoid residual bad actions; it learned to **abandon “limping”** during self-play yet **“donk-bets” far more than humans do**; and — like Libratus — its **code was never released**, leaving only pseudocode for independent verification, because poker is played commercially.

6.4.5 Compute & accessibility

Pluribus’s compute story is the one most people remember, and it genuinely inverts its predecessors’. The blueprint was trained in **eight days on a single 64-core server** for about **12,400 core-hours** and under 512 GB of memory — roughly **\$144** at cloud spot prices — and at the table Pluribus runs on **two CPUs (28 cores) and under 128 GB, with no GPUs at any point**, taking one to thirty-three seconds per decision and playing about twice as fast as a human. Set against the field the contrast is almost comic: AlphaGo used 1,920 CPUs and 280 GPUs, Deep Blue 480 custom chips, and Libratus around fifteen million core-hours to build its blueprint and a roughly hundred-CPU cluster to play. Pluribus reached a *harder* milestone — more players, a larger game — for on the order of a thousandth of Libratus’s training compute. Where DeepStack concentrated its cost offline and Libratus paid heavily at both ends, Pluribus is **cheap at both ends**, and that, as much as the six-player result, is the paper’s thesis.

This reframes accessibility entirely. For the first time a superhuman poker system was reproducible, in principle, by a single well-equipped researcher rather than a supercomputing centre — the authors explicitly present it as a rebuttal to the worry that frontier game-AI would belong only to teams with millions of dollars of hardware. The collapse is not magic but algorithmic: the compounding of depth-limited search (which the authors estimate saves at least five orders of magnitude over solving to the end), Linear CFR, and aggressive pruning and memory thrift. The one caveat is that the artefact itself stays closed — no code — so “accessible” describes the *method*, demonstrated at laptop scale by Modicum, more than a downloadable program.

6.4.6 Strengths and limitations

Pluribus’s signal strength is simply that it is **first**: the first AI to reach superhuman performance in any widely recognized benchmark game with more than two players or two teams, and in poker’s most popular form. The win was decisive and rigorous — **+48 mbb/g ($p = 0.028$)** against five elite pros at the table, and **+32 mbb/g ($p = 0.014$)** with five copies against a lone pro — measured with the

AIVAT variance reducer (which cut variance about ninefold) over tens of thousands of hands, against thirteen professionals who had each won over a million dollars and who had days to hunt for weaknesses; the win rate barely wavered. A later rematch even beat **Linus Loeliger**, widely regarded as the best six-max cash player alive. And, like Libratus, Pluribus is **robustness-first**: it plays a fixed strategy, never models or adapts to opponents, and does not even know their identities, so it cannot be lured into a counter-exploitable adjustment — a discipline that, with **no human data and no domain knowledge**, keeps the result clean.

The limitations are precisely what it gave up to get there. Pluribus has **no safety guarantee whatsoever** in the six-player setting — no Nash convergence, no exploitability bound — so its superhuman status is an *empirical* fact about thirteen strong humans over tens of thousands of hands, not a theorem; a sufficiently coordinated table, or simply a different game, carries no assurance. It leans on **unsafe** search, still **rounds off-tree bets on the first round**, and remains **purely tabular and abstraction-based**, with nothing learned that generalizes across situations — the blueprint is a giant lookup table, not a model. The authors add that the whole approach may not survive where players can **communicate and collude**, which poker largely forbids. And its 48-mbb/g six-handed win rate, though decisive, is not the same currency as Libratus’s 147 heads-up — different game, five opponents, higher variance. Most of these gaps are picked up by later systems; the one this thesis singles out — that **multiplayer success came with no safety guarantee at all** — is the explicit target of Contribution 2.

6.4.7 Legacy and modern relevance

Two ideas outlast the poker specifics. The first is technical: **depth-limited search made sound under hidden information by giving each leaf a small menu of selectable continuation strategies rather than a single value** — a clean way to look only a little way ahead in a game where, naïvely, you cannot. The second is economic, and the field absorbed it most deeply: Pluribus is the canonical demonstration that **a harder problem can be solved with a thousandfold less compute through better algorithms and search at decision time**, rather than more hardware. Noam Brown has since pointed to exactly this — that adding real-time search was worth as much as an enormous scaling of the precomputed strategy — as an early, concrete instance of the “**test-time compute**” argument now central to reasoning models such as o1. In the architectural arc of this chapter, Pluribus is the point where the *player-count* barrier falls; the *generalization* barrier it leaves standing — everything is still tabular and hand-abstracted — is what **ReBeL** and **Student of Games** dismantle next, by replacing the blueprint with learned belief-state values.

Several of Pluribus’s parts remain directly reusable: the **continuation-strategy (multi-valued leaf)** construction for robust depth-limited search in any hidden-information or multi-agent setting; **Linear (discounted) CFR**, now a standard convergence accelerator and the launch point for later discounted-CFR variants; **regret-based pruning** for scaling CFR; **AIVAT** as a low-variance evaluator; and the *add-the-action-and-re-solve* handling of off-tree bets inherited from Libratus. What is superseded is the surrounding tabular edifice — hand-tuned abstraction and a lookup-table blueprint — which the learned representations of ReBeL and SoG dissolve, along with the *unsafe* search those belief-state methods make

unnecessary. The honest verdict mirrors Libratus’s: **superseded as a deployed architecture, but vindicated and sharpened as a thesis**. For this dissertation specifically, Pluribus is the keystone of Contribution 2 — the empirical proof that Nash-and-search methods *work* in N-player imperfect-information games while offering **no safety guarantee at all**, which is exactly the gap a theory of multi-agent safe exploitation must close.

6.5 ReBeL (2020)

Pluribus broke the player-count barrier, but it did so while remaining exactly what Libratus was — a giant, hand-abstracted lookup table with nothing learned that transfers from one situation to the next — and it bought its six-handed win by giving up safety altogether, leaning on *unsafe* search with no exploitability guarantee of any kind. ReBeL (Brown, Bakhtin, Lerer & Gong, 2020; Facebook AI Research) steps back from six players to two and asks the opposite question: **what if, instead of hand-crafting an abstraction and precomputing a blueprint, we ran AlphaZero — self-play reinforcement learning plus search, at both training and test time — in a game of hidden information, recovering the very guarantees Pluribus discarded while throwing the abstraction out entirely?** Its answer, *Recursive Belief-based Learning*, is the first algorithm to make reinforcement learning *and* search provably sound in imperfect-information games. The trick is to recast the game as a “perfect-information” game over **public belief states** — probability distributions over what each player might be holding, given common knowledge — on which value and policy functions are well-defined; an AlphaZero-style loop then trains a neural value (and policy) network on those states, with counterfactual regret minimization (CFR) solving depth-limited subgames at the leaves. ReBeL provably converges to a Nash equilibrium in any two-player zero-sum game, beat the top human heads-up specialist Dong Kim by 165 mbb/g (milli-big-blinds per game, the win-rate unit from the previous sections) over 7,500 hands while using *far less* domain knowledge than any prior poker AI — and, unlike the closed Libratus and Pluribus, its implementation (for Liar’s Dice) was **open-sourced**.

At a glance	ReBeL (2020)
Players	2 (heads-up) — <i>guarantees</i> are two-player zero-sum; the algorithm generalizes to more players but without the guarantees
Game type	General 2p0s imperfect-information; evaluated on HUNL poker + Liar’s Dice (and turn endgame hold’em). Reduces to an AlphaZero-like algorithm in perfect-information games
Blueprint (offline)?	No stored blueprint — the offline product is a <i>learned PBS value (+ policy) network</i> from self-play, not a strategy table

At a glance	ReBeL (2020)
Neural component	PBS value network + (optional) PBS policy network; MLP (GeLU/LayerNorm), 6×1536 hidden for poker, input = belief over each player's 1,326 hands + board + pot + bet flag
Search mechanism	CFR (CFR-D / CFR-AVG; also FP/FLOP) solving a depth-limited subgame rooted at a PBS, with the learned value net supplying (iteration-dependent) leaf values — at both training and test time
Abstraction?	None — no card/information abstraction (lossy or lossless); the value net replaces it. Keeps only a small (9) hand-chosen bet-size menu, with off-tree bets added live
Perfect-info too?	No (presented/evaluated as imperfect-information), with the nuance that it <i>degenerates</i> to AlphaZero-style search if private information is removed — full unification is Student of Games' claim
Compute	GPU-trained: full HUNL used ~ 90 DGX-1 nodes $\times$ 8 V100 GPUs for self-play data generation (a contrast with Pluribus's CPU-only $\sim \$150$); CFR on a single CPU thread; play < 2 s/hand, ~ 5 s/decision
Key innovation	Public belief states + an AlphaZero-style self-play loop training a PBS value/policy net with CFR run in belief space: the first <i>sound</i> RL+Search for imperfect-information games, recovering a provable 2p0s Nash guarantee with no abstraction or blueprint

6.5.1 The gap it closed

The previous three systems converged on one recipe — a precomputed strategy plus real-time search — and Pluribus pushed it to six players, but every one of them was, underneath, a *tabular* object: a strategy or blueprint stored over a hand-engineered abstraction, with nothing learned that generalizes to a situation never explicitly solved. DeepStack alone had a neural component, yet even it leaned on hand-crafted features, a 1,000-bucket clustering at the network's input, and an abstraction on the river. Meanwhile the most successful paradigm in all of game AI — AlphaZero's marriage of self-play reinforcement learning with search, which learns its own evaluation from scratch and reuses it both to train and to play — had

been *unavailable* for imperfect information. The open question ReBeL answers is the one Noam Brown calls the field’s “holy grail”: **can the AlphaZero recipe be made to work, soundly, in games of hidden information?**

The reason it could not, before, is subtle and is the conceptual crux of the whole chapter. AlphaZero assumes each state has a single well-defined value: a chess position is worth what it is worth, regardless of how often you reach it. Hidden information destroys that assumption. The paper makes it vivid with a modified Rock–Paper–Scissors in which scissors wins (or loses) double: the equilibrium is to throw rock and paper 40% of the time and scissors 20%, and at that equilibrium every action has expected value zero. If Player 1 runs the one-ply lookahead search that works in chess — substituting the equilibrium value at the leaves — every move looks equally good, so the searcher might settle on “always rock”, whereupon the opponent switches to “always paper” and rock’s true value collapses from 0 to -1 . **In an imperfect-information game the value of an action depends on the probability with which it is played**, so a state defined by the sequence of actions alone has no unique value — and AlphaZero-style search is simply unsound. Pluribus had patched the symptom with selectable continuation strategies at the leaves; ReBeL cures the disease by changing what a “state” *is*. It redefines the state to include the probability distribution over the hidden information — a *public belief state* — on which, as the next sections show, values become well-defined again and the entire AlphaZero machinery can be ported across. In doing so it also dissolves the two crutches DeepStack and the abstraction systems still leaned on: the hand-crafted abstraction and the precomputed blueprint both disappear, replaced by a value function learned purely from self-play.

6.5.2 Architecture

ReBeL is best read as **AlphaZero for imperfect information**: a self-play loop that trains neural value and policy networks, where the “search” used during both training and play is CFR solving a depth-limited subgame — but everything operates on *public belief states* rather than raw game states, and the leaf evaluator is a learned value network rather than a rollout (Figure 6.4).

The loop has three moving parts. **Self-play reinforcement learning** drives the whole thing: starting from a root PBS, ReBeL constructs a depth-limited subgame, solves it, records the solved values and policy as training targets, samples a leaf to become the next root, and continues to the end of the game — then retraining the networks and iterates, exactly as AlphaZero alternates self-play with network training. **Search** is the inner solve: ReBeL runs CFR (specifically CFR-D, “CFR with decomposition”, or its variant CFR-AVG) over the depth-limited subgame, and on every CFR iteration it sets each leaf’s value by querying the **learned PBS value network** — so the leaf values shift from iteration to iteration, which is precisely what keeps the search sound under hidden information. The **neural networks** are the learned intuition: a value network that maps a PBS to the values of the player’s possible hands, and an optional policy network that warm-starts CFR to cut the number of iterations needed.

The architectural signatures are all *absences* relative to the earlier systems. There is **no precomputed blueprint** — the only artefacts carried out of training are the two networks. There is **no abstraction**

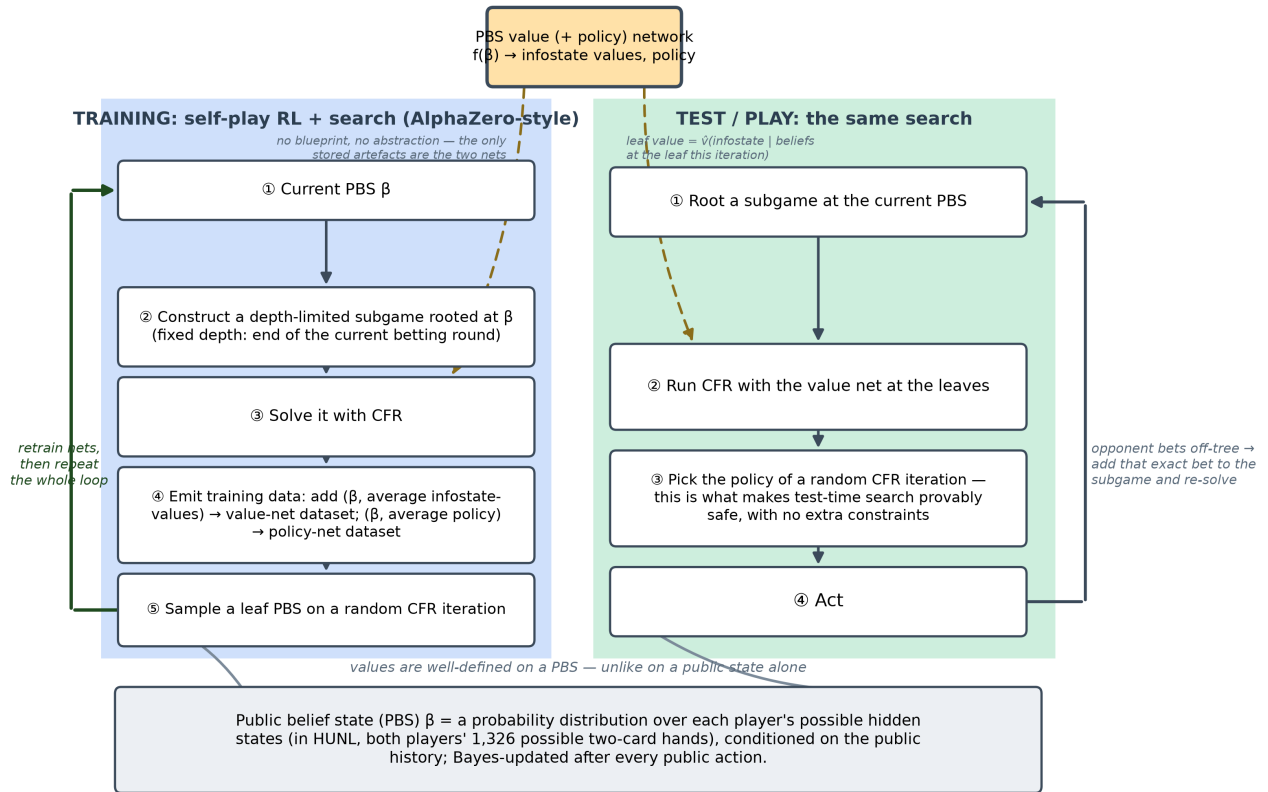


Figure 4: ReBeL’s AlphaZero-style loop: self-play training (left) and test-time play (right) share one PBS value/policy network, both solving depth-limited subgames rooted at the public belief state (shared definition box, bottom).

of any kind: where DeepStack, Libratus, and Pluribus all bucketed strategically similar hands, ReBeL computes a distinct policy for every infostate, feeding the network only the raw belief distribution over both players’ 1,326 possible hands plus the board, pot, and a single “has anyone bet this round” flag. And the CFR search runs on a **single CPU thread** with no abstraction at all; the heavy compute is the GPU self-play that trains the value net (a balance discussed under *Compute*). The classic building blocks of the earlier steps are still visible — CFR (Step 3) is the search engine and neural value approximation (Step 5) is the leaf evaluator — but they are now fused into a single self-play-plus-search loop, with the public belief state (the section below) as the representation that makes the fusion possible.

6.5.3 Key innovation: public belief states and learned values in belief space

ReBeL’s contribution is one conceptual move with three technical consequences. The move is to stop searching over *states* and start searching over *beliefs about states*.

The **public belief state (PBS)** is the heart of it. The Meta AI blog gives the cleanest intuition. Take a card game and modify it so the players cannot see their own cards — only an impartial referee can; on each turn a player announces, for every card they *might* hold, the probability with which they would take each action, and the referee samples the move for the player’s true card. Because all players’ strategies are assumed common knowledge, everyone can track, via Bayes’ rule, the probability that each player holds each possible hand. This modified game is **strategically identical** to the original, yet it contains *no private information*: its state — the vector of those probabilities — is fully observed by everyone. ReBeL calls that state a public belief state: formally, a joint probability distribution over the players’ possible infostates, given the common public observations. Crucially, viewing imperfect-information games as continuous-state perfect-information games this way is an old idea (it goes back to work on cooperative POMDPs); ReBeL’s achievement is being the first to combine it with self-play reinforcement learning in an *adversarial* setting.

The first consequence is that **values become well-defined again**. An imperfect-information subgame rooted at a mere *public state* has no unique value, because its worth depends on the hidden hands that reach it — which is exactly why heuristic search failed. But a subgame rooted at a *public belief state* does have one: in two-player zero-sum games every PBS β carries a unique value $V_i(\beta)$ with $V_1(\beta) = -V_2(\beta)$, defined by both players playing a Nash equilibrium in the subgame from β . This is precisely the property AlphaZero relies on in chess, now recovered for poker — and it *formalizes and generalizes DeepStack’s belief-conditional counterfactual values*, which were the first instance of a PBS value function but were trained in a much more ad-hoc way.

The second consequence is that the AlphaZero loop can run, with **CFR as the search algorithm**. One could in principle run AlphaZero’s Monte-Carlo tree search directly on the belief game, but the belief representation is a very high-dimensional continuous space (in the blog’s small card example the action space alone has 156 dimensions), and tree search is hopeless there. The saving grace is that in two-player zero-sum games these belief representations are *convex* optimization problems, and CFR is effectively a gradient-based solver for them. So ReBeL searches with CFR rather than MCTS — the one substantive

difference from AlphaZero. A technical subtlety the paper proves (Theorem 1) is that the quantities CFR actually needs, the per-infostate values, are a *supergradient* of the concave PBS value function, so ReBeL learns an infostate-value network rather than a single scalar per PBS.

The third consequence — and the headline — is that the whole thing is **provably sound**, recovering the guarantee Pluribus surrendered. The deepest result concerns *test time*. During self-play ReBeL can assume both players’ policies are common knowledge, so it always knows the true PBS; but against a real opponent it does not know their policy and therefore does not know which PBS it is in, which naively breaks search. ReBeL’s elegant fix is to run *the same algorithm* at test time and **act according to the policy of a randomly chosen CFR iteration**. The paper proves (Theorem 3) that this yields safe search — a Nash equilibrium *in expectation* — with no extra constraints, where every prior “safe search” method bolted on constraints so costly they were never fully used in a competitive bot. Concretely, with a PBS value network whose error is at most δ and T CFR iterations per subgame, ReBeL plays an ε -Nash equilibrium — one no opponent can exploit for more than ε — with

$$\varepsilon = \delta C_1 + \frac{\delta C_2}{\sqrt{T}},$$

for game-specific constants C_1, C_2 . The structure deliberately echoes DeepStack’s $k_1\epsilon + k_2/\sqrt{T}$ — a value-error term plus an ordinary CFR-convergence term — but where DeepStack’s bound governed the exploitability of a single re-solve, ReBeL’s governs *convergence to a Nash equilibrium of the whole game*, and both terms now scale with the learned value’s error δ : as the value network improves, $\delta \rightarrow 0$ and ReBeL’s play approaches an exact Nash equilibrium. This is the precise sense in which ReBeL *returns to two-player zero-sum and recovers the theoretical guarantees Pluribus gave up* — the same $1/\sqrt{T}$ CFR backbone that proved DeepStack and Libratus safe, now wrapped around a self-play-learned belief-state value instead of a hand-built abstraction.

6.5.4 Caveats, dead-ends, and what the paper under-describes

ReBeL inverts the usual pattern of this chapter: the *theory* is in the main text and the *engineering honesty* — especially the precise sense in which it uses “far less domain knowledge” — is relegated to the appendices.

The most important detail is what ReBeL deliberately **throws away**, catalogued in Appendix D. Every prior top poker AI, DeepStack included, used **information abstraction** to bucket strategically similar hands; ReBeL uses *none*, lossy or lossless, computing a unique policy per infostate from the raw belief distribution. DeepStack trained its value net on *randomly generated* PBSs drawn from a hand-tuned sampler; ReBeL generates its training PBSs purely from self-play, arguing that random sampling “would be like learning a value function for Go by randomly placing stones on the board” — and indeed it shows (Figure 2) that a value net trained on random beliefs “fails to learn anything valuable.” Prior agents precomputed exact all-in equity tables and solved *to the end of the game* on the third betting round (using river abstraction to make that tractable); ReBeL does neither, always solving only to the end

of the *current* round and learning even the all-in values itself. The upshot is honest but double-edged: ReBeL must learn *six* “layers” of values where DeepStack needed three, which *increases* the surface for error propagation — a real cost accepted in exchange for shedding the abstraction. And it is not domain-knowledge-free: it keeps a hand-chosen menu of at most eight or nine bet sizes (perturbed during training), the one concession to tractability, with off-tree bets handled by adding them to the subgame and re-solving, à la Libratus and Pluribus.

Several other asterisks matter. The clean theorems rest on **idealizations**: Theorem 2’s convergence assumes a perfect function approximator, and the guarantees assume players’ policies are common knowledge (the §6 result removes the *test-time* version of that assumption, but the training analysis still idealizes the network). The variant the agent actually runs for efficiency, a modified **CFR-AVG**, is by the authors’ own admission *not known to be theoretically sound* — “whether or not this modified form of CFR-AVG is theoretically sound remains an open question” — even though it performs well in poker, a familiar gap between the proven algorithm and the shipped one. The safe-search guarantee also depends on picking a *random* CFR iteration, which could land on a terrible early one; this is mitigated only by using Linear CFR, which down-weights early iterations. And as with its predecessors, essentially all of the architecture, hyperparameters, and the all-important compute figures live in the appendices rather than the six pages of main text — the paper is, by design, a theory paper.

6.5.5 Compute & accessibility

ReBeL’s cost profile is the mirror image of Pluribus’s, and the contrast is the sharpest in the chapter. Where Pluribus’s intuition was a blueprint solved by **CPU-only CFR on a single 64-core server for about \$150**, ReBeL’s intuition is a neural network, and training it is **GPU-cluster-scale**. The bottleneck is generating self-play data — sequential CFR solves whose every iteration evaluates all leaf nodes through the network — so the paper parallelizes data generation across “up to 128 machines with 8 GPUs each”, and the full HUNL agent in particular used **about 90 DGX-1 nodes, each with eight 32 GB Nvidia V100 GPUs** (on the order of 700 GPUs), feeding a single training machine over 1,750 epochs of millions of examples apiece. This is squarely a deep-learning training bill, not a CFR bill: the CFR search itself runs on a single CPU thread. At the table, though, ReBeL is cheap and DeepStack-like — under two seconds per hand on average and never more than five seconds per decision, with preflop subgames cached to go faster still.

The accessibility story splits in two, and it is the cleanest contrast with the earlier poker systems. On the one hand ReBeL’s *method* is the most general and least hand-tuned of the four, and — decisively — its **implementation was open-sourced** (for Liar’s Dice), where the code for both Libratus and Pluribus was kept closed. That open release is a genuine step toward reproducibility that the CMU systems never offered. On the other hand, the headline *poker* result was *not* released — the authors withheld it on the explicit grounds that ReBeL “can compute a policy for arbitrary stack sizes and arbitrary bet sizes in seconds”, making it a ready-made cheating tool — and reproducing that result at full strength required a large V100 fleet most groups did not have. So ReBeL is “open” in a way its predecessors were not, but the open artefact is the research game, not the superhuman poker bot.

6.5.6 Strengths and limitations

ReBeL’s signal strength is **generality with a guarantee**. It is the first algorithm to make the AlphaZero RL+Search paradigm *sound* in imperfect-information games, it provably converges to a Nash equilibrium in any two-player zero-sum game, and it backs the theory with results: it beat the prior champions Slumbot (+45 mbb/g) and BabyTartanian8 (+9 mbb/g), drove the local-best-response probe to a large loss, and beat the top human specialist Dong Kim by 165 mbb/g over 7,500 hands — all while using *far less domain knowledge than any prior poker AI*, with no card abstraction and the most general representation of the four. The same algorithm, unchanged, converges to approximate Nash in Liar’s Dice, demonstrating that this is a *framework* and not a poker program; in perfect-information games it gracefully reduces to an AlphaZero-like method. And it recovers, for the neural era, the safety that Pluribus had to abandon — search that is provably safe at test time with no bolt-on constraints.

The limitations are precise and they set up the final system. ReBeL’s guarantees are **two-player zero-sum only**: the unique PBS value, the convexity that licenses CFR-as-search, and the soundness proofs all lean on that structure, and although the algorithm has been run with more players, the theory does not follow it there — the scorecard’s “2 players” is a statement about the *guarantee*, not the code. Its most concrete scaling wall is that the **PBS grows with the amount of hidden information**: the network’s input scales with the number of infostates in a public state, so games with great strategic depth but little common knowledge — the paper names Recon Blind Chess — blow the representation up, and adding players only makes the belief space larger. It assumes the **exact rules of the game are known** (a MuZero-style extension to unknown dynamics is flagged as future work), it inherits a residual **value-approximation error** δ that bounds how close to Nash it actually plays, and its strongest empirical result rests on a **single human opponent** over 7,500 hands rather than a multi-pro field.

6.5.7 Legacy and modern relevance

Strip away the poker and ReBeL’s enduring idea is a representational one: **convert hidden information into a belief state and the full power of self-play reinforcement learning plus search transfers across**. Public belief states are now the standard substrate for sound search in imperfect-information games, and ReBeL is the cleanest statement of “AlphaZero for imperfect information” — search woven into *both* training and play. Its direct descendant is **Student of Games** (2023), which carries the belief-state-plus-sound-search idea further into a single algorithm spanning perfect *and* imperfect information; the same RL-plus-search-with-learned-models lineage also runs into **CICERO** (2022), Meta’s human-level Diplomacy agent, which plans with learned models in a seven-player, mixed-motive, natural-language game far outside ReBeL’s two-player guarantees. More broadly, ReBeL belongs to the line of work — DeepStack, Libratus, Pluribus, and now ReBeL — that Noam Brown points to as an early, concrete instance of **test-time compute**: spending computation on search at decision time, guided by learned intuition, rather than baking everything into one precomputed policy, the same thesis now central to reasoning models.

Several subsystems remain directly reusable. The **PBS representation** itself is the most important

— a belief distribution over opponents’ hidden states given common knowledge — and it is the natural seed for belief-based opponent modelling: this thesis’s Contribution 1 extends PBS to carry beliefs about the opponent’s *strategy type*, not just their cards. The **self-play-trained value/policy function on belief states**, the pattern of **CFR-as-search with a learned leaf evaluator**, the **random-iteration “safe search for free”** trick, and the simple fast equilibrium finder **FLOP** the paper introduces are all transferable. What is superseded is mostly DeepStack’s hand-engineered scaffolding, which ReBeL’s no-abstraction self-play dissolves, and then ReBeL itself as the *general* framework, which Student of Games subsumes. The honest verdict is that **ReBeL is a living stepping stone, not an endpoint**: as a deployed poker artefact it was never released and is now eclipsed, but as an idea — public belief states plus AlphaZero-style RL and search — it is foundational and very much alive. For this thesis specifically it cuts two ways at once: it is the starting point for Contribution 1’s richer belief state, and it is the proof that the soundness Pluribus abandoned *can* be recovered — just not yet beyond two players, which is exactly the frontier Contribution 2 sets out to cross.

6.6 Student of Games / SoG (2023)

ReBeL recovered soundness for two-player zero-sum poker and discarded both the abstraction and the blueprint — but it remained an imperfect-information method that merely *reduced* to AlphaZero in perfect-information games rather than unifying the two, solved a *fixed* depth-limited subgame, and tied its test-time search to its training procedure. Student of Games (Schmid et al., 2023; Google DeepMind, with collaborators at the University of Alberta and EquiLibre Technologies in Prague) closes the last gap in the chapter by asking the most ambitious question of all: **what if a single algorithm, learning from self-play with no human data, could play chess, Go, heads-up poker, and Scotland Yard — growing its own search tree as needed and remaining provably sound for perfect- and imperfect-information games alike?** Its answer fuses the chapter’s two great lineages — it is, in first author Martin Schmid’s words, “AlphaZero and DeepStack in a single big unified algorithm.” The engine is **Growing-Tree counterfactual regret minimization (GT-CFR)**: an anytime search that builds the game tree incrementally, guided by a policy network, with a learned value network evaluating the leaves it has not yet expanded. On perfect-information subtrees GT-CFR behaves like AlphaZero’s MCTS; on imperfect-information ones it behaves like CFR; and a single soundness theorem covers both. The system reaches **strong amateur-to-professional play in chess and Go, beats Slumbot — the strongest openly available heads-up no-limit bot — and defeats the state-of-the-art Scotland Yard agent**, all while being **proven to converge toward minimax-optimal play as computation and network accuracy grow**. It shares several DeepStack authors (Bowling, Moravčík, Burch, Schmid) — the Alberta lineage that began this chapter, here in contrast to ReBeL’s Facebook/FAIR provenance — and it first appeared in 2021 under the name *Player of Games* before being renamed for its 2023 *Science Advances* publication.

At a glance	Student of Games (2023)
Players	2 (two-player zero-sum) — guarantees <i>and</i> evaluation are 2p0s (Scotland Yard’s detectives count as one team); the underlying formalism is more general, but Nash is “less meaningful” beyond 2p0s
Game type	Both perfect- and imperfect-information — the only unified system: chess + Go (perfect) and HUNL poker + Scotland Yard (imperfect)
Blueprint (offline)?	No — the offline product is a learned value+policy network, not a stored strategy table (as in ReBeL)
Neural component	A single counterfactual value-and-policy network (CVPN) : one net outputs per-infostate counterfactual <i>values</i> and a prior <i>policy</i> , for all game stages
Search mechanism	GT-CFR — alternates a CFR regret-update phase (CVPN values at the leaves) with an AlphaZero-style PUCT expansion phase that <i>grows</i> the public tree; run inside continual re-solving, at both training and test time
Abstraction?	None of the card/information kind (the CVPN replaces it); keeps only a small <i>randomized betting</i> (action) menu in poker (20,000 $\rightarrow$ 4–5 actions), like ReBeL
Perfect-info too?	Yes — the distinguishing row; the only system in the chapter demonstrated on perfect-information games, with the <i>same</i> algorithm and soundness covering both classes
Compute	TPU-trained, deliberately matched to AlphaZero’s budget (Google TPuv4; Go the most expensive); reported <i>relative</i> to AlphaZero, no single dollar figure; search is $O(kT^2)$ ($O(T)$ for perfect-info); the full agent/code was not released
Key innovation	Growing-Tree CFR + sound self-play: one self-play-with-search algorithm, sound for both game classes , that grows its tree incrementally with a CVPN at the leaves and provably converges to Nash

6.6.1 The gap it closed

For seventy years, the two great traditions of game AI ran on separate tracks. One — minimax, alpha-beta, Monte-Carlo tree search, and ultimately AlphaZero — conquered *perfect-information* games by combining search with a learned value function, and its single most general expression, AlphaZero, mastered chess, shogi, and Go with one algorithm. The other — counterfactual regret minimization and the four poker systems of this chapter — conquered *imperfect-information* poker through game-theoretic reasoning. But the tracks never met. AlphaZero could not play poker: its Monte-Carlo tree search is *unsound* under hidden information, for the reason the ReBeL section made precise — the value of a state depends on the probability with which you reach it, so values cannot simply be backed up from local subtrees. And the poker systems could not play Go: they were built around the betting structure of one game. Even ReBeL, the most general of them, was presented and guaranteed only for imperfect information, *reducing* to AlphaZero in the perfect-information case rather than genuinely spanning both.

The open question Student of Games answers is whether a *single* algorithm — one search procedure, one network architecture, one self-play loop — can be **sound and strong across both game classes at once**. The difficulty is not merely engineering. A perfect-information search wants to expand the one best line deeply and read a single value off each leaf; an imperfect-information search must keep a *distribution* over actions (so as not to leak private information) and must reason about a *belief* over hidden states at every leaf. Reconciling these in one algorithm means finding a search that grows a tree the way AlphaZero does — guided, asymmetric, anytime — while computing the *game-theoretically sound* quantities that CFR does. Student of Games closes the gap by building exactly that search, GT-CFR, on top of the public-belief- state representation ReBeL had established, and by training its networks through a *sound self-play* loop whose targets come from the search itself. The result is the chapter’s capstone: the point where the abstraction-to-neural and offline-to-search arcs are joined by a third — perfect and imperfect information, unified.

6.6.2 Architecture

Student of Games is, structurally, **AlphaZero’s self-play loop with Monte-Carlo tree search replaced by GT-CFR and with the value/policy network defined over public belief states** — an offline phase that learns a network by self-play and an online phase that searches with it, but where the same search now runs in both phases (Figure 6.5).

The representation is inherited from the ReBeL lineage: a **public belief state** $\beta = (s_{\text{pub}}, r)$ pairs the public state (in poker, the betting history and board) with a **range** r — a pair of distributions over the information states each player could privately occupy. Perfect-information games are simply the degenerate case in which every public state has exactly *one* information state and the belief is a point mass, which is precisely why one representation can serve both classes. On this representation sit the two components. The **counterfactual value-and-policy network (CVPN)** is a *single* network that, given a belief state, emits both a vector of counterfactual values (one per information state, per player) and a prior policy — one network doing both jobs, for every stage of the game, where DeepStack used

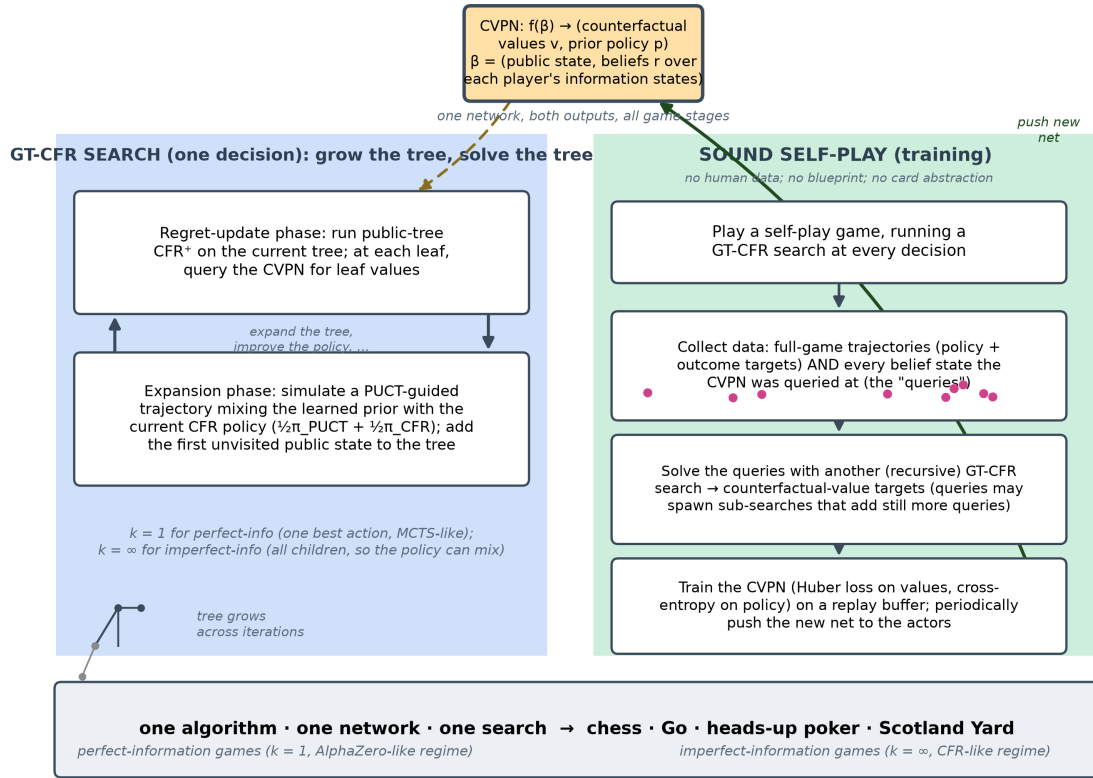


Figure 5: Student of Games: the GT-CFR search loop (left) and the sound self-play training loop that feeds it (right) share one CVPN; the footer strip names the unification across perfect- and imperfect-information games.

separate value-only networks per round. The **GT-CFR search** is the engine that turns that network into a strategy, and the **sound self-play** loop is what trains the network from the search’s own output. Offline, actors play self-play games, run a GT-CFR search at every decision, and emit two kinds of training data — full-game trajectories and the belief states the network was queried at during search — while trainers fit a new CVPN and periodically push it back. Online, the agent runs the very same GT-CFR search (via continual re-solving) to choose each move. The classic building blocks are all visible — CFR (Step 3) is the search’s inner loop, value-and-policy approximation (Step 5) is the CVPN, and public belief states and decomposition (the DeepStack/ReBeL lineage) are the representation — but the binding novelty is GT-CFR, the search that *grows* its tree, and the sound self-play that keeps every search consistent with every other.

6.6.3 Key innovation: Growing-Tree CFR and sound self-play

Student of Games’ contribution is a search algorithm that does for imperfect information what Monte-Carlo tree search did for perfect information — grow a tree intelligently toward the lines that matter — *without* sacrificing the soundness that hidden information demands. Two ideas make it work, and a single theorem makes it general.

The first idea is **growing the tree by alternating two phases**. AlphaZero’s MCTS expands one node per simulation and never looks back, because in a perfect-information game the value of a node, once computed, never changes — the future does not alter the past. Under imperfect information this fails: as Schmid puts it, observing an opponent’s *future* action changes your *belief* about the private state they held in the past, so changing the strategy anywhere in the tree ripples everywhere. A search therefore cannot just expand-and-evaluate; after each expansion it must re-solve the *whole* tree to keep it self-consistent. GT-CFR does exactly this. Each iteration runs two phases in turn. The **regret-update phase** runs several iterations of public-tree CFR over the *current* tree, and wherever the tree stops — at its frontier leaves — it queries the CVPN for the counterfactual values of the subgame below, exactly as depth-limited solving uses a learned value at the horizon. The **expansion phase** then grows the tree: it simulates a single trajectory from the root, choosing actions by a PUCT rule that *mixes* the network’s prior with the current CFR policy, and appends the first public state it reaches that is not yet in the tree. Iterating — “expand the tree, improve the policy, expand, improve” — yields an *anytime* search that, like MCTS, pours computation into the relevant lines, but that, unlike MCTS, is solving for a game-theoretically sound strategy at every step.

The second idea is the **single knob that adapts this one search to both game classes**. When it expands, AlphaZero adds only the single most promising action, which is ideal when optimal play can be deterministic. But optimal imperfect-information play is generally *stochastic* — you must mix actions to avoid leaking information — so committing to one action is wrong. GT-CFR therefore expands the top- k actions by prior, and sets $k = 1$ for perfect-information games (one good line suffices; the search is then essentially AlphaZero’s) and $k = \infty$ — all children — for imperfect-information games (so the policy can mix over them). With $k = \infty$ the search even gains a *finite-time* guarantee on policy quality, not merely an in-the-limit one. This one switch is the technical heart of the unification: the same code reduces to

MCTS-like search on a chess position and to CFR-like iteration on a poker decision.

The third element is **sound self-play**, which trains the CVPN. Like AlphaZero, the agent is trained to predict the result of its own search: each network query made during a search defines a subgame, that subgame is re-solved by *another* GT-CFR search, and the network is trained toward the search’s (higher-quality) output — values by a Huber loss, policy by cross-entropy, with full-game outcomes supplying the remaining value targets. The word *sound* is the load-bearing one: every search used to generate data must be consistent with the network and with the other searches along the trajectory (it must not be assembling fragments of two different equilibria), which is enforced by running the searches on a safe re-solving auxiliary game. This is the imperfect-information analogue of AlphaZero’s bootstrapped self-play, and it is what makes the learned values actually converge to the game-theoretic ones.

The pairing comes with a guarantee that completes the chapter’s family of soundness bounds. Theorem 1 states that the regret of GT-CFR’s average policy after T iterations splits into a term that accumulates the value network’s ϵ -error over the tree’s *frontier* (the leaves the CVPN evaluates) and an ordinary CFR regret term over the tree’s *interior* (the nodes where CFR runs); dividing by T , the average policy’s exploitability is bounded by

$$\text{exploitability}(\bar{\pi}^T) \lesssim \underbrace{|\mathcal{F}| \epsilon}_{\text{value-net error (frontier)}} + \underbrace{\frac{|\mathcal{N}| UA}{\sqrt{T}}}_{\text{CFR convergence (interior)}},$$

with $|\mathcal{F}|$ and $|\mathcal{N}|$ the frontier and interior sizes, U the largest value gap, and A the maximum number of actions. The shape is by now familiar — a value-error term plus a $1/\sqrt{T}$ CFR-convergence term, the direct structural heir of DeepStack’s $k_1\epsilon + k_2/\sqrt{T}$ and ReBeL’s $\delta C_1 + \delta C_2/\sqrt{T}$ — but two things are distinctive. First, the coefficients are the *sizes of the growing tree’s frontier and interior*, so the bound is literally a statement about incremental tree growth: adding nodes over time costs nothing in convergence order. Second, and decisively, **this same statement holds whether the tree is a perfect-information game tree or an imperfect-information public tree** — it is the one soundness guarantee that spans both classes. A companion result, Theorem 2, then shows that invoking GT-CFR recursively through continual re-solving over a whole episode keeps the agent sound, with exploitability growing only *linearly* in the game length (a factor of roughly $5D + 2$ for D re-solving steps) — the property that lets Student of Games survive Scotland Yard’s twenty-four-round horizon. Together the theorems say what AlphaZero could only assume: as the network improves ($\epsilon \rightarrow 0$) and search deepens ($T \rightarrow \infty$), play converges to a Nash equilibrium — now for poker and chess alike.

6.6.4 Caveats, dead-ends, and what the paper under-describes

True to the chapter’s pattern, the *Science Advances* main text is the clean, high-level story and the engineering honesty lives in the supplement, the theorems being stated there “only informally” with the architectures, hyperparameters, full pseudocode, compute, and proofs all relegated to the Supplementary Text.

The headline caveat is openly owned: **Student of Games is weaker than AlphaZero in chess and Go given the same resources**, and the gap is not small in Go — its strongest configuration won just 2 of 400 games against a mid-training AlphaZero, i.e. it lost 99.5%, even while crushing the classical program Pachi by over 1,100 Elo. The authors call this “the price of SoG’s generality” and hypothesize the cause precisely: CFR is simply less efficient than Monte-Carlo tree search on perfect-information games, so the unified search pays for its breadth with peak strength. Schmid is candid that this is by design — the goal is one algorithm “better than humans” at everything, not the best chess engine — but it means the unification is a proof of *soundness and competence* across classes, not of dominance in either.

Two scaling walls are acknowledged as the real limits. The first is the **belief-space blow-up**, which Schmid names as the main limitation: the CVPN must *enumerate the information states per public state*, so its cost scales with the number of private states a player could occupy — manageable for poker’s 1,326 hands, but “prohibitively expensive in some games”, where the representation “falls apart”. This is exactly ReBeL’s PBS-input limitation inherited intact; the paper floats approximating it with a generative model that *samples* world states rather than enumerating them, but does not build one. The second is the **known-model requirement**: like AlphaZero and ReBeL, Student of Games needs a perfect simulator of the rules, placing it, in Schmid’s words, “behind the AlphaZero lineage” relative to MuZero, which *learns* its model.

Several smaller asterisks round out the picture. The system still keeps a **randomized betting abstraction** in poker (about twenty thousand actions reduced to four or five), the one piece of domain knowledge it has not shed, flagged as removable future work. Its search is **quadratic in the number of iterations** ($O(kT^2)$ network calls, since the tree grows each step), an inefficiency the authors note could be parallelized away but was not. The hyperparameter c — how many regret updates to run per expansion — is admittedly **under-explored** (“ $c = 1$ is not always best in practice ... we hope to explore this more thoroughly”). And the clean training-convergence argument is **idealized**, holding only “asymptotically, as $T \rightarrow \infty$ and with very large (exponential) memory”, so practice rests on the network’s finite capacity. None of these dents the result, but together they mark Student of Games as a first, deliberately general proof of concept rather than a tuned, scalable product.

6.6.5 Compute & accessibility

Student of Games’ cost is concentrated **offline, in TPU-scale self-play training**, and the paper reports it *relative to AlphaZero rather than in absolute terms* — a deliberate choice to enable a fair comparison without claiming a hardware record. The AlphaZero baseline used 3,500 concurrent actors each on a single Google TPUv4 over 800,000 training steps, and Student of Games “was trained using a similar amount of TPU resources”, with per-domain training running from roughly a million steps (Go) to seventeen million (Scotland Yard). Schmid’s own gloss is that chess and poker are trainable “on a few GPUs”, while Go “by far the hardest” consumed a large TPU pool — chiefly because the hardware was available, not because the method demanded it. There is, pointedly, **no single dollar figure** of the kind Pluribus made famous: because the comparison is pinned to *network-call budget* rather than wall-clock, the honest summary is “comparable to AlphaZero, and substantial” — indeed the authors list reducing this compute among their

open problems. At play time the search is anytime and tunable, but its $O(kT^2)$ growth (reducing to $O(T)$ for perfect-information games, where a node need only be evaluated once) means strong configurations are not cheap either.

On accessibility the verdict is mixed and worth stating precisely. The **full Student of Games agent and its trained networks were not released** — there is no code-availability statement beyond data in the paper and supplement — so on the headline-artefact axis it sits with the closed Libratus and Pluribus rather than the open ReBeL. What *is* open is the surrounding ecosystem: the same DeepMind/Alberta group maintains **OpenSpiel**, which ships the CFR family and the benchmark games themselves (Leduc, Liar’s Dice, Scotland Yard), so the components and environments are reproducible even though the assembled system is not. In practice this makes Student of Games an *open method on an open substrate with a closed flagship* — a reasonable middle ground, but one that leaves full replication of the chess/Go/poker/Scotland-Yard results beyond a typical group’s reach.

6.6.6 Strengths and limitations

Student of Games’ signal strength is **generality with soundness** — a combination no prior system in the chapter offered. It is the first algorithm to be *provably sound across both perfect- and imperfect-information games* (Theorems 1–2), and it backs the theory with breadth no competitor matched: in a single design it reaches expert-to-professional level in **chess and Go**, **beats Slumbot**, the strongest openly available heads-up no-limit bot (by about +7 mbb/g — the win-rate unit from the previous sections, which this paper writes as mbb/hand — and crucially it is *not* exploited by the local-best-response probe that catches weaker bots, which it beats by +434 mbb/g), and **defeats PimBot**, the state-of-the-art Scotland Yard agent built on Monte-Carlo tree search with hand-tuned heuristics, even when PimBot is given ten million search simulations to Student of Games’ few hundred. It uses **no human data, no precomputed blueprint, and no card abstraction**, learning everything from self-play; it **scales** with compute (more search and more training both lower exploitability, the empirical face of Theorem 1); and it demonstrably *refines its policy at test time through search*, “unlike any pure reinforcement-learning algorithm” — the clearest statement in the chapter of search-as-inference.

The limitations are the precise shape of its ambition. It is **weaker than the specialists** in their home domains — markedly so in Go — the acknowledged price of one algorithm for everything. Its guarantees, like ReBeL’s, hold only for **two-player zero-sum** play (Scotland Yard’s detectives are pooled into a single team to fit), so the *multiplayer* safety gap Pluribus exposed remains untouched even here. Its **belief representation does not scale** to games with vast private-state spaces, and it **requires a known model** of the environment, the two walls its own authors flag first. It retains a small **betting abstraction**, its search is **quadratic** in iterations, and — uniquely awkward for a generality claim — the **flagship code was never released**. The honest reading is that Student of Games proves the unification is *possible and sound*, and pays for that proof in peak strength, scale, and reproducibility.

6.6.7 Legacy and modern relevance

Student of Games is the capstone of this chapter’s argument, and it lands all three of its arcs at once. Across the five systems the field moved from **hand-built abstraction to learned neural approximation** (DeepStack and ReBeL dissolved the abstraction that Libratus and Pluribus depended on), and from **solving everything offline to learning-and-searching** (the blueprint shrank from Libratus’s petabytes to a value network and a real-time solve). Student of Games adds the third and final unification — **perfect and imperfect information in one algorithm** — and in doing so reframes the whole arc. The conceptual takeaway is that AlphaZero’s recipe (self-play, a learned value-and-policy network, and a tree grown by guided search) was never specific to perfect information; it only needed a *sound* search over *beliefs* rather than states. Swap Monte-Carlo tree search for GT-CFR and run it over public belief states, and the same recipe spans chess and poker with a single convergence guarantee. That is the chapter’s central thesis made literal: one algorithm, one network, one search, four very different games.

Its relationships to the surrounding frontier are clarifying. Student of Games is the *general* framework ReBeL pointed toward — it grows its tree instead of fixing a depth-limited subgame, decouples test-time search from training (so, like AlphaZero, it can simply search harder at test time for stronger play), and is validated across game classes rather than on imperfect information alone. Its most visible *unmet* neighbour is **MuZero**: where MuZero removed AlphaZero’s need for a known model by learning the dynamics, Student of Games still assumes a perfect simulator, so the natural next step — uniting GT-CFR with a learned model and a generative belief sampler — would dissolve both of its named limitations at once. And it sits one axis away from **CICERO**, Meta’s human-level Diplomacy player, which carries search-plus-learning into a seven-player, mixed-motive, natural-language game: a vivid reminder that Student of Games unifies *information structure* but not *player count*. Several of its parts are already reusable in their own right — **GT-CFR** as an anytime, policy-guided growing-tree search for any game class; the **CVPN** as a single value-and-policy head over belief states; **sound self-play** as a way to bootstrap targets from mutually consistent sub-searches; and a **domain-independent continual re-solving** that no longer leans on poker’s special structure.

For this thesis specifically, Student of Games is both a ceiling and a signpost. As the most general PBS-based sound-search framework, it is the natural context for **Contribution 1**: its belief — a distribution over the opponent’s possible information states — is exactly the substrate to be *enriched* with beliefs about the opponent’s strategy *type*, turning a Nash-seeking belief into an adaptive one. And its careful restriction of all guarantees to two-player zero-sum play — the paper is explicit that Nash is “less meaningful” beyond it — re-states, from the most advanced vantage point in the literature, the exact gap **Contribution 2** targets: unifying the *information structure* of games did nothing to close the *multi-agent safety* gap that Pluribus first exposed. That is the cleanest possible hand-off from this chapter to the rest of the dissertation, and the synthesis that follows draws the five systems together — an evolution-delta table of what each added and gave up, the evolution diagram, and the component-reuse map — to make the open problems, and the contributions that address them, explicit.

6.7 Synthesis

With all five systems in hand, the synthesis draws them together — not to crown a winner, but to make three things explicit: the shape of the seven-year arc, the machinery the systems share, and the open problems that hand this chapter off to the rest of the dissertation. The per-system scorecards already supply the dimension-by-dimension detail, so what follows aggregates rather than re-derives them.

6.7.1 The arc in one read

The progression is best read not as a climb but as a walk across the three axes of the introduction, in which every step forward on one axis was paid for somewhere else. DeepStack and Libratus opened the decade by attacking the same weakness — the lossy abstraction-and-blueprint paradigm — from opposite directions: DeepStack threw the paradigm out and replaced it with learned values and continual re-solving, while Libratus kept it and added a provably safe real-time repair. Both stayed two-player and neither reported the other’s headline result, so even at the start “better” was already multi-dimensional. Pluribus then advanced the *player-count* axis to six, the first superhuman result in any benchmark game beyond two players or teams, but did so by giving up safety entirely and remaining tabular — a sideways move on the representational axis and a backward one on guarantees. ReBeL reversed both of those concessions, returning to two players to *recover* a soundness guarantee while pushing the representational axis the furthest yet — public belief states with self-play-trained value and policy networks, no abstraction and no blueprint — and Student of Games completed the picture by adding the third axis, unifying perfect- and imperfect-information play in one algorithm, at the cost of peak strength in each. The honest one-line summary is that the field traded *hand-built breadth* for *learned generality* and *offline precomputation* for *real-time search*, and that no single system dominates on every axis at once.

The table below captures that walk as a ledger of what each system added and what it cost, which is the useful comparison for an evolutionary reading; the fixed per-dimension values (players, game type, blueprint, neural component, search, abstraction, perfect-information, compute, key innovation) are in each section’s scorecard and are not repeated here.

System (year)	What it added	What it gave up / what it cost
DeepStack (2017)	First <i>sound</i> depth-limited search under hidden information; learned counterfactual values in place of a stored strategy; discards the full-game abstraction-and-blueprint paradigm	Still keeps a sparse betting abstraction inside its look-ahead; never tested head-to-head against prior bots; ~175 CPU-core-years of offline value labelling

System (year)	What it added	What it gave up / what it cost
Libratus (2017/18)	First head-to-head <i>and</i> human win, via real-time nested safe subgame solving with a provable bound; an overnight self-improver that patches its own holes	Keeps and depends on abstraction + a petabyte blueprint; no neural generalization; supercomputer-scale (~25M core-hours)
Pluribus (2019)	First superhuman multiplayer (six-player) play; famously cheap (~\$150 on a single server)	Drops <i>all</i> safety guarantees (no N-player bound); relies on <i>unsafe</i> search; remains fully tabular and abstracted
ReBeL (2020)	“AlphaZero for imperfect information”: public belief states + learned value/policy + CFR in belief space; <i>recovers</i> the 2p0s guarantee; eliminates abstraction <i>and</i> blueprint; open-sources Liar’s Dice	Guarantees retreat to two players; the PBS blows up where common knowledge is scarce; GPU-cluster training; needs a known model
Student of Games (2023)	Unifies perfect- and imperfect-information play in one sound algorithm (GT-CFR + a single value-and-policy net + sound self-play)	Weaker than the specialists (markedly at Go); guarantees still 2p0s; belief-space blow-up; known-model requirement; flagship code unreleased

6.7.2 What carries forward

Almost nothing in these systems is wholly new; each is a recombination of a small set of building blocks, most of them introduced in Steps 3–5. **CFR and CFR** (Step 3) are the inner solver of every system; **Monte-Carlo CFR** (Step 3) trains the blueprints of Libratus and Pluribus; **card and action abstraction** (Step 4) underpin Libratus and Pluribus and survive only as a vestigial betting menu in ReBeL and Student of Games; and **neural value — then value-and-policy — approximation** (Step 5) is precisely what lets DeepStack, ReBeL, and Student of Games dissolve the abstraction the others depend on. Onto these the chapter layers its own native primitives, each introduced by one system and reused by its successors: **depth-limited solving**, **continual re-solving**, **safe nested subgame solving**, the **public belief state**, and **Growing-Tree CFR**. Read down the list of *first appearances* and the chapter’s evolution restates itself in components: the blocks accumulate, the abstraction-era ones fade, and the learned ones take over.

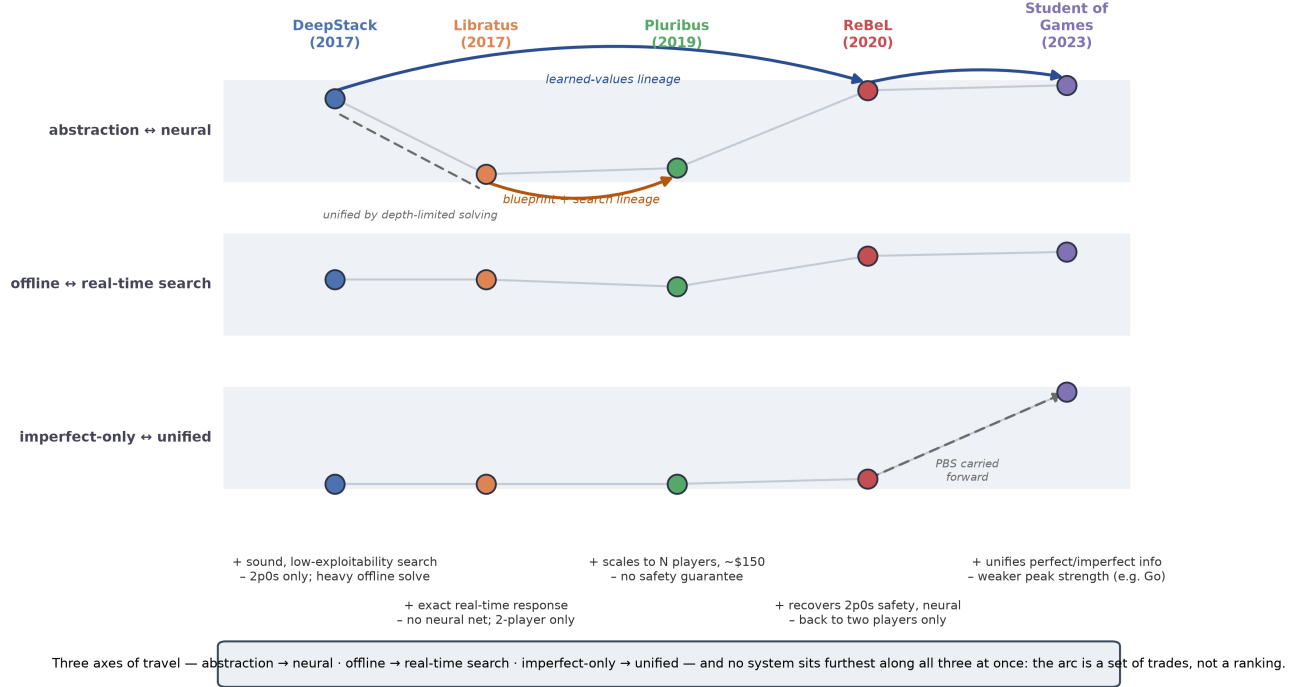


Figure 6: The seven-year arc as trade-offs: the five systems placed along three axes (abstraction $\rightarrow$ neural; offline $\rightarrow$ real-time search; imperfect-only $\rightarrow$ unified), with lineage arrows and a per-system capability-gained/given-up tag.

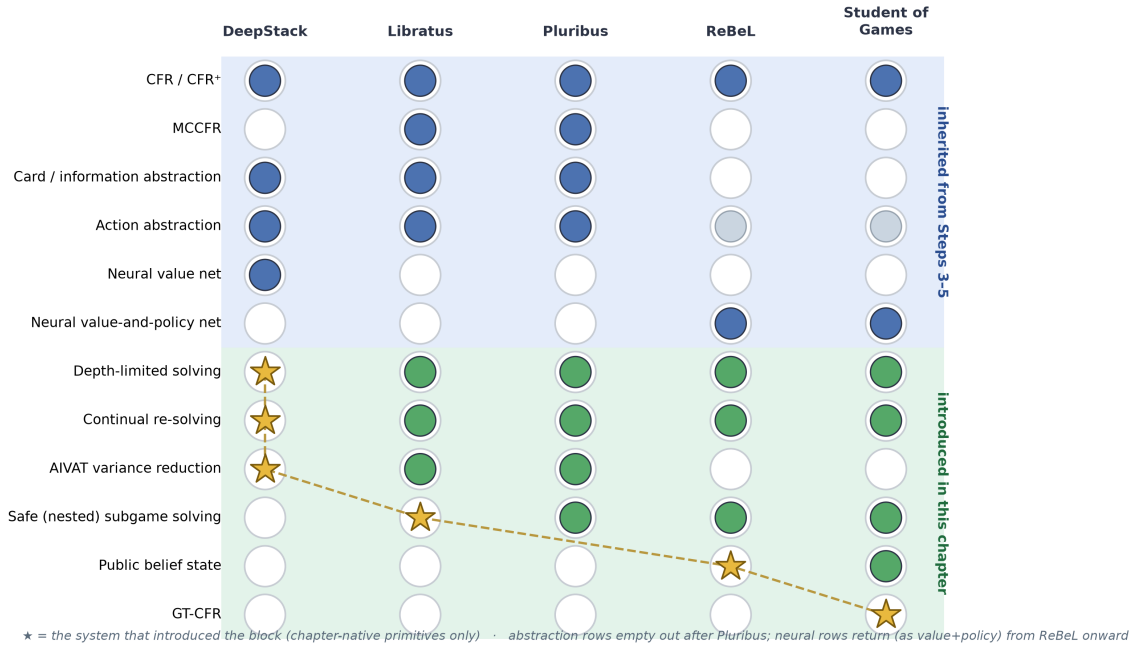


Figure 7: Component-reuse map: Step 3–5 building blocks and chapter-native primitives (rows) against the five systems in chronological order (columns); stars mark the system that introduced each native primitive, connected by a staircase of first appearances.

6.7.3 Why this matters for our research

This chapter is the state of the art the dissertation builds on and departs from, and its single most important contribution to that work is a framing one. Every system surveyed here is **opponent-blind by design**: each computes a worst-case-robust strategy and plays it unconditionally, and the two multiplayer- and human-tested systems make the stance explicit — they refuse to model or adapt to opponents so as never to be counter-exploited, and Pluribus does not even know who it is playing. Read as a survey, then, the chapter is a catalogue of five world-class systems that share exactly one omission: none adapts to the specific, fallible opponent in front of it. That shared omission is the opening this entire dissertation occupies, and the rest of this subsection reads the chapter’s assets against the three planned contributions.

For Contribution 1 (Behavioral Adaptation Framework). The chapter’s most direct gift is the **public belief state (PBS)** introduced by ReBeL and inherited by Student of Games — a distribution over the hidden states an opponent could occupy given common knowledge, which Contribution 1 widens to carry beliefs about the opponent’s *strategy type*, not merely their cards. Two less obvious points sharpen this. First, the PBS was invented to make *search sound*, and soundness is orthogonal to opponent modelling — so building adaptation on a PBS inherits the entire re-solving machinery and its convergence guarantees *unchanged*, letting an adaptive belief be updated inside a provably sound search rather than bolted on outside one; adaptation gets to reuse not just a data structure but the proofs attached to it. Second, every system in the chapter shares a **two-tier architecture** — a cheap global policy (a blueprint, or a learned value network) refined by a real-time local solve — and the dissertation’s own design, “start from a safe strategy and improve against weak opponents,” is that same two-tier shape with the second tier’s objective changed from *accuracy* to *bounded exploitation*. The chapter thus supplies both the substrate for adaptation and the system skeleton it slots into.

For Contribution 2 (Multi-Agent Safe Exploitation). The keystone is Pluribus: superhuman six-player poker reached with *no* safety guarantee of any kind, which is exactly the gap a theory of multi-agent safe exploitation must close — a point the chapter states outright. A subtler reading turns the systems’ own caveat into the contribution’s justification. Pluribus and Student of Games both restrict their guarantees to two-player zero-sum play and concede that beyond it a Nash equilibrium is neither unique, nor efficiently computable, nor even a guarantee against losing. If equilibrium buys no safety in the multiplayer setting to begin with, then declining to exploit a visibly weak opponent forfeits a guarantee one never possessed — which reframes deliberate exploitation from a reckless departure from safety into the *rational objective* once safety is unattainable by equilibrium alone. That is the cleanest motivation the literature offers for the multi-agent exploitation Contribution 2 pursues.

For Contribution 3 (Evaluation Methodology). The chapter standardizes on **exploitability** — the worst-case loss to a best-responding adversary — as the universal yardstick, applied identically to all five systems, and that uniformity is itself the template for a cross-system evaluation framework. It also hands the dissertation two instruments, repurposed. The **local-best-response (LBR) probe**, used throughout the chapter to *certify* unexploitability, becomes a *safety stress-test* when pointed at an adaptive agent — does chasing a weak opponent make you counter-exploitable? — which is precisely the adversarial,

teaching-attack-style evaluation Step 8 calls for. And **AIVAT**, the learned-value variance reducer behind DeepStack’s and Pluribus’s reported results, is the instrument that makes a small but *growing* adaptation edge statistically visible against the brutal variance of card games, where raw winnings would mislead.

Further leverage points (candidate — to keep or trim against final length). The following are compact extrapolations rather than claims already made by the systems; each is a one-line seed for later steps.

- **A reusable proof shape (C2).** Every guarantee in the chapter has the same form — a value-approximation term plus a $1/\sqrt{T}$ convergence term (DeepStack’s $k_1\epsilon + k_2/\sqrt{T}$, Libratus’s 2Δ , ReBeL’s $\delta C_1 + \delta C_2/\sqrt{T}$, SoG’s $|\mathcal{F}|\epsilon + |\mathcal{N}|UA/\sqrt{T}$) — giving Contribution 2 a ready template for bounding the exploitability *incurred* by a deliberate deviation.
- **Depth-limited exploitation (C1/C2).** Pluribus’s multi-valued leaves (k selectable continuation strategies) are structurally a menu of opponent behaviours at the horizon; reinterpreting that menu as opponent *types* yields tractable exploitation past the search depth — the exact problem Step 8’s “adapting beyond the depth limit” takes up.
- **A build-vs-cite map (methodology).** The “Compute & accessibility” subsections sort the field into what is reproducible at PhD scale (Pluribus’s single-server CFR, ReBeL’s single-thread CPU solver, the open OpenSpiel substrate, small games) versus what is cite-only (Libratus’s supercomputer, SoG’s TPU pool) — the implicit rationale for building on the cheap, open lineage (the ReBeL-Lite-on-Leduc task) rather than reproducing a flagship.
- **A redirected mechanism (framing).** Libratus’s overnight self-improvement is in-match strategy revision aimed at its *own* weaknesses; the dissertation runs the same observe-then-revise loop aimed at the *opponent’s*, so the contribution is redirecting a proven superhuman mechanism, not inventing one.
- **A frontier-narrative hook (framing).** All five systems are now cited as early instances of “test-time compute” — spending search at decision time rather than baking everything into a precomputed policy — so framing real-time adaptation as test-time *adaptation* compute connects this work to the contemporary reasoning-model story.
- **Consistency-as-safety (C2).** Student of Games’ “sound self-play” requires every local search to stay *consistent with* a safe reference, conceptually the same constraint as the KL-regularized (PiKL) exploitation Contribution 2 will use — an analogy worth borrowing.
- **Abstraction transferred to type-space (C3).** Step 4’s lesson that lossy abstraction creates exploitable seams carries from state-space to opponent-*type* space: a too-coarse type model is exploitable the way a too-coarse card abstraction is, predicting a failure mode for Contribution 3 to catch.

6.7.4 Open problems and the hand-off to Steps 7–15

Four open problems fall directly out of the synthesis, and each is the seed of a later step. The first is **opponent-blindness** itself: the entire frontier computes a fixed, worst-case-optimal strategy and plays it unconditionally, leaving the gap between static equilibrium and dynamic, opponent-aware play wide open — the subject of Step 7 (inferring opponent behaviour from action traces) and the inference half of

Contribution 1. The second is **multiplayer safety**: Pluribus proved that Nash-and-search methods *win* in N-player imperfect-information games while offering no safety guarantee at all, and Student of Games confirmed that even the most general sound-search framework still restricts its guarantees to two players — so a tractable theory of *safe exploitation beyond two-player zero-sum* (Step 8, Contribution 2) remains genuinely open. The third is **adaptation beyond the depth limit**: the chapter’s real-time solvers all look only a little way ahead, and carrying an opponent model soundly past that horizon — rather than discarding it at the leaf — is an unsolved problem Step 8 takes up directly. The fourth is **real-time compute budgets**: the systems’ costs range over six orders of magnitude, and an adaptive agent that must re-solve *and* re-estimate an opponent online inherits the tightest version of that constraint.

These problems define Phase D (Steps 7–8), the thesis-critical core, and they ramify through the phases that follow — multi-agent dynamics and coalition formation (Phase E), the data-driven behavioural pipelines that ground the models in real traces (Phase F), and the cross-domain evaluation framework that closes the plan (Phase G). The five systems of this chapter are, collectively, the platform this dissertation stands on and the foil it defines itself against: it inherits their belief-state representations, their real-time search, and their exploitability discipline, and it sets out to add the one capability they all, by design, leave out — the ability to notice that an opponent is not playing optimally, and to do something about it without becoming exploitable in turn.

Step 7 Summary — Opponent Modeling in Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

7	Step 7 — Opponent Modeling in Imperfect-Information Games	1
7.1	1. Why Opponent Modeling — the money Nash leaves on the table	2
7.2	2. The Bayesian Core — belief, evidence, response	3
7.3	3. Seeing Through a Keyhole — the partial-observability problem	5
7.4	4. Three Models, One Interface	7
7.5	5. The Consistency Problem and the Sequence-Form Fix	8
7.6	6. When a Model Is Confident and Wrong	10
7.7	7. From Model to Money — best response, the ceiling, and a self-inflicted leak	12
7.8	8. Adapting to Change — non-stationary opponents	14
7.9	9. Connections and Forward Pointers	15

7 Step 7 — Opponent Modeling in Imperfect-Information Games

This is a ground-up chapter on opponent modeling: the problem, the mathematics, the family of methods, and a set of controlled experiments run on two small poker games. It serves two purposes — a self-contained refresher while later steps build on it, and a primary source for the Step 15 public synthesis. It is written to be read on its own; no prior familiarity with the project’s code is assumed. All experimental numbers reported here were measured on reproducible runs of the two testbeds (Kuhn Poker and Leduc Hold’em) and are bounded, wherever possible, by *exact* analytical references rather than simulated ones.

Where this sits in the thesis. A game-theoretic agent from the earlier steps computes a *Nash equilibrium* — a strategy that cannot be beaten in the long run. This step builds the opposite capability: a **sensor** that watches how a *specific* opponent actually plays and turns those observations into an estimate of their strategy, so the agent can deviate from equilibrium to punish that opponent’s mistakes. That sensor is the first half of the thesis’s Behavioral Adaptation Framework (Contribution #1). The second half — how to exploit *safely*, without opening a hole in your own play — is formalized in Step 8.

7.1 1. Why Opponent Modeling — the money Nash leaves on the table

A **Nash equilibrium** strategy is built never to lose in the long run, *no matter who it plays*. In a two-player zero-sum game that guarantee is real and valuable: an equilibrium strategy has a fixed *value*, and no opponent — however clever — can push you below it. But the same property is also a limitation. An equilibrium strategy plays **identically** against a world champion and against someone who folds every single time you bet. It never adapts, because adaptation is exactly what it was designed not to need.

That refusal to adapt has a price, and the price is measurable. **Opponent modeling** is the act of watching a specific opponent’s actions, forming a belief about their strategy, and **deviating from Nash to exploit the pattern** — bluffing more against someone who folds too much, value-betting thinner against someone who calls too much.

The cleanest way to see the value is a *rock-paper-scissors* picture. Your opponent secretly throws rock 70% of the time. The safe strategy is to randomize evenly and break even forever. But if you *notice* the bias, you throw paper more and start winning. Two things make this the whole step in miniature: you must **infer** the bias from a noisy stream of throws (a few aren’t enough), and if you **over-commit** to paper you become predictable and they crush you with scissors. Inferring the bias is opponent modeling; knowing how far to lean is the exploitation-versus-safety tradeoff that runs through everything below.

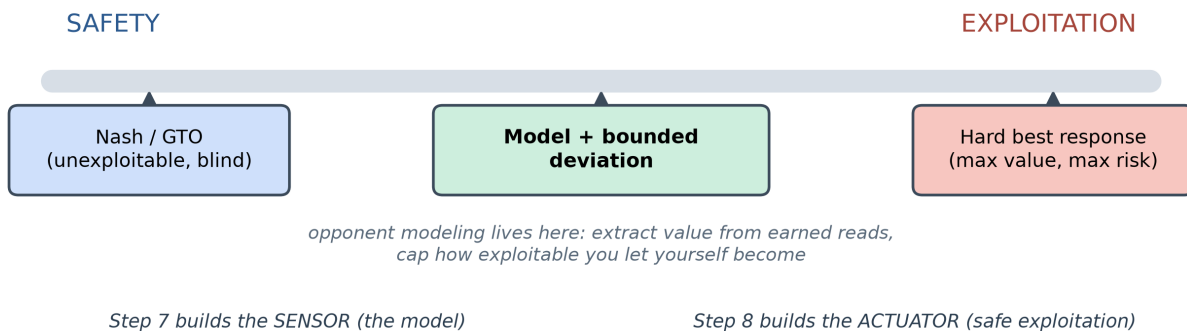


Figure 1: The safety-exploitation dial: pure equilibrium play is unexploitable but blind; a hard best response extracts the most value but is maximally risky. Opponent modeling operates in between. Step 7 builds the sensor (the model); Step 8 builds the actuator (safe exploitation).

7.1.1 How much is at stake — measured on Kuhn Poker

Kuhn Poker is the smallest non-trivial poker game: a three-card deck (Jack, Queen, King), one card each, a single betting round. It is small enough to solve *exactly*, which lets us compute the value of modeling rather than guess at it. For each of several fixed opponent styles we computed three exact quantities:

- **Nash EV** — what an equilibrium strategy earns against that opponent (per hand);
- **Best-response EV** — what a strategy that *knows the opponent perfectly* earns: the exact ceiling on exploitation;

- **the gap** between them — the money equilibrium play leaves on the table.

Opponent (Kuhn)	Nash EV	Best-response EV	Exploitation gap
Calling station (never folds)	+0.119	+0.333	0.215
Rock (only commits with the King)	-0.060	+0.167	0.227
Maniac (always bets/calls)	+0.121	+0.333	0.213
Nash (equilibrium play)	-0.055	-0.052	0.004

Against the three exploitable styles the gap is enormous — **0.11 to 0.28 per hand**, on a game whose entire equilibrium value is about one twentieth of a chip. Against a Nash opponent the gap is essentially **zero**, exactly as theory demands: you cannot exploit an equilibrium, because best response cannot beat the game value. (The small negative Nash EVs are Kuhn’s known first-player disadvantage of $-1/18 \approx -0.056$ — that is the game, not an error.)

Two lessons are already visible. First, **modeling is worth doing**: the value is large and real. Second, **exploitation is directional** — the exact best response to a rock *bluffs more* (it bets the weakest hand into an open pot, because the rock folds everything but the King). The rest of this chapter is about earning that value without giving it back.

Read more: Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*.

7.2 2. The Bayesian Core — belief, evidence, response

Underneath every method in this chapter is one loop, and it is worth stating in plain language before any notation. You are a detective. You start with a hunch about how the opponent plays (a **prior**). Each action they take is a clue: it makes some explanations more likely and others less. You fold the clue into your suspicion (the **posterior**), and repeat. Opponent modeling *is* this loop:

belief about their strategy $\rightarrow$ see an action $\rightarrow$ update belief $\rightarrow$ best-respond $\rightarrow$ repeat.

Formally, the update is Bayes’ rule. If σ denotes a candidate opponent strategy and a the action just observed,

$$P(\sigma \mid a) \propto P(\sigma) \cdot P(a \mid \sigma),$$

in words: **posterior** $\propto$ **prior** $\times$ **how well that strategy explains what they just did**. An action a candidate calls impossible drives its probability toward zero; an action it predicts well boosts it. Over many hands the belief concentrates on the best-fitting explanation.

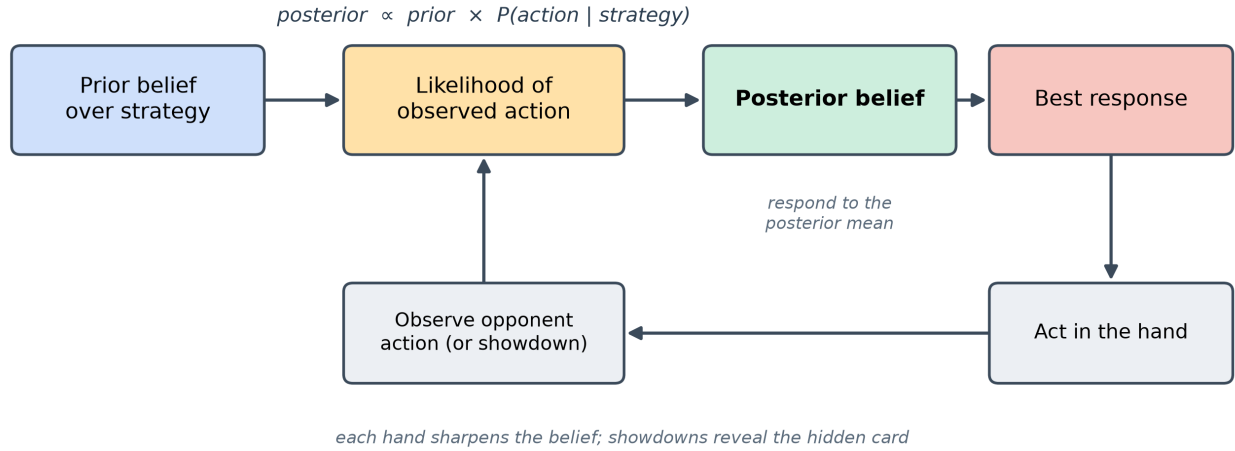


Figure 2: The opponent-modeling loop: a prior belief is multiplied by the likelihood of each observed action to give a posterior, which drives a best response; the outcome feeds the next observation.

7.2.1 The type “zoo”

The simplest way to make this concrete is to reason over a small menu of predefined opponent **types** rather than the whole infinite space of strategies. Throughout the chapter we use a recurring cast, each of which serves double duty — as an opponent to play against, and as a hypothesis the detector reasons over:

Type	Behavior
Calling station	Never bets, never folds — checks when it can, calls any bet.
Rock (tight-passive)	Commits chips only with the best hand; folds everything else. The most exploitable type.
Maniac (loose-aggressive)	Always bets or calls, regardless of hand.
Nash	Balanced equilibrium play; mixes its actions; unexploitable.

7.2.2 The update, worked by hand

The mechanics are easiest to trust when you turn the crank once yourself. Suppose three candidate types bet a particular hand with probability 0.8, 0.5, and 0.1 respectively, and you start from a uniform prior $(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$. You observe a **bet**:

$$\text{posterior} \propto \left(\frac{1}{3} \cdot 0.8, \frac{1}{3} \cdot 0.5, \frac{1}{3} \cdot 0.1\right) = (0.267, 0.167, 0.033) \xrightarrow{\text{normalize}} (0.571, 0.357, 0.071).$$

Observe a **second** bet and multiply by (0.8, 0.5, 0.1) again:

$$(0.711, 0.278, 0.011).$$

The belief piles onto the bet-happy type, and it would do so *faster* the more the candidates disagree. That “how much they disagree” intuition returns in Section 6 as the reason some opponents are identified in four hands and others take two hundred.

7.2.3 Why Dirichlet, and why you only ever need the mean

Two facts from the literature shape every practical model. First, when you estimate an action distribution by counting, the natural prior is the **Dirichlet** distribution, because it is the *conjugate* prior of the multinomial: updating a Dirichlet with observed counts yields another Dirichlet, with the counts simply **added** to the prior’s pseudo-counts. No integral, no simulation — the posterior mean of action i is

$$\mathbb{E}[p_i] = \frac{\alpha_i + n_i}{\sum_j (\alpha_j + n_j)},$$

where α_i are prior pseudo-counts (which double as *smoothing*, keeping no action at exactly zero) and n_i are observed counts. This closed form *is* the “continuous” model of Section 4.

Second — and this is the single most reused result in the area — to choose an optimal response you never need the whole posterior distribution over strategies; you only need its **mean**. Your expected payoff against a *distribution* of opponent strategies equals your payoff against the single averaged strategy. That collapses an intractable integral over strategy space into “best-respond to one averaged opponent,” and it is what makes Bayesian exploitation practical. (Section 5 revisits this: responding to the mean is payoff-optimal but, as it turns out, can fail to *converge* to the opponent’s true strategy — the subtlety the 2025 work resolves.)

Read more: Ganzfried, S. & Sun, Q. (2016/2018). “Bayesian Opponent Exploitation in Imperfect-Information Games.” *IEEE CIG*. (Theorem 2.1: respond to the posterior mean.)

7.3 3. Seeing Through a Keyhole — the partial-observability problem

There is a reason opponent modeling in poker is harder than counting frequencies in rock-paper-scissors, and it deserves its own section because it shapes every model that follows.

In poker you observe **actions** — bet, call, fold — but **not** the private hand that produced them. The same “bet” can come from a monster or a bluff. The one exception is a **showdown**, where cards are revealed at the end of a hand that goes to the end without a fold. So every hand falls into one of two evidential categories:

- **Showdown** — **cause and effect**. You see both the opponent’s actions *and* the private hand behind them. You can attribute the behavior to the exact situation that produced it. This is the gold-standard observation.
- **Fold** — **effect only**. The hand ends without revealing cards. You saw *what* they did but not *what they held*, so you cannot be sure which situation to credit.

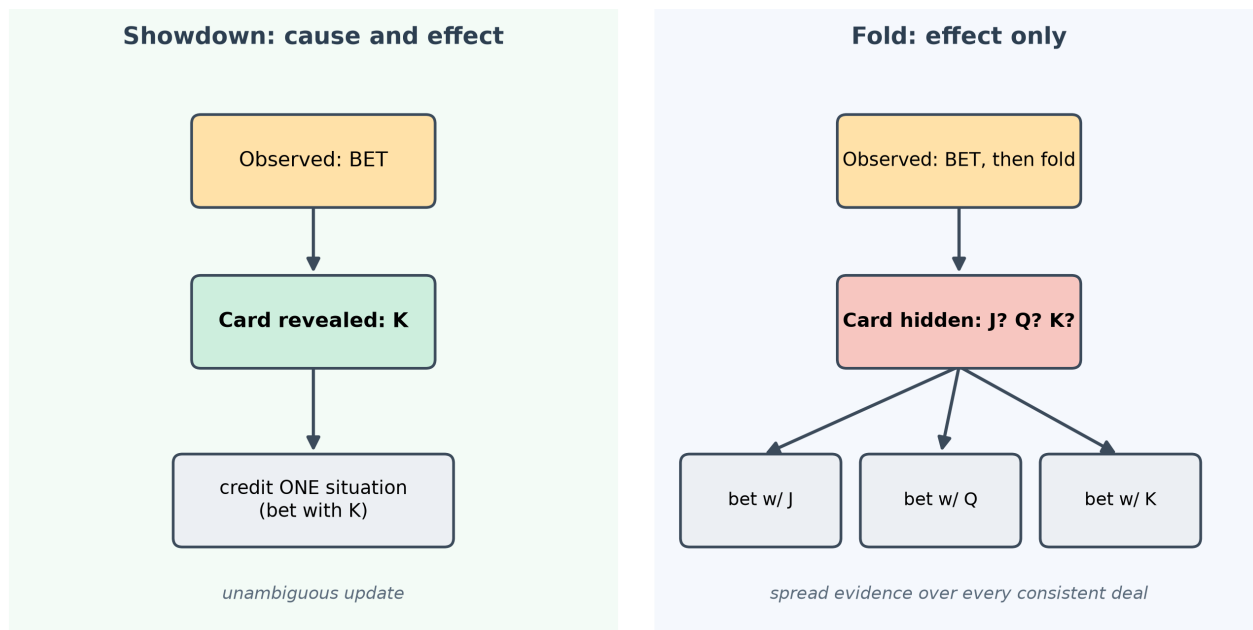


Figure 3: Partial observability: a showdown reveals the opponent’s private card, pinning the action to one situation; a fold hides it, forcing the model to spread the evidence across every hand the opponent might have held.

The consequence is a genuine theoretical limit, not an implementation nuisance: **without ever observing the opponent’s private information, you cannot learn beyond your prior**. If folds are all you ever see, more hands do not help — the evidence is fundamentally ambiguous. Showdowns are precisely the information that breaks the ambiguity.

How does a model cope with a fold? By reasoning over **all the hands the opponent might have held**. Concretely, it enumerates the deals consistent with what was actually seen, replays the opponent’s decisions under each hypothetical private card, and spreads the evidence across them — weighting each hypothesis by how plausible it is. A showdown collapses that set to a single possibility; a fold leaves it spread. This “marginalize over the hidden card” move is the technical heart of every model in the next two sections, and — not coincidentally — the formal object the consistency theory of Section 5 is built around.

7.4 4. Three Models, One Interface

There is no single “opponent model.” There is a family, trading off convergence speed, robustness to surprises, interpretability, and compute. This project implements and compares three points on that spectrum, deliberately built behind **one shared interface** — each consumes a stream of observed hands and emits a predicted opponent strategy — so that the *same* downstream best response can be applied to all three and the comparison is apples-to-apples.

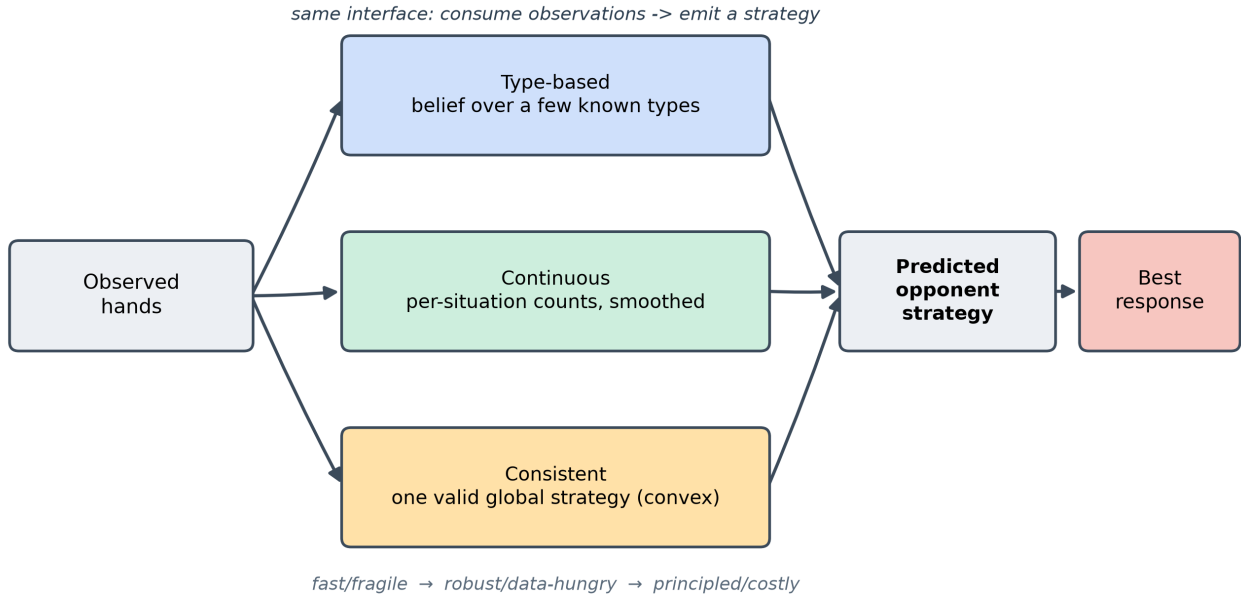


Figure 4: Three models on one interface: a discrete belief over types, a free-form per-situation count, and a globally-consistent sequence-form estimate — all emit a predicted opponent strategy consumed by the same best-response step.

- **Type-based.** Maintain a belief (a probability distribution) over a fixed menu of opponent types, update it by Bayes’ rule (Section 2), and report the posterior-weighted average strategy. It converges *extremely* fast **when the opponent is in the menu**, and its output is interpretable (“80% rock, 20% maniac”). Its weakness is equally clear: if the real opponent resembles none of the types, the model has no honest way to say so (Section 6).
- **Continuous.** Drop the menu. Estimate the opponent’s action probabilities *directly at each situation* by counting what they did there, smoothed by a Dirichlet prior (Section 2). It can represent **any** strategy, including blends and novel styles, so it is the robust choice against unknown opponents. The cost is data: it learns each situation independently, with no structural sharing, so it needs many more observations — and, as Section 7 shows, that hunger has teeth on the larger game.
- **Consistent.** Estimate a single *globally consistent* strategy — one guaranteed to be a valid strategy over the whole game tree — rather than each situation in isolation. This is the most principled model and the most recent; it is important enough, and central enough to the thesis, to get its own section (Section 5).

The following table is the mental map to carry forward:

Model	Represents	Converges	Robust to out-of-menu?	Interpretable	Compute
Type-based	a few known types	fastest	no	high (named types)	cheap
Continuous	any per-situation strategy	slower (data-hungry)	yes	medium (per-situation)	cheap
Consistent	one valid global strategy	provably to the truth	yes	medium	expensive (optimization)

The obvious question — *which one should the thesis use?* — does not have a single answer, and that is itself a finding. The natural target is a **hybrid**: lean on structural priors (types) when data is sparse, and converge toward the consistent estimate as evidence accumulates. This chapter builds and measures the endpoints so that the hybrid has firm ground to stand on.

Read more: Bard, N. (2013). “Online Implicit Agent Modelling.” *AAMAS* — the explicit-vs-implicit axis that frames this taxonomy.

7.5 5. The Consistency Problem and the Sequence-Form Fix

This section is the theoretical frontier of the step and the piece the thesis most directly extends. It answers a question the earlier models quietly beg: *if a model fits the observed behavior well, have we actually recovered the opponent’s true strategy?* The surprising answer is **not necessarily** — **even with infinite data** — and there is a principled fix.

7.5.1 The flaw: fitting the mean is not the same as finding the truth

The classical Bayesian recipe responds to the posterior **mean** strategy (Section 2). Call an opponent-modeling method **consistent** if its estimate converges to the opponent’s true strategy σ^* as observations accumulate against a fixed opponent. One would hope the classical recipe is consistent. It is not.

The cleanest counterexample is rock-paper-scissors. Suppose the modeler reasons over a set of *sampled* candidate strategies and always responds to some average of them. If the true strategy $\sigma^* = (0.8, 0.1, 0.1)$ lies **outside** the region those samples can combine to, the model — being always a weighted average of the samples — simply **cannot reach it**, no matter how much data arrives. That much is intuitive. The deeper result is that the method can fail **even when the truth lies inside** the achievable region: with the true strategy sitting at the center of three samples that average to it, the posterior weight on the single best-fitting sample grows without bound relative to the others, so asymptotically the belief

collapses onto one sample rather than settling on the true mixture. Fitting the data ever more tightly, the model converges to the *wrong* strategy.

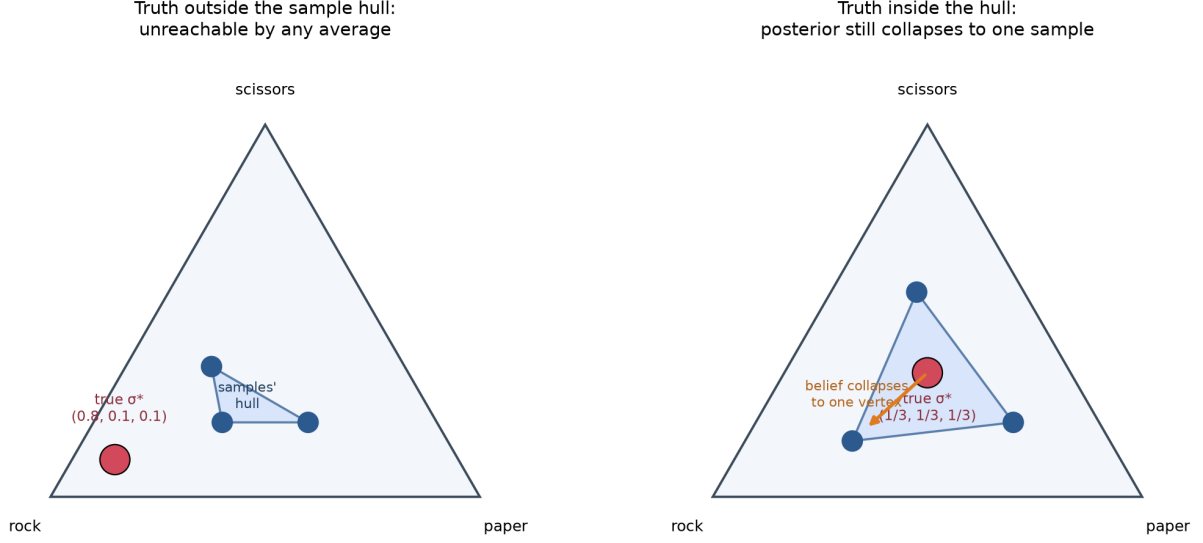


Figure 5: Why fitting the mean can miss the truth: the modeler can only ever produce points in the shaded hull of its samples. A true strategy outside the hull is unreachable; even one inside is abandoned as the posterior collapses onto a single vertex.

This matters because it undercuts the intuitive safety net “just collect more data.” Consistency is not automatic; it has to be engineered.

7.5.2 The fix: a convex program in sequence form

The remedy reformulates the estimate so that “a valid strategy” and “fits the observations” become constraints in a single, well-behaved optimization. The key change of variables is the **sequence form**: instead of per-situation action probabilities, represent the opponent’s strategy by **realization weights** y_r over action *sequences*. The legal strategies are then exactly those satisfying a set of **linear** constraints $Fy = f$, $y \geq 0$ — polynomial in the size of the game tree, rather than exponential.

Partial observability (Section 3) enters through an **observability function**: for each observed hand, the set of trajectories consistent with what you actually saw (a fold hides the opponent’s card and admits many trajectories; a showdown reveals it and admits one). The likelihood of an observation is a sum of realization weights over that consistent set. Putting a Dirichlet prior on the weights and maximizing the log-posterior gives the program

$$\max_y \sum_r (\alpha_r - 1) \log y_r + \sum_t \log \left(\sum_{r \in o(\ell_t)} q_r y_r \right) \quad \text{s.t.} \quad Fy = f, y \geq 0,$$

where $o(\ell_t)$ is the consistent-trajectory set for observation t and q_r are (normalized) chance probabilities. The decisive property: when $\alpha_r \geq 1$ this objective is **concave** — the first term is concave, the second

is a log of an affine function, and the constraints are affine — so it is a **convex** problem with **no local optima**. Any solver that finds a local optimum has found the global one. A standard **projected gradient** scheme suffices: take a gradient step, then project back onto the constraint set. In pseudocode the loop is unremarkable, which is the point —

```
initialize y feasible (F y = f, y >= 0)
repeat until converged:
  g <- gradient of the (negative) log-posterior at y
  z <- y - eta * g                                # gradient step
  y <- project z onto { y : F y = f, y >= 0 }    # convex projection
```

This method returns the **mode** of the posterior (its single most probable point), not the mean. That is the crucial trade: the mode is *consistent* — under mild conditions (the truth has positive prior density, distinct strategies produce distinguishable observations, and every opponent situation is visited infinitely often) the estimate provably converges to σ^* — whereas the payoff-optimal mean is generally intractable to compute exactly. So the design axis underneath the whole step is **mean versus mode**: payoff-optimal-but-intractable versus tractable-and-consistent.

7.5.3 What we found, honestly

We implemented this consistent estimator and verified it on Kuhn strategy recovery, where it performs as advertised: its recovered strategy sits very close to the truth (total-variation distance roughly **0.004 to 0.021**), matching or beating the continuous model’s recovery on the same game. That confirms the machinery and the theory on the small testbed.

We did **not** run it inside the online exploitation loop, nor on the larger game. The reason is the reason the literature itself flags: the estimate is the solution of an optimization that is **re-solved as observations accumulate**, and that cost grows with history — from a fraction of a second early on to many seconds per refit once tens of thousands of hands are in hand. For a real-time, hand-by-hand match that is impractical in its naive form. Rather than a gap, we treat this as the *empirical answer* to a question the step poses explicitly — *is a per-update convex solve fast enough for real-time play?* — namely **not without incremental methods** (warm-starting each solve from the last, caching the per-hand terms), which is exactly the kind of approximation Step 8 takes up. The understanding and the recovery result are what the thesis needs from this model now; the online engineering is future work.

Read more: Ganzfried, S. (2025). “Consistent Opponent Modeling in Imperfect-Information Games.” *arXiv:2508.17671*.

7.6 6. When a Model Is Confident and Wrong

Section 5 was about a subtle failure with infinite data. This section is about a blunt failure with finite data, and it is the one that most directly motivates *safe* exploitation. It is a different failure from inconsistency:

here the problem is a **misspecified menu** — an opponent the type-based model simply cannot represent — and how the model behaves when cornered.

We built a hidden opponent that is a 50/50 per-action blend of the rock and the maniac — deliberately **none** of the four candidate types. A reasonable guess is that the posterior would split its belief between the two nearest types. It does not. A product-of-likelihoods posterior concentrates on the *single best explanation*, so the belief lurches from one type to another and finally commits, hard, to **Nash** — the one candidate that assigns real probability to *both* betting and checking a middling hand, and so is never fatally contradicted. The model ends up **confident** (posterior near 1.0 on a single type) and **wrong** (that type is not what it is playing).

This exposes a trap worth stating plainly: the posterior is a **relative** quantity. “Ninety-five percent Nash” means “best fit *among these four candidates*,” not “good fit in absolute terms.” Measuring the winner’s *absolute* fit gives the game away — the same Nash hypothesis that scores well against a genuine Nash opponent scores far worse against this blend, while reporting equal confidence on paper. A small stereotype menu has no honest way to say “none of the above.”

7.6.1 How long can it stay wrong? A robustness sweep

Because confidently-wrong beliefs are dangerous, we stress-tested the phenomenon over **300 random seeds of 500 hands each**, separating two things a naive metric conflates: *slow convergence* from *falling after convergence*.

Measurement (300 seeds x 500 hands)	Result
Correct long-run winner by hand 500	300 / 300 (100%)
Ever falls back after taking the lead for good	0 / 300 (never)
A wrong type still leading past hand 100	40 / 300 (~13%)
A wrong type still leading past hand 200	14 / 300 (~5%)
Hand at which the truth locks in for good	median 23 - 90th pct 125 - worst 461

The good news: the belief eventually lands on the closest representable strategy every time, and once it locks in it never falls. The sobering news is the middle rows: in a meaningful minority of runs the model held a **confident wrong belief for well over a hundred hands** — long enough that an agent best-responding hard to it would have been adjusting to beat a phantom. This is the empirical face of the exploitation-versus-safety tension: a model can be trusted *eventually*, but “eventually” is sometimes 200 hands away, and acting on the interim belief as if it were certain is how you hurt yourself.

The honest fix is not a bigger menu — it is to stop asking “which single type?” and to lean on models that can represent blends (the continuous and consistent models of Sections 4-5), and to **scale how hard you exploit to how well-earned the read is**. That principle is the through-line into the next two sections and into Step 8.

7.7 7. From Model to Money — best response, the ceiling, and a self-inflicted leak

A model is only useful if acting on it wins. To measure that cleanly we feed each model’s estimate into the **same** exact best response and play full matches on both games, bracketing every result between two analytical yardsticks computed by hand-verified math, not simulation:

- **Nash EV** — what safe equilibrium play earns against that opponent (you cannot do worse than this if the read is useless);
- **ceiling** — the exact best-response value, the most that is extractable if you knew the opponent perfectly.

A good exploiter sits between the two and climbs toward the ceiling. First, a sanity check that the apparatus is trustworthy: the solver reproduces Kuhn’s exact equilibrium value of $-1/18$ (residual exploitability ≈ 0.002 , essentially unexploitable), and an exact best response beats uniform play by the predicted margins (+0.500 on Kuhn, +2.087 on Leduc).

Kuhn — both models reach the ceiling.

Opponent	ceiling	type-based	continuous
Rock	0.167	0.168	0.168
Maniac	0.333	0.337	0.330
Nash	-0.055	-0.055	-0.053

(A pure “always fold to a bet” opponent is even more exploitable — a ceiling of 0.975, reached to within 0.966 — but it is not one of the recurring types defined above, so it is omitted here for consistency.)

Leduc — the model class starts to matter. Leduc Hold’em adds a second betting round and a shared community card, roughly two orders of magnitude more situations than Kuhn — still exactly solvable, but large enough to separate the models.

Opponent	ceiling	type-based	continuous
Level-1	3.056	3.061	2.672
Maniac	2.177	2.199	2.038
Calling station	1.464	1.451	1.434
Rock	0.937	0.912	0.848
Nash	-0.083	-0.085	-0.175

Two results carry the message, and the second is the important one:

1. **You cannot exploit an equilibrium.** Against the Nash opponent every model earns approximately the (negative) game value and never more. The ceiling *is* essentially the game value. This confirms the exploitation elsewhere is real and not an artifact of the harness.

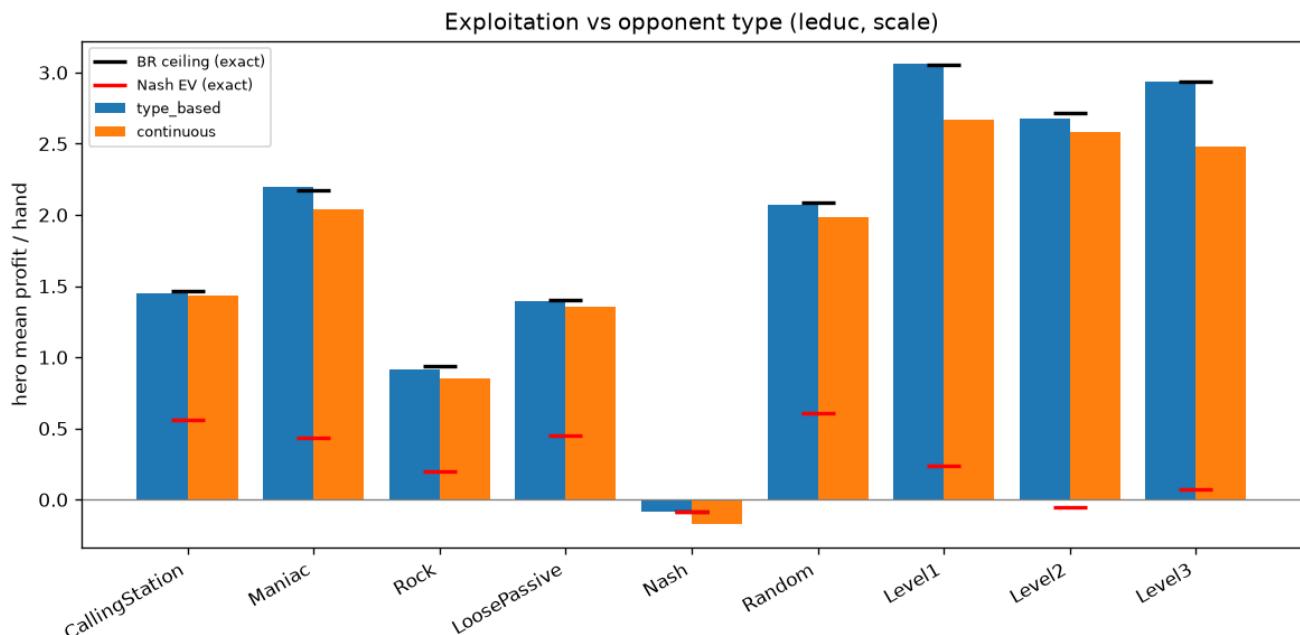


Figure 6: Exploitation against each Leduc opponent: the type-based model (which fits these opponents) hugs the exact best-response ceiling, while the continuous model tracks close but sits below for the hardest-to-fit types — and dips below the safe baseline against Nash.

2. **A confident-but-underfit model makes *you* exploitable.** On Leduc the continuous model *loses* to Nash — -0.175 against a -0.083 ceiling. With imperfect data over Leduc’s many situations it best-responds to a *wrong* estimate of an opponent who cannot be exploited at all, and in doing so opens a leak in its **own** play. The undersampling that merely costs it a little against exploitable opponents turns actively harmful against an unexploitable one.

This is the exploitation-versus-safety tension in a single number. When the model class fits, best response extracts the full theoretical value; when it does not, the agent both leaves money on the table *and* — more dangerously — hands some back. That second cost is precisely what Step 8’s safety mechanism (bounding the deviation from Nash by the model’s own confidence) exists to prevent.

The differences here are not seed luck. Across five seeds the standard error of per-hand profit is tiny relative to the effects: the type-based model is statistically indistinguishable from the ceiling on every type in both games, and the continuous model’s Leduc shortfall and its Nash self-leak are many standard errors wide.

Read more: Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM EC* — the safety half of the dial, and the anchor for Step 8.

7.8 8. Adapting to Change — non-stationary opponents

Everything so far assumed a fixed opponent. Real opponents drift and adapt, and a model that learned patiently for ten thousand hands is worse than useless the moment its subject changes style — it is now *confidently* describing a person who no longer exists. This is the open frontier the thesis is positioned to attack, and the step includes a first controlled probe.

The full adaptive agent runs the loop of Section 2 end to end: **observe** hands, **update** the model, periodically rebuild the hero strategy as a best response to the current estimate (optionally blended toward Nash for safety), and **act**. To handle change, it adds a detector: a lightweight statistical monitor on the opponent’s aggression that watches for a regime shift and, on firing, **resets** the model and drops back to safe play while it re-learns.

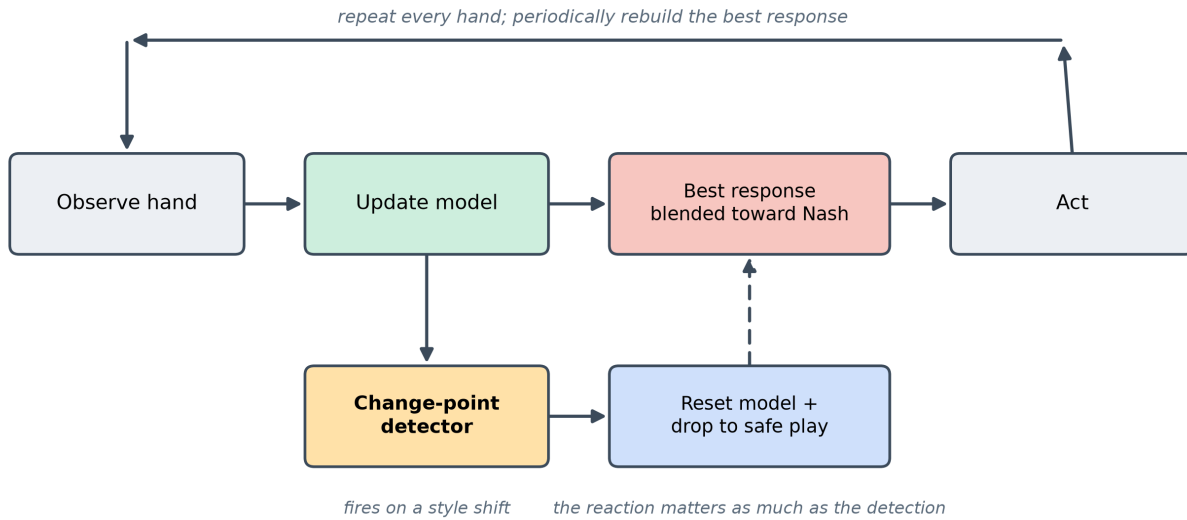


Figure 7: The adaptive exploitation loop: observe, update the model, best-response (blended toward Nash for safety), and act; a change-point detector can reset the model when the opponent’s style shifts.

We switched the opponent’s style at the midpoint of a 20,000-hand match and compared a **static** model (never forgets) against one with **change-point forgetting**. The result is **scenario-dependent**, and that is exactly the finding:

Game	Style switch (at the midpoint)	static (after switch)	change-point (after switch)
Kuhn	rock -> maniac	-0.116	+0.226
Leduc	rock -> maniac	+1.940	+0.525

- **On Kuhn, forgetting wins.** The strategy learned against a rock is bluff-heavy; unleashed on a maniac who calls everything, it *actively loses* (the static model goes negative). Detecting the switch and re-learning recovers to a healthy profit.

- **On Leduc, forgetting loses.** The maniac there leaks over two chips a hand, so a continuously-adapting model exploits it handsomely without any reset — while the detector, too eager, fires dozens of **false alarms** during the stable stretches (around sixty resets against a single true change), each one throwing away hard-won data and dropping to safe play.

The lesson is not “change detection is good” or “bad” — it is that **the reaction to a detected change matters as much as the detection**. A trigger-happy detector paired with a full reset-to-safe can cost more than staleness when the new opponent is exploitable enough that staleness is cheap. Cheaper, gentler responses — partial forgetting instead of a hard reset, a less nervous detector — are the clear next step, and they connect directly to Step 8’s confidence-scaled exploitation. (This experiment was run at a single seed, so read the *direction* as robust and the exact magnitudes as illustrative.)

7.9 9. Connections and Forward Pointers

What this step establishes. A good opponent model is *necessary but not sufficient* for profitable, safe adaptation. When the model class fits the opponent, best response reaches the exact extractable ceiling — the full theoretical value of modeling is realized. But under partial observability and limited data, an underfit model both leaves money on the table and, against an opponent who cannot be exploited, best-responds to a phantom and *loses*. Non-stationarity adds a second edge: forgetting a stale model helps only when staleness is actively harmful, and a careless detector’s false alarms can cost more than they save. One principle recurs across all of it: **exploitation must be scaled to how well-earned the read is**.

Backward connections. This step is the mirror image of the equilibrium work that preceded it. Where an equilibrium strategy *ignores* the opponent’s identity by design, an opponent model is the machinery that *uses* it — the two are the endpoints of the safety-exploitation spectrum. The sampling machinery that earlier steps used to traverse a game tree according to the *current* strategy reappears here to reason over an opponent’s actions according to the *inferred* model. And the idea of grouping many concrete situations into a few representative buckets — abstraction — is exactly what the type-based model does in strategy space.

Forward to Step 8 and the thesis. This step built the **sensor**; Step 8 builds the **actuator** — the mechanism that turns a model into *safe* exploitation, bounding how far you deviate from equilibrium by how much your read has earned. The continuous model’s self-inflicted leak against Nash (Section 7) is the empirical case for that mechanism; the confident-but-wrong sweep (Section 6) sets its budget; the consistency theory (Section 5) is the principled backbone the framework extends.

Open questions carried forward.

- *Consistency at real-time speed.* The consistent model is principled and accurate but its per-update convex solve does not yet fit an online match. Incremental solving (warm starts, cached terms) or Step 8’s subgame approximations are the routes to a usable real-time version.

- *Non-stationarity, properly.* All three models assume a stationary opponent for their guarantees. Confidence-scaled forgetting and better change signals are the immediate next work; a full theoretical treatment is a thesis-scale target.
- *Out-of-menu opponents.* Well-specified detection is the easy case. The interesting regime is opponents *outside* any menu — mixtures, drift, adversaries — where the non-parametric and sequence-form models should separate from the type-based one, and where an explicit “none of my types fit” signal becomes its own research question.
- *From one opponent to many.* Against several opponents at once, the jointly-optimal response is not the combination of individual best responses, and the convex guarantee of Section 5 no longer holds. Multi-opponent modeling is the bridge to the later multi-agent steps.

Read more: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems*, Ch. 7 “Learning and Teaching” — the learning-in-repeated-games framing under all of the above, including the tension that your actions both *exploit* and *teach* the opponent.

Step 8 Summary — Safe Exploitation in Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

8	Step 8 — Safe Exploitation in Imperfect-Information Games	1
8.1	1. Why Safe Exploitation — the other half of the dial	2
8.2	2. Exploitation as Constrained Optimization	3
8.3	3. What “Safe” Means — three definitions, in increasing realism	5
8.4	4. The Engine — sequence-form LP and constraint generation	6
8.5	5. Restricted Nash Response and the Bang-Bang Frontier	7
8.6	6. Ganzfried on Kuhn — safe <i>and</i> profitable (the core result)	10
8.7	7. Real-Time Safety — subgame gadgets (SES)	12
8.8	8. At Scale — Kuhn works; Leduc breaks globally, holds locally	13
8.9	9. Connections and Forward Pointers	16

8 Step 8 — Safe Exploitation in Imperfect-Information Games

This is a ground-up chapter on *safe* opponent exploitation: the problem, the mathematics, the family of methods, and a set of controlled experiments run on two small poker games. It serves two purposes — a self-contained refresher while later steps build on it, and a primary source for the Part-1 thesis synthesis. It is written to be read on its own; no prior familiarity with the project’s code is assumed. All experimental numbers reported here were **measured** on reproducible runs of the two testbeds (Kuhn Poker and Leduc Hold’em) and are bounded, wherever possible, by *exact* analytical references rather than simulated ones. Where a run contradicted what theory led me to expect, I say so and reconcile it — those gaps are the most instructive parts of the step.

Where this sits in the thesis. Step 7 built a **sensor**: a model that watches how a specific opponent plays and estimates their strategy. It also exposed the danger of acting on that sensor naively — a confident-but-underfit model, best-responded to, can open a hole in your own play (on Leduc it actually *lost* to an unexploitable Nash opponent). Step 8 builds the **actuator**: the mechanism that turns a model into profit **without becoming exploitable**. That actuator is the second half of the thesis’s Behavioral Adaptation Framework (Contribution #1), and pinning down *exactly where its guarantees depend on the two-player zero-sum assumption* is the launch point for the multi-agent extension (Contribution #2).

8.1 1. Why Safe Exploitation — the other half of the dial

A **Nash equilibrium** strategy is built never to lose in the long run. In a two-player zero-sum game that guarantee is exact: the strategy has a fixed *value* v^* , and no opponent, however clever, can push you below it. Opponent modeling (Step 7) buys you the ability to *deviate* from that safe strategy to punish a specific opponent’s mistakes. The obvious way to cash a model in is to compute the **best response** to it and play that. The trouble is that a best response is usually **wildly exploitable itself**: to punish “you always fold to a bet” it stops bluffing certain hands entirely, and a smarter opponent — or the same opponent, done pretending — walks straight through the hole it opened.

Picture a **dial**. Turn it all the way to *safety* and you play Nash: unbeatable, but you never punish a weak opponent. Turn it all the way to *exploitation* and you best-respond hard to your current read: maximum profit **if** the read is right and the opponent stays put, but a gaping hole if you are wrong, your sample was small, or you were being *sandbagged* (deliberately fed a weak style to bait a big deviation). **Safe exploitation** bolts a *governor* onto that dial: lean toward exploitation as far as you like, but never past the point where a worst-case adversary could drag you below your safe baseline.

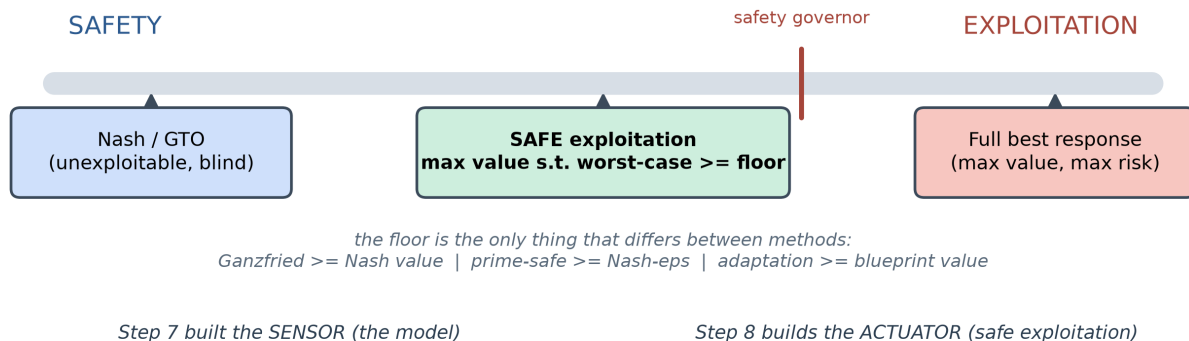


Figure 1: The safety-exploitation dial with a governor. Pure Nash is unexploitable but blind; a full best response extracts the most value but is maximally risky. Safe exploitation operates in between, capped by a floor on the worst-case value. Step 7 built the sensor (the model); Step 8 builds the actuator (safe exploitation). The only thing that differs between methods is where the floor sits.

The reason this is not paranoia is a number from Step 7 and again from this step: the full best response to the tight “Rock” style on Kuhn earns **+0.167 per hand**, but its **worst-case value is −0.5** — an adversary who best-responds back can take half a chip a hand off it. Nash’s worst-case, by contrast, is the game value itself. Safe exploitation is the discipline of keeping most of that +0.167 upside while refusing the −0.5 downside.

What “safe” means, informally. Your *expected value over the match* must never fall below a chosen floor, **no matter what the opponent does**. It is a statement about the worst case, not about any single hand — you will still lose hands; the guarantee is on the long-run adversarial average. The whole design question of the step is *which floor*, and how to compute a strategy that respects it while extracting

as much as possible above it.

Read more: Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Trans. Economics and Computation* — the paper that first made “exploit but never lose to the baseline” a theorem.

8.2 2. Exploitation as Constrained Optimization

The single most useful idea in this step is that **every safe-exploitation method is the same optimization problem** with one part swapped out. In words:

maximize the hero’s expected value against the opponent model, **subject to** a safety floor on the hero’s worst-case value.

To make that a *computable* program, represent the hero’s strategy in **sequence form**: instead of per-situation action probabilities, use the **realization plan** x , which assigns a weight to each of the hero’s action *sequences* (root-to-her chains of their own choices), subject to the linear **treeplex** constraints (the empty sequence has weight 1; at each information set the children’s weights sum to the parent’s; all weights are non-negative). The reason to pay this change-of-variables tax is decisive: against a **fixed** opponent policy, the hero’s expected value is **linear** in x ,

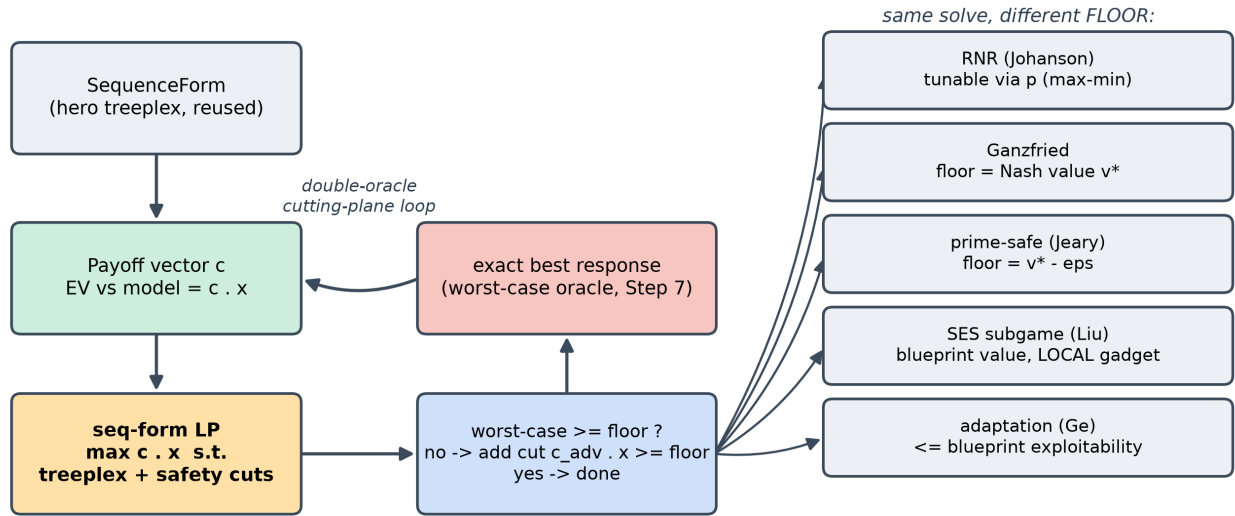
$$\text{EV}(x) = \sum_{\text{terminals } z} [\text{chance}(z) \cdot \text{opp-reach}(z) \cdot u_{\text{hero}}(z)] x_{\text{seq}(z)} = c \cdot x,$$

because the hero’s own action-probability product along any line *is* x at that line’s sequence. So “maximize EV against the model” is a **linear objective** $c \cdot x$, the treeplex is a set of **linear constraints**, and the whole method is a **linear program**. A full best response is simply $\max_x c \cdot x$ over the treeplex — which lets us cross-check the LP against Step 7’s exact best-response code (they agree to 10^{-6}).

The safety floor is a constraint on the **worst-case** value — “for *every* opponent σ' , $\text{EV}(x, \sigma') \geq \text{floor}$.” That inner “for every opponent” is itself a best-response (a minimax), so we do not write it out as one giant bilinear program; we solve it by **constraint generation** (a double-oracle / cutting-plane loop): solve the LP, read the hero policy, call an **exact best response as the worst-case oracle**, and if the worst case is below the floor, add the single linear cut it implies and re-solve. This is finite (there are finitely many pure best responses), transparent, and debuggable.

The pay-off of seeing it this way is conceptual economy: **five methods, one engine**. They differ only in the floor, which is the subject of the next section. And because both the objective (the payoff vector) and the safety oracle reuse Step 7’s *validated* best-response code, the whole of Step 8 rests on one already-trusted primitive rather than a new pile of math.

Read more: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems*, §3.4 (computing equilibria) and §4.6 (computing best responses) — the sequence-form machinery underneath



one validated primitive (Step 7's exact best response) powers BOTH the objective and every safety check

Figure 2: One LP engine, five safety floors. The sequence-form treeplex gives a linear objective (EV vs the model); a constraint-generation loop calls an exact best response as the worst-case oracle and adds a safety cut until the floor is met. RNR, Ganzfried, prime-safe, SES and adaptation are the SAME solve with a different floor — and one validated primitive (Step 7's best response) powers both the objective and every safety check.

every LP in this chapter.

8.3 3. What “Safe” Means — three definitions, in increasing realism

“Safety” sounds absolute, but it is not one thing. Three definitions matter, and the line of research is essentially a *weakening* of the demand to make it achievable in practice.

(a) Ganzfried safety (2015): never earn less than the game value. With a *perfect* Nash equilibrium σ^* as baseline, require

$$\text{EV}(x, \sigma') \geq v^* \quad \text{for all opponents } \sigma'.$$

This is the strongest and cleanest notion. Its proof rests on the **minimax theorem**: in a two-player zero-sum game a Nash strategy *guarantees* the value v^* against any opponent, so one can deviate toward exploiting the model within the “slack” the opponent’s mistakes create (their *gifts*) and never fall below v^* . The catch is the premise: a *perfect* Nash equilibrium, which is uncomputable in any large game.

(b) Prime-safe / -safety (Jeary & Turrini 2023): correct for an imperfect baseline. Every real baseline is an **-equilibrium** (from abstraction and finite compute — the subject of Step 4), so it is itself exploitable by some amount ε . Anchoring to the exact v^* is then unjustified; prime-safe lowers the floor by exactly the baseline’s own exploitability,

$$\text{floor} = v^* - \varepsilon, \quad \varepsilon = \text{exploitability}(\text{baseline}) \geq 0,$$

i.e. “never earn less than the worst case of the strategy you were going to play anyway.” The ε must be **measured**, not assumed — a point the experiments take literally.

(c) Adaptation safety (Ge et al. 2024): be no more exploitable than your blueprint. The most practical notion. An exploiting strategy is *adaptation-safe* iff

$$\text{exploitability}(x) \leq \text{exploitability}(\text{blueprint}) \iff \text{worst-case}(x) \geq \text{worst-case}(\text{blueprint}).$$

Because the blueprint is already ε -exploitable, this is strictly weaker than Ganzfried’s absolute floor (weaker by exactly ε), and therefore *achievable* where strict safety is not. Its danger is the mirror image: if the blueprint is *terrible*, “no worse than the blueprint” is trivially satisfied by almost anything — so adaptation safety is only meaningful with a reasonable baseline (an open requirement I flag rather than resolve).

Safety notion	Informal floor	Needs	Weakness
Ganzfried (2015)	$\geq v^*$	a <i>perfect</i> Nash baseline	perfect Nash is uncomputable at scale
Prime-safe (2023)	$\geq v^* - \varepsilon$	the baseline’s measured exploitability	you must measure ε honestly
Adaptation (2024)	$\leq$ blueprint exploitability	any blueprint	vacuous if the blueprint is bad

8.3.1 Where the two-player zero-sum assumption hides — the thesis attack point

Every floor above rests on one fact: **in a two-player zero-sum game, a Nash strategy secures the value v^* against any opponent** ($\min_{\sigma'} \text{EV}(\sigma^*, \sigma') = v^*$). That is what makes “deviate toward the model but never below the floor” a coherent, enforceable constraint, and it comes straight from the minimax theorem. In an $N > 2$ -**player** game this collapses: a Nash strategy does *not* guarantee a fixed value against arbitrary opponents (the others can coordinate, and their payoffs no longer sum to the negative of yours), so there is no single v^* to anchor to. **That is the open problem for Contribution #2** — whether an N -player analogue of the value guarantee exists, or whether a structural assumption (e.g. a coalition structure) must restore an anchor. This step does not solve it; it makes the failure *precise*, which is exactly what a thesis chapter needs.

Read more: Johanson, M., Zinkevich, M. & Bowling, M. (2007). “Computing Robust Counter-Strategies.” *NeurIPS* — Restricted Nash Response, the tunable ancestor of all of the above. · Jeary, J. & Turrini, P. (2023). “Safe Opponent Exploitation for γ -Equilibrium Strategies,” *arXiv:2307.12338*. · Ge, Z. et al. (2024). “Safe and Robust Subgame Exploitation....,” *ICML*.

8.4 4. The Engine — sequence-form LP and constraint generation

This section is the “how it is built.” The one primitive the whole step adds is a **HeroTreeplex** that (i) enumerates the hero’s sequences and treeplex constraints (reusing Step 7’s sequence-form code), (ii) builds the payoff vector c for a fixed opponent by one tree traversal, and (iii) solves the safety-constrained LP with SciPy’s HiGHS solver. Everything else is the constraint-generation loop, which is short enough to state in full:

```

c_model <- payoff vector of EV vs the opponent model      # linear objective
cuts     <- {}                                             # discovered safety cuts
repeat:
  x <- argmax c_model . x  s.t.  treeplex(x)  and  all cuts      # one LP solve
  pol <- behavioral strategy read out of x
  wc  <- worst_case_value(pol)                                # = -(opponent's exact best response) : the oracle

```

```

if wc >= floor - tol: return pol      # safe: done
adv <- opponent's exact best response to pol
add the cut  c(adv) . x >= floor - slack  to cuts      # a valid lower bound on the worst cas

```

Each generated cut is the payoff vector against a *specific* adversary best response; it is a valid linear lower bound on the true worst case, so adding cuts monotonically tightens the relaxed safety constraint toward the real one. Reading a behavioral policy back out of a realization plan x is the ratio $\beta(I, a) = x_{Ia} / x_{\text{seq}(I)}$ (uniform where the parent weight is ~ 0).

The **only** things that change between the five methods are the `floor` and, for the subgame method (§7), a set of *pins* that hold the hero’s play outside a chosen subgame equal to the blueprint. Ganzfried passes `floor = v*`; prime-safe passes `floor = v* -` ; adaptation passes `floor = worst_case(blueprint)`; RNR reformulates the objective as a max-min in a parameter p (§5).

One practical wrinkle worth recording (it cost real debugging on the runs): an *approximate* Nash’s self-play value can sit a hair **above** the game’s true achievable max-min, so requiring the exact `wc floor` can make the LP eventually infeasible and, on an unlucky cut path, return an *unsafe* strategy. The fix is a small feasibility slack on the cuts (`floor - slack`, with `slack = 5·10-6`, kept independent of the convergence tolerance) so the true max-min strategy always stays feasible. This is the kind of numerical detail that never appears in the papers but decides whether the method actually returns a safe strategy.

Read more: the constraint-generation / double-oracle idea traces to McMahan, Gordon & Blum (2003), “Planning in the Presence of Cost Functions Controlled by an Adversary,” *ICML* — the general recipe for “optimize against a worst case you discover as you go.”

8.5 5. Restricted Nash Response and the Bang-Bang Frontier

The oldest principled method, **Restricted Nash Response** (Johanson 2007), is the conceptual ancestor of all the rest and introduces the *tunable knob*. Its idea: compute the hero’s equilibrium against a *p-restricted* opponent — one forced to play the fixed model with probability p and free to play adversarially with probability $1 - p$. Sweeping p from 0 to 1 traces a path from Nash ($p = 0$) to full best response ($p = 1$). Because the free component best-responds to the hero, RNR is a **max-min**:

$$\text{RNR}(p) = \arg \max_x \left[p \cdot \text{EV}(x, \text{model}) + (1 - p) \cdot \min_{\sigma'} \text{EV}(x, \sigma') \right],$$

which we solve with the same cutting-plane machinery plus an auxiliary worst-case variable.

A flag worth keeping. The project’s own step notes originally described RNR as a *naive behavioral blend*, $(1-p) \cdot \text{Nash} + p \cdot \text{BR}$. That is a fine intuition tool but it is **not** Johanson’s algorithm; we implemented *both* and labelled them, because on a small game they behave completely differently. The naive blend traces a smooth line (Figure below, left): profit and exploitability rise together, near-proportionally, from Nash to full BR.

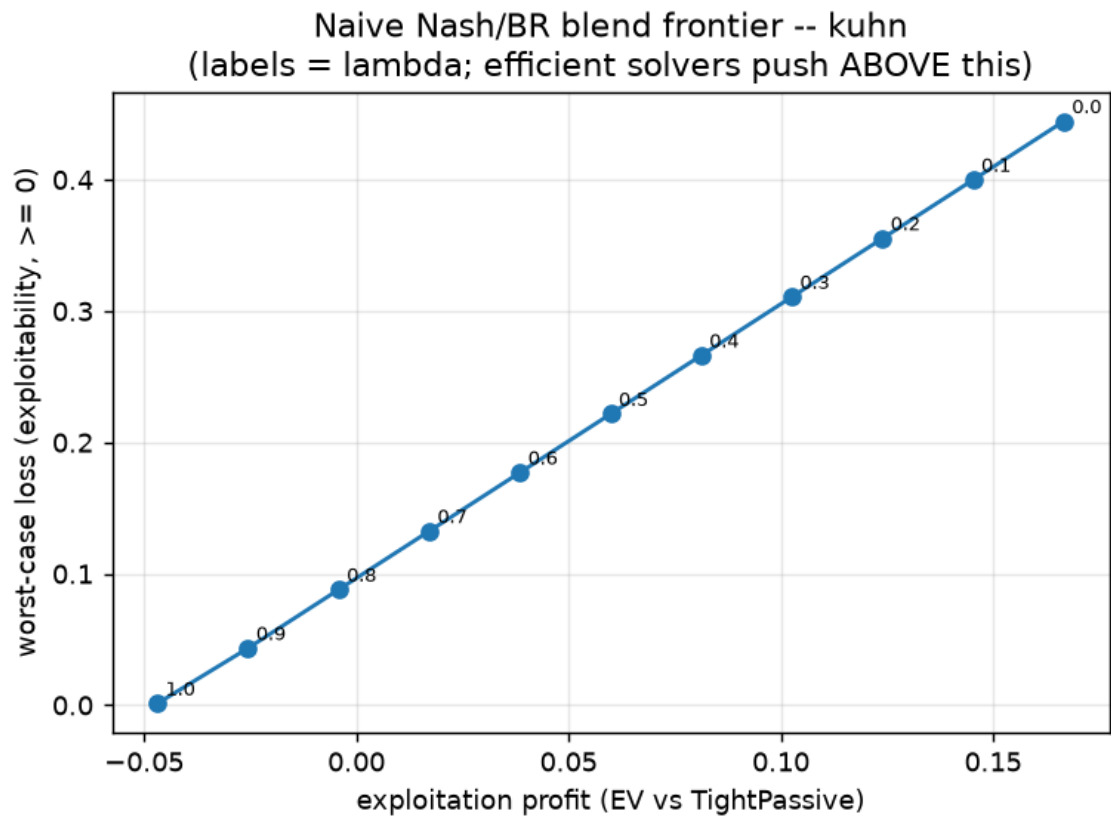


Figure 3: The naive Nash/best-response blend on Kuhn: a smooth exploitation-safety frontier where each increment of profit costs a near-proportional increment of your own worst-case loss.

The measured surprise — canonical RNR is bang-bang. I predicted the canonical sweep would be a smooth, monotone frontier that dominates the blend everywhere. It is not. On Kuhn versus the Rock, canonical RNR returns the **same safe strategy** for all $p \in [0, 0.6]$ (EV -0.044 , exploitability ≈ 0) and then **jumps straight to the full best response** at $p \approx 0.7$ (EV $+0.167$, exploitability 0.444). There are no intermediate points.

p	canonical RNR — EV	canonical — exploitability	naive blend — EV	naive — exploitability
0.0	-0.044	~ 0.000	-0.047	0.000
0.3	-0.044	~ 0.000	$+0.017$	0.133
0.6	-0.044	~ 0.000	$+0.081$	0.266
0.7	$+0.167$	0.444	$+0.103$	0.311
1.0	$+0.167$	0.444	$+0.167$	0.444

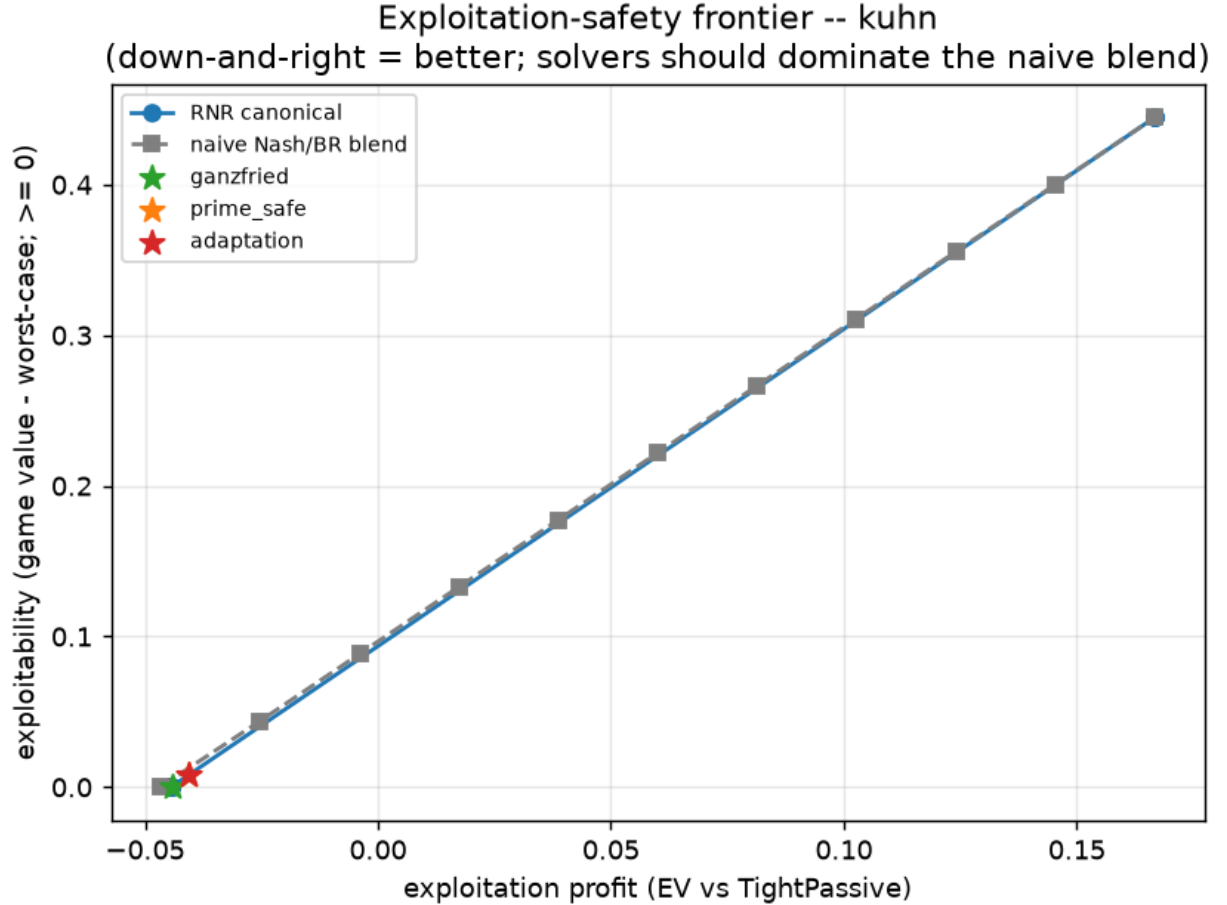


Figure 4: The exploitation-safety frontier on Kuhn with the LP operating points. The canonical RNR “curve” is only an interpolation between two achieved clusters (the safe corner and the full-BR corner); the smooth line is the dominated naive blend; the stars (Ganzfried, prime-safe, adaptation) sit at the efficient safe corner.

Why (*I checked before trusting the story*). The RNR objective is **linear** in x over a **polytope**, so its optimum is a **vertex** and switches vertices only when p crosses a critical ratio. Kuhn’s strategy polytope is tiny — few vertices — so the transition is a single jump. Johanson’s smooth curve is a *large-game* phenomenon (many vertices), or comes from the *data-biased* variant that makes p per-information-set. The naive blend looks smooth only because it linearly interpolates two fixed strategies — and it is **dominated at the safe corner** (at exploitability ≈ 0 the LP methods reach EV -0.044 versus the blend’s -0.047). The headline lesson survives — “choose *where* to deviate, not *how much* uniformly” — but the mechanism (“a smooth RNR dial”) is a big-game artifact, and in a small game the honest picture is a discrete safe-vertex $\rightarrow$ BR-vertex switch whose threshold *moves with how exploitable the opponent is*. That last point recurs in §6: against a very exploitable opponent, even $p = 0.5$ is already past the jump.

Read more: Johanson, M. & Bowling, M. (2009). “Data Biased Robust Counter Strategies.” *AISTATS* — makes the p knob depend on how much data supports each part of the strategy, which is what smooths the frontier in practice.

8.6 6. Ganzfried on Kuhn — safe *and* profitable (the core result)

The central experiment solves each method against a *perfect* model of each opponent type and scores it on **profit** (EV vs that opponent) and **safety** (worst-case value), with the Nash floor at $v^* = -0.056$ (Kuhn’s known first-player value, $\approx -1/18$). All numbers are exact (full-tree), from the scale run.

Opponent	metric	nash	full_br	rnr_0.5	ganzfried	prime_safe	adaptation
Rock (Tight-Passive)	EV	-0.047	$+0.167$	-0.044	-0.044	-0.040	-0.040
	worst-case	-0.056	-0.500	-0.056	-0.056	-0.063	-0.063
Maniac (LooseAggr.)	EV	$+0.118$	$+0.333$	$+0.278$	$+0.131$	$+0.151$	$+0.151$
	worst-case	-0.056	-0.167	-0.111	-0.056	-0.063	-0.063
AlwaysBeEV	EV	$+0.113$	$+0.333$	$+0.333$	$+0.115$	$+0.131$	$+0.131$
	worst-case	-0.056	-0.167	-0.167	-0.056	-0.063	-0.063
AlwaysPass (most exploitable)	EV	$+0.146$	$+0.975$	$+0.975$	$+0.222$	$+0.266$	$+0.266$
	worst-case	-0.056	-0.500	-0.333	-0.056	-0.063	-0.063

Opponent	metric	nash	full_br	rn timer_0.5	ganzfried	prime_safe	adaptation
Nash (control)	EV	−0.056	−0.055	−0.056	−0.056	−0.055	−0.055
	worst-case	−0.056	−0.333	−0.056	−0.056	−0.063	−0.063

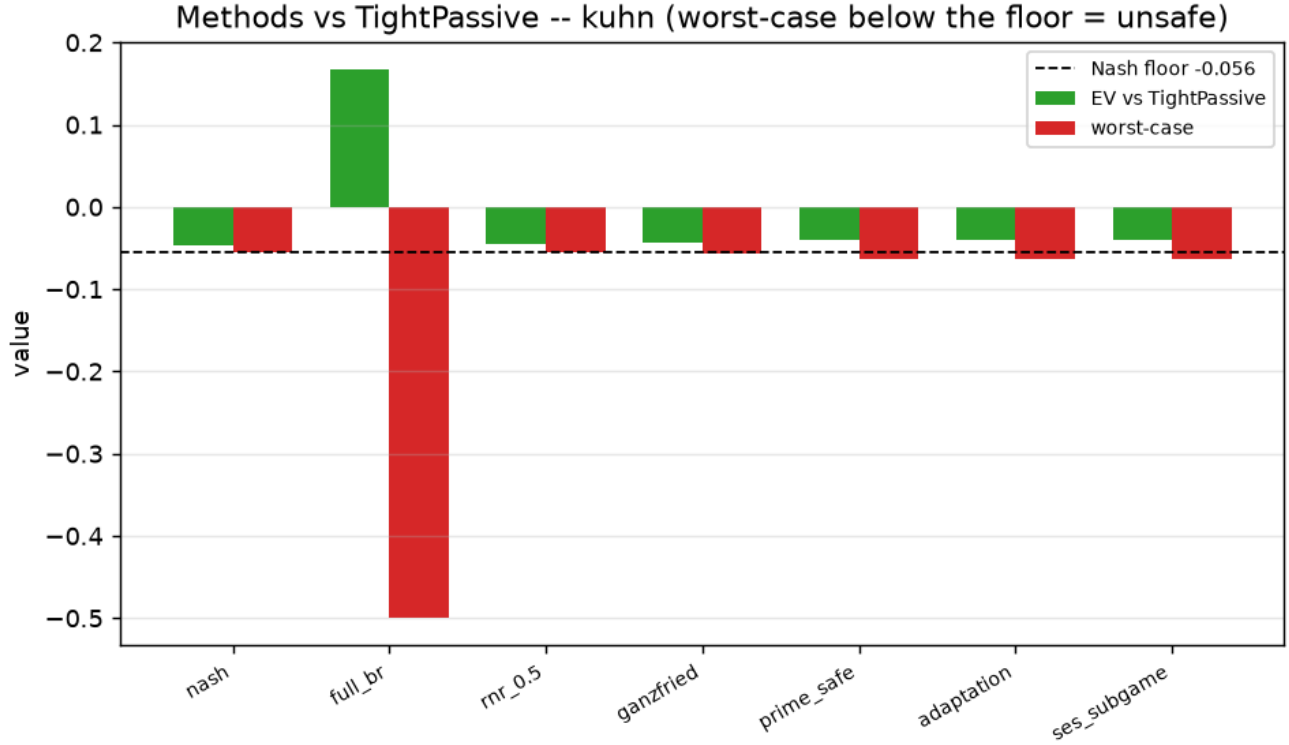


Figure 5: Methods versus the Rock on Kuhn: green is EV against the opponent, red is worst-case value, the dashed line is the Nash floor. Only the full best response’s worst-case (red) plunges to -0.5 , far below the floor; every principled method hugs the floor while still exploiting.

Three results carry the chapter.

1. **The full best response is the cautionary tale.** It always wins the most against a fixed model — up to **+0.975** against *AlwaysPass* — but its worst-case collapses to **−0.5**. It is the single unsafe method in the table, and it is unsafe against *every* opponent, because an adversary can always best-respond back to the hole it opens.
2. **Ganzfried is the safe-and-profitable sweet spot.** Against *every* opponent its worst-case stays at the Nash floor (safe within 10^{-3}) **and** it beats Nash’s own EV on every exploitable type — most vividly **+0.222** versus Nash’s **+0.146** against *AlwaysPass*, and **+0.131** versus **+0.118** against the *Maniac*. This is the result the step exists to produce: *you can exploit meaningfully while provably never dropping below equilibrium value*.
3. **A single global p is not a safety setting.** *rn timer_0.5* is safe against the Rock but has *already*

jumped to the full best response against the highly exploitable **AlwaysPass** / **AlwaysBet** (identical EV to **full_br**, worst-case -0.33 / -0.17 , unsafe). This is the §5 bang-bang threshold moving with opponent exploitability — and the concrete reason Ganzfried, which constrains the *value* rather than a *knob*, is the better primitive.

Prime-safe and adaptation spend a measured -budget. They lower the floor from v^* to $v^* - \varepsilon$, and the run *measured* the early-stopped-CFR baseline’s exploitability at $\varepsilon = 0.0074$; their worst-case comes out at $-0.063 = v^* - 0.008$ across every opponent, matching the -adjusted floor. Spending that budget, they earn a little more than Ganzfried ($+0.266$ versus $+0.222$ on **AlwaysPass**). They appear “unsafe” in the table only because the flag compares to v^* ; against their *own* floor they are safe by construction. (Prime-safe and adaptation coincide here because, for this baseline, $v^* - \varepsilon = \text{worst-case}(\text{blueprint})$ — the two floors are literally equal, which the run confirms rather than a bug.)

A measurement-resolution note, because it bit me. In the *smoke* run (30 000 CFR iterations) even the Nash policy is flagged “unsafe” — its worst-case is -0.0568 , i.e. 0.0013 below the exact v^* , just over the 10^{-3} tolerance. That is pure CFR approximation error, not a safety failure: at scale (200 000 iterations) the shortfall drops to 3×10^{-4} and Nash is correctly flagged safe. The lesson for the harness is that the safe-flag tolerance must exceed the baseline’s own approximation error, or the baseline trips its own test.

Read more: Ganzfried, S. & Sandholm, T. (2015), *op. cit.* — Theorem 1 is exactly the guarantee this table exhibits on Kuhn (worst-case $\geq v^*$), and its minimax step is the 2-player-zero-sum dependency of §3.

8.7 7. Real-Time Safety — subgame gadgets (SES)

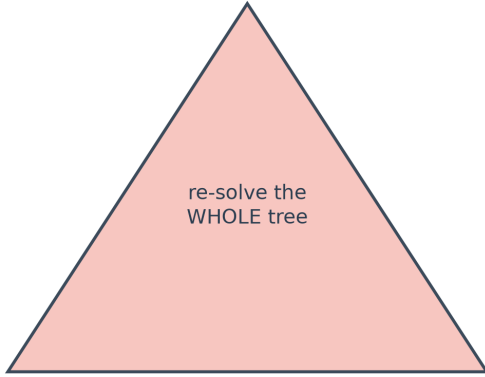
Ganzfried’s guarantee is *global*: the safety constraint is enforced over the **whole** game tree. That is fine on Kuhn, but in a real game you cannot re-solve the entire tree every time the model updates. The **Safe Exploitation Search** idea (Liu et al. 2022) makes safety a **local** property of a single **subgame**: play the Nash blueprint everywhere, but at a chosen subgame re-optimize the hero’s play to exploit the model — with a **gadget** that guarantees the local deviation can never make the hero worse off *globally* than the blueprint.

In sequence form the gadget is realized directly on the treplex: **pin** every hero sequence *outside* the subgame to its blueprint realization weight (so the hero plays the blueprint everywhere except the subgame), and set the safety floor to the blueprint’s own worst-case value. Because “play the blueprint inside the subgame too” is always feasible and attains exactly that floor, the LP can only do at least as well — the local exploit is safe by construction. The one requirement is that the subgame be **downward-closed** (once you are in it you stay in it), which holds for natural choices like “Leduc round 2” or “after a King flops.”

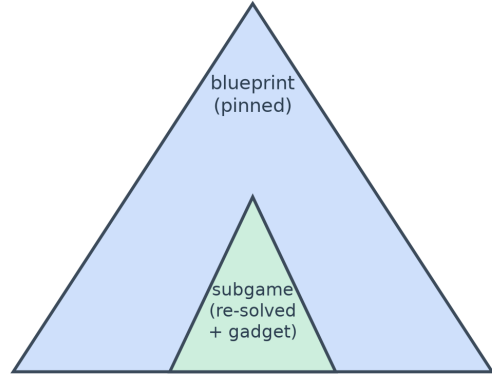
The importance of “local” is not aesthetic; §8 shows it is the difference between a solve that converges

GLOBAL safety (Ganzfried / prime-safe / adaptation)

LOCAL safety (SES subgame gadget)



*Leduc: capped at 40 iters, NOT converged
worst-case -0.64 .. -1.33 (grossly unsafe)*



*Leduc: converged in 194-350 iters
worst-case ~ -0.13 (near-safe), +0.25..+0.68 vs weak types*

Figure 6: Global versus local safety. Global methods re-solve the whole tree; on Leduc their constraint-generation loop did not converge within the iteration budget and left grossly unsafe strategies. The subgame method pins play outside a chosen subgame to the blueprint and re-solves only that subgame with a gadget — a far smaller problem that did converge and stayed near-safe.

and one that does not.

Read more: Liu, W. et al. (2022). “Safe Opponent-Exploitation Subgame Refinement.” *NeurIPS* (the gadget). · Ge, Z. et al. (2024), *op. cit.* — OX-Search bounds exploitation loss *per information set*, hardening the same idea against “teaching” attacks. · Milec, D., Kovařík, V. & Lisý, V. (2025). “Adapting Beyond the Depth Limit,” *arXiv:2501.10464* — using opponent-model information *past* the search horizon.

8.8 8. At Scale — Kuhn works; Leduc breaks globally, holds locally

Leduc Hold’em adds a second betting round and a shared community card — roughly two orders of magnitude more situations than Kuhn, still exactly solvable, but large enough to stress the methods. The Leduc run used an iteration-capped configuration (a 40-iteration budget on the constraint-generation loop, tolerance 10^{-2} , and the subgame set to the King-flop), recording for each cell whether the solve **converged** or hit the cap. Game value $v^* = -0.086$.

Opponent	metric	prime_safe /				
		nash	full_br	ses_subgame	ganzfried	adaptation
Rock	EV	+0.201	+0.937	+0.247	+0.624	+0.635
	worst-case	-0.089	-1.633	-0.130	-0.838	-0.744

Opponent	metric	nash	full_br	ses_subgame	ganzfried	prime_safe / adaptation
	converged?			(194 it)	capped	capped
Maniac	EV	+0.438	+2.177	+0.682	+1.806	+1.842
	worst-case	−0.089	−1.100	−0.130	−0.638	−0.657
	converged?			(211 it)	capped	capped
CallingStation	EV	+0.559	+1.464	+0.663	+1.342	+1.363
	worst-case	−0.089	−1.000	−0.130	−0.915	−0.686
	converged?			(350 it)	capped	capped
LoosePassive	EV	+0.449	+1.405	+0.559	+1.306	+1.309
	worst-case	−0.089	−4.200	−0.133	−1.239	−1.334
	converged?			(400 it)	capped	capped

The headline finding is negative and empirical: global safe-exploitation does not scale, even to Leduc. I predicted Ganzfried would be safe on Leduc as it is on Kuhn. Instead the **global** solvers (ganzfried, prime_safe, adaptation, and rnr_0.5) all **hit the 40-iteration cap without converging**, leaving worst-case values of **−0.64 to −1.33** — grossly unsafe. The reason, which I confirmed rather than assumed: the cutting-plane loop adds one adversary cut per iteration, and on Leduc’s larger tree the set of relevant pure best responses is large, so 40 cuts nowhere near pin the true worst case; the master LP keeps returning optimistic-but-unsafe strategies. This was flagged as the number-one “likely to break” item *before* the run, and it broke exactly there.

The positive half is the subgame method. SES **converged** on three of four exploitable opponents (194–350 iterations), because it re-solves only the small King-flop subgame with the rest of the tree pinned — a far smaller LP with a far smaller adversary set. It extracts real value (**+0.25 to +0.68** versus the weak types, beating Nash) at a worst-case of **−0.13**, an order of magnitude closer to safe than the global methods. This is the *global-versus-local safety* distinction of §7 appearing as a measured fact on a game as small as Leduc — the concrete argument for real-time subgame methods and against a naive global solve.

I keep myself honest on SES, though: its residual exploitability (0.043) still exceeds the 0.01 tolerance, so it too is flagged unsafe, and on **LoosePassive** it ran the full 400 iterations without converging. Whether that residual 0.04 is the gadget legitimately bounding to an already-below- v^* blueprint (Nash’s own worst-case is −0.089, itself 0.003 below v^*), a convergence-tolerance artifact, or a small leak in the outside-subgame pinning is the top item to resolve before calling SES *provably* safe.

8.8.1 The teaching attack — and why realized profit is the wrong lens

The last experiment is the deception stress test the whole safety machinery is meant to survive. A deceptive opponent plays the weak Rock bait for 10 000 hands, then switches to a strong Nash “reveal” for 10 000 more; a Step-7 model feeds each solver every 500 hands (5 seeds).

method	mean/hand (all)	mean/hand (after switch)	safety violations / seed
full_br	+0.051	−0.061	40, 40, 40, 40, 40
ganzfried	−0.048	−0.055	0, 0, 0, 0, 0
adaptation	−0.046	−0.055	40, 40, 40, 40, 40
nash	−0.051	−0.061	0, 0, 0, 0, 0

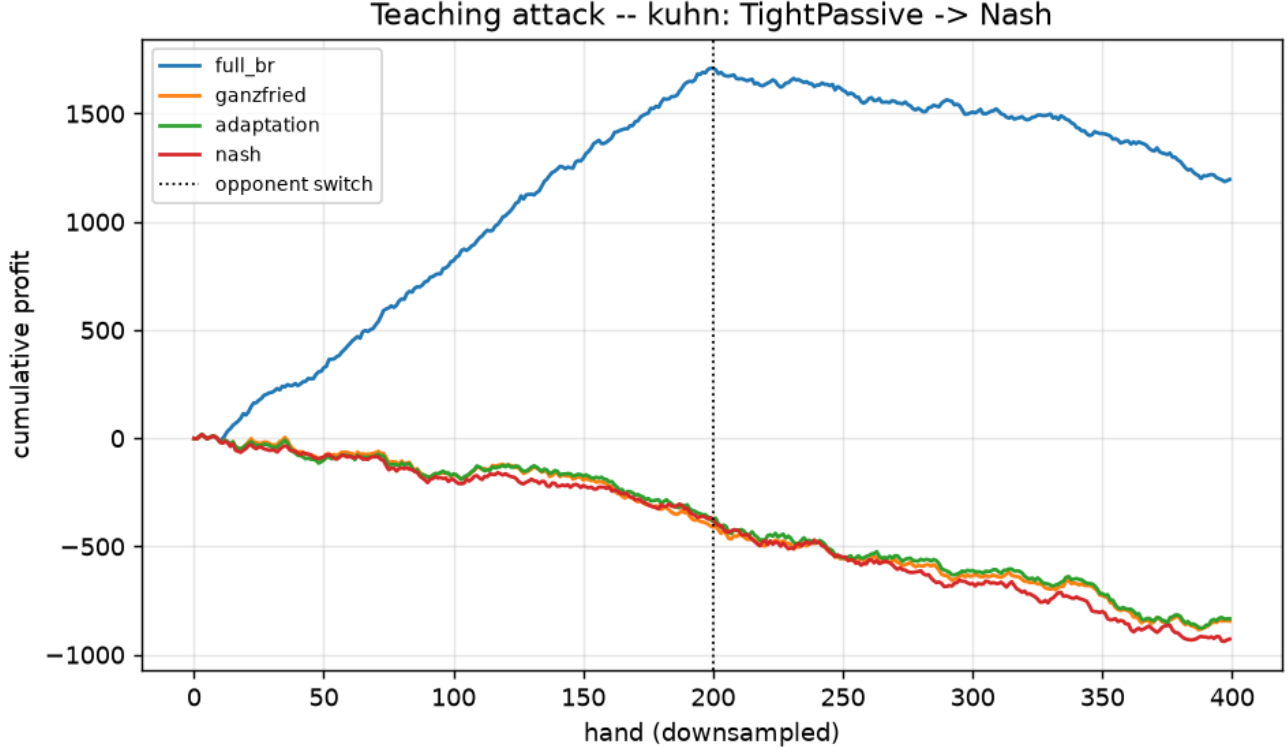


Figure 7: Teaching attack on Kuhn, cumulative profit. The full best response (blue) climbs on the bait to about +1700, then only drifts down after the switch — it ends far ahead, because a Nash “revealer” claws back only about the game value per hand. The safe methods refuse the bait and pay the first-player tax throughout.

The naive story — “the teaching attack punishes the greedy exploiter” — **did not hold in realized profit**, and understanding why is instructive. The full best response ends *hugely net-positive* (+0.051/hand overall): it banks a windfall during the bait phase and, because the “reveal” is only a *Nash* opponent, that opponent claws back barely more than the game value per hand, so the windfall is never repaid within 10 000 hands. The signal that *does* separate safe from unsafe is the **exact worst-case / safety-violation count**: the full best response violated the Nash floor at **40 of 40** refits, Ganzfried at **0 of 40**. A strategy’s worst case is realized by an opponent that *best-responds to it* — and a stationary Nash reveal simply is not that adversary. The corrected takeaway, which I now believe is the right design lesson: **measure safety by the worst case, not by realized profit against a benign opponent**; to punish the greedy exploiter *in profit*, the reveal must be an adaptive counter-exploiter, not a fixed Nash.

(That **adaptation** shows 40 violations while **nash** shows 0 is not a bug either: adaptation deliberately targets a floor *below* v^* , so it trips the v^* -referenced counter by design.)

8.9 9. Connections and Forward Pointers

What this step establishes. Safe exploitation is one idea — *maximize value against the model subject to a safety floor* — and on a fully solvable game it works exactly as the theory says: Ganzfried is safe against every opponent while beating equilibrium value on every exploitable one, where the naive best response earns more but is ruinously exploitable. Prime-safe and adaptation extend the guarantee to the imperfect baselines any real system has, by spending a *measured* -budget below the game value. But the step’s most valuable result is negative and empirical: on a game as small as Leduc the *global* safe-exploitation solve does not converge within a practical budget, while the *local* subgame method does — the theory-to-practice gap between global and local safety, measured rather than asserted.

Backward connections. This step is the exact complement of the equilibrium and opponent- modeling work before it. Step 7’s sensor produces the opponent model that becomes this step’s *objective*; Step 7’s exact best response becomes this step’s *worst-case oracle*; and the sequence-form representation from the CFR and consistency work becomes the *variable* the LP optimizes. The continuous model’s self-inflicted leak against Nash on Leduc (Step 7, §7) is the empirical motivation that this step’s safety floor is designed to prevent — and, indeed, Ganzfried never leaks to Nash here.

Forward to the thesis. The Kuhn results confirm the actuator is sound; the Leduc non-convergence is the concrete argument for the two scalable paths the thesis must choose between — an **exact one-shot dual LP** for the worst-case constraint, or a commitment to **local / subgame** safety (SES, OX-Search) as the real-time mechanism. Either way, the safety floor is what turns Step 7’s fragile sensor into a deployable adaptive agent.

Open questions carried forward.

- *Scalable safety.* Replace the cutting-plane loop with an exact dual LP, or adopt subgame safety wholesale; confirm whether the Leduc global solvers are merely *slow* or *structurally* stuck by re-running with a large budget.
- *The SES gadget’s residual exploitability.* Determine whether the 0.04 gap over tolerance is a blueprint-bounding effect, a tolerance artifact, or a pinning leak — this decides whether the subgame method is provably or only approximately safe.
- *A punishing teaching attack.* Replace the Nash reveal with an adaptive counter-exploiter so realized profit corroborates the worst-case view, and a safe method can be shown to *out-earn* the greedy one under deception.
- *From two players to N .* Every guarantee here rests on the two-player zero-sum fact that a Nash strategy secures the value v^* against any opponent — an anchor that vanishes for $N > 2$. Finding an N -player analogue (or a structural substitute) is the open problem this step makes precise, and the heart of Contribution #2.

Read more: Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems*, Ch. 7 “Learning and Teaching” — the repeated-game framing in which your actions both *exploit* and *teach* the opponent, which is exactly the tension the teaching attack (§8) probes and the multi-agent extension must confront.

Step 9 Summary — Multi-Agent Reinforcement Learning

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

9 Step 9 — Multi-Agent Reinforcement Learning: Coordination, Competition, and Communication	1
9.1 1. Why multi-agent RL is a different problem	2
9.2 2. Markov games — the bridge from Steps 2–8 to MARL	2
9.3 3. The family of methods	4
9.4 4. Independent learning and its failure modes	5
9.5 5. Centralized training, decentralized execution (CTDE)	10
9.6 6. PSRO — game theory over a population of policies	12
9.7 7. Learned communication (CommNet)	16
9.8 8. LOLA — modeling the opponent as a learner	16
9.9 9. Honest notes, limitations, and where this hands off	18
9.10 Key takeaways for the thesis synthesis	19

9 Step 9 — Multi-Agent Reinforcement Learning: Coordination, Competition, and Communication

This is a ground-up chapter on multi-agent reinforcement learning (MARL): the problem that makes it different from everything before it, the mathematics that frames it, the family of methods that attack it, and a set of controlled experiments run on small, exactly-solvable games. It serves two purposes — a self-contained refresher for later steps that build on it, and a primary source for the Part-1 thesis synthesis. It is written to be read on its own; no prior familiarity with the project’s code is assumed. **All experimental numbers reported here were measured** on reproducible runs of the testbeds and, wherever possible, are bounded by *exact* analytical references (Nash equilibria, exact best-response values) rather than by other simulations. Where a run contradicted what theory led me to expect, I keep the original expectation and reconcile it with what happened — those gaps are the most instructive parts of the step.

Where this sits in the thesis. Steps 2–8 lived entirely inside **two-player zero-sum** games: there is a value v^* , a Nash strategy secures it against *any* opponent, and CFR provably converges to it. Step 9 is the pivot into the **multi-agent** world, where those three comforts weaken or vanish. It carries three thesis hooks. **LOLA** — differentiating through an opponent’s *learning step* — is *dynamic* opponent modeling,

the moving-target complement to Step 7’s static read (Contribution #1). **PSRO** — a game played over a *population of policies* — is both the framework for safe exploitation where there is no minimax theorem (Contribution #2) and a general multi-agent *evaluation methodology* (Contribution #3). The place where every two-player guarantee breaks — the missing minimax anchor for $N > 2$ — is named here and left open for the thesis to attack.

9.1 1. Why multi-agent RL is a different problem

In single-agent RL (Steps 1 and 6) an agent learns by trial and error against a **fixed** world. Formally it faces a stationary Markov decision process: the transition law $P(s' | s, a)$ and reward $R(s, a)$ do not change while the agent trains, which is exactly what lets a fixed optimal value function Q^* exist for the agent to converge toward. In Steps 2–8 we went one level up and computed **equilibria** — strategies that are optimal against a *perfectly rational* opponent who has already finished reasoning.

Multi-agent RL sits between these two and is harder than both. Several agents **learn at the same time in a shared world**, so from any one agent’s seat the “environment” — which now *includes the other agents* — **keeps changing** as everyone updates. This is **non-stationarity**, and it breaks the assumption underneath single-agent RL. From agent i ’s perspective the effective transition and reward depend on the other agents’ policies π_{-i} , and those are moving every update, so the target Q^* that agent i chases is itself in motion. It also creates two problems that simply do not exist for a lone agent: **coordination** (how do cooperating agents learn to act together without being told how?) and **credit assignment** (when the team succeeds, whose actions mattered?).

A picture to hold onto: **learning to dance with a partner who is also learning to dance**. If your partner’s steps were fixed, you could memorize a routine that fits them — that is single-agent RL. If your partner were a flawless professional, you could study their known style and prepare the perfect counter — that is equilibrium computation. But when *both* of you are improving in real time, every adjustment you make changes what they should do, and vice-versa; you step on each other’s toes, over-correct, and oscillate — until you either lock into a shared rhythm (a coordinated equilibrium) or cycle forever (as in Matching Pennies, §4). The entire field is the art of *stacking the deck so the synchronization happens reliably*.

Read more: Zhang, K., Yang, Z. & Başar, T. (2021). “Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms.” *Handbook of RL and Control* — §1–2 for non-stationarity and the taxonomy used here.

9.2 2. Markov games — the bridge from Steps 2–8 to MARL

Steps 2–8 reasoned about **extensive-form games (EFGs)**: a game *tree*, **information sets** grouping histories a player cannot distinguish, and counterfactual values feeding a regret solver (CFR). Step 9

Single-agent RL assumes a fixed world; here the 'world' is a partner who is also learning.

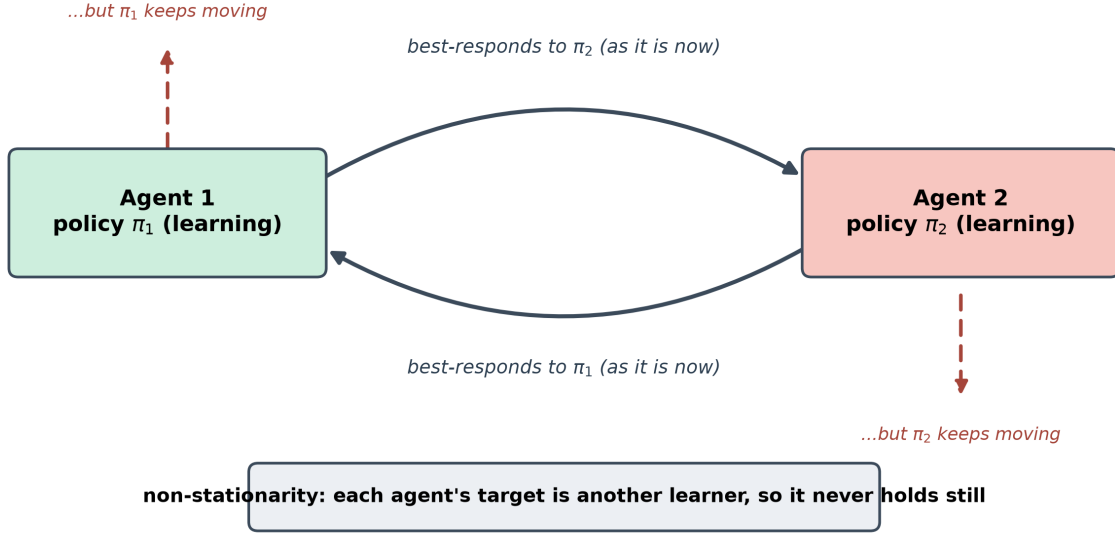


Figure 1: Non-stationarity as two partners learning to dance at once. Each agent optimizes against the other’s current policy (the solid arrows), but that policy is itself moving because the other agent is optimizing back (the dashed arrows). The target each one chases is never still, so naive learners cycle or over-correct rather than converge. Every method in this chapter is a different way to make the moving target hold still long enough to learn against.

reasons about **Markov (stochastic) games**, the standard MARL formalism. The two connect cleanly, and stating the connection is what keeps the switch from feeling like starting over.

A **Markov game** is the tuple

$$(\mathcal{S}, \{\mathcal{A}_i\}_{i=1}^N, P, \{R_i\}_{i=1}^N, \{\Omega_i\}, O, \gamma),$$

with states $s \in \mathcal{S}$, one action set $\mathcal{A}_i$ per agent, a transition law $P(s' \mid s, a_1, \dots, a_N)$ driven by the **joint** action, per-agent rewards $R_i(s, a_1, \dots, a_N)$, and (in the partially observed case) observations $o_i = O_i(s)$. Each agent has a policy $\pi_i(a_i \mid o_i)$; together they form a **joint policy** $\pi = (\pi_1, \dots, \pi_N)$. The special cases are exactly the previous steps:

- $N = 1$ recovers an ordinary **MDP** (Step 1).
- $N = 2$, $R_1 = -R_2$, a single state gives a **matrix game** (this step’s testbed, §4).
- Sequential, imperfect-information, zero-sum gives an **EFG** (Steps 2–8): a Markov game whose “state” is a *history* and whose partial observability is precisely an **information set**.

What survives the bridge is the *vocabulary*. A policy still maps what-you-know to a distribution over actions, so the EFG’s behavioral strategy at an information set is exactly the Markov-game policy $\pi_i(a_i \mid o_i)$; the information set becomes the observation o_i ; and the counterfactual value of a history becomes a centralized critic’s estimate at a state. What does **not** survive is the *guarantees*. CFR’s counterfactual

decomposition needs the tree and perfect recall, which general Markov games (loops, simultaneous moves, $N > 2$) do not provide, so “average strategy $\rightarrow$ Nash” no longer holds; MARL falls back to gradient/value learning whose convergence is not guaranteed (hence the cycling of §4). Most consequentially, the **minimax value anchor** disappears: in two-player zero-sum a Nash strategy guarantees v^* against any opponent — the fact that made Step 8’s safe exploitation coherent — and for $N > 2$ there is no such single value. That missing anchor is the precise gap Contribution #2 inherits.

Read more: Littman, M. L. (1994). “Markov Games as a Framework for Multi-Agent Reinforcement Learning.” *ICML* — the paper that introduced this framing and the minimax-Q algorithm.

9.3 3. The family of methods

Every method in this step is a different structural answer to “the other agents are learning while I learn.” Five families matter here.

Approach	What it does	Reach for it when	Main weakness
Independent Learning (IL)	each agent runs its own single-agent RL, treating others as environment	a quick baseline; near-stationary settings	non-stationarity $\rightarrow$ cycling, coordination failure
MADDPG (CTDE)	each agent’s critic sees all agents’ obs+actions at training; each actor sees only its own obs at execution	mixed cooperative-competitive tasks; the canonical CTDE template	critic input grows with N ; the actor update is fiddly
MAPPO (CTDE)	plain PPO with a centralized value $V(\text{global state})$ and shared parameters	cooperative MARL where you want a simple, strong baseline	on-policy sample cost; “simple” but tuning-sensitive
PSRO	maintain a population of policies; solve a meta-Nash over it; train a best response to that mixture; add it; repeat	competitive/general games; when you want a game-theoretic convergence target	a full best response per round; an approximate oracle weakens the guarantee

Approach	What it does	Reach for it when	Main weakness
LOLA	optimize assuming the opponent takes one learning step ; differentiate <i>through</i> their update	2-player differentiable games where naive learning fails (IPD)	assumes you know and can differentiate the opponent's update
CommNet	agents broadcast a differentiable message ; each receives the mean of the others' and feeds it into its policy	cooperative tasks with partial observability	mean pooling discards <i>who</i> said what

Two axes cut across the table (Figure below). First, **what is centralized, and when?** IL centralizes nothing; CTDE centralizes the *critic/value at training only* (execution stays decentralized); PSRO centralizes a whole *meta-game solve* between training rounds; CommNet centralizes *information at execution* through a learned channel. Second, **is the opponent modeled as static or as a learner?** Everything except LOLA treats the opponent's strategy as fixed-while-you-respond; **LOLA** anticipates the opponent's *next update* — dynamic, not static.

Historically the line runs: learn to talk (CommNet, 2016) → centralize the critic (MADDPG, 2017) → bring game theory to a population (PSRO, 2017) → factorize the value (QMIX, 2018) → look ahead at the opponent's learning (LOLA, 2018) → and then discover that the simple thing often wins (MAPPO, 2022).

Read more: Albrecht, S. V., Christianos, F. & Schäfer, L. (2024). *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches* (MIT Press), Ch. 8–9 — a current textbook treatment of exactly this taxonomy.

9.4 4. Independent learning and its failure modes

The cleanest way to see *why* the rest of the field exists is to run the control that fails. Independent learning drops each agent into its own gradient loop and pretends the others are part of the floor. On four canonical 2×2 matrix games — each a different qualitative case — it produces four different fates.

The games and their analytic Nash equilibria (ground truth): **Prisoner's Dilemma** (a dominant strategy: Defect strictly beats Cooperate, so the unique Nash is mutual Defect); **Matching Pennies** (zero-sum; the unique Nash is the fully mixed $(\frac{1}{2}, \frac{1}{2})$; value 0); **Stag Hunt** (two pure Nash — the payoff-dominant (Stag, Stag) and the risk-dominant (Hare, Hare) — plus a mixed one); **Battle of the Sexes** (two pure Nash that disagree on which to pick, plus a mixed one).

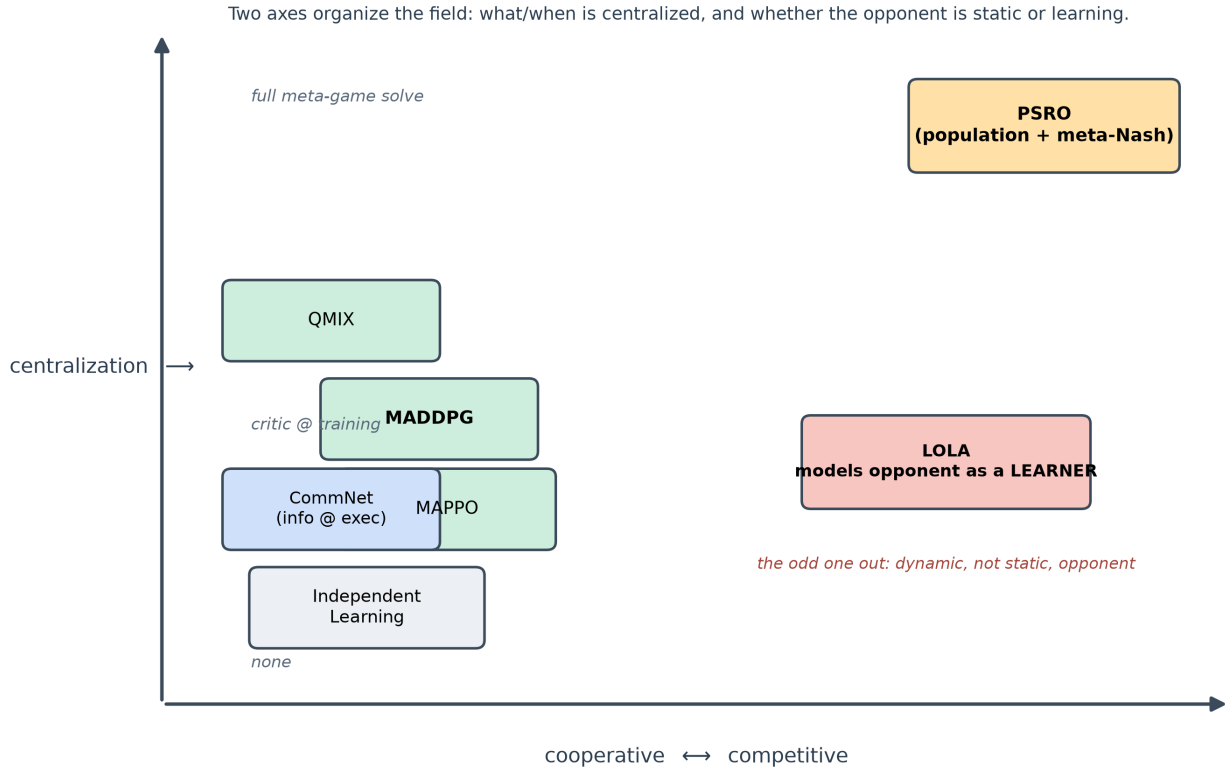


Figure 2: The method family on two axes. Horizontal: how competitive vs cooperative the target setting is (IL and PSRO span competition; MADDPG/MAPPO/QMIX/CommNet target cooperation; LOLA is the mixed-motive bridge). Vertical: how much is centralized and when (nothing for IL; the critic at training for CTDE; a full meta-game solve for PSRO; information at execution for CommNet). LOLA is the outlier that models the opponent as a learner rather than a fixed strategy.

Running two independent gradient learners (exact gradients, so the dynamics are clean and noise-free) gives, measured:

Game	Measured outcome	NashConv	Matches analytic Nash?
Prisoner’s Dilemma	$x \rightarrow [0.001, 0.999]$ (Defect)	0.0013	yes — the dominant-strategy equilibrium
Stag Hunt	all seeds $\rightarrow$ (Hare, Hare)	0.0013	yes — the <i>risk-dominant</i> pure Nash
Battle of the Sexes	all seeds $\rightarrow$ one pure Nash	0.0013	yes — a pure Nash
Matching Pennies	does not converge (last iterate drifts to the boundary)	1.4–1.8	no — it never settles

Three of the four converge to a genuine Nash, and the interesting details are in the “how.” In Stag Hunt the learners reliably pick the **risk-dominant** equilibrium (Hare) rather than the payoff-dominant one (Stag) even though Stag pays more — a unilateral move toward Stag is punished, so gradient dynamics slide to the safe corner. In Battle of the Sexes every seed lands on the *same* pure equilibrium; the learners solve the coordination problem but not in a seed-diverse way. These are the coordination and equilibrium-selection problems made concrete.

Matching Pennies is the headline: it **never converges**, which is exactly the point.

Reconciliation (kept prediction $\rightarrow$ what actually happened). I predicted Matching Pennies would trace a clean orbit at roughly constant radius around $(\frac{1}{2}, \frac{1}{2})$. Two learners, run two ways, both failed to converge — but neither orbited cleanly. The projected-gradient (IGA) learner used in the exploration scripts *spirals outward*: the measured distance-to-Nash grew across time-windows from 0.30 to 0.48. The softmax-logit learner used in the implementation *drifts to the corners* (final profiles like $x = [0.96, 0.04]$, $y = [0.03, 0.97]$, $\text{NashConv} \approx 1.8$). The **lesson is unchanged and arguably sharper**: under naive simultaneous learning the *last iterate* does not converge in a game with only a mixed equilibrium — the thing that converges is the *time-average* (§6). The specific “energy-preserving orbit” mental model was the part that was wrong; the mechanism is a slow divergence toward the boundary, which is if anything a stronger argument for the population/averaging machinery that follows.

Read more: Singh, S., Kearns, M. & Mansour, Y. (2000). “Nash Convergence of Gradient Dynamics in General-Sum Games.” *UAI* — the analysis of why gradient ascent cycles rather than converges in games like Matching Pennies.

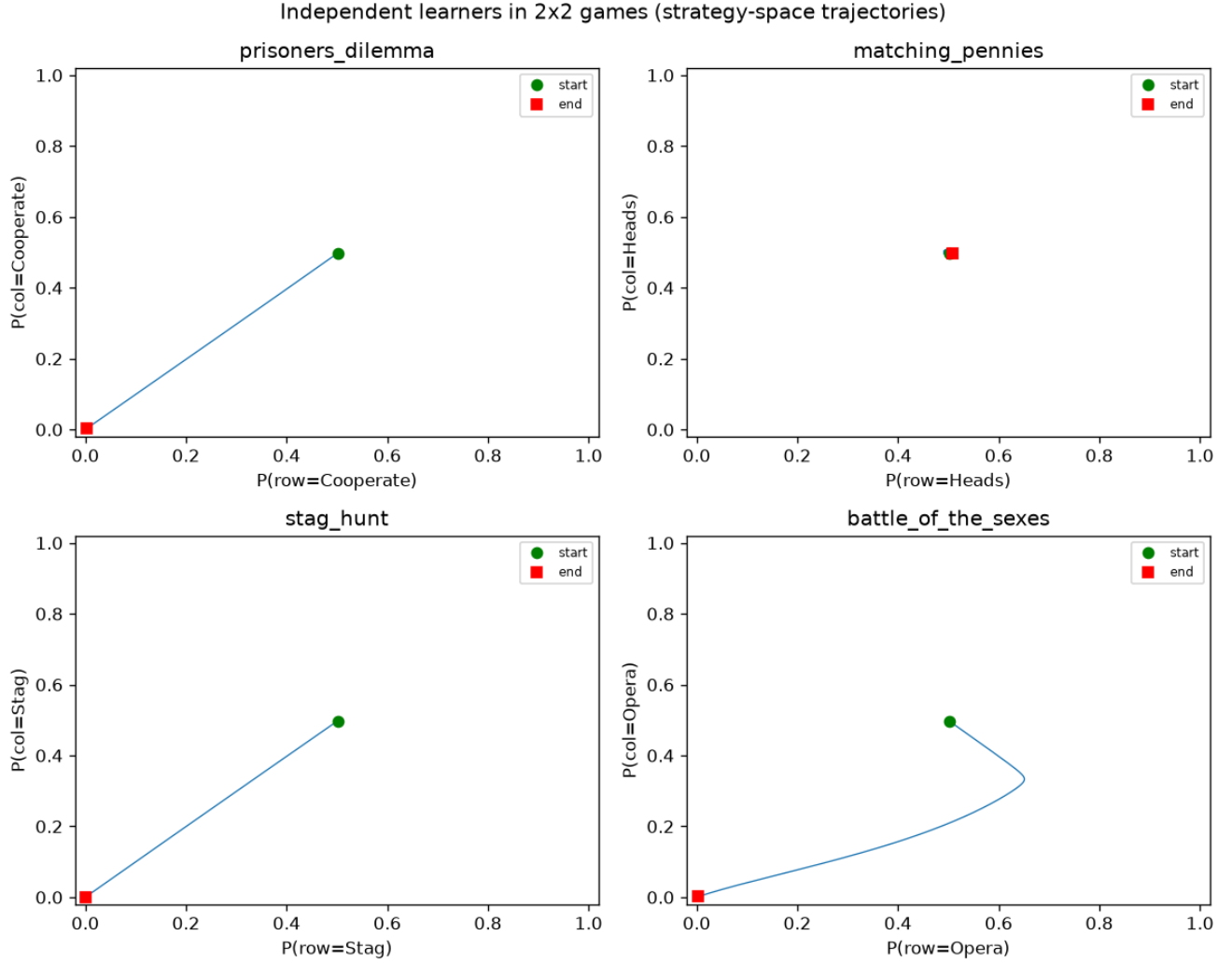


Figure 3: Independent learners on the four matrix games: Prisoner's Dilemma collapses to mutual defection, Stag Hunt and Battle of the Sexes settle on a pure equilibrium, and Matching Pennies fails to converge (its trajectory drifts away from the mixed Nash rather than settling on it). The contrast is the visceral case for coordination machinery.

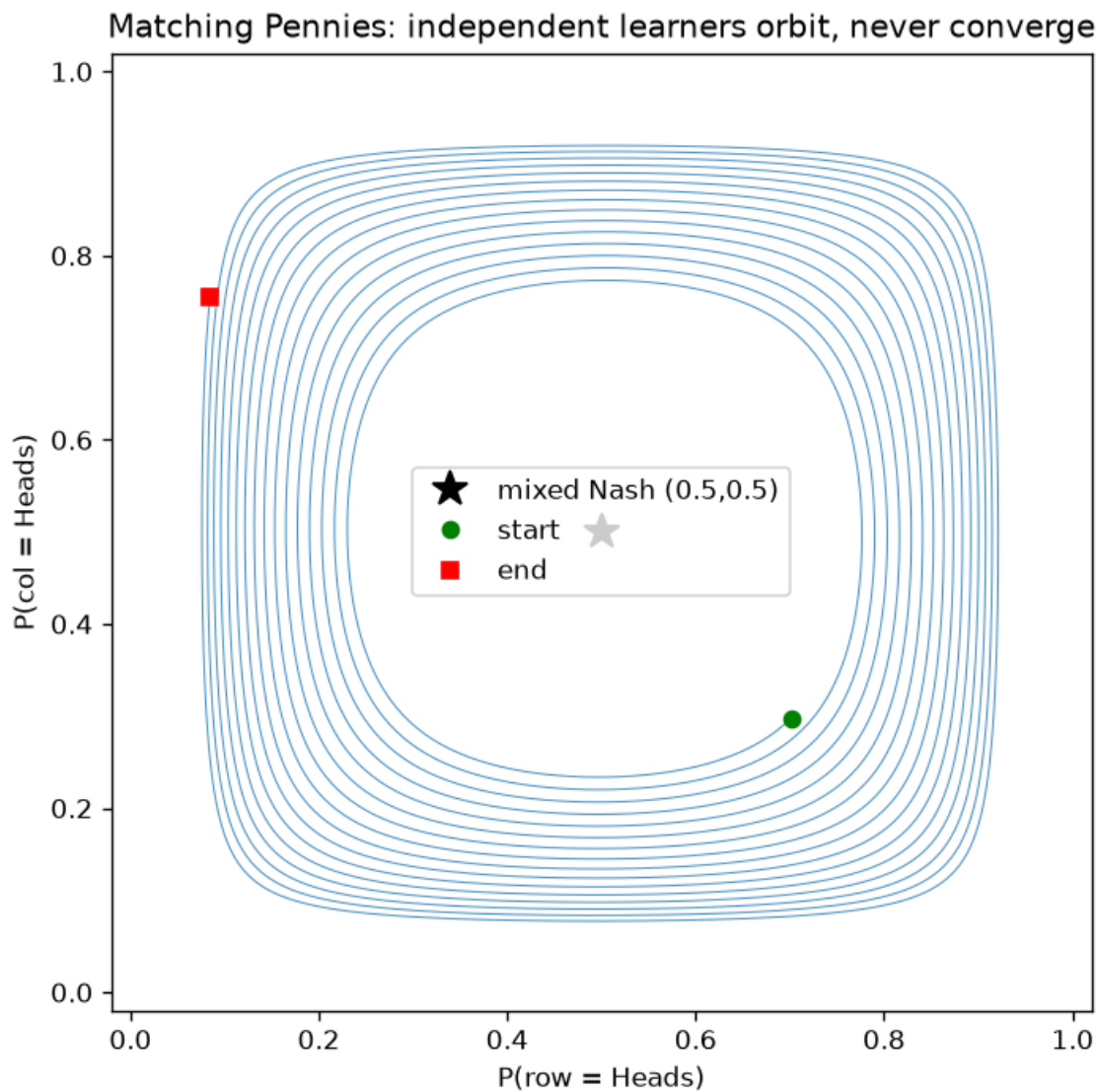


Figure 4: Zoom on non-stationarity: the distance-to-Nash for Matching Pennies does not shrink (it grows across time-windows, 0.30 to 0.48), while the Prisoner's Dilemma distance-to-(Defect,Defect) collapses to zero. More compute traces a bigger divergence, not convergence — non-stationarity is structural, not a budget problem.

9.5 5. Centralized training, decentralized execution (CTDE)

The first structural fix keeps execution realistic — each agent still acts on its own observation — while giving the *learner* a privileged view during training. The archetype is **MADDPG** (Lowe et al., 2017): each agent’s **actor** $\pi_i(a_i | o_i)$ sees only its own observation, but a **centralized critic** $Q_i(s, a_1, \dots, a_N)$ sees the global state and *every* agent’s action. The asymmetry is the whole idea. A per-agent critic that watches only o_i faces a non-stationary, partially-observed world, so identical inputs map to different returns and its value target is noisy; a centralized critic conditioned on everything sees a target that is (near-)deterministic, so it is a far lower-variance teacher. **MAPPO** (Yu et al., 2022) is the minimal version: plain PPO with a single **centralized value function** $V(\text{global state})$ instead of per-agent $V(o_i)$, and it is often the strongest baseline — a caution against over-engineering.

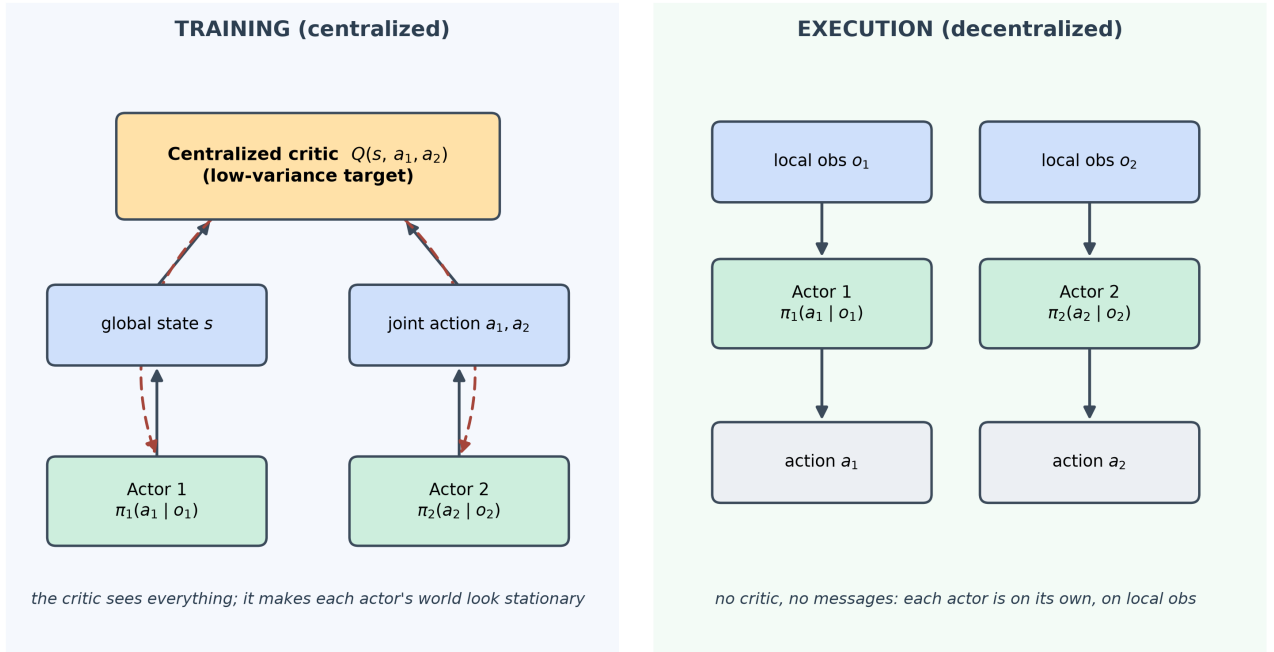


Figure 5: CTDE architecture. During training a centralized critic (or value function) sees the global state and the joint action and produces low-variance value targets; during execution each actor acts on its own local observation with no access to the critic and no message passing. The training/execution asymmetry is what makes the world look stationary to the learner without cheating at showtime.

Does the centralized critic actually have lower variance? Measured on a one-step cooperative “referential” task (a speaker sees a target, a listener does not, both must name it), and trained to convergence:

Critic	Final residual (value loss)
centralized $Q(s, \text{joint } a)$	3.2×10^{-11}
independent $Q_i(o_i)$	0.077

The centralized critic drives its residual to essentially zero — it can see the target and the joint action, so the reward is a deterministic function of its inputs — while the independent critic cannot see the target and is stuck predicting the base rate. This is the CTDE variance-reduction claim, confirmed cleanly.

But variance reduction is **not** the same as solving coordination, and this is where a prediction broke.

Reconciliation (kept prediction $\rightarrow$ what actually happened). I predicted that on the Claus-Boutilier **climbing game** — a stateless cooperative matrix game whose optimum (11) is flanked by -30 miscoordination penalties, with a “safe” attractor at 5 — the centralized critic would escape the trap and reach the optimum, beating independent learners. It did not. Measured greedy rewards: **independent learners 7, MADDPG 5, MAPPO 7** (optimum 11, safe 5). No method reached the optimum, and MADDPG actually *underperformed* both IL and MAPPO. The honest reading: a centralized critic lowers value-target variance (confirmed above) but that is **not sufficient** to overcome relative over-generalization plus the hard-exploration risk of the -30 penalties — the agents will not try the risky joint action long enough to discover the 11. MADDPG’s below-IL result specifically flags its discrete counterfactual-baseline actor update as the piece to scrutinize next. The lesson survives in a chastened form: **CTDE buys a better critic, not automatic coordination.**

A methodological note worth carrying forward: both CTDE effects above were **invisible at the fast “smoke” configuration** — there the critic losses were near-equal and the communication benefit (§7) was zero. They appeared only once trained at the larger “scale” configuration. The smoke config proves the code runs; the phenomena need training to convergence.

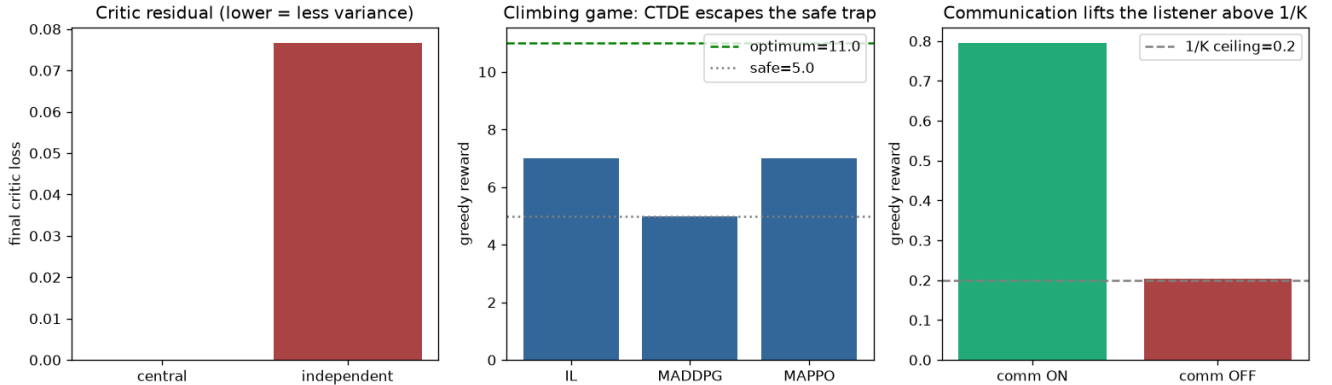


Figure 6: Cooperative CTDE and communication results (scale config). Left: the centralized critic’s residual is orders of magnitude below the independent critic’s. Middle: on the climbing game, no method reaches the optimum (11); MADDPG (5) trails IL and MAPPO (7) — CTDE reduces critic variance but does not by itself solve hard-exploration coordination. Right: communication lifts the listener far above the $1/K$ guessing ceiling (see §7).

Read more: Lowe, R. et al. (2017). “Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments.” *NeurIPS* (MADDPG); and Yu, C. et al. (2022). “The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games.” *NeurIPS* (MAPPO).

9.6 6. PSRO — game theory over a population of policies

The second structural fix is the through-line of the step and the direct descendant of Step 2’s iterated best response. **PSRO** (Policy-Space Response Oracles; Lanctot et al., 2017) treats whole policies as the atoms of a higher game. It maintains a **population** of policies per player, builds the empirical **meta-game** payoff matrix between the populations, solves a **meta-Nash** over it, trains a **best response** (the “oracle”) to the opponent’s meta-Nash mixture, and adds that response to the population — repeat. It unifies self-play, fictitious play, and the double-oracle method under one framework, and, crucially for this project, its progress metric is *exploitability* — the very same NashConv used throughout Steps 2–8.

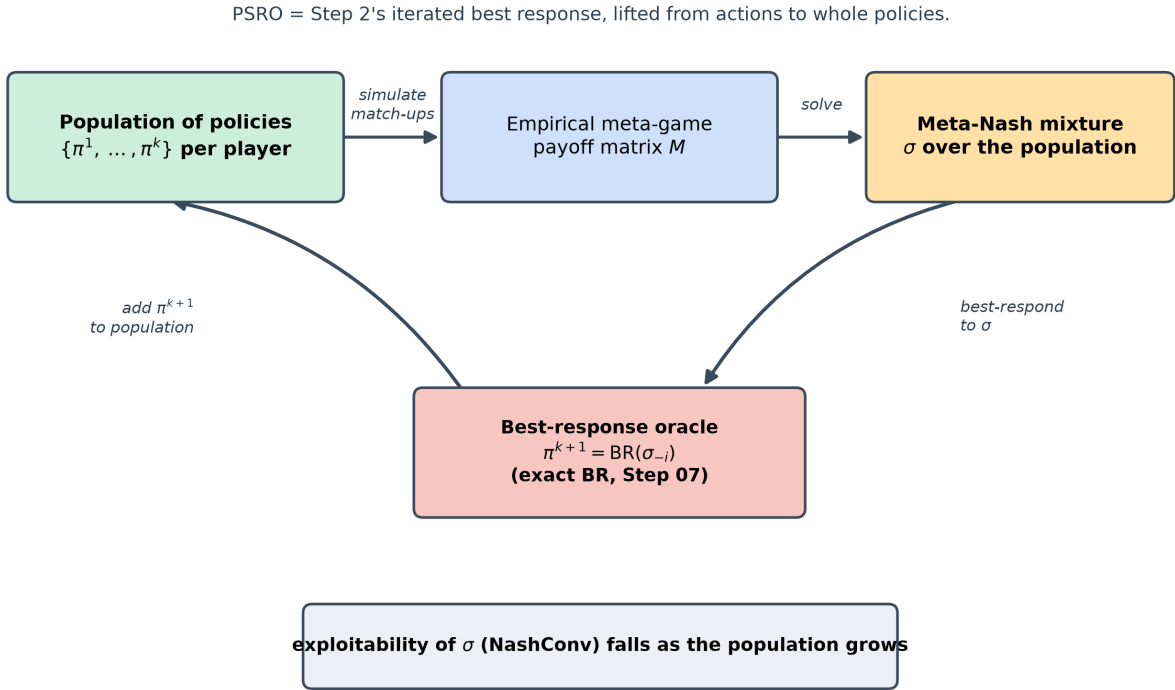


Figure 7: The PSRO double-oracle loop. From the current populations, build the meta-game payoff matrix, solve its meta-Nash mixture, then call a best-response oracle against the opponent’s mixture and add the new policy to the population. Exploitability of the meta-Nash mixture is expected to fall as the population grows. In this project the oracle is Step 07’s exact best response, so PSRO’s convergence is measured with the same exploitability yardstick as the game-theory steps.

Two facts make the implementation exact rather than approximate. First, the oracle reuses Step 07’s **exact best response** on Kuhn and Leduc. Second, the opponent’s meta-Nash mixture over *behavioral policies* is, by Kuhn’s theorem (these games have perfect recall), realization- equivalent to a **single behavioral policy**; collapsing the mixture that way lets the exact best-response engine apply directly.

Why a population and not just the last self-play policy? Because self-play converges in the *average*, not the last iterate. Measured on Kuhn with fictitious-play self-play: the **average**-iterate exploitability

fell from 0.24 to 0.031 over 200 iterations, while the **last**-iterate exploitability kept oscillating between 0.33 and 0.83. Trusting the latest policy would be trusting a number that never settles; PSRO’s meta-mixture is the population-level version of the averaging that CFR and fictitious play rely on.

PSRO convergence, measured (exploitability = NashConv of the meta-Nash mixture in the full game):

Game	Exploitability trajectory	Verdict
Kuhn Poker	$0.917 \rightarrow \sim 2 \times 10^{-16}$ by round 6	converges to machine zero
matrix (Matching Pennies)	$2.0 \rightarrow 0$ by round 2	converges
Rock–Paper–Scissors (exploration)	$2.0 \rightarrow 0.017$; population $\rightarrow$ $\{R,P,S\}$, mixture $\rightarrow$ uniform	converges
Leduc Hold'em	$4.75 \rightarrow 2.16$ over 20 rounds	decreases, but far above target
Goofspiel ($K = 3$)	$1.33 \rightarrow 0$	converges
Goofspiel ($K = 4$)	oscillates $1.4 \leftrightarrow 2.0$	does not settle

The Kuhn / matrix / RPS results are textbook: on the small games the population quickly spans the strategies needed and exploitability collapses. Two results did not go as predicted.

Reconciliation 1 (Leduc). I predicted PSRO would drive Leduc exploitability below 0.5 within 20 iterations. Measured, it fell from 4.75 to 2.16 — a clear, roughly monotone decline, but nowhere near 0.5. This is **genuine slow convergence**, not a bug: Kuhn hit machine zero in 6 rounds, but Leduc’s game tree is far larger, and a population of 20 *pure* best responses is simply too small to closely approximate its mixed Nash. The “ < 0.5 in 20” target was optimistic. The lesson — exploitability decreases as the population grows — holds; the *rate* is the scaling wall, and it rhymes with Step 8’s global-vs-local scaling finding.

Reconciliation 2 (Goofspiel $K = 4$). I predicted non-increasing exploitability. At $K = 3$ it converged to 0 cleanly; at $K = 4$ it oscillates between ~ 1.4 and ~ 2.0 and does not settle. This is the one result I cannot yet fully explain, and per the workflow I am **documenting it, not fixing it**. Two concrete suspects for a follow-up session: the Goofspiel PSRO driver never de-duplicates best-response policies (so the meta-game can stall on repeats), and a pure-strategy population is likely too weak to represent the larger game’s mixed meta-Nash. Flagged as an open code item, not a validated result.

Read more: Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning.” *NeurIPS* (PSRO); and McMahan, H. B., Gordon, G. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* (the double-oracle method PSRO generalizes).

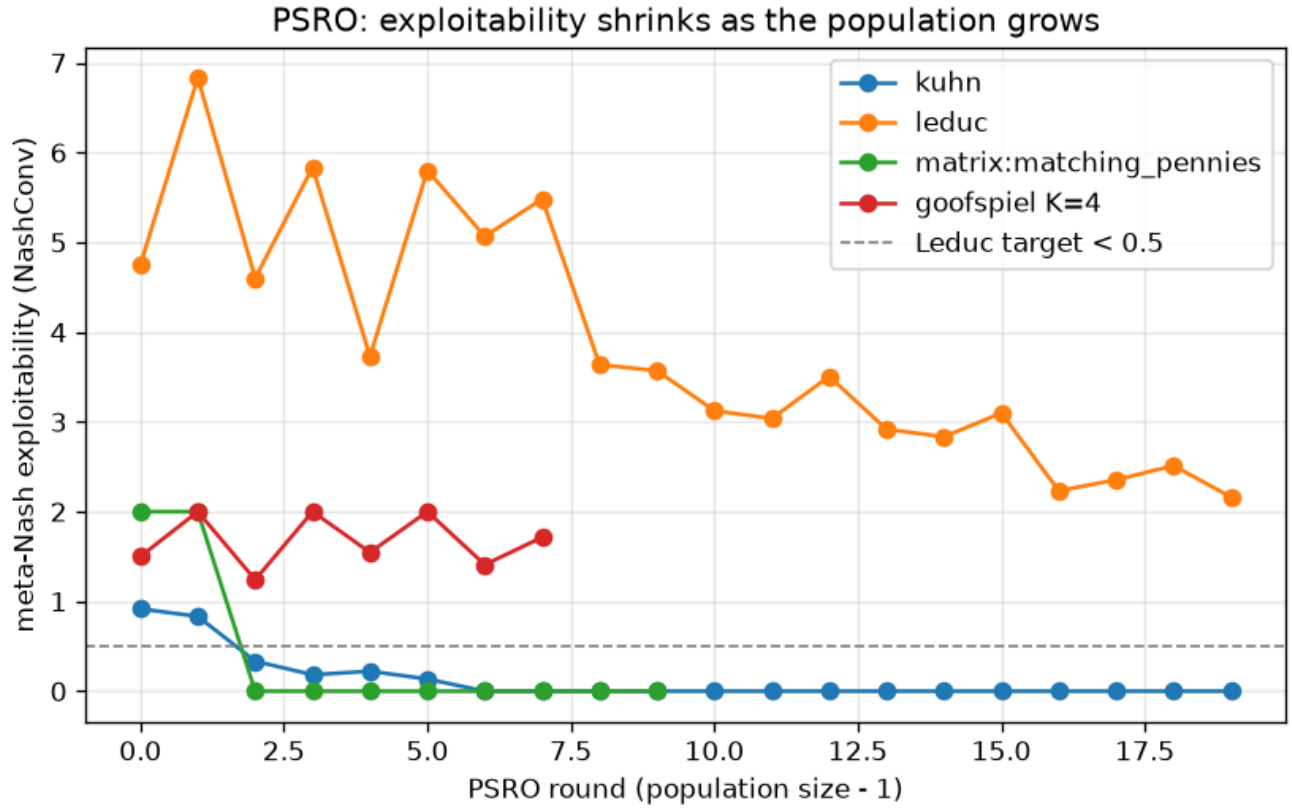


Figure 8: PSRO exploitability vs population size across games (scale config). Kuhn, the matrix game, and Rock-Paper-Scissors collapse to (near) zero within a handful of rounds; Leduc declines steadily but stays well above the 0.5 target after 20 rounds (a scaling wall for a pure-strategy population); Goofspiel $K=4$ oscillates rather than settling (a flagged anomaly). The exact best-response oracle guarantees convergence in principle; the rate is what size controls.

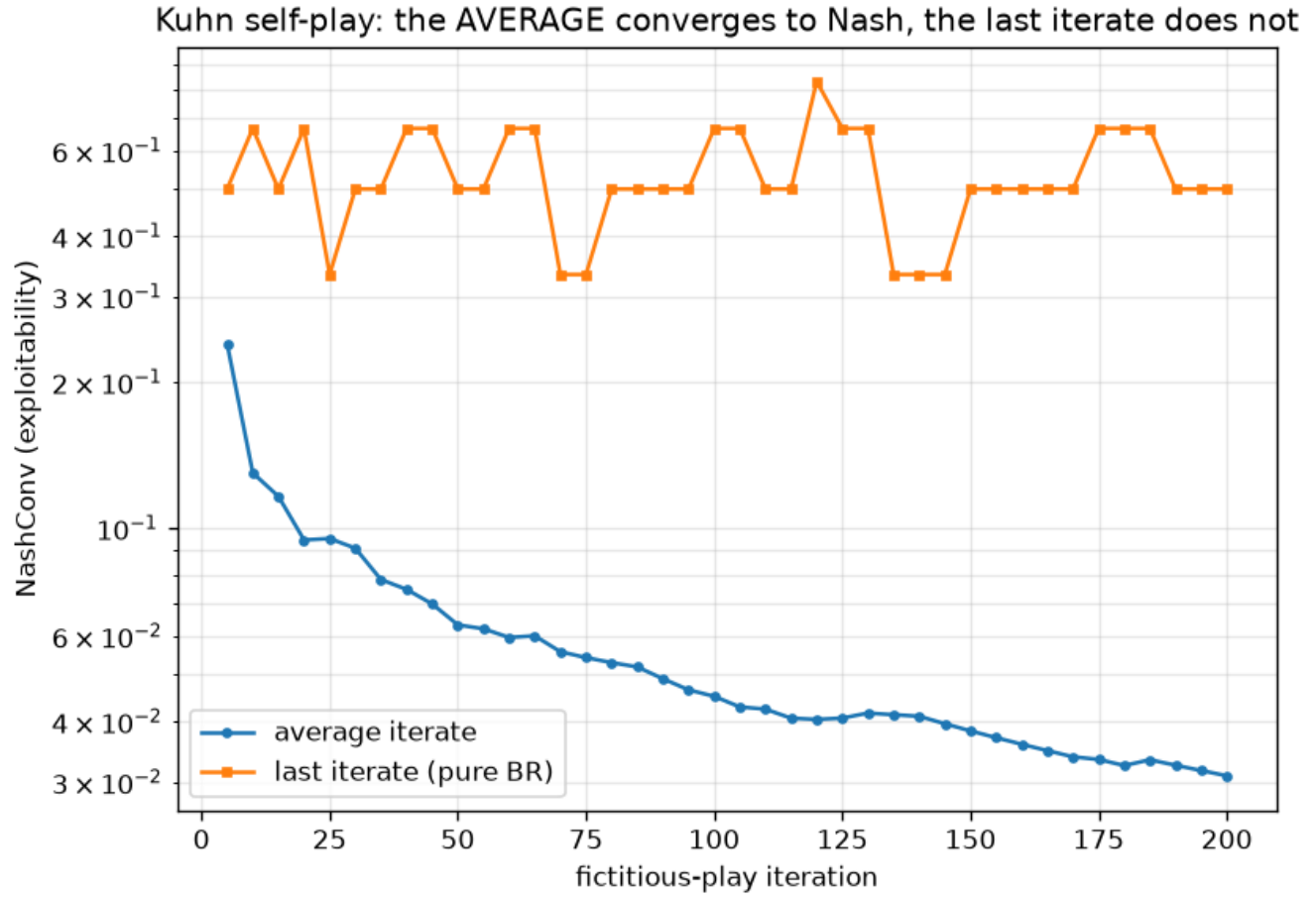


Figure 9: Self-play on Kuhn: the average-iterate exploitability (NashConv) falls steadily toward zero while the last-iterate exploitability keeps oscillating. This is why self-play and PSRO rely on averaging over a population rather than trusting the most recent policy.

9.7 7. Learned communication (CommNet)

CTDE centralizes information at *training*; communication centralizes it at *execution*, through a channel the agents **learn** rather than one a designer builds. In **CommNet** (Sukhbaatar et al., 2016) each agent encodes its observation to a hidden vector, emits a **message**, and then receives the **mean** of the other agents’ messages as extra input to its policy. The whole thing is trained end-to-end, so the protocol emerges: on the referential task, the speaker (who alone sees the target) must learn to encode it and the listener must learn to decode it.

The test is sharp because without a channel the listener is capped at pure guessing, $1/K$. Measured (scale config, $K = 5$, so the guessing ceiling is 0.2):

Channel	Greedy team reward
communication ON	0.795
communication OFF	0.204

With the channel the listener climbs well above the $1/K$ ceiling; without it, it sits exactly at the ceiling. Communication is doing real work — and note the reconciliation from §5 applies here too: at the smoke configuration both numbers were 0.24 (the channel had not yet learned to carry information), and the benefit appeared only after training at scale.

Read more: Sukhbaatar, S., Szlam, A. & Fergus, R. (2016). “Learning Multiagent Communication with Backpropagation.” *NeurIPS* (CommNet).

9.8 8. LOLA — modeling the opponent as a learner

Every method so far treats the opponent’s strategy as fixed while you respond. **LOLA** (Learning with Opponent-Learning Awareness; Foerster et al., 2018) is the exception, and it is the one most directly connected to the thesis. Instead of optimizing against the opponent’s *current* policy, each agent optimizes against the policy the opponent will hold *after one learning step*, and differentiates *through* that step. The extra term is a mixed second derivative — how the opponent’s update depends on *my* parameters — and it is what turns self-interested agents cooperative.

The classic demonstration is the memory-1 **Iterated Prisoner’s Dilemma**, where each agent’s policy is five cooperation probabilities and the expected discounted return has a closed form via the stationary Markov chain over outcome pairs. Naive gradient learners, each maximizing its own return against the other’s current policy, converge to **mutual defection**; LOLA learners, each accounting for the other’s upcoming update, reach **mutual cooperation**.

Measured (per-step discounted return; full cooperation ≈ 3 , mutual defection ≈ 1):

Learners	Return
naive vs naive	1.04
LOLA vs LOLA	2.82

The direction is exactly the LOLA result — cooperation emerges where naive learning defects. (My prediction of ≈ 3 was slightly high; the measured 2.82 is near-cooperation, and the exploration run with a larger look-ahead reached ~ 2.9 .) A built-in sanity check confirms the mechanism: setting the look-ahead learning rate to zero makes LOLA’s gradient reduce exactly to the naive gradient, so the cooperation comes specifically from the second-order look-ahead term.

Conceptually this is **dynamic** opponent modeling: Step 7 inferred an opponent’s *current* strategy; LOLA anticipates their *learning trajectory*. Combining the two — a static read that seeds a dynamic look-ahead — is a candidate for Contribution #1, not a solved thing.

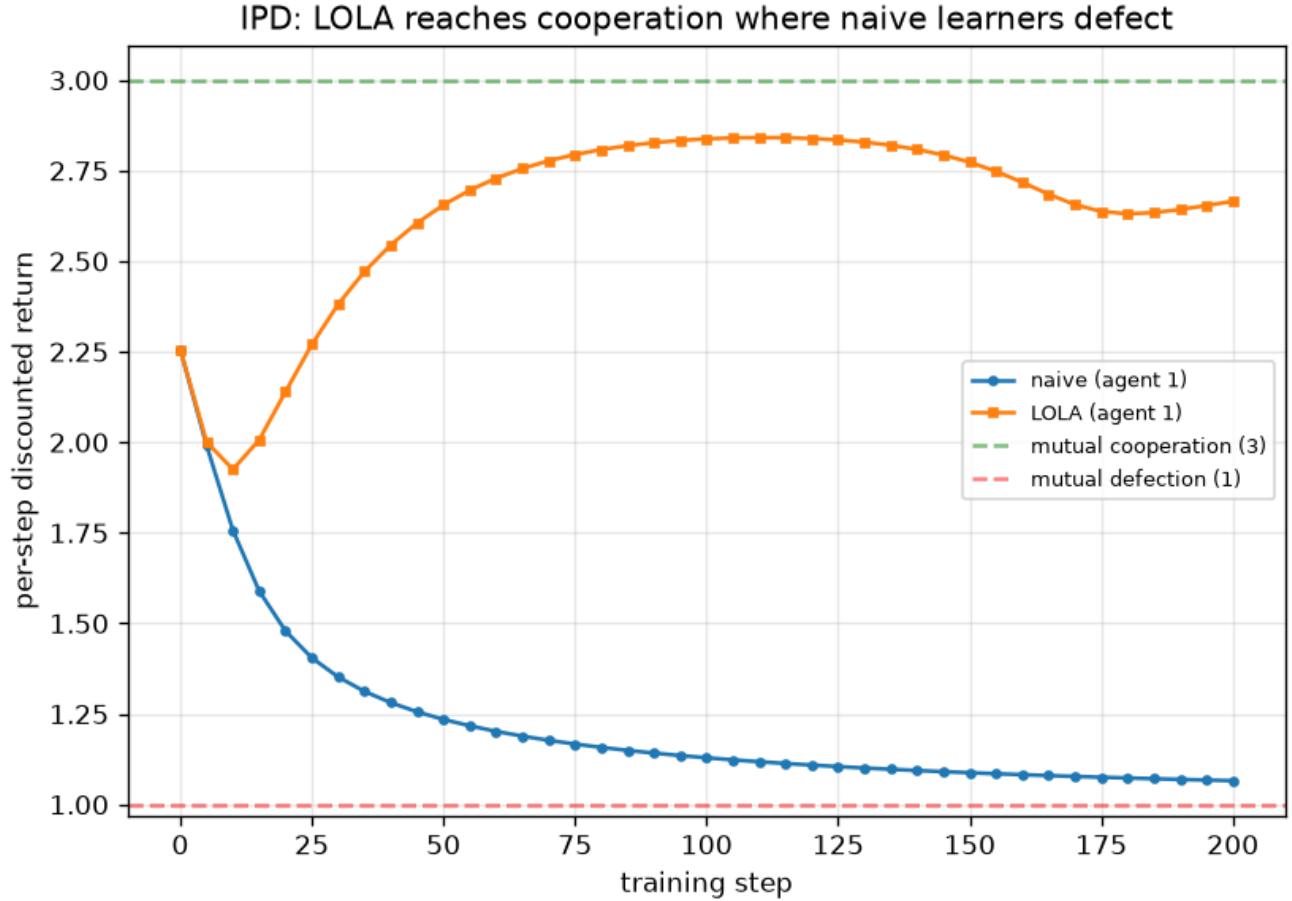


Figure 10: LOLA vs naive learners on the Iterated Prisoner’s Dilemma: naive learners’ per-step return collapses toward mutual defection (~ 1), while LOLA learners’ return climbs toward mutual cooperation (~ 2.8). Anticipating the opponent’s next learning step is what reshapes the dynamics from defection to cooperation.

Read more: Foerster, J. et al. (2018). “Learning with Opponent-Learning Awareness.” *AA-MAS* (LOLA).

9.9 9. Honest notes, limitations, and where this hands off

What held up. The confirmed results are the backbone: independent learning converges on dominant/coordination games and *fails* on Matching Pennies; PSRO drives exploitability to (near) zero on Kuhn, the matrix game, and RPS with an exact oracle; self-play converges in the average but not the last iterate; the centralized critic has orders-of-magnitude lower residual variance; learned communication lifts the listener above the guessing ceiling; and LOLA turns IPD defectors into cooperators. Taken together they trace the intended arc from “why naive learning breaks” to “the structural fixes that repair it.”

What did not, and why it matters. Four honest caveats travel forward. (1) Matching Pennies diverges toward the boundary rather than orbiting — a sharper version of the non-convergence lesson. (2) PSRO on Leduc converges *slowly* (exploitability ~ 2.16 after 20 rounds, not < 0.5) — the pure-strategy-population scaling wall. (3) Goofspiel $K = 4$ oscillates rather than converging — a flagged, unexplained code anomaly (documented, not fixed). (4) On the climbing game no method reached the optimum and MADDPG underperformed independent learners — a centralized critic reduces variance but does not by itself solve hard-exploration coordination. The two code items (Goofspiel $K = 4$; MADDPG’s counterfactual baseline) should be investigated before those pieces are reused. And a methodological point: the neural effects were invisible at the fast smoke config and only emerged at scale, so the scale numbers are the ones the claims rest on.

Trust. Every equilibrium target is *exact* (analytic Nash for the matrix games; Step 07’s exact best response and NashConv for Kuhn/Leduc/Goofspiel), so the game-theoretic results are bounded by ground truth rather than by other simulations. The neural results are qualitative inequalities (central $<$ independent; comm ON $>$ comm OFF), not precise values, and are seed- and version-sensitive by construction. The experiment PNGs cited above are generated from the committed JSON artifacts (see `../figures/README.md`); a few were not saved on the run and are marked “to close.”

Backward and forward connections. Backward: PSRO is Step 2’s iterated best response lifted to a population, and it reuses Step 07’s exact best-response engine wholesale; the Leduc scaling wall echoes Step 8’s global-vs-local finding. Forward: the three thesis hooks are now concrete — LOLA as dynamic opponent modeling (Contribution #1), PSRO’s meta-game as an evaluation methodology (Contribution #3), and, above all, the **missing $N > 2$ minimax anchor** (Contribution #2), the single place where every two-player guarantee from Steps 2–8 stops applying.

Open questions. Can an approximate (RL) oracle push Leduc PSRO much closer to Nash, and at what cost to the convergence guarantee? What breaks the Goofspiel $K = 4$ oscillation — de-duplication, a mixed-strategy population, or a different meta-solver? Why does a centralized critic *hurt* on the climbing game, and does a counterfactual/COMA baseline fix it? And the thesis question underneath all of them: with no minimax value for $N > 2$, what does “safe” even mean for a population, and can PSRO’s

meta-game supply a usable substitute?

9.10 Key takeaways for the thesis synthesis

- **Non-stationarity is *the* problem**, and it is structural, not a compute-budget issue — Matching Pennies diverges no matter how long you train.
- **CTDE centralizes the critic at training and decentralizes the actor at execution**; measured, it delivers a near-zero-variance critic (3.2×10^{-11} vs 0.077) — but variance reduction alone did not solve the climbing game, so it is necessary, not sufficient.
- **PSRO is the game-theory MARL bridge**: measured, it drives Kuhn to machine-zero exploitability and RPS to uniform; Leduc declines but hits a scaling wall — the same wall that motivates the thesis’s scalable methods.
- **Self-play/PSRO succeed in the average/population, not the last iterate** (Kuhn average NashConv $0.24 \rightarrow 0.031$ while the last iterate oscillates) — the reason a population exists.
- **LOLA reframes the opponent as a learner** and turns IPD defection into cooperation ($1.04 \rightarrow 2.82$) — the dynamic complement to Step 7’s static opponent model (Contribution #1).
- **The $N > 2$ minimax gap is the open door to the thesis** (Contribution #2): the vocabulary of Steps 2–8 crosses into MARL, but the safety anchor does not.

Step 10 Summary — Population-Based Training and Evolutionary Game Theory

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

10 Step 10 — Population-Based Training and Evolutionary Game Theory	1
10.1 1. Why populations — and why structure decides everything	2
10.2 2. The family of methods	2
10.3 3. Replicator dynamics — where a population can rest	3
10.4 4. The spinning top — transitive vs cyclic structure	4
10.5 5. The AlphaStar-style PBT league	7
10.6 6. Diversity — is the population actually diverse?	9
10.7 7. EGTA — evaluating the population, and a surprise	11
10.8 8. Honest notes, limitations, and where this hands off	13
10.9 Key takeaways for the thesis synthesis	14

10 Step 10 — Population-Based Training and Evolutionary Game Theory

This is a ground-up chapter on training and evaluating **populations** of agents: the evolutionary mathematics that says whether a population can settle at all, the transitive/cyclic structure that decides whether self-training converges or spins, and a small AlphaStar-style **league** built on a solvable poker game. It serves two purposes — a self-contained refresher for later steps and a primary source for the thesis’s population-level contributions. It is written to be read on its own. **All experimental numbers reported here were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (analytic ESS/Nash for the matrix games; Step 07’s exact best-response exploitability for Leduc). Where a run contradicted what theory led me to expect, I keep the original expectation and reconcile it with what happened — those gaps are the most instructive parts of the step.

Where this sits in the thesis. Steps 2-8 solved *one* fixed game; Step 9 added a *second* learner. Step 10 goes to a *whole population* that trains against itself. It carries three thesis hooks. **Main exploiters** are automated opponent modelers — the population lift of Step 7’s static read (Contribution #1). The **AlphaStar league** is a population-level safe-exploitation mechanism, but one whose guarantee is *missing* — the exact place Contribution #2 lives. And **EGTA / meta-Nash** is the population evaluation

10.1 1. Why populations — and why structure decides everything

Self-play has a famous failure mode: a player that only ever trains against its current self can go around in circles. If the game is **rock-paper-scissors**, “get better” has no meaning — Rock beats Scissors, loses to Paper, and improving against one opponent just makes you worse against another. The single most important idea in this step is that games have two kinds of structure mixed together: a **transitive** part (a genuine skill ladder — there *is* a better player) and a **cyclic** part (a wheel of counters — there is only *what beats what*). Whether a self-training population converges to something good or spins forever is decided by *which part dominates*, not by the learning rate.

A picture to hold onto: a **dojo**. The **main** students are what you are trying to produce. You keep a room full of **sparring partners** whose entire job is to find and punish a specific weakness in the current students (exploiters). And you keep a **museum of past champions** — frozen snapshots — so nobody wins by forgetting how to beat an old style. Training is then a loop: students spar, the strong ones get copied and slightly mutated (population-based training), and periodically a copy of everyone is frozen into the museum. The league is that dojo, made mathematical.

Read more: Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML* — the spinning-top geometry of transitive vs cyclic structure; and Jaderberg, M. et al. (2017). “Population Based Training of Neural Networks.” *arXiv:1711.09846*.

10.2 2. The family of methods

Every method here is a different way to turn “a population” into a training signal or an evaluation.

Approach	What it does	Reach for it when	Main weakness
Self-play	train the latest policy against (a copy of) itself	transitive games; a quick baseline	cycles / forgets in cyclic games; last iterate does not converge
PSRO	population + meta-Nash + best-response oracle; add the BR, repeat	competitive games; want a game-theoretic target	a full best response per round; the <i>mixture</i> is not the answer (see §7)
PBT	train many agents; copy the fitter ones (exploit) + perturb hyper-parameters (explore)	any population; online hyper-parameter search	can collapse diversity or regress (see §5)

Approach	What it does	Reach for it when	Main weakness
AlphaStar league	PBT with three agent <i>types</i> (main / main-exploiter / league-exploiter) + freezing + PFSP	non-transitive games where naive self-play cycles	many agents, much compute; heuristic, no safety guarantee

Two evaluation tools cut across them. **Replicator dynamics** (§3) are the continuous idealization of selection — they say where a population *flows* and where it can *rest*. The **spinning-top decomposition** (§4) measures how transitive vs cyclic a game (or a population) is. Together they are the diagnostic that predicts, *before* you spend the compute, whether the league in §5 will settle or spin.

Historically the line runs: population-based training as online hyper-search (PBT, 2017) → game theory over a policy population (PSRO, 2017) → the geometry that explains *why* you need a population (spinning-top, 2019) → and the full three-type league that beat human pros (AlphaStar, 2019), evaluated with empirical game theory (EGTA, Tuyls et al., 2020).

10.3 3. Replicator dynamics — where a population can rest

The **replicator equation** is the simplest model of selection: a strategy’s share grows in proportion to how much it beats the *population average*,

$$\dot{x}_i = x_i [f_i(x) - \bar{f}(x)], \quad \bar{f}(x) = \sum_j x_j f_j(x).$$

Its rest points are exactly Nash equilibria; the *stable* rest points are **evolutionarily stable strategies (ESS)** — mixes that, once adopted by the whole population, cannot be invaded by a small mutant share. This is the continuous-time idealization of what a PBT league does discretely: copy the fitter agents (selection) and perturb them (mutation).

Run on four canonical symmetric games (deterministic dynamics, so the outcome is unambiguous), the measured results match theory exactly:

Game	Analytic reference	Measured final	Converged?
Prisoner’s Dilemma	Defect dominates; ESS (0, 1)	[0.0, 1.0]	yes
Hawk-Dove	interior ESS $p(\text{Hawk}) = V/C = 0.5$	[0.5, 0.5] (orbit radius 0.0)	yes

Game	Analytic reference	Measured final	Converged?
Rock-Paper-Scissors	Nash = uniform; no ESS	orbits centre, radius 0.095	no
Stag Hunt	two pure ESS (basin-dependent)	$[0.8, 0.2] \rightarrow [1, 0]$; $[0.2, 0.8] \rightarrow [0, 1]$	yes

Prisoner’s Dilemma collapses to the dominant strategy; Hawk-Dove *reaches* the interior 0.5 ESS; Stag Hunt picks a different pure ESS depending on the basin its start lands in. **Rock-Paper-Scissors never converges** — its interior fixed point is a *centre*, so the population orbits it forever. That last row is the whole motivation for the rest of the step: a cyclic game has no stable population.

Read more: Hofbauer, J. & Sigmund, K. (1998). *Evolutionary Games and Population Dynamics* (Cambridge) — replicator dynamics, ESS, and the RPS centre; and the Bloembergen–Tuyts survey (JAIR, 2015) connecting replicator dynamics to multi-agent learning.

10.4 4. The spinning top — transitive vs cyclic structure

If cyclic games are the problem, we need to *measure* how cyclic a game is. The **spinning-top decomposition** (Balduzzi et al.) splits a game’s payoff matrix into a **transitive** component (a skill ladder, captured by per-strategy ratings) and a **cyclic** component (what is left over — the rock-paper-scissors part). The name comes from the shape: real games are widest (most cyclic) among mediocre strategies and narrow to a point (purely transitive) at the extremes of skill.

A subtlety worth flagging: the raw step suggested an **SVD rank-1** decomposition, which wrongly reports Rock-Paper-Scissors as ≈ 0.707 transitive — a rank-1 skew-symmetric approximation simply cannot represent a pure cycle. The implementation uses the **combinatorial-Hodge** (ratings-difference) method instead, which correctly gives RPS a transitive ratio of 0.0. Measured:

Population	Transitive (Hodge)	Cyclic	Structure
Rock-Paper-Scissors	0.0	1.0	purely cyclic
Pure skill ladder	1.0	0.0	purely transitive
PSRO-Leduc	0.41-0.46	0.89-0.91	mostly cyclic (27 three-cycles)
best-response			
meta-game			
League snapshot	0.94-0.98	—	mostly transitive
meta-game			

The two *real* populations are the headline — and they disagree, on the same game.

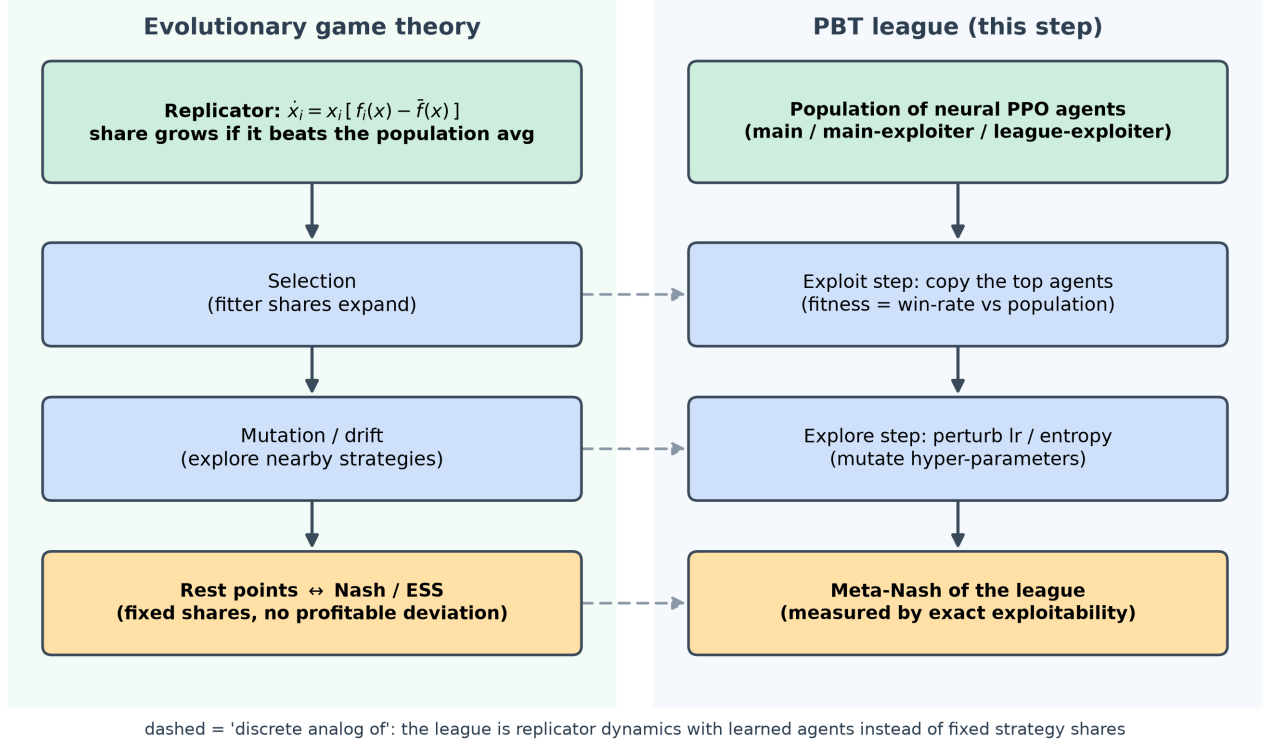


Figure 1: Evolutionary game theory as the continuous idealization of a PBT league: replicator selection copying the fitter agents, mutation/drift perturbing learning rate and entropy, and replicator rest points the meta-Nash / ESS the league is trying to reach. The league is replicator dynamics with learned agents instead of fixed strategy shares.

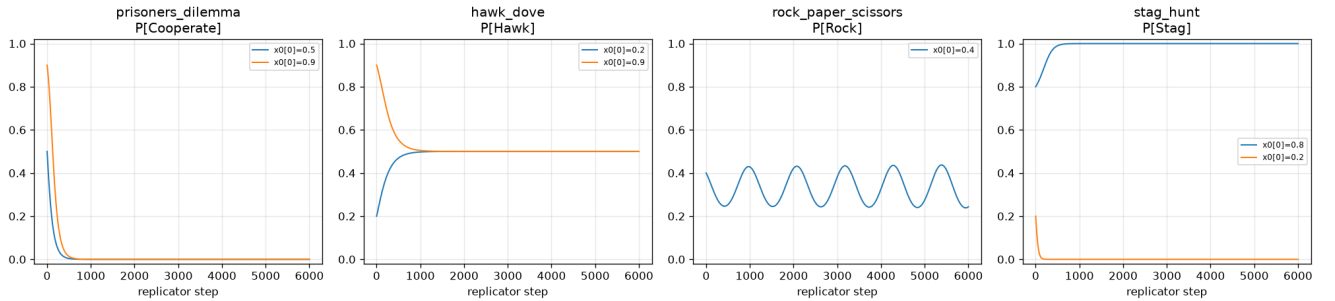
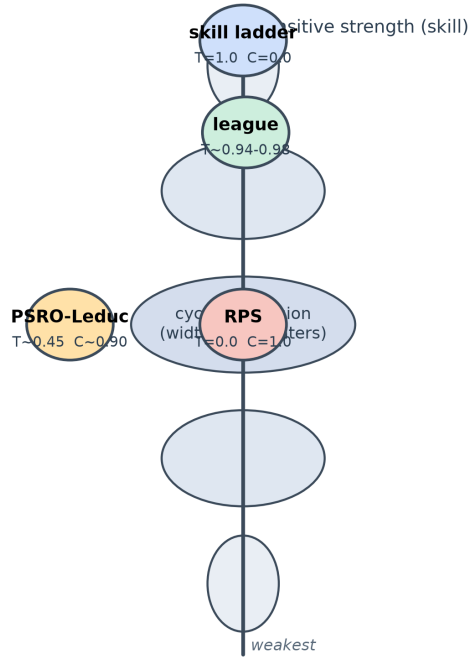


Figure 2: Replicator phase portraits: Prisoner's Dilemma $\rightarrow$ all-Defect, Hawk-Dove $\rightarrow$ the 0.5 interior ESS, Rock-Paper-Scissors $\rightarrow$ a closed orbit that never converges, and Stag Hunt $\rightarrow$ two basins (all-Stag or all-Hare). The non-converging RPS orbit is why populations in cyclic games need explicit machinery to avoid spinning.



same game, different populations: a best-response population (PSRO) sits in the wide cyclic belly; a training-snapshot population (league) climbs the transitive spine.

Figure 3: The spinning top: a vertical transitive axis (skill — there is a better player) and a cyclic dimension (width — the number of counters, rock-paper-scissors structure). Rock-Paper-Scissors sits in the widest cyclic belly (transitive ratio 0.0); a pure skill ladder sits on the transitive spine (1.0); the PSRO-Leduc best-response meta-game sits in the wide cyclic belly (~ 0.45 transitive), while the league’s snapshot meta-game climbs the transitive spine ($\sim 0.94-0.98$).

Reconciliation (kept prediction $\rightarrow$ what actually happened). I framed poker as a skill ladder and expected Leduc’s meta-game to be mostly transitive. Measured, it depends entirely on *which population you decompose*. A population of **best responses** (PSRO) is mostly **cyclic** (≈ 0.45 transitive, 27 three-cycles): the best response beats the current mixture, a newer best response beats *that*, and so on — Balduzzi’s spinning-top in action. A population of **training-trajectory snapshots** (the league) is mostly **transitive** (≈ 0.94 - 0.98), because later snapshots are usually stronger than earlier ones, forming a ladder. Neither is a bug; the transitive/cyclic ratio is a property of the *population*, and choosing how you build the population is choosing whether you see a wheel or a ladder.

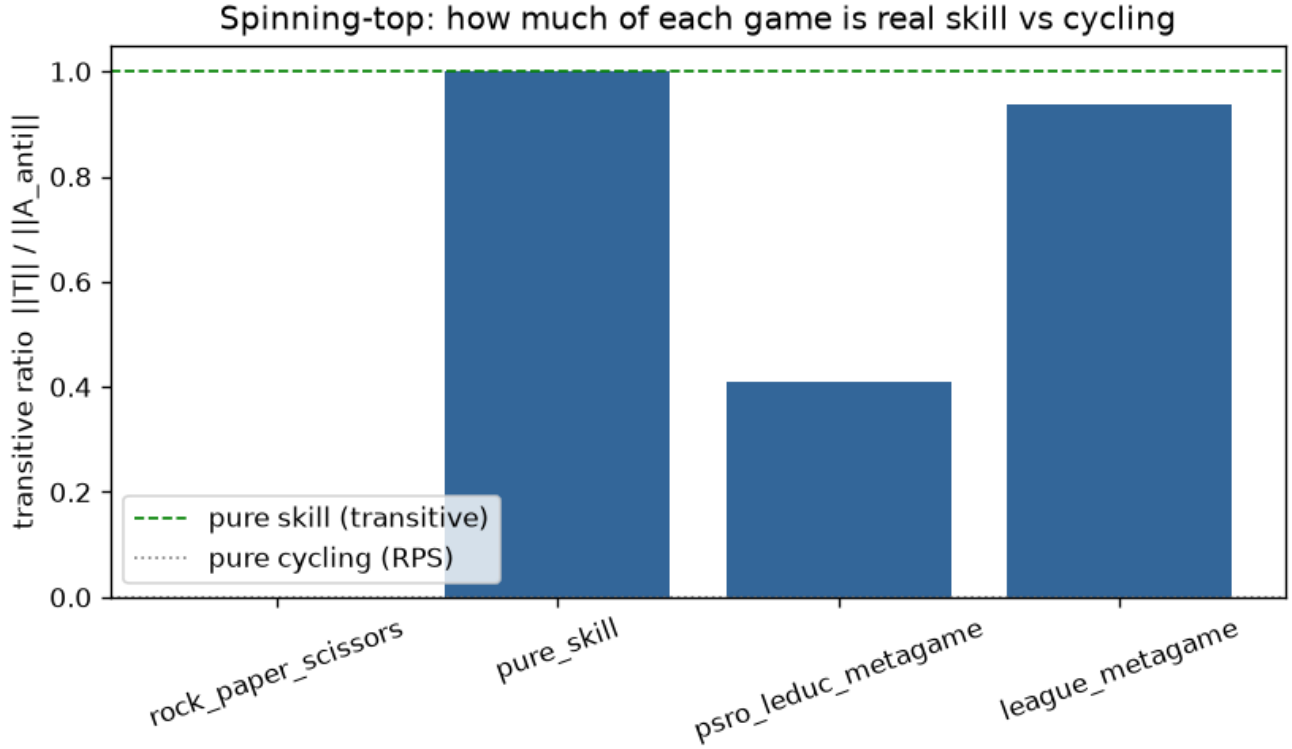


Figure 4: Transitive ratio across four populations: RPS (0.0, purely cyclic), a pure skill ladder (1.0), the PSRO-Leduc best-response meta-game (~ 0.41 - 0.46 , mostly cyclic), and the league snapshot meta-game (~ 0.94 - 0.98 , mostly transitive). Same game, opposite structure, depending on how the population is built.

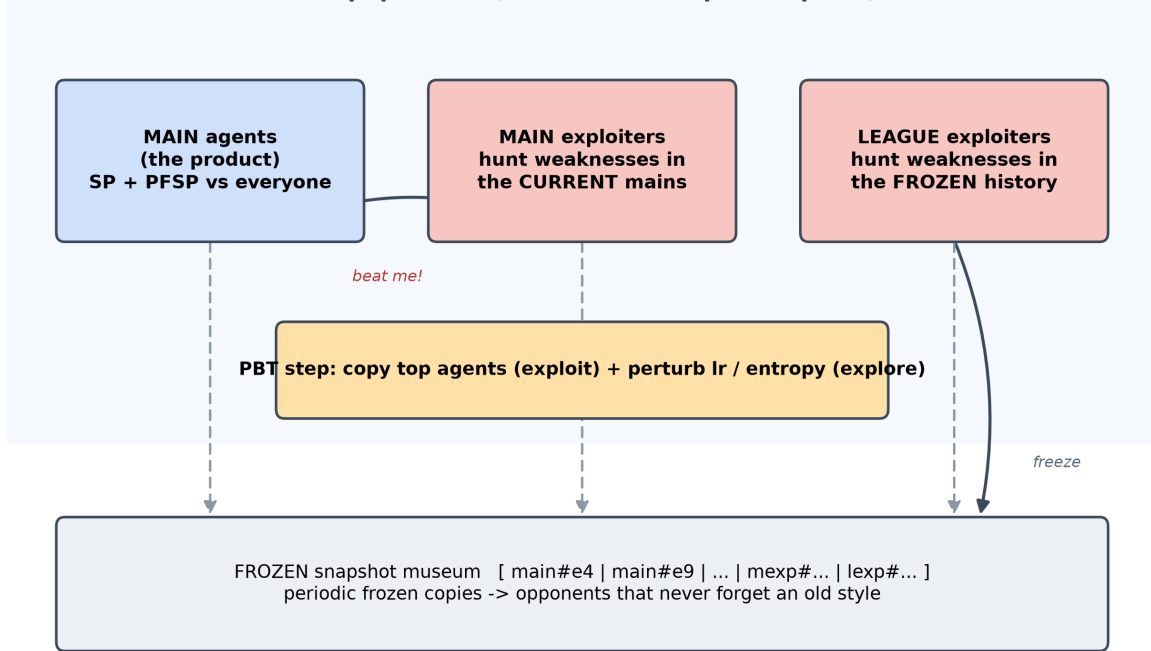
Read more: Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML*; and Czarnecki, W. M. et al. (2020). “Real World Games Look Like Spinning Tops.” *NeurIPS*.

10.5 5. The AlphaStar-style PBT league

With the diagnostic in hand, we build the dojo. The league trains neural PPO agents on **Leduc Hold'em** with three agent types: **main** agents (the product; they play everyone via PFSP self-play), **main ex-**

plotters (hunt weaknesses in the *current* mains), and **league plotters** (hunt weaknesses anywhere in the *frozen history*). Population-based training copies the top agents (exploit) and perturbs their learning rate / entropy (explore); periodically a frozen snapshot of each agent is added to the museum; **PFSP** matchmaking samples each opponent with probability rising in how hard it is to beat. Critically, every neural network is extracted to a **tabular** policy so Step 07's **exact** best response measures its exploitability — the same NashConv used since Step 2.

PFSP matchmaking: sample each opponent with probability rising in how hard it is to beat (focus training where you are losing).



measured: main exploitability falls 4.73 -> ~1.21 then regresses to ~2.05 (scale, 120 epochs); best frozen snapshot 1.31 beats PSRO 2.16 and self-play 3.68.

Figure 5: The league: main agents (the product), main plotters (hunt weaknesses in the current mains), and league plotters (hunt weaknesses in the frozen history), with periodic freezing into a snapshot museum and PFSP matchmaking that focuses training where the agent is losing. PBT copies the top agents and perturbs their hyper-parameters.

Does it improve? Measured, over two configs (smoke: 7 agents, 15 epochs; scale: 8 agents, 120 epochs, 48 frozen snapshots):

Metric	Smoke	Scale
min-main exploitability	4.67 → 3.04 (ends at min)	4.73 → min ≈ 1.21 → 2.05
meta-Nash exploitability	4.73 → 3.04	5.01 → min ≈ 1.32 (plateau 1.60) → 2.96
final Elo (live agents)	1176-1211	1198-1210

Reconciliation (kept prediction → what actually happened). I predicted a *monotone*

decrease in exploitability. Smoke’s 15 epochs oblige — a clean drop that ends at its minimum. But scale’s 120 epochs tell the real story: exploitability falls steeply to a minimum near epoch 60 (min-main ≈ 1.21 , meta-Nash bottoming ≈ 1.32 then holding a ≈ 1.60 plateau), and then **regresses** back up to ≈ 2.05 / ≈ 2.96 by epoch 119. The best agents are the *frozen snapshots* from mid-run; the *live* main agents get worse late, chasing their exploiters (churn / partial forgetting). This is only visible once training is long enough — a running league is not a monotonically improving one. The remedy (untested here) is best-snapshot retention / population regularization. Methodologically it echoes Step 9: **scale reveals what smoke hides**.

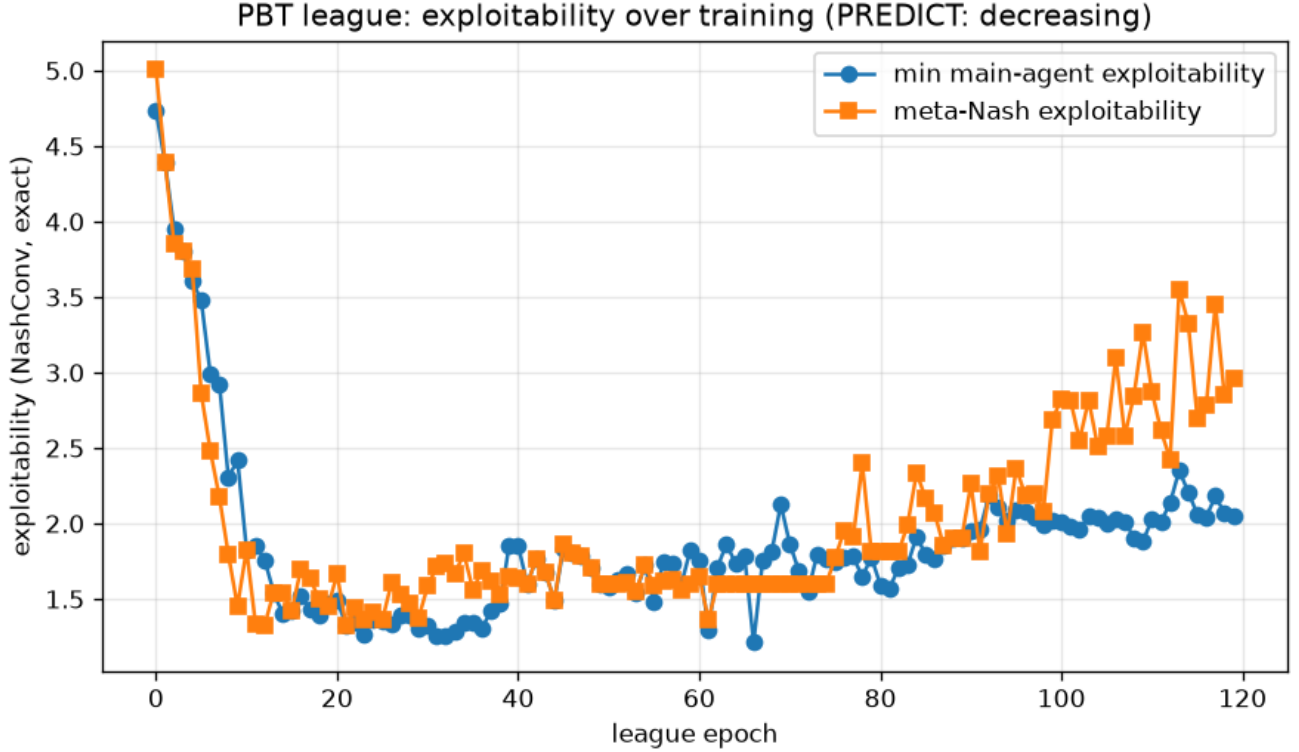


Figure 6: League exploitability over 120 epochs (scale): both min-main and meta-Nash exploitability fall to a minimum near epoch 60 and then regress upward. The frozen snapshots capture the strong mid-run agents; the live agents degrade late under exploiter pressure.

10.6 6. Diversity — is the population actually diverse?

A league is only as good as the *variety* of strategies it holds. We measure three things: the **effective population size** (participation ratio of the meta-Nash weights), **behavioral clustering** (are the policies actually different?), and **exploit coverage**. Measured, the population is only weakly diverse: participation ratio rises from 1.0 (smoke) to 1.9 (scale), but behavioral clustering collapses everything into a **single cluster** at both scales — even at scale, where the maximum pairwise behavioral distance (0.48) exceeds the clustering threshold (0.30), single-linkage merges the chain. Diversity here is *weight-level* (the

meta-Nash spreads support over a few agents), not *behavior-level* (the agents play near-identically).

The mechanism is clearest on a fast toy — a mini-PBT on matrix games:

Game	Diversity over generations
Prisoner’s Dilemma (transitive)	collapses to 0 (everyone $\rightarrow$ Defect)
Rock-Paper-Scissors (cyclic)	churns forever (0.07-0.29), never settles

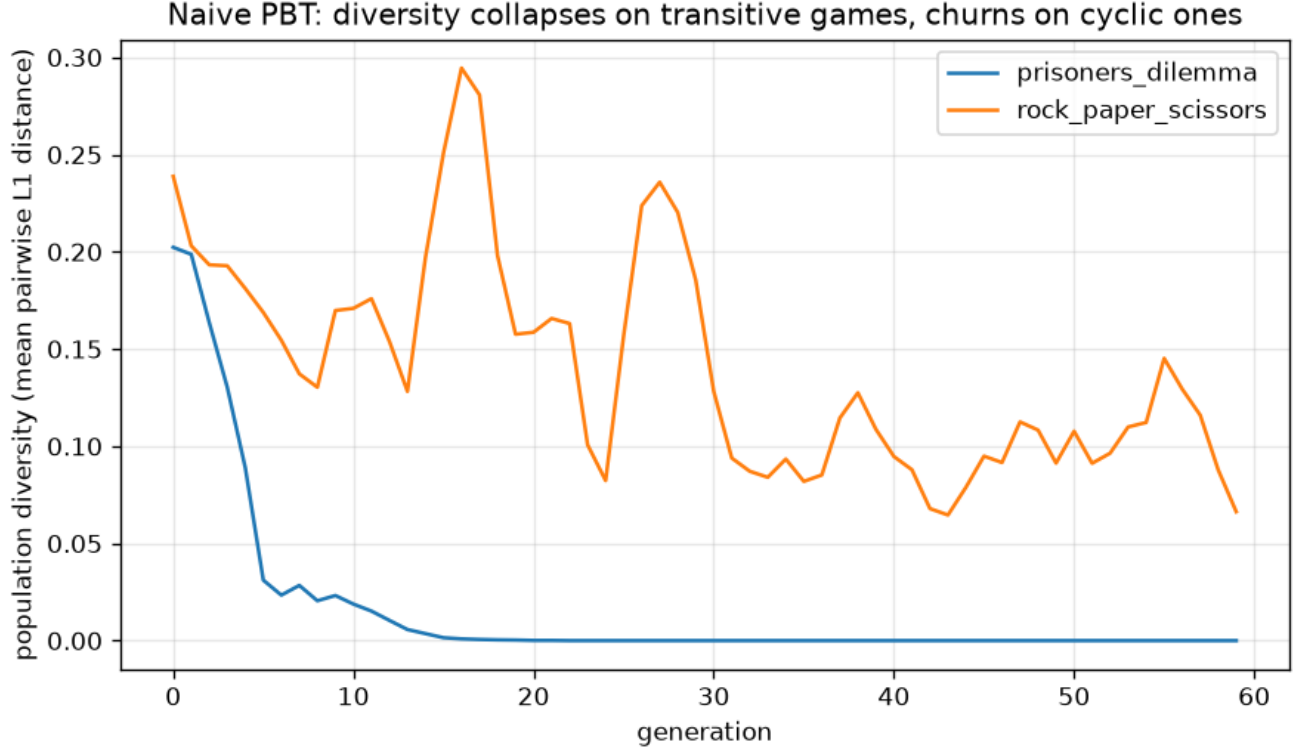


Figure 7: Mini-PBT diversity: on the transitive Prisoner’s Dilemma the population collapses to a single strategy (diversity $\rightarrow$ 0), while on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29) as it chases the wheel of counters. Game structure, not population size, decides whether diversity survives.

Transitive games *kill* diversity (there is one best, everyone converges to it); cyclic games *force* it (there is no best, so the population keeps churning). The Leduc league sits near the transitive end, which is exactly why its diversity is thin — and why the AlphaStar diversity benefits reported at $\sim$ 600 agents do not materialize at this scale.

Read more: Vinyals, O. et al. (2019). “Grandmaster level in StarCraft II using multi-agent reinforcement learning.” *Nature* (AlphaStar; the league and PFSP).

10.7 7. EGTA — evaluating the population, and a surprise

The last tool is **empirical game-theoretic analysis (EGTA)**: treat whole policies as the “strategies” of a higher game, play every pair to fill an empirical payoff matrix, solve its **meta-Nash** mixture, and score it. It is the population lift of exploitability — Nash of a game whose atoms are policies.

EGTA = Nash of a game whose 'strategies' are whole policies -- the population lift of exploitability.

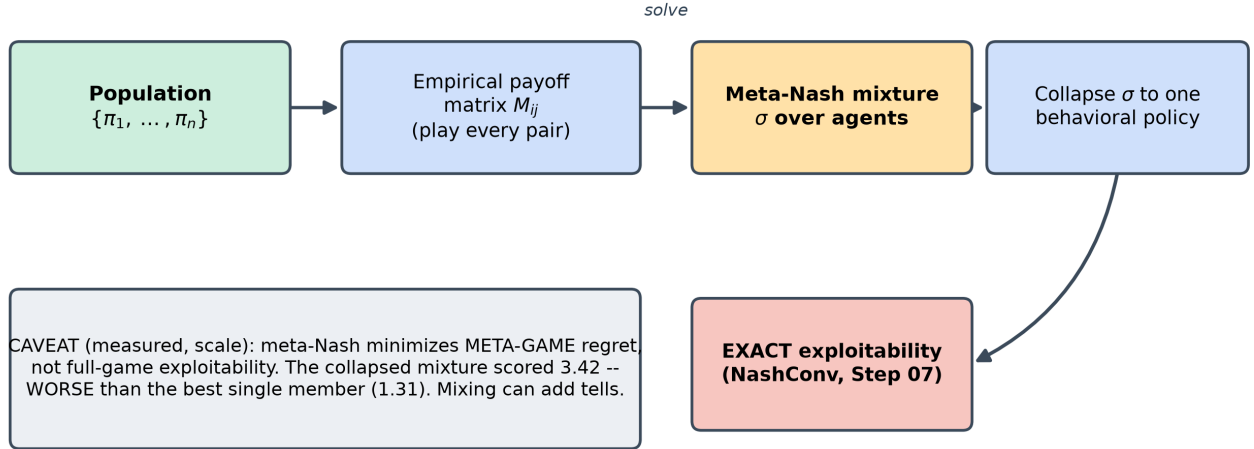


Figure 8: The EGTA pipeline: play every pair of agents to build an empirical payoff matrix, solve its meta-Nash mixture, collapse the mixture to a single behavioral policy, and measure its EXACT full-game exploitability. The measured caveat: the meta-Nash minimizes meta-game regret, not full-game exploitability, so the mixture can be more exploitable than the best single member.

The expectation (from the raw step’s checklist) was that the meta-Nash of the league would be *less* exploitable than any single member. Measured:

Config	meta-Nash exploitability	best individual	meta-Nash best?
Smoke	2.665	2.665	yes (all weight on the best agent)
Scale	3.418	1.305	no

Reconciliation (kept prediction → what actually happened). I expected the meta-Nash mixture to be at least as unexploitable as its best member. Smoke confirmed it trivially — the meta-Nash put *all* weight on the single best agent, so meta = best = 2.665. At scale the meta-Nash spreads weight (0.645 on one agent plus a tail), and the collapsed behavioral mixture scores 3.418 — **worse** than the best single snapshot (1.305). This is not a mixing bug (the identical code path gave meta = best in smoke). The meta-Nash minimizes **meta-game regret** — doing well *against the population* — which is a *different objective* from minimizing

full-game exploitability; and a realization-weighted mixture of behavioral policies can be *more* exploitable than its components, because the blend introduces information-set “tells” that an exact best responder punishes. The lesson inverts: what a league should *ship* is a selected, best-response-robust member — not the meta-Nash mixture.

How does the league compare to the alternatives on Leduc exploitability?

Method (scale)	Final exploitability
CFR-Nash (the floor)	0.0099
League — best individual	1.305
PSRO (exact best-response oracle)	2.163
Self-play	3.683
League — meta-Nash mixture	3.418

The league’s **best individual** is the strongest learned result — beating exact PSRO and self-play — though all learned methods remain far above the CFR-Nash floor. The league’s **mixture** is the weakest, worse even than self-play (the §7 reconciliation). Ship the member, not the mixture.

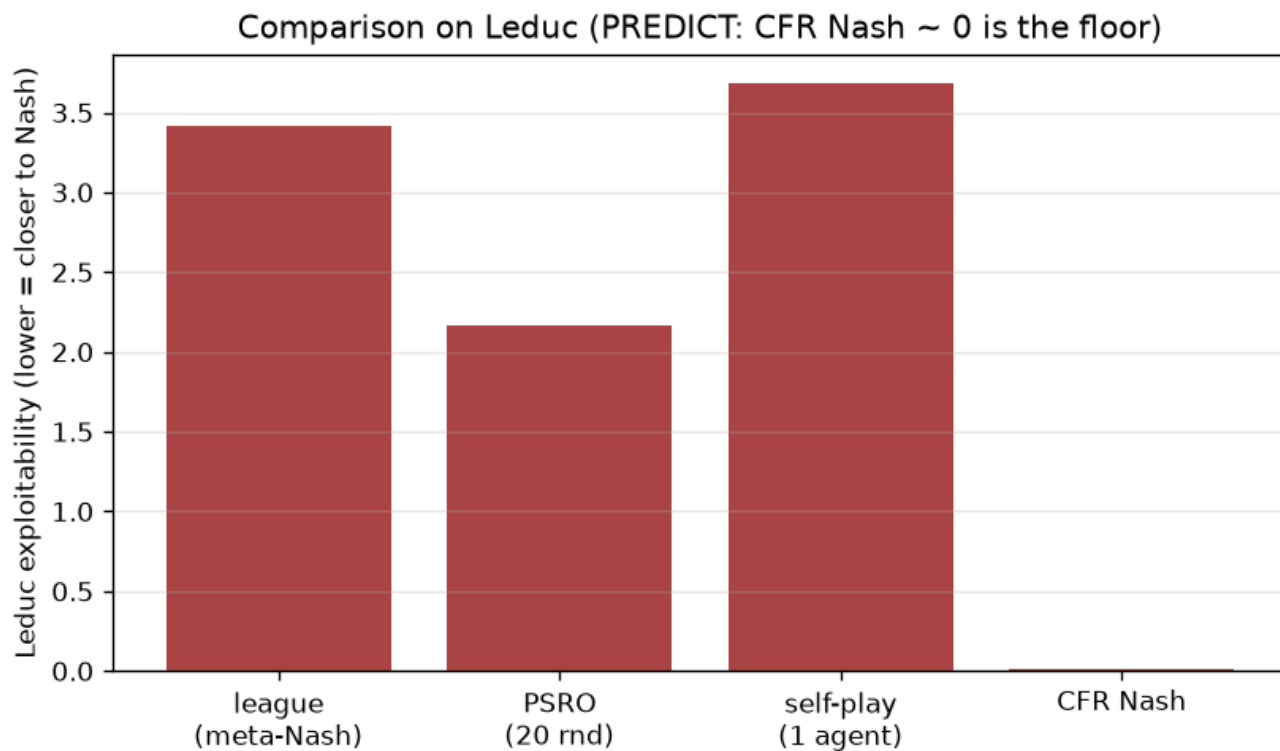


Figure 9: Final Leduc exploitability by method (scale): CFR-Nash floor ~ 0.01 ; the league’s best individual (1.31) beats PSRO (2.16) and self-play (3.68); the league’s meta-Nash mixture (3.42) is the weakest learned result.

Read more: Lanctot, M. et al. (2017). “A Unified Game-Theoretic Approach to Multia-

gent Reinforcement Learning.” *NeurIPS* (PSRO); and Tuyls, K. et al. (2020). “Bounds and dynamics for empirical game-theoretic analysis.” *AAMAS/JAAMAS* (EGTA).

10.8 8. Honest notes, limitations, and where this hands off

What held up. The confirmed backbone: replicator dynamics reproduce every analytic ESS/Nash, including RPS’s non-converging orbit; the Hodge spinning-top decomposition cleanly separates skill from cycles and correctly labels the pure cases; the league drives strong *early* improvement and produces a best individual (1.305) that beats PSRO (2.163) and self-play (3.683); and EGTA gives a working population-level exploitability. Together they trace the arc from “which games can a population even settle in?” to “what happens when a population trains itself?”

What did not, and why it matters. Three honest caveats travel forward. (1) The league’s exploitability **regresses late** at scale ($4.73 \rightarrow \approx 1.21 \rightarrow \approx 2.05$) — a running league is not a monotonically improving one; the best agents are frozen snapshots. (2) The **meta-Nash mixture is more exploitable than its best member** at scale (3.42 vs 1.31) — mixing behavioral policies does not guarantee robustness, so ship a member, not the mixture. (3) Leduc’s meta-game is **cyclic as a best-response population** but **transitive as a snapshot population** — structure is a property of the population you build. And a methodological point: the late regression and the mixing failure are both invisible at the fast smoke config — scale reveals what smoke hides.

Trust. Every evolutionary target is *analytic* (ESS/Nash of the matrix games), and every league exploitability is Step 07’s *exact* NashConv on tabular-extracted policies — so the game-theoretic results are bounded by ground truth. The neural results are a single PBT run per config: the *directions* (early improvement, best-individual < PSRO < self-play, late regression, meta-Nash > best member) are the trustworthy claims, not the third-decimal magnitudes. The experiment PNGs above are produced from the committed JSON artifacts (see `../figures/README.md`); the conceptual diagrams are generated by the `make_*_figure.py` scripts.

Backward and forward connections. Backward: the league is Step 9’s PSRO made asynchronous with neural oracles, and it reuses Step 07’s exact best response wholesale; the “meta-Nash of a population” is Step 2’s Nash lifted one level. Forward: main exploiters are automated opponent modelers (Contribution #1); EGTA/meta-Nash is the evaluation methodology (Contribution #3); and the league’s missing guarantee — it can regress and its mixture can be exploitable — is the population form of the **missing $N > 2$ safety anchor** (Contribution #2). The transitive/cyclic diagnostic predicts Step 11’s FFA coalition games will be strongly cyclic, so naive PBT there will cycle.

Open questions. Does best-snapshot retention / population regularization remove the late regression (a training fix), or is it inherent to the three-type design (a design fix)? Should population evaluation report a selected member or a diversity-regularized meta-solver instead of the meta-Nash mixture? And underneath both: with no minimax value for a population, what does “safe” mean, and can the league’s exploiter mechanism be given a guarantee rather than remaining heuristic?

10.9 Key takeaways for the thesis synthesis

- **Structure decides convergence.** The transitive/cyclic (spinning-top) ratio is a *pre-training diagnostic*: transitive games settle (and kill diversity), cyclic games spin (and force diversity). Measured: RPS 0.0 transitive, skill ladder 1.0, PSRO-Leduc ≈ 0.45 (cyclic), league snapshots ≈ 0.94 -0.98 (transitive).
- **The league produces strong individuals but carries no guarantee.** Best snapshot 1.305 beats PSRO 2.163 and self-play 3.683 — yet exploitability *regresses* late ($\rightarrow 2.05$) and the meta-Nash *mixture* (3.42) is worse than its best member (1.31).
- **Meta-Nash optimizes meta-game regret, not full-game exploitability** — the two disagree, so population evaluation must score the collapsed mixture directly and ship a selected member.
- **Exact evaluation is the anchor.** Extracting neural policies to tabular and grading them with Step 07’s exact best response makes every population claim ground-truthed — the same NashConv since Step 2.
- **The population-safety gap is the open door (Contribution #2):** the AlphaStar exploiter mechanism is the population analog of Step 8’s safe exploitation, but it is heuristic — it neither guarantees monotone improvement nor a non-exploitable mixture.

Step 11 Summary — Dynamic Coalition Formation in Competitive FFA Games (So Long Sucker)

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

11 Step 11 — Dynamic Coalition Formation in Competitive FFA Games	1
11.1 Why the third player changes everything	2
11.2 The coalition detector — reading alliances off the moves	2
11.3 Shapley credit for a competitive game	3
11.4 Coalition-aware MAPPO — and when coalitions actually emerge	7
11.5 EGTA and the spinning top — is SLS a wheel or a ladder?	9
11.6 Honest notes, limitations, and where this hands off	10
11.7 Key takeaways for the thesis synthesis	11

11 Step 11 — Dynamic Coalition Formation in Competitive FFA Games

This is a ground-up chapter on the thing that only exists once a game has a **third player**: temporary **alliances** that form, get exploited, and get betrayed. It builds the first coalition-aware reinforcement-learning treatment of **So Long Sucker (SLS)** — the 1950 game Nash, Shapley, Shubik & Hausner designed to study exactly this — and uses it to internalize the jump from $N = 2$ to $N \geq 3$, where **Nash and exploitability stop being tractable and stop being meaningful**, so “did it work?” can no longer be a single number. It serves two purposes — a self-contained primer for later steps and a primary source for the thesis’s multi-agent contributions. It is written to be read on its own. **All experimental numbers reported here were measured** on reproducible runs and, wherever possible, are bounded by *exact* references (textbook Shapley/core values for the cooperative-game toys; an exact 2-player minimax solver for the SLS endgame). Where a run contradicted what I expected — including a real engine bug — I keep the original expectation and reconcile it with what happened; those gaps are the most instructive parts of the step.

Where this sits in the thesis. Steps 2-10 all leaned on a 2-player game with an *exact* best-response oracle. Step 11 removes it and enters the frontier. Three thesis hooks live here. The **coalition detector** lifts opponent modeling from “what kind of player is this?” to “who is allied with whom?” (Contribution #1). The **safe baseline loses its Nash anchor** — with an empty core there is no stable allocation,

so “safe” must become behavioral/population-based (piKL), the gap this step frames but does not close (Contribution #2). And the **EGTA meta-game + Shapley credit** replaces exploitability, which has no meaning against a coalition (Contribution #3).

11.1 Why the third player changes everything

With two players, a zero-sum game has a *value*: there is a minimax-optimal strategy, and “how far from optimal are you?” (exploitability) is a single, meaningful number that anchored every step since Step 2. Add a third player and the ground shifts. Now two players can **gang up** on the third, and the interesting question is not “what is the equilibrium?” but “who allies with whom, for how long, and who betrays first?” Nash equilibrium in a 4-player free-for-all is both **intractable** to compute and **strategically empty** — it says nothing about the coalitions that actually decide the game.

A picture to hold onto: a **dinner party** with four guests and one house. Everyone pairs up to get things done, but the smartest move is to have quietly betrayed your partner one turn *before* they betray you. So Long Sucker makes this literal: you hold coloured chips, you place them into other players’ piles (a handshake) or capture their piles (the knife), and you are eliminated when you run out. The whole alliance is **encoded in the moves themselves** — there is no separate negotiation phase to model, which is exactly why a detector can read the coalition straight off the move stream.

The consequence for methodology is the single most important idea in the step: **at $N \geq 3$ you trade exact evaluation for empirical evaluation**. No exploitability; instead win-rate, a coalition score from the detector, and the cyclic ratio of an empirical meta-game — all anchored by the one subgame that *is* exactly solvable, the 2-player endgame.

Read more: Nash, Shapley, Shubik & Hausner (1950s), *So Long Sucker* (the game itself); and De Carufel, J. & Jerade, M. — the 2-player SLS endgame analysis that gives the one exact anchor.

11.2 The coalition detector — reading alliances off the moves

The first tool generalizes opponent modeling. Instead of inferring a hidden *type* or *hand* (Step 7), the detector infers a hidden *social structure*. It watches the move log and accumulates two matrices: **help** (player i placed a chip into player j ’s pile) and **harm** (player i captured player j ’s pile). Their difference is **net support**; the pair with the highest mutual net support is the strongest coalition.

Does it work? We script two players to systematically help each other and harm the rest, tell the detector nothing, and ask it to name the coalition. Measured, it recovers the planted $\{0, 1\}$ alliance **exactly**, with a well-separated score (10.0 on the allied pair, 0 or -1 everywhere else). The alliance is fully legible from chip placement alone.

The alliance is encoded in the moves themselves - no separate negotiation phase to model.

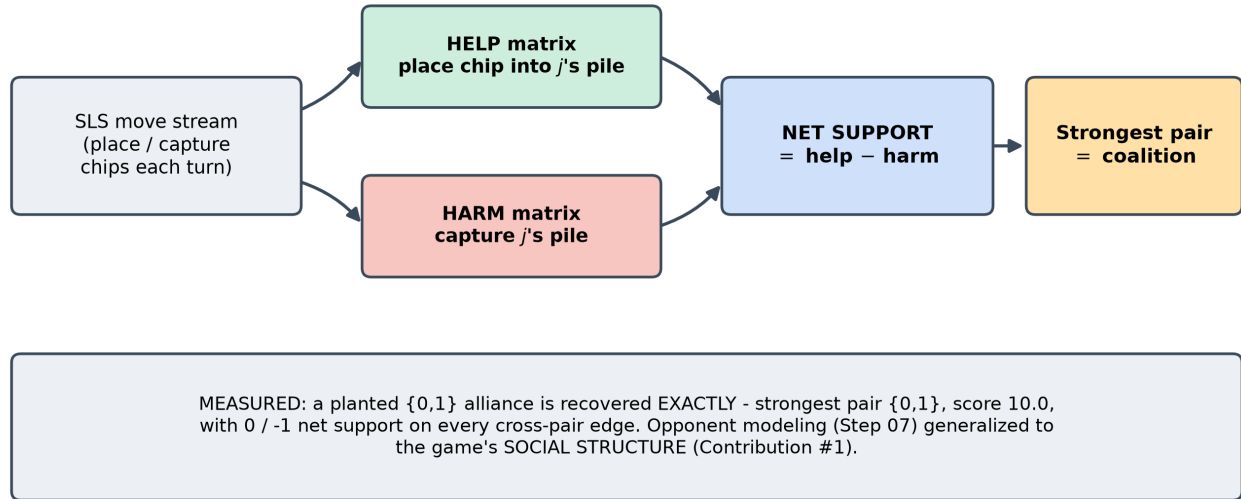


Figure 1: The coalition detector: from the SLS move stream, placing a chip into another player’s pile counts as HELP and capturing a pile counts as HARM. Accumulated into help/harm matrices and differenced into net support, a reciprocal alliance shows up as a strong mutual edge. Opponent modeling (Step 7) lifted from “what hand?” to “who is allied with whom?” (Contribution #1).

Read more: the Step 07 opponent-modeling engine (this repo) — the “observe actions -> update beliefs” principle the detector re-derives as help/harm matrices.

11.3 Shapley credit for a competitive game

If we want an agent to *learn* to form coalitions, we need a reward signal that says “you contributed to a coalition.” The classical tool is the **Shapley value**: the unique fair way to split a coalition’s worth among its members, averaging each member’s marginal contribution over all join orders. Two textbook toys pin down what “fair” and “stable” mean, and both reproduce exactly:

Cooperative-game toy	Shapley value	Core (stability)
Glove game	$(2/3, 1/6, 1/6)$	non-empty (a stable split exists)
3-player majority	$(1/3, 1/3, 1/3)$	empty (no stable split)

The **empty core** of the majority game is the conceptual heart of the step: it is a game where *any* allocation can be overturned by some coalition, so cooperation is *inherently unstable*. That is the SLS situation — and the reason N-player “safe” play cannot be anchored to a stable equilibrium (Contribution #2).

SLS is competitive, not cooperative — there is no shared pot to split — so we **redefine the coalition value** as *the probability that a member of the coalition wins*, estimated by Monte-Carlo rollouts. Each player’s credit is then the Shapley value of that win-probability function. Measured on SLS positions: a genuinely symmetric position gives near-equal credit (spread 0.013), and an asymmetric $[8, 8, 1, 1]$ position

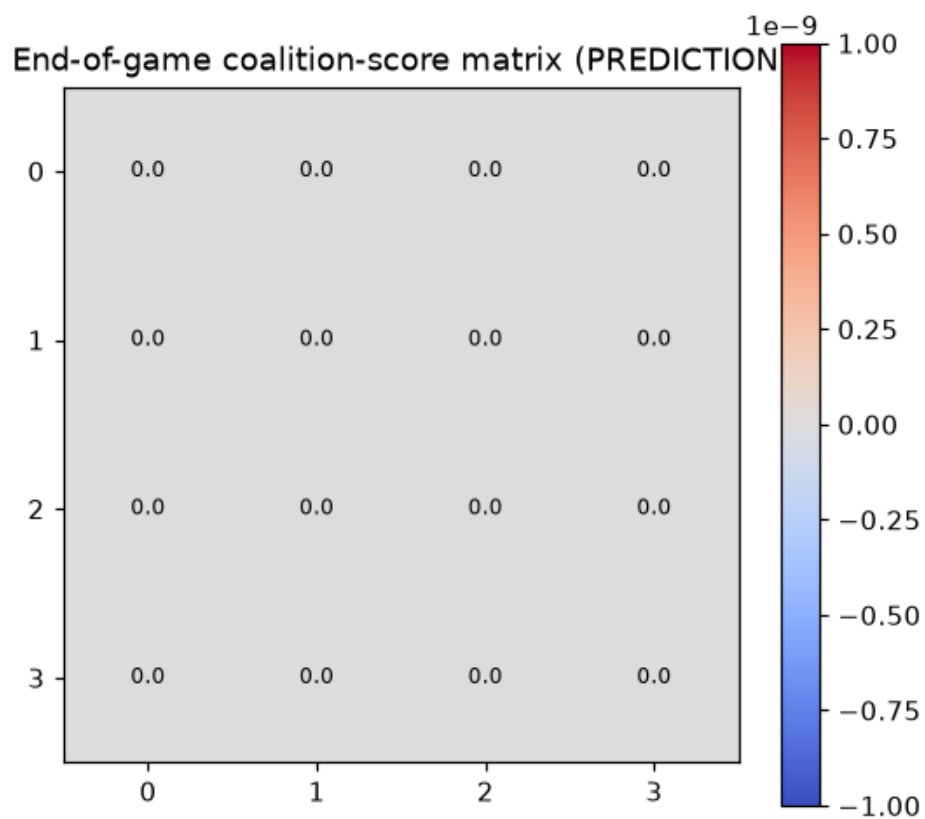


Figure 2: Coalition graph inferred purely from chip placement: the planted $\{0,1\}$ alliance appears as a strong reciprocal help edge; cross-pair edges are neutral or hostile. The detector cleanly recovers a coalition it was never told about.

Shapley = the UNIQUE fair split; adapted to a competitive game via win-probability as the value.

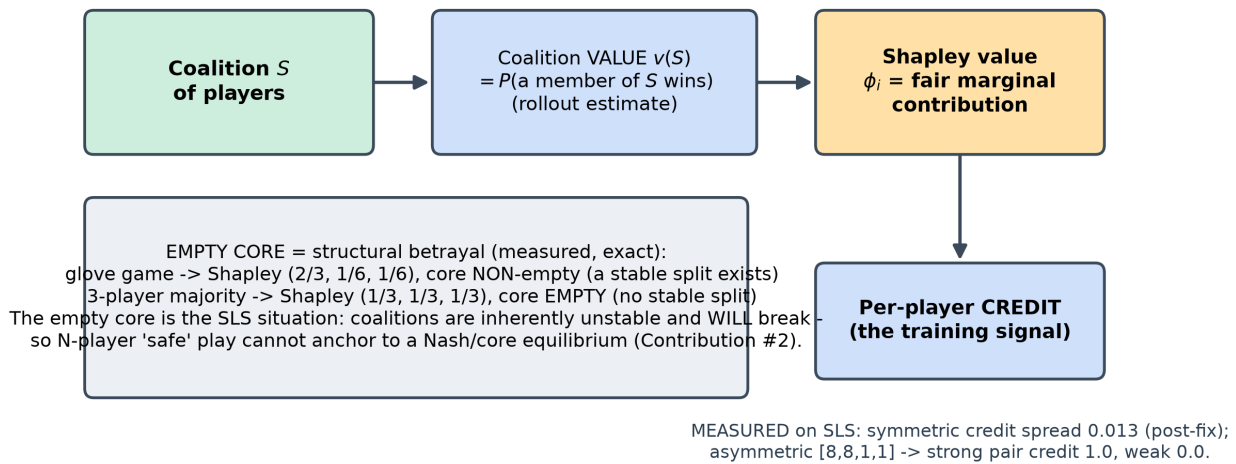


Figure 3: Shapley credit adapted to a purely competitive game: the coalition’s “value” is redefined as the PROBABILITY a coalition member wins (estimated by rollouts), and each player’s Shapley value of that win-probability function is their credit. The empty core of the majority game (no stable allocation) is the structural signature of SLS coalitions - they will break.

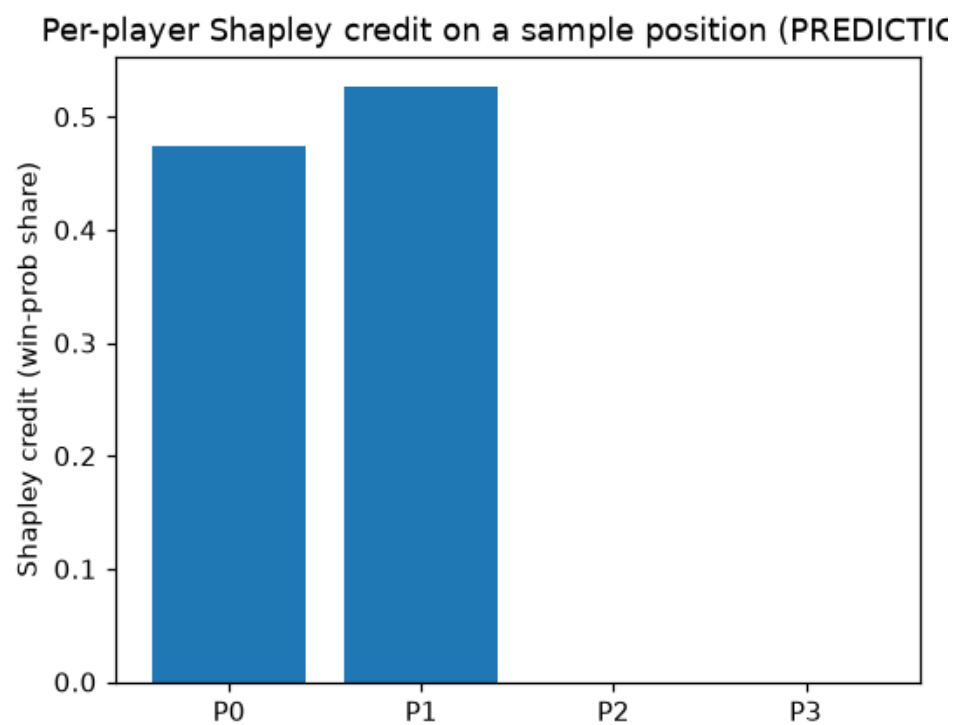


Figure 4: Shapley credit attribution on SLS positions: near-flat across seats in the symmetric case (spread 0.013, post-fix), and fully concentrated on the strong pair in the asymmetric $[8,8,1,1]$ case. Credit tracks real contribution once the engine tie-break is unbiased.

Game Theory (core, Shapley, nucleolus); and Wang et al. on Shapley-based credit assignment in MARL.

11.4 Coalition-aware MAPPO — and when coalitions actually emerge

Now we train. Each SLS seat is a masked, episodic PPO agent learning by self-play. The reward is a **blend**:

$$r = (1 - \alpha) r_{\text{sparse}} + \alpha (\text{Shapley coalition credit}),$$

where r_{sparse} is the sparse winner-takes-all reward and the credit term is the win-probability Shapley value from the *Shapley credit* section (a cheap critic-value **proxy**, or an expensive rollout **counterfactual**). The blend weight α dials between “just win” and “form coalitions.”

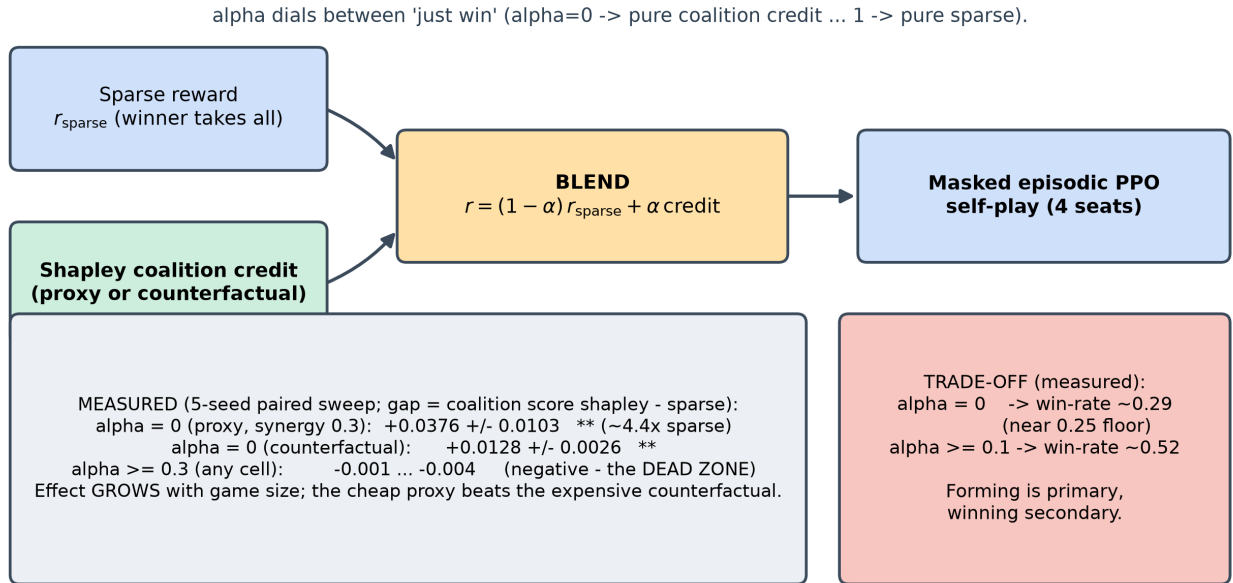


Figure 5: Coalition-aware MAPPO reward blend: the sparse winner-takes-all signal is mixed with the Shapley coalition credit by weight alpha. Measured, alpha is the dominant knob: coalitions emerge significantly only at low alpha (heavy credit weight), and every alpha ≥ 0.3 suppresses the signal. The cheap critic-value proxy beats the expensive counterfactual credit.

Do coalition-aware agents form coalitions more than sparse agents? The honest answer required a **5-seed paired sweep** over $\alpha \times$ credit-mode $\times$ synergy, because a single config misled me badly (below). Measured (paired gap = coalition score of Shapley agents minus sparse agents; ** = significant at $> 2 \times$ SE):

Regime	Paired gap (Shapley - sparse)
scale, proxy, $\alpha = 0$, synergy 0.3	$+0.0376 \pm 0.0103$ ($\sim 4.4\times$ the sparse baseline)
scale, counterfactual, $\alpha = 0$	$+0.0128 \pm 0.0026$
any tier, $\alpha \geq 0.3$	$-0.001 \dots -0.004$ (negative)
smoke, proxy, $\alpha = 0$, synergy 0.1	$+0.0024 \pm 0.0008$ (tiny)

Reconciliation (kept prediction $\rightarrow$ what actually happened). My single-config runs used the default $\alpha = 0.3$ and showed the coalition signal collapse at scale — which I first read as “the proxy credit is too weak once training is longer.” The sweep overturned that completely: **α is the dominant knob, and 0.3 is a dead zone.** Coalitions emerge significantly *only at low α* (at $\alpha \approx 0$ the Shapley agent beats sparse by $+0.038$, $\sim 4.4\times$), while *every $\alpha \geq 0.3$* cell is negative — the sparse winner-takes-all term drowns the coalition signal. Two further surprises: the effect **grows with game size** (opposite to my “smoke-positive / scale-null” read, which was an artifact of holding $\alpha = 0.3$ at both tiers), and the **cheap proxy beats the expensive counterfactual**. So the primary thesis signal (coalition-forming) is real and robust — I had simply measured it in the one regime where the sparse term hides it. The fix is “weight the coalition credit heavily,” not “compute a truer credit.”

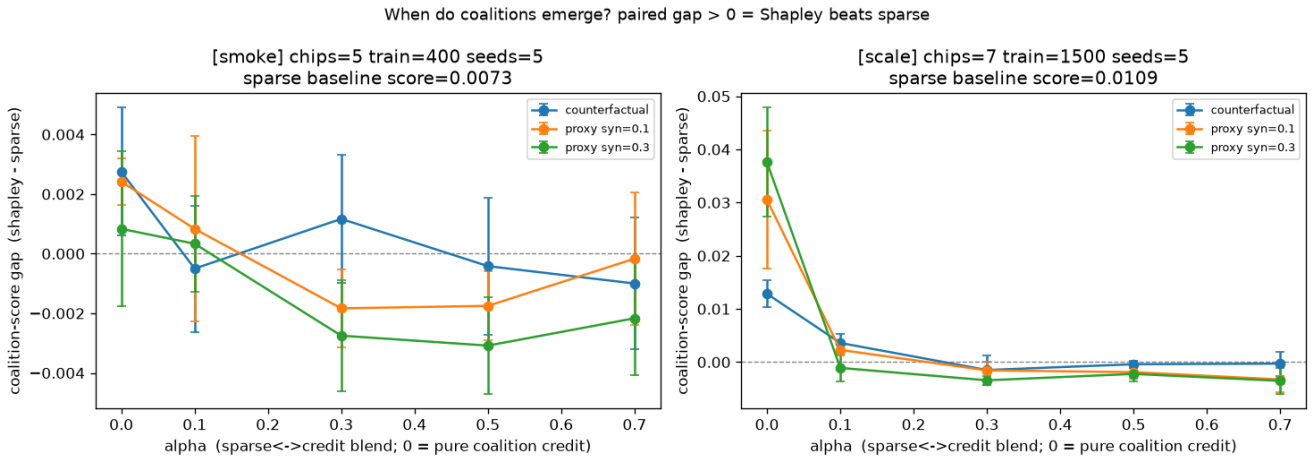


Figure 6: Paired coalition-score gap across the alpha x credit x synergy sweep (5 seeds, error bars): large and significant only in the low-alpha regime (peaking at $+0.038$ with the proxy at $\alpha=0$), negative for every $\alpha \geq 0.3$. The earlier null came from measuring in the $\alpha=0.3$ dead zone.

There is no free lunch: pure coalition credit ($\alpha = 0$) drops win-rate to ~ 0.29 (near the 0.25 random floor), while $\alpha \geq 0.1$ keeps it ~ 0.52 . Coalition-*forming* is the primary target and winning is secondary — exactly the raw step’s framing, now quantified.

Read more: the Step 09 MARL stack (this repo) for MAPPO with a centralized critic; and Bakhtin et al. (2022) on **piKL** — regularizing toward a behavioral prior instead of Nash, the N-player safe-play recipe this step points at.

11.5 EGTA and the spinning top — is SLS a wheel or a ladder?

The final tool evaluates the *population*. **Empirical game-theoretic analysis (EGTA)** treats whole strategies as the atoms of a meta-game, plays every pair to fill an empirical payoff matrix, and analyzes its structure. To reuse Step 9’s meta-Nash solver and Step 10’s **spinning-top** (Hodge) decomposition — both 2-player tools — the 4-player payoff *tensor* is **projected** to a pairwise matchup matrix, then split into a **transitive** (skill-ladder) component and a **cyclic** (rock-paper-scissors) component.

EGTA = Nash of a game whose 'strategies' are whole policies (exploitability has no meaning vs a coalition).

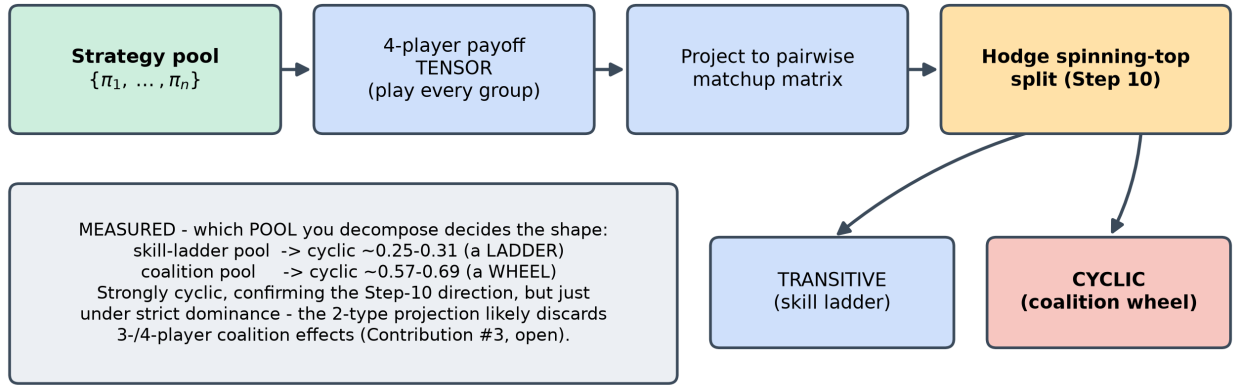


Figure 7: The EGTA + spinning-top pipeline for SLS: play every pair of strategies to fill a payoff tensor, project the 4-player tensor to a pairwise matchup matrix, and Hodge-decompose it into transitive (skill ladder) and cyclic (coalition counters) parts. Measured caveat: the 2-type projection likely discards 3-/4-player coalition effects, so the cyclic ratio is a lower bound.

Step 10 predicted FFA coalition games would be strongly cyclic. Measured, it depends entirely on **which population you decompose** — the same lesson Step 10 taught:

Population	Cyclic ratio	Structure
Skill-ladder pool	0.25-0.31	transitive-dominant (a ladder)
Coalition pool (ally/betray strategies)	~ 0.57-0.69	strongly cyclic (a wheel)

Reconciliation (kept prediction -> what actually happened). I expected a large cyclic component and, at first, saw a near-perfect skill ladder (cyclic ~ 0.07). That was partly the seat-0 bug (the *Shapley credit* section) and partly **pool composition**: the default baseline pool *is* a skill ladder. A coalition pool (fixed-ally + betrayer strategies) pushes the cyclic ratio to ~ 0.57-0.69 — a large non-transitive component that strongly confirms the *direction*

of the prediction, but stays **honestly short of strict dominance** (cyclic² just under 0.5). The prime suspect for the residual is that the **2-type projection discards 3-/4-player coalition effects** — a tensor-native decomposition is the open question. Neither reading is a bug; which population you build decides whether SLS looks like a wheel or a ladder.

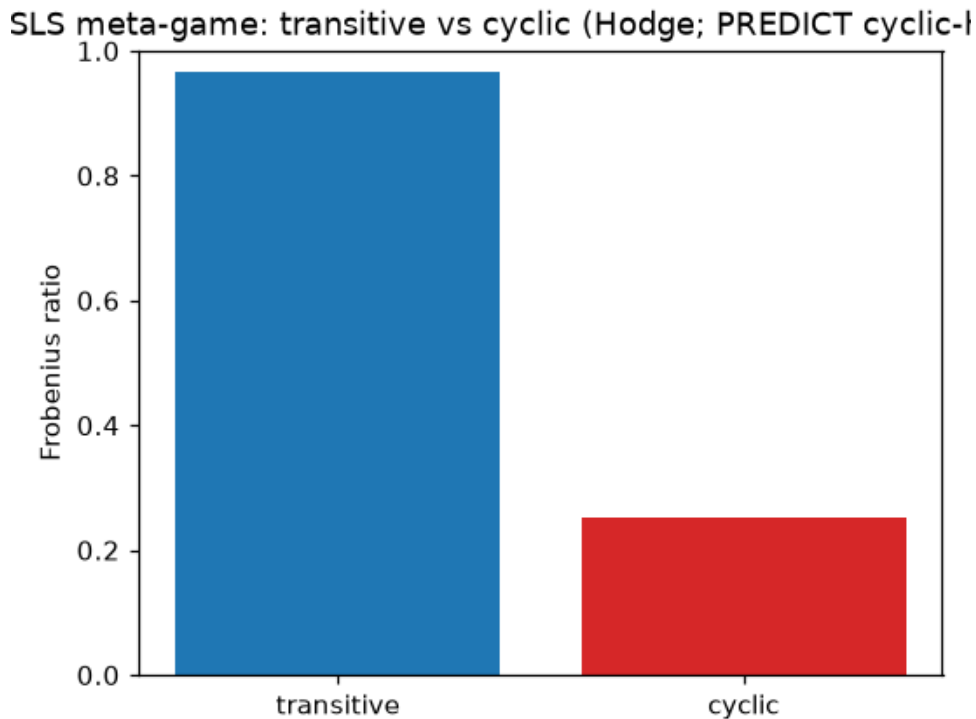


Figure 8: Spinning-top transitive/cyclic ratios: the skill-ladder pool is transitive-dominant (cyclic ~ 0.25 - 0.31), while the coalition pool is strongly cyclic (~ 0.57 - 0.69). Coalition play injects large non-transitivity, confirming the Step-10 direction while staying just under strict cyclic dominance.

Read more: Balduzzi, D. et al. (2019). “Open-ended Learning in Symmetric Zero-sum Games.” *ICML* (the spinning top); and Lanctot, M. et al. (2017), the PSRO / EGTA line (Step 9-10 stack).

11.6 Honest notes, limitations, and where this hands off

What held up. The exact anchors are solid: the SLS engine matches the 2-player minimax endgame with **zero** mismatches; the detector recovers a planted coalition **exactly** ($\{0, 1\}$, score 10.0); the Shapley code reproduces the glove and majority toys — *including their core* — to four decimals. And the headline learning claim survives a 5-seed sweep: coalition-aware training produces significantly more coalition behavior ($+0.038$, $\sim 4.4\times$) in the low- α regime.

What did not, and why it matters. Two honest corrections travel forward. (1) The “symmetric” game was not symmetric — a most-chips **tie-break bug** handed seat 0 $\sim 2\times$ its fair share and inflated the hero win-rate from a true ~ 0.41 to a false ~ 0.87 ; fixed, but it proves the engine’s *rules*, not its solver,

are where the risk lives. (2) “Coalitions don’t emerge at scale” was a **mis-set blend weight** ($\alpha = 0.3$ is a dead zone), not a failure of the method. A methodological echo of Steps 9-10: **a single config hides what a seeded sweep reveals.**

Trust. Every exact target (endgame minimax, cooperative-GT toys) is deterministic and reproducible; the training claims rest on a 5-seed paired sweep with error bars, so the *directions* are the trustworthy claims, not third-decimal magnitudes. The standing caveat is **engine fidelity**: the engine is certified against its own ruleset, not a transcription of De Carufel & Jerade — and the tie-break bug shows that reconciliation is not cosmetic. (The committed `scale_results.json` is a pre-fix artifact, cited only as evidence of the bug; authoritative scale numbers come from the sweep.)

Backward and forward connections. Backward: the detector is Step 7’s opponent model lifted to social structure; the meta-Nash and spinning-top are Steps 9-10 reused on the projected meta-game; the empty core makes Step 8’s “safe = bounded deviation from Nash” impossible, forcing piKL’s behavioral prior. Forward: the behavioral-prior safety gap points at **Step 12** (language / negotiation, CICERO / Welfare-Diplomacy), and the EGTA-tensor evaluation is the multi-agent generalization of exploitability that **Step 14** inherits (Contribution #3).

Open questions. Does the SLS engine match De Carufel & Jerade’s exact rules (the tie-break bug says this matters)? Does a 3-/4-player-aware EGTA projection push the cyclic ratio past strict dominance? Is the cheap proxy credit “good enough” in general, or only on SLS? And underneath all of it: with no Nash value and an empty core, **what does “safe” even mean** for an N-player coalition setting — the Contribution #2 gap this step frames but does not close.

11.7 Key takeaways for the thesis synthesis

- **At $N \geq 3$, exact evaluation is gone.** Nash is intractable and strategically empty in 4-player FFA; evaluation becomes empirical (win-rate + coalition score + EGTA cyclic ratio), anchored by the one exactly-solvable subgame (the 2-player minimax endgame, 0 mismatches).
- **Coalitions can be *learned* — but only if you weight the credit.** The Shapley coalition signal beats sparse reward by +0.038 ($\sim 4.4\times$, 5 seeds) at low α , and is *suppressed* at $\alpha \geq 0.3$; the effect grows with game size, and the cheap proxy beats the expensive counterfactual.
- **Coalition-forming costs winning.** $\alpha = 0$ drops win-rate to the ~ 0.25 floor; $\alpha \geq 0.1$ restores ~ 0.52 — forming is primary, winning secondary, quantified.
- **Empty core $\Rightarrow$ structural instability.** The majority game’s empty core is the SLS situation: no stable allocation exists, so coalitions *will* break — which is why N-player “safe” play needs a behavioral prior (piKL), not a Nash/core anchor (Contribution #2).
- **Which population you decompose decides ladder-vs-wheel** (the Step-10 lesson, confirmed): a skill-ladder pool looks transitive (~ 0.3 cyclic), a coalition pool looks strongly cyclic (~ 0.57 - 0.69).
- **Engine rule fidelity outranks solver code.** Both remaining weaknesses (the seat bias, the sub-dominant cyclic ratio) live in the engine’s simplifications, not in the Shapley/EGTA math — which

passed its textbook checks exactly.

Step 12 Summary — Sequence Models and LLM Agents in Strategic Settings

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

12 Step 12 — Sequence Models and LLM Agents in Strategic Settings	1
12.1 Why reframe reinforcement learning as sequence prediction	1
12.2 The luck-versus-skill trap	2
12.3 ARDT — conditioning on what you can guarantee	2
12.4 What return conditioning actually does in poker	3
12.5 Language models as strategic agents	4
12.6 Exploitation, adaptation, and the limits of the toy game	6
12.7 Key takeaways for the thesis synthesis	7

12 Step 12 — Sequence Models and LLM Agents in Strategic Settings

12.1 Why reframe reinforcement learning as sequence prediction

Classical RL optimises: estimate a value function, back up rewards, improve the policy. The Decision Transformer (Chen et al., 2021) proposes something different — treat the whole problem as **supervised sequence modelling**. Feed a GPT-style model a trajectory of triples

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

where $\hat{R}_t = \sum_{t' \geq t} r_{t'}$ is the *return-to-go*, and train it to predict a_t from everything up to s_t . There is no Bellman backup and no policy-improvement step. At inference you simply **condition**: ask for a high return-to-go and the model produces the actions that historically preceded such returns.

The appeal for this thesis is that it works entirely **offline**, on a fixed dataset — the regime Step 13’s Playtech logs live in, where self-play is unavailable.

Read more: Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modelling*, NeurIPS 2021 (arXiv:2106.01345).

12.2 The luck-versus-skill trap

The reframing hides an assumption: that the return-to-go is something the agent *earned*. In a stochastic environment it is not. Paster et al. (2022) show that return-conditioned policies systematically chase luck, and the smallest possible example makes it obvious.

Consider a one-step bandit. Action A pays 0.5 deterministically. Action B pays 1.0 with probability 0.4 and 0 otherwise, so $\mathbb{E}[B] = 0.4 < 0.5$. Action A is EV-optimal. But the *only* way a trajectory ever achieves a return of 1.0 is to take B **and get lucky**. Condition on “return = 1.0” and you recover exclusively B-trajectories:

$$P(a = B \mid \hat{R} = 1.0) = 1.00 \quad (\text{measured})$$

The return-conditioned learner confidently selects the action with the *worse* expected value. This is not a bug in the model; it is what conditioning on an outcome means when the outcome is partly noise.

Read more: Paster, McIlraith & Ba, *You Can’t Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments*, NeurIPS 2022 (arXiv:2205.15967).

12.3 ARDT — conditioning on what you can guarantee

The fix is to condition not on the return that *happened* but on the return the protagonist can **guarantee against a worst-case opponent** — the minimax return-to-go. ARDT (Tang et al., 2024) estimates it with **expectile regression**, which minimises the asymmetric squared loss

$$L_{\text{ER}}^{\alpha}(u) = \mathbb{E}_u[|\alpha - \mathbf{1}(u > 0)| \cdot u^2]$$

whose limits give the operators you need: $\alpha \rightarrow 0$ recovers the **minimum** and $\alpha \rightarrow 1$ the **maximum**. The trajectories are relabelled with the estimated minimax return and a standard DT is trained on the relabelled data.

Two details of the published method matter, and both were confirmed by reading the source rather than the summary. First, the pessimistic side is **low** α (the raw step’s claim that $\tau = 0.9$ is pessimistic is inverted); the paper itself runs $\alpha = 0.01$. Second — the important one — the relabel target is

$$\tilde{R}_t = \tilde{Q}_{\nu}(s_t, a_t)$$

a state-**action** value produced by two *coupled* estimators alternately fitted with α and $1 - \alpha$ losses. A state-only value $V(s)$ cannot distinguish “this state is bad” from “*this action* in this state is bad”, which is exactly the discrimination the method relies on.

Measured on Kuhn with a deliberately simplified state-only proxy, the relabel target moves monotonically with τ as theory requires — but exploitability is *lowest* on the optimistic side, which is the signature of

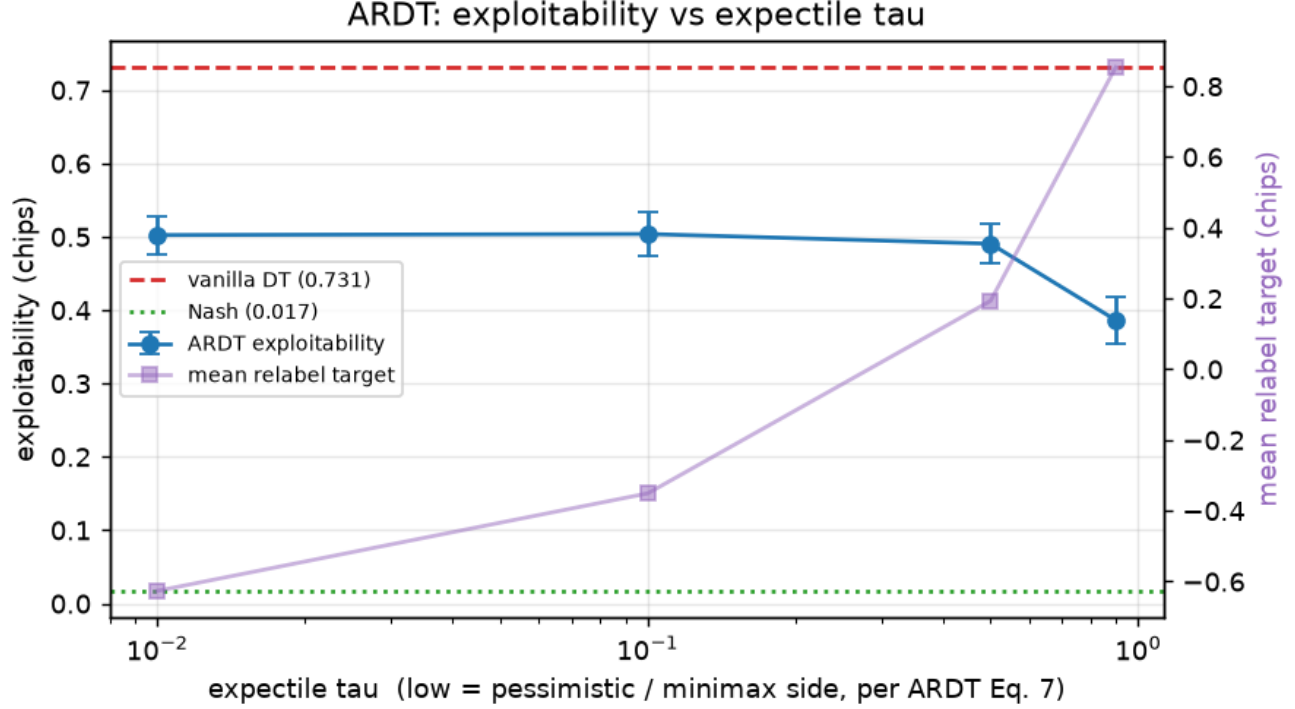


Figure 1: ARDT exploitability against the expectile tau, with the mean relabel target on the second axis.

the missing action argument rather than evidence against ARDT.

Read more: Tang, Zhang, Gu et al., *Adversarially Robust Decision Transformer*, NeurIPS 2024 (arXiv:2407.18414) — Section 3 and Algorithm 1.

12.4 What return conditioning actually does in poker

Kuhn Poker’s equilibrium and exploitability are exactly computable, so the question can be settled rather than argued. The result is that conditioning **changes** the policy substantially but does not **steer** it: exploitability is flat across the range of real target returns, with a sharp collapse at one specific value — the modal payoff, which in Kuhn is the payoff of folding.

The natural explanation is that in a four-payoff game the *magnitude* of the return encodes which betting line was played, while its *sign* encodes who held the better card. Conditioning therefore selects the shape of the hand rather than the quality of the play.

Leduc Hold’em tests that explanation with fifteen payoff values instead of four, two betting streets and a board card. **The collapse disappears exactly as the explanation predicts — and steering still does not appear** (Pearson $r = +0.062$). So the payoff-alphabet account explains the collapse but not the failure. The failure is more fundamental: in a zero-sum imperfect-information game the realised return is dominated by the opponent’s private card and actions, which the protagonist does not control. No amount of payoff granularity makes an uncontrollable signal steerable.

This is the argument that carries into Step 13. On fixed logs, a return-conditioned Decision Transformer is

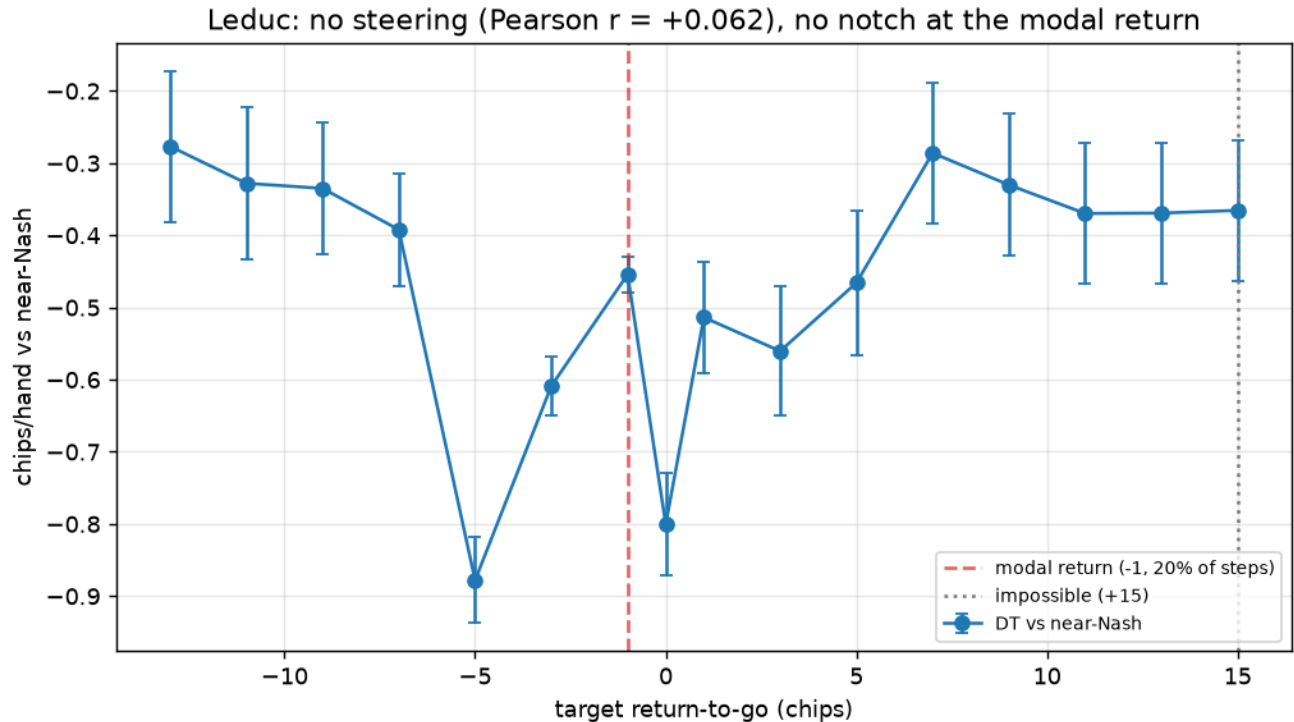


Figure 2: Decision Transformer performance against the conditioned target return on Leduc Hold’em, with standard errors.

the wrong instrument — and the value of ARDT’s relabeling is precisely that it replaces an uncontrollable conditioning target with a controllable one.

12.5 Language models as strategic agents

The second paradigm skips training altogether: describe the rules in English and let a language model play. Measured against the same exact ruler, LLMs are **honestly exploitable** — but the interesting result is *where* they fail.

They get hand **ranking** right and mixing **frequencies** wrong. Every backend tested value-bets the King with probability 1.00 where equilibrium mixes at 0.68, and none bluffs the Jack at all under a plain prompt. A 7B model matches a 20B model, because the game rewards correct mixing rather than knowledge or scale.

Decomposing the loss per decision overturns the obvious reading. The King “failure” — the largest visible deviation from equilibrium — costs **0.1%** of the total loss, because over-betting a hand that is never behind is nearly free. A single Queen decision costs **41.4%**. Deviation magnitude and cost are almost uncorrelated, which is the concrete argument for reporting per-decision diagnostics rather than a single exploitability number in the Step 14 evaluation framework.

Asking a model what it intends is *worse* than measuring what it does. Scored as strategies, the frequencies these models **state** are 2.6× and 4.0× more exploitable than the strategies they actually **play**. One model

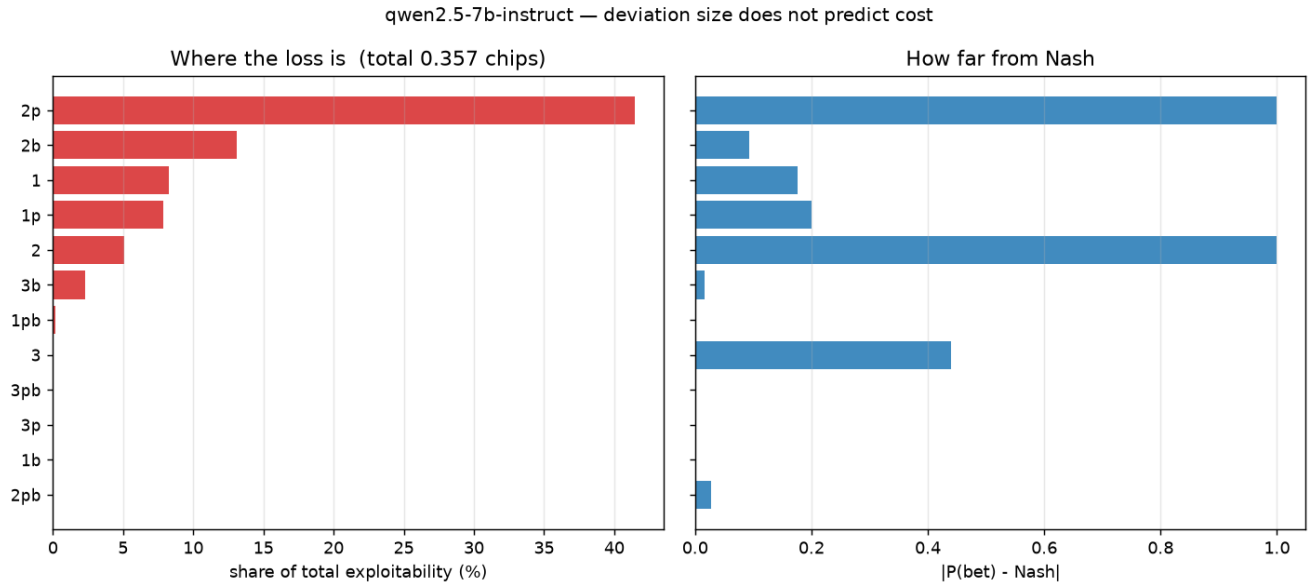


Figure 3: Each information set’s share of total exploitability, beside its deviation from equilibrium.

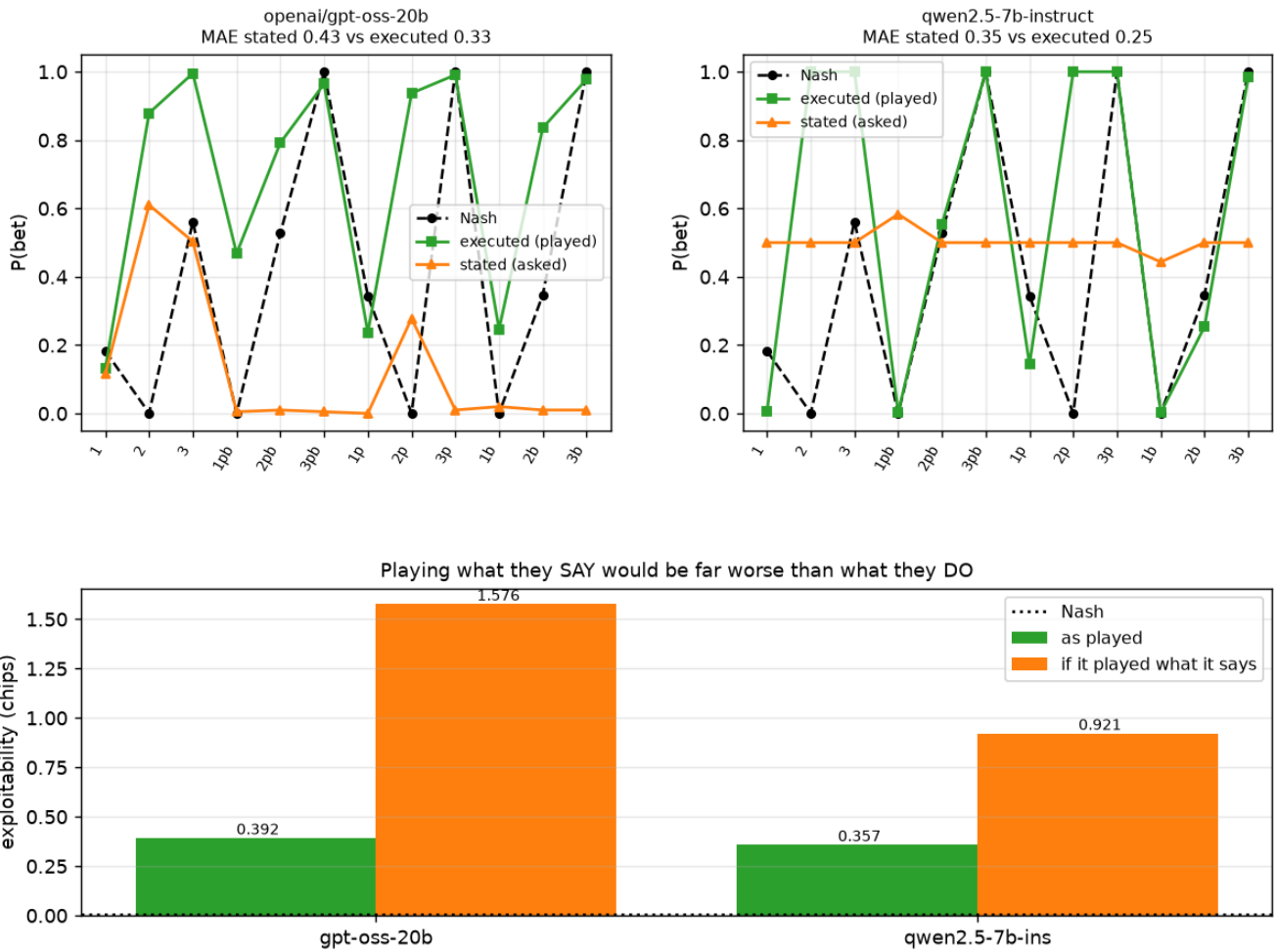


Figure 4: Stated versus executed betting frequencies for two models, with the exploitability of each strategy.

answers “50%” almost everywhere; the other answers “near 0%” almost everywhere, including with the strongest hand. Behavioural probing is not merely more convenient than introspection here — it is more accurate.

Read more: Guertler et al., *TextArena: A Framework for Text-Based Game Environments*, 2025 (arXiv:2504.11442).

12.6 Exploitation, adaptation, and the limits of the toy game

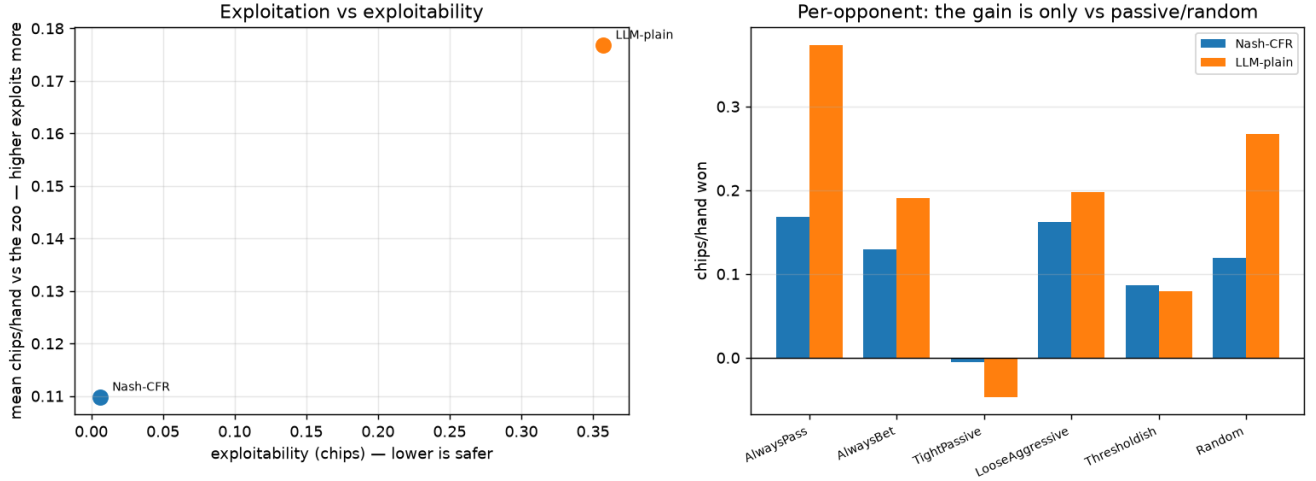


Figure 5: Exploitation against exploitability, with the per-opponent breakdown.

Exploitability measures how a perfect adversary punishes you; it says nothing about how well you punish weakness. Measured against a zoo of deliberately exploitable archetypes, the language model **exploits 61% harder than equilibrium play while being 59× more exploitable** — but the entire gain comes from passive and random opponents. Against the two most competent archetypes it does *worse* than equilibrium. That is the safe-exploitation trade-off of the thesis’s second contribution, observed rather than assumed.

Given the observed history of a session in context, the model does **not** adapt: it captures 83% of the available exploitation against a trivially passive opponent from the first half of the session onward, and never improves (mean learning -0.22). What looks like opponent modelling is a fixed loose-aggressive prior.

Finally, the toy game flatters these models. One street and one board card later, the LLM is statistically indistinguishable from the Decision Transformer, cannot beat weak opponents at all, and wants to take an illegal action with roughly a quarter of its probability mass. That failure is a *single* misconception rather than diffuse confusion — it wants to **fold when checking is free**, almost exclusively on weak unpaired hands against a high board, while never once attempting to raise past the betting cap. Any optimism drawn from the toy game should be read as a property of the toy game.

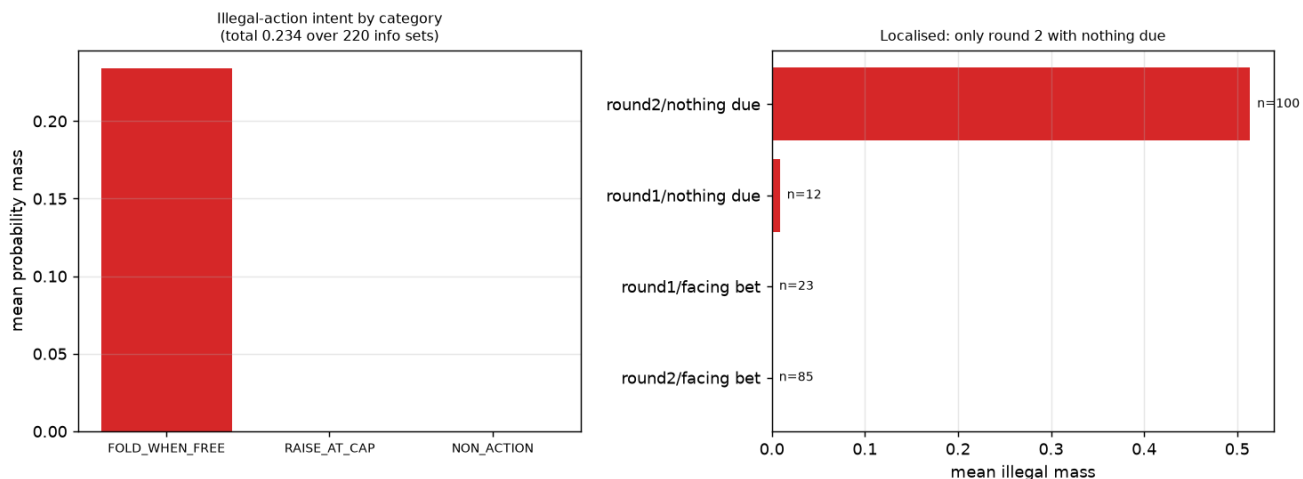


Figure 6: Illegal-action intent by category and by situation on Leduc Hold'em.

12.7 Key takeaways for the thesis synthesis

- **Conditioning on a quantity the agent does not control cannot steer it.** Demonstrated on two games; it is why ARDT's relabeling — not the Decision Transformer itself — is what transfers to fixed poker logs.
- **The published ARDT relabels with a state-action value.** Reproducing the method's benefit requires $\tilde{Q}(s, a)$, not $V(s)$ — the identified, evidence-backed fix for Step 13.
- **A scalar score ranks methods but does not diagnose them.** One decision carried 41.4% of the loss while the most conspicuous deviation carried 0.1%.
- **Measure behaviour, not self-report.** Verbalised strategy was substantially worse than played strategy for every model tested.
- **In-context opponent adaptation did not emerge**, so an explicit opponent model is required rather than assumed — directly relevant to the first thesis contribution.
- **Measurement protocol is part of the result.** At greedy decoding a language model plays a pure strategy and every measured frequency degenerates to 0 or 1; four separate findings in this step turned out to be measurement artefacts, each caught by a cheap check against an exactly computable quantity.